```
//// FILE: backend/app/main.py  (301 lines)

"""GeM BidVerify AI - HTTP API.

Run from the backend folder:   uvicorn app.main:app --reload --port 8000
Interactive docs:              http://localhost:8000/docs
All endpoints live under /api and (except /api/auth/* and /api/health) need a Bearer token.
"""
from typing import List, Literal, Optional

from fastapi import Depends, FastAPI, File, Form, Header, Query, Request, UploadFile
from fastapi.middleware.cors import CORSMiddleware
from fastapi.responses import FileResponse, JSONResponse
from pydantic import BaseModel

from . import db, services
from .services import ServiceError

app = FastAPI(title="GeM BidVerify AI", version="2.0.0",
              description="AI-assisted bid compliance verification for GeM procurement officers.")
# The API uses bearer tokens (no cookies), so a permissive CORS policy is safe for local use.
app.add_middleware(CORSMiddleware, allow_origins=["*"], allow_methods=["*"], allow_headers=["*"])


@app.on_event("startup")
def _startup():
    db.init_db()


@app.exception_handler(ServiceError)
async def _service_error(_: Request, exc: ServiceError):
    return JSONResponse(status_code=exc.status, content={"detail": exc.message})


def current_user(authorization: Optional[str] = Header(None)) -> dict:
    if not authorization or not authorization.lower().startswith("bearer "):
        raise ServiceError(401, "Authentication required.")
    return services.user_from_token(authorization.split(" ", 1)[1].strip())


# ---------------------------------------------------------------------------- request models
class LoginIn(BaseModel):
    username: str
    password: str


class OtpRequestIn(BaseModel):
    username: Optional[str] = None
    email: Optional[str] = None


class Verify2FAIn(BaseModel):
    username: Optional[str] = None
    email: Optional[str] = None
    otp: str


class ResetPasswordIn(BaseModel):
    username: Optional[str] = None
    email: Optional[str] = None
    otp: str
    new_password: str


class RegisterIn(BaseModel):
    username: str
    full_name: Optional[str] = None
    email: str
    password: str
    department: Optional[str] = None
    phone: Optional[str] = None


class RequirementIn(BaseModel):
    requirement_code: str
    requirement_name: str
    is_mandatory: bool = True
    verification_source: Optional[str] = None
    min_condition: Optional[str] = None
```

```
    weight: float = 10
    applicable_bidder_type: Optional[str] = "ALL"


class TenderIn(BaseModel):
    tender_id: Optional[str] = None
    title: str
    department: Optional[str] = ""
    tender_value: Optional[float] = None
    deadline: Optional[str] = None
    description: Optional[str] = ""
    requirements: List[RequirementIn] = []


class BidderIn(BaseModel):
    company_name: str
    cin: Optional[str] = None
    pan: Optional[str] = ""
    gstin: Optional[str] = None
    registered_address: Optional[str] = None
    contact_email: Optional[str] = None
    contact_phone: Optional[str] = None
    is_msme: bool = False
    msme_reg_no: Optional[str] = None
    is_startup: bool = False
    startup_dipp_no: Optional[str] = None


class BidIn(BaseModel):
    tender_id: int
    bidder_id: int


class ComplianceUpdateIn(BaseModel):
    points_awarded: float
    status: Literal["VERIFIED", "MISMATCH", "NON_COMPLIANT"]
    explanation: Optional[str] = None


class DecisionIn(BaseModel):
    decision: Literal["APPROVED", "REJECTED", "REQUIRES_CLARIFICATION"]
    comments: Optional[str] = ""
    clarification_details: Optional[str] = None


def _dump(model) -> dict:
    return model.model_dump() if hasattr(model, "model_dump") else model.dict()


# ---------------------------------------------------------------------------- health + auth
@app.get("/api/health")
def health():
    from . import extraction
    return {"status": "ok", "ocr_available": extraction.ocr_available()}


@app.post("/api/auth/login")
def login(body: LoginIn, request: Request):
    return services.login(body.username, body.password, request.client.host if request.client else "unknown")


@app.post("/api/auth/send-otp")
def send_otp(body: OtpRequestIn):
    return services.send_otp(body.username, body.email)


@app.post("/api/auth/verify-2fa")
def verify_2fa(body: Verify2FAIn):
    return services.verify_otp(body.username, body.email, body.otp)


@app.post("/api/auth/reset-password")
def reset_password(body: ResetPasswordIn):
    return services.reset_password(body.username, body.email, body.otp, body.new_password)


@app.post("/api/auth/register")
def register(body: RegisterIn):
    return services.register(_dump(body))
```

```
@app.get("/api/auth/me")
def me(user: dict = Depends(current_user)):
    return services._public_user(user)


# ---------------------------------------------------------------------------- dashboard
@app.get("/api/dashboard")
def dashboard(user: dict = Depends(current_user)):
    return services.dashboard()


# ---------------------------------------------------------------------------- tenders / bidders
@app.get("/api/tenders")
def list_tenders(user: dict = Depends(current_user)):
    return services.list_tenders()


@app.get("/api/tenders/{tender_ref}")
def get_tender(tender_ref: str, user: dict = Depends(current_user)):
    return services.get_tender(tender_ref)


@app.post("/api/tenders")
def create_tender(body: TenderIn, user: dict = Depends(current_user)):
    return services.create_tender(_dump(body), user)


@app.post("/api/tenders/{tender_id}/requirements")
def add_requirement(tender_id: int, body: RequirementIn, user: dict = Depends(current_user)):
    return services.add_requirement(tender_id, _dump(body), user)


@app.get("/api/bidders")
def list_bidders(user: dict = Depends(current_user)):
    return services.list_bidders()


@app.post("/api/bidders")
def create_bidder(body: BidderIn, user: dict = Depends(current_user)):
    return services.create_bidder(_dump(body), user)


# ---------------------------------------------------------------------------- bids
@app.get("/api/bids")
def list_bids(tender_id: Optional[int] = Query(None), user: dict = Depends(current_user)):
    return services.list_bids(tender_id)


@app.get("/api/bids/{bid_ref}")
def get_bid(bid_ref: str, user: dict = Depends(current_user)):
    return services.get_bid(bid_ref)


@app.post("/api/bids")
def create_bid(body: BidIn, user: dict = Depends(current_user)):
    return services.create_bid(body.tender_id, body.bidder_id, user)


@app.post("/api/bids/{bid_ref}/documents")
def upload_document(bid_ref: str, file: UploadFile = File(...), classified_type_hint: Optional[str] = Form(None)
    ,
                    user: dict = Depends(current_user)):
    return services.upload_document(bid_ref, file.filename, file.file.read(), classified_type_hint, user)


@app.delete("/api/bids/{bid_ref}/documents/{doc_id}")
def delete_document(bid_ref: str, doc_id: int, user: dict = Depends(current_user)):
    return services.delete_document(bid_ref, doc_id, user)


@app.get("/api/documents/{doc_id}/file")
def download_document(doc_id: int, user: dict = Depends(current_user)):
    path, name = services.document_file(doc_id)
    return FileResponse(path, filename=name)


@app.put("/api/bids/{bid_ref}/compliance/{result_id}")
```

```
def update_compliance(bid_ref: str, result_id: int, body: ComplianceUpdateIn, user: dict = Depends(current_user)
    ):
    return services.update_compliance_result(bid_ref, result_id, _dump(body), user)


@app.post("/api/bids/{bid_ref}/verify")
def verify_bid(bid_ref: str, user: dict = Depends(current_user)):
    return services.verify_bid(bid_ref, user)


@app.post("/api/bids/{bid_ref}/decision")
def decision(bid_ref: str, body: DecisionIn, user: dict = Depends(current_user)):
    return services.submit_decision(bid_ref, _dump(body), user)


# ---------------------------------------------------------------------------- integrations (simulated)
@app.get("/api/integrations/status")
def integrations_status(user: dict = Depends(current_user)):
    return services.integrations_status()


@app.post("/api/integrations/test-gst")
def test_gst(gstin: str, user: dict = Depends(current_user)):
    from . import adapters
    return adapters.lookup_gst(gstin)


@app.post("/api/integrations/test-pan")
def test_pan(pan: str, user: dict = Depends(current_user)):
    from . import adapters
    return adapters.lookup_pan(pan)


@app.post("/api/integrations/test-debarment")
def test_debarment(company_name: str, pan: str = "", user: dict = Depends(current_user)):
    from . import adapters
    return adapters.check_debarment(company_name, pan)


# ---------------------------------------------------------------------------- audit / notifications / admin
@app.get("/api/audit")
def audit(user: dict = Depends(current_user)):
    return services.audit_logs()


@app.get("/api/audit/verify")
def audit_verify(user: dict = Depends(current_user)):
    return services.audit_verify()


@app.get("/api/notifications")
def notifications(user: dict = Depends(current_user)):
    return services.notifications()


@app.post("/api/notifications/{notification_id}/read")
def read_notification(notification_id: int, user: dict = Depends(current_user)):
    return services.mark_notification_read(notification_id)


@app.get("/api/admin/export")
def export_data(user: dict = Depends(current_user)):
    return services.export_data()


@app.post("/api/admin/reset-demo")
def reset_demo(user: dict = Depends(current_user)):
    return services.reset_demo(user)


//// FILE: backend/app/config.py  (60 lines)

"""Runtime configuration.

Everything can be overridden with environment variables; the defaults are safe for a
local demo (SQLite file next to the code, OTP shown in the console).
"""
import os
import secrets
```

```
from pathlib import Path

BASE_DIR = Path(__file__).resolve().parent.parent
DATA_DIR = Path(os.getenv("GEM_DATA_DIR", BASE_DIR / "data"))
UPLOAD_DIR = DATA_DIR / "uploads"
DB_PATH = Path(os.getenv("GEM_DB_PATH", DATA_DIR / "gem_bidverify.db"))
for _p in (DATA_DIR, UPLOAD_DIR):
    _p.mkdir(parents=True, exist_ok=True)


def _load_secret() -> str:
    """JWT signing key: from GEM_SECRET_KEY, else generated once and kept in data/secret.key."""
    env = os.getenv("GEM_SECRET_KEY")
    if env:
        return env
    key_file = DATA_DIR / "secret.key"
    if not key_file.exists():
        key_file.write_text(secrets.token_hex(32))
    return key_file.read_text().strip()


SECRET_KEY = _load_secret()
TOKEN_TTL_HOURS = 8

OTP_TTL_SECONDS = 300
OTP_MAX_ATTEMPTS = 5
LOGIN_MAX_FAILURES = 5
LOGIN_LOCK_SECONDS = 300
MIN_PASSWORD_LENGTH = 6

MAX_UPLOAD_MB = 15
ALLOWED_EXTENSIONS = {".pdf", ".png", ".jpg", ".jpeg", ".tif", ".tiff", ".txt"}

# E-mail (optional). Without SMTP_HOST the OTP is printed in the server console.
SMTP_HOST = os.getenv("GEM_SMTP_HOST", "")
SMTP_PORT = int(os.getenv("GEM_SMTP_PORT", "587"))
SMTP_USER = os.getenv("GEM_SMTP_USER", "")
SMTP_PASSWORD = os.getenv("GEM_SMTP_PASSWORD", "")
SMTP_FROM = os.getenv("GEM_SMTP_FROM", SMTP_USER or "noreply@example.gov.in")
# For local demos the OTP is also returned to the browser when no mail server is set up.
SHOW_OTP_IN_RESPONSE = os.getenv("GEM_SHOW_OTP", "false" if SMTP_HOST else "true").lower() == "true"

DEMO_OFFICER_EMAIL = os.getenv("GEM_DEMO_OFFICER_EMAIL", "officer@example.gov.in")

# OCR
OCR_LANG = os.getenv("GEM_OCR_LANG", "eng")
TESSERACT_CMD = os.getenv("TESSERACT_CMD", "")
_WIN_TESSERACT = Path(r"C:\Program Files\Tesseract-OCR\tesseract.exe")
if not TESSERACT_CMD and _WIN_TESSERACT.exists():
    TESSERACT_CMD = str(_WIN_TESSERACT)

# If true a GSTIN whose check digit is wrong is treated as a failure (real GSTINs always pass).
STRICT_GSTIN_CHECKSUM = os.getenv("GEM_STRICT_GSTIN", "false").lower() == "true"


//// FILE: backend/app/db.py  (214 lines)

"""SQLite storage: schema, small helpers, hash-chained audit log, first-run demo data."""
import hashlib
import json
import sqlite3
from contextlib import contextmanager
from datetime import datetime, timezone
from pathlib import Path

from . import config, security

SCHEMA = """
CREATE TABLE IF NOT EXISTS users (
  id INTEGER PRIMARY KEY AUTOINCREMENT, username TEXT UNIQUE NOT NULL, full_name TEXT NOT NULL,
  role TEXT NOT NULL, department TEXT, email TEXT, phone TEXT, password_hash TEXT NOT NULL,
  is_active INTEGER NOT NULL DEFAULT 1);
CREATE TABLE IF NOT EXISTS otps (
  key TEXT PRIMARY KEY, otp_hash TEXT NOT NULL, email TEXT, expires_at REAL NOT NULL, attempts INTEGER NOT NULL
    DEFAULT 0);
CREATE TABLE IF NOT EXISTS tenders (
  id INTEGER PRIMARY KEY AUTOINCREMENT, tender_id TEXT UNIQUE NOT NULL, title TEXT NOT NULL, department TEXT,
  tender_value REAL, deadline TEXT, status TEXT NOT NULL DEFAULT 'ACTIVE', description TEXT, created_at TEXT NOT
     NULL);
```

```
CREATE TABLE IF NOT EXISTS requirements (
  id INTEGER PRIMARY KEY AUTOINCREMENT, tender_pk INTEGER NOT NULL REFERENCES tenders(id) ON DELETE CASCADE,
  requirement_code TEXT NOT NULL, requirement_name TEXT NOT NULL, is_mandatory INTEGER NOT NULL DEFAULT 1,
  verification_source TEXT, min_condition TEXT, weight REAL NOT NULL DEFAULT 10, applicable_bidder_type TEXT DEF
    AULT 'ALL');
CREATE TABLE IF NOT EXISTS bidders (
  id INTEGER PRIMARY KEY AUTOINCREMENT, company_name TEXT NOT NULL, cin TEXT, pan TEXT, gstin TEXT,
  registered_address TEXT, contact_email TEXT, contact_phone TEXT, is_msme INTEGER DEFAULT 0, msme_reg_no TEXT,
  is_startup INTEGER DEFAULT 0, startup_dipp_no TEXT);
CREATE TABLE IF NOT EXISTS bids (
  id INTEGER PRIMARY KEY AUTOINCREMENT, bid_id TEXT UNIQUE NOT NULL,
  tender_pk INTEGER NOT NULL REFERENCES tenders(id), bidder_pk INTEGER NOT NULL REFERENCES bidders(id),
  submission_date TEXT NOT NULL, status TEXT NOT NULL, compliance_score INTEGER DEFAULT 0, risk_level TEXT DEFAU
    LT 'LOW',
  ai_recommendation TEXT, ai_explanation TEXT, risk_score INTEGER DEFAULT 0, risk_factors TEXT DEFAULT '[]',
  recommended_action TEXT);
CREATE TABLE IF NOT EXISTS documents (
  id INTEGER PRIMARY KEY AUTOINCREMENT, bid_pk INTEGER NOT NULL REFERENCES bids(id) ON DELETE CASCADE,
  file_name TEXT NOT NULL, file_type TEXT, file_size_kb INTEGER, stored_path TEXT, classified_type TEXT,
  classification_confidence REAL, ocr_status TEXT, extracted_fields TEXT DEFAULT '[]', created_at TEXT NOT NULL)
    ;
CREATE TABLE IF NOT EXISTS compliance_results (
  id INTEGER PRIMARY KEY AUTOINCREMENT, bid_pk INTEGER NOT NULL REFERENCES bids(id) ON DELETE CASCADE,
  requirement_code TEXT NOT NULL, requirement_name TEXT, status TEXT NOT NULL, points_awarded REAL, max_points R
    EAL,
  evidence_document TEXT, confidence_score REAL, rule_applied TEXT, explanation TEXT, flags TEXT DEFAULT '[]',
  is_mandatory INTEGER DEFAULT 1, officer_override INTEGER DEFAULT 0);
CREATE TABLE IF NOT EXISTS decisions (
  id INTEGER PRIMARY KEY AUTOINCREMENT, bid_pk INTEGER UNIQUE NOT NULL REFERENCES bids(id) ON DELETE CASCADE,
  officer_name TEXT, decision TEXT NOT NULL, comments TEXT, clarification_details TEXT, decided_at TEXT NOT NULL
    );
CREATE TABLE IF NOT EXISTS audit_logs (
  id INTEGER PRIMARY KEY AUTOINCREMENT, bid_id INTEGER, user_name TEXT, action TEXT NOT NULL, entity_type TEXT,
  details TEXT, rule_applied TEXT, ip_address TEXT, timestamp TEXT NOT NULL, prev_hash TEXT, entry_hash TEXT);
CREATE TABLE IF NOT EXISTS notifications (
  id INTEGER PRIMARY KEY AUTOINCREMENT, title TEXT, message TEXT, category TEXT, is_read INTEGER DEFAULT 0,
  link TEXT, created_at TEXT NOT NULL);
CREATE TABLE IF NOT EXISTS connectors (
  id TEXT PRIMARY KEY, name TEXT, type TEXT, mode TEXT, status TEXT, latency_ms INTEGER, badge TEXT);
CREATE TABLE IF NOT EXISTS evaluations (
  id INTEGER PRIMARY KEY AUTOINCREMENT, bid_pk INTEGER NOT NULL, evaluated_at TEXT NOT NULL, score INTEGER,
  risk_level TEXT, recommendation TEXT);
"""


def now_iso() -> str:
    return datetime.now(timezone.utc).isoformat(timespec="milliseconds").replace("+00:00", "Z")


@contextmanager
def connect():
    conn = sqlite3.connect(config.DB_PATH, timeout=30)
    conn.row_factory = sqlite3.Row
    conn.execute("PRAGMA foreign_keys=ON")
    try:
        yield conn
        conn.commit()
    except Exception:
        conn.rollback()
        raise
    finally:
        conn.close()


def rows(conn, sql, params=()):
    return [dict(r) for r in conn.execute(sql, params).fetchall()]


def row(conn, sql, params=()):
    r = conn.execute(sql, params).fetchone()
    return dict(r) if r else None


def loads(value, default):
    try:
        return json.loads(value) if value else default
    except (TypeError, ValueError):
        return default
```

```
# ---------------------------------------------------------------------------------------
# Tamper-evident audit log: every entry stores SHA-256(previous hash + entry content).
# Changing or deleting an old row breaks the chain, which verify_audit_chain() detects.
# ---------------------------------------------------------------------------------------
def _entry_hash(prev_hash: str, entry: dict) -> str:
    payload = json.dumps(entry, sort_keys=True, ensure_ascii=False)
    return hashlib.sha256((prev_hash + payload).encode()).hexdigest()


def add_audit(conn, action, details, entity_type="BID", bid_id=None, user_name="System",
              rule_applied=None, ip_address="127.0.0.1", timestamp=None):
    last = row(conn, "SELECT entry_hash FROM audit_logs ORDER BY id DESC LIMIT 1")
    prev = last["entry_hash"] if last and last["entry_hash"] else "GENESIS"
    entry = {"bid_id": bid_id, "user_name": user_name, "action": action, "entity_type": entity_type,
             "details": details, "rule_applied": rule_applied, "ip_address": ip_address,
             "timestamp": timestamp or now_iso()}
    conn.execute(
        "INSERT INTO audit_logs (bid_id,user_name,action,entity_type,details,rule_applied,ip_address,timestamp,p
    rev_hash,entry_hash)"
        " VALUES (:bid_id,:user_name,:action,:entity_type,:details,:rule_applied,:ip_address,:timestamp,:prev,:h
    )",
        {**entry, "prev": prev, "h": _entry_hash(prev, entry)})


def verify_audit_chain(conn) -> dict:
    prev, count = "GENESIS", 0
    for r in conn.execute("SELECT * FROM audit_logs ORDER BY id"):
        r = dict(r)
        entry = {k: r[k] for k in ("bid_id", "user_name", "action", "entity_type", "details",
                                   "rule_applied", "ip_address", "timestamp")}
        if r["prev_hash"] != prev or r["entry_hash"] != _entry_hash(prev, entry):
            return {"valid": False, "entries_checked": count, "first_broken_id": r["id"]}
        prev, count = r["entry_hash"], count + 1
    return {"valid": True, "entries_checked": count, "first_broken_id": None}


# ---------------------------------------------------------------------------------------
# Schema + demo data
# ---------------------------------------------------------------------------------------
def init_db():
    with connect() as conn:
        conn.executescript(SCHEMA)
        if not row(conn, "SELECT id FROM users LIMIT 1"):
            seed_demo_data(conn)


def _b(v):
    return 1 if v else 0


def seed_demo_data(conn):
    """Load the demo tenders, bidders, bids and users that ship with the project."""
    seeds = json.loads((Path(__file__).parent / "seed_data" / "seeds.json").read_text(encoding="utf-8"))

    for u in seeds["seedUsers"]:
        conn.execute(
            "INSERT INTO users (id,username,full_name,role,department,email,phone,password_hash,is_active) VALUE
    S (?,?,?,?,?,?,?,?,1)",
            (u["id"], u["username"], u["full_name"], u["role"], u["department"], config.DEMO_OFFICER_EMAIL,
             u.get("phone"), security.hash_password(u["password"])))

    for t in seeds["seedTenders"]:
        conn.execute(
            "INSERT INTO tenders (id,tender_id,title,department,tender_value,deadline,status,description,created
    _at) VALUES (?,?,?,?,?,?,?,?,?)",
            (t["id"], t["tender_id"], t["title"], t["department"], t["tender_value"], t["deadline"], t["status"]
    ,
             t.get("description"), t["created_at"]))
        for r in t["requirements"]:
            conn.execute(
                "INSERT INTO requirements (tender_pk,requirement_code,requirement_name,is_mandatory,verification
    _source,min_condition,weight,applicable_bidder_type) VALUES (?,?,?,?,?,?,?,?)",
                (t["id"], r["requirement_code"], r["requirement_name"], _b(r["is_mandatory"]), r["verification_s
    ource"],
                 r["min_condition"], r["weight"], r.get("applicable_bidder_type", "ALL")))

    for b in seeds["seedBidders"]:
        conn.execute(
```

```
            "INSERT INTO bidders (id,company_name,cin,pan,gstin,registered_address,contact_email,contact_phone,i
    s_msme,msme_reg_no,is_startup,startup_dipp_no) VALUES (?,?,?,?,?,?,?,?,?,?,?,?)",
            (b["id"], b["company_name"], b.get("cin"), b.get("pan"), b.get("gstin"), b.get("registered_address")
    ,
             b.get("contact_email"), b.get("contact_phone"), _b(b.get("is_msme")), b.get("msme_reg_no"),
             _b(b.get("is_startup")), b.get("startup_dipp_no")))

    for bid in seeds["seedBids"]:
        ra = bid["risk_assessment"]
        conn.execute(
            "INSERT INTO bids (id,bid_id,tender_pk,bidder_pk,submission_date,status,compliance_score,risk_level,
    ai_recommendation,ai_explanation,risk_score,risk_factors,recommended_action) VALUES (?,?,?,?,?,?,?,?,?,?,?,?
    ,?)",
            (bid["id"], bid["bid_id"], bid["tender_id"], bid["bidder_id"], bid["submission_date"], bid["status"]
    ,
             bid["compliance_score"], bid["risk_level"], bid["ai_recommendation"], bid["ai_explanation"],
             ra["risk_score"], json.dumps(ra["risk_factors"]), ra["recommended_action"]))
        for d in bid["documents"]:
            conn.execute(
                "INSERT INTO documents (id,bid_pk,file_name,file_type,file_size_kb,stored_path,classified_type,c
    lassification_confidence,ocr_status,extracted_fields,created_at) VALUES (?,?,?,?,?,?,?,?,?,?,?)",
                (d["id"], bid["id"], d["file_name"], d["file_type"], d["file_size_kb"], None, d["classified_type
    "],
                 d["classification_confidence"], d["ocr_status"], json.dumps(d["extracted_fields"]), d["created_
    at"]))
        for c in bid["compliance_results"]:
            conn.execute(
                "INSERT INTO compliance_results (id,bid_pk,requirement_code,requirement_name,status,points_award
    ed,max_points,evidence_document,confidence_score,rule_applied,explanation,flags) VALUES (?,?,?,?,?,?,?,?,?,?
    ,?,'[]')",
                (c["id"], bid["id"], c["requirement_code"], c["requirement_name"], c["status"], c["points_awarde
    d"],
                 c["max_points"], c["evidence_document"], c["confidence_score"], c["rule_applied"], c["explanati
    on"]))
        conn.execute("INSERT INTO evaluations (bid_pk,evaluated_at,score,risk_level,recommendation) VALUES (?,?,
    ?,?,?)",
                     (bid["id"], bid["submission_date"], bid["compliance_score"], bid["risk_level"], bid["ai_rec
    ommendation"]))

    for a in seeds["seedAuditLogs"]:
        add_audit(conn, a["action"], a["details"], a.get("entity_type", "BID"), None, a["user_name"],
                  a.get("rule_applied"), a.get("ip_address", "127.0.0.1"), a["timestamp"])
    for n in seeds["seedNotifications"]:
        conn.execute("INSERT INTO notifications (id,title,message,category,is_read,link,created_at) VALUES (?,?,
    ?,?,?,?,?)",
                     (n["id"], n["title"], n["message"], n["category"], _b(n["is_read"]), n.get("link"), n["crea
    ted_at"]))
    for c in seeds["seedConnectors"]:
        conn.execute("INSERT INTO connectors (id,name,type,mode,status,latency_ms,badge) VALUES (?,?,?,?,?,?,?)"
    ,
                     (c["id"], c["name"], c["type"], c["mode"], c["status"], c["latency_ms"], "Simulated"))


def reset_demo_data():
    """Drop everything and reload the demo data (uploaded files are removed as well)."""
    with connect() as conn:
        for table in ("evaluations", "decisions", "compliance_results", "documents", "bids", "requirements",
                      "tenders", "bidders", "audit_logs", "notifications", "connectors", "otps", "users"):
            conn.execute(f"DELETE FROM {table}")
        conn.execute("DELETE FROM sqlite_sequence")
        seed_demo_data(conn)
        add_audit(conn, "DATA_RESET_DEFAULT", "Demo data restored to the shipped baseline.", "SYSTEM", None, "Sy
    stem")
    for f in config.UPLOAD_DIR.glob("**/*"):
        if f.is_file():
            f.unlink()


//// FILE: backend/app/security.py  (64 lines)

"""Password hashing, JWT tokens and one-time passwords (standard library + PyJWT)."""
import base64
import hashlib
import hmac
import secrets
import time

import jwt
```

```
from . import config

_ITERATIONS = 200_000


def hash_password(password: str) -> str:
    salt = secrets.token_bytes(16)
    digest = hashlib.pbkdf2_hmac("sha256", password.encode(), salt, _ITERATIONS)
    return "pbkdf2_sha256${}${}${}".format(
        _ITERATIONS, base64.b64encode(salt).decode(), base64.b64encode(digest).decode()
    )


def verify_password(password: str, stored: str) -> bool:
    try:
        _, iterations, salt_b64, digest_b64 = stored.split("$")
        salt = base64.b64decode(salt_b64)
        expected = base64.b64decode(digest_b64)
        actual = hashlib.pbkdf2_hmac("sha256", password.encode(), salt, int(iterations))
        return hmac.compare_digest(actual, expected)
    except (ValueError, TypeError):
        return False


def create_token(user: dict) -> str:
    now = int(time.time())
    payload = {
        "sub": str(user["id"]),
        "username": user["username"],
        "iat": now,
        "exp": now + config.TOKEN_TTL_HOURS * 3600,
    }
    return jwt.encode(payload, config.SECRET_KEY, algorithm="HS256")


def decode_token(token: str) -> dict:
    """Return the token payload or raise jwt.InvalidTokenError."""
    return jwt.decode(token, config.SECRET_KEY, algorithms=["HS256"])


def generate_otp() -> str:
    return str(100000 + secrets.randbelow(900000))


def hash_otp(otp: str) -> str:
    return hmac.new(config.SECRET_KEY.encode(), otp.encode(), hashlib.sha256).hexdigest()


def mask_email(email: str) -> str:
    if "@" not in email:
        return email
    local, domain = email.split("@", 1)
    if len(local) <= 2:
        return f"{local}*@{domain}"
    return local[:2] + "*" * max(2, len(local) - 3) + local[-1] + "@" + domain


//// FILE: backend/app/services.py  (644 lines)

"""Business logic. Every function returns plain dictionaries shaped exactly like the frontend expects."""
import json
import random
import re
import smtplib
import time
import uuid
from datetime import datetime, timedelta, timezone
from email.message import EmailMessage
from pathlib import Path

from . import adapters, config, db, extraction, rules, security
from .db import connect, loads, now_iso, row, rows
from .extraction import PAN_RE, GSTIN_RE


class ServiceError(Exception):
    def __init__(self, status: int, message: str):
        super().__init__(message)
        self.status, self.message = status, message
```

```
def _num(x):
    """JSON-friendly number: 10.0 -> 10, 7.5 -> 7.5"""
    if x is None:
        return None
    return int(x) if float(x).is_integer() else round(float(x), 2)


# ============================================================================================
# Authentication
# ============================================================================================
_login_failures = {}  # username -> (count, locked_until)


def _public_user(u: dict) -> dict:
    return {"id": u["id"], "username": u["username"], "full_name": u["full_name"], "role": u["role"],
            "department": u["department"], "email": u["email"], "phone": u["phone"], "is_active": bool(u["is_act
    ive"])}


def _session(u: dict) -> dict:
    return {"access_token": security.create_token(u), "token_type": "bearer", "user": _public_user(u)}


def _find_user(conn, username=None, email=None):
    if username:
        u = row(conn, "SELECT * FROM users WHERE username=? OR lower(email)=lower(?)", (username, username))
        if u:
            return u
    if email:
        return row(conn, "SELECT * FROM users WHERE lower(email)=lower(?)", (email,))
    return None


def user_from_token(token: str) -> dict:
    import jwt
    try:
        payload = security.decode_token(token)
    except jwt.InvalidTokenError:
        raise ServiceError(401, "Session expired or invalid. Please sign in again.")
    with connect() as conn:
        u = row(conn, "SELECT * FROM users WHERE id=? AND is_active=1", (int(payload["sub"]),))
    if not u:
        raise ServiceError(401, "Account not found or disabled.")
    return u


def login(username: str, password: str, ip: str = "unknown") -> dict:
    count, locked_until = _login_failures.get(username, (0, 0))
    if locked_until > time.time():
        raise ServiceError(429, "Too many failed attempts. Try again in a few minutes.")
    with connect() as conn:
        u = row(conn, "SELECT * FROM users WHERE username=? AND is_active=1", (username,))
        if not u or not security.verify_password(password, u["password_hash"]):
            count += 1
            lock = time.time() + config.LOGIN_LOCK_SECONDS if count >= config.LOGIN_MAX_FAILURES else 0
            _login_failures[username] = (0 if lock else count, lock)
            db.add_audit(conn, "LOGIN_FAILED", f"Failed sign-in attempt for '{username}'.", "AUTH", None, userna
    me, ip_address=ip)
            conn.commit()
            raise ServiceError(401, "Invalid username or password.")
        _login_failures.pop(username, None)
        db.add_audit(conn, "OFFICER_LOGIN", f"Officer '{u['full_name']}' authenticated via GeM console.", "AUTH"
    ,
                     None, u["full_name"], ip_address=ip)
        return _session(u)


def _send_mail(to: str, subject: str, body: str):
    msg = EmailMessage()
    msg["From"], msg["To"], msg["Subject"] = config.SMTP_FROM, to, subject
    msg.set_content(body)
    with smtplib.SMTP(config.SMTP_HOST, config.SMTP_PORT, timeout=20) as smtp:
        smtp.starttls()
        if config.SMTP_USER:
            smtp.login(config.SMTP_USER, config.SMTP_PASSWORD)
        smtp.send_message(msg)
```

```
def send_otp(username=None, email=None) -> dict:
    with connect() as conn:
        u = _find_user(conn, username, email)
        target = (u["email"] if u else None) or email or ""
        masked = security.mask_email(target) if target else "your registered e-mail"
        generic = {"success": True, "message": f"If the account exists, an OTP was sent to {masked}",
                   "masked_email": masked, "email_masked": masked}
        if not u:
            return generic  # do not reveal whether the account exists
        otp = security.generate_otp()
        conn.execute("INSERT OR REPLACE INTO otps (key,otp_hash,email,expires_at,attempts) VALUES (?,?,?,?,0)",
                     (u["username"], security.hash_otp(otp), u["email"], time.time() + config.OTP_TTL_SECONDS))
        db.add_audit(conn, "OTP_REQUESTED", f"OTP requested for '{u['username']}'.", "AUTH", None, u["full_name"
    ])
    body = (f"Your One-Time Password for GeM BidVerify AI is: {otp}\n\n"
            f"It is valid for {config.OTP_TTL_SECONDS // 60} minutes. If you did not request it, ignore this e-m
    ail.")
    if config.SMTP_HOST and u["email"]:
        try:
            _send_mail(u["email"], "[GeM BidVerify AI] Your one-time password", body)
        except Exception as exc:  # network / credentials problem
            raise ServiceError(502, f"Could not send the OTP e-mail: {exc}")
    else:
        print(f"[GeM 2FA] OTP for {u['username']}: {otp}  (no SMTP configured - shown here for local demo)")
    result = dict(generic)
    if config.SHOW_OTP_IN_RESPONSE:
        result["otp"] = otp
    return result


def _check_otp(conn, u: dict, otp: str):
    rec = row(conn, "SELECT * FROM otps WHERE key=?", (u["username"],))
    if not rec:
        raise ServiceError(400, "No active OTP found. Please request a new OTP.")
    if time.time() > rec["expires_at"]:
        conn.execute("DELETE FROM otps WHERE key=?", (u["username"],))
        raise ServiceError(400, "OTP has expired. Please request a new one.")
    if security.hash_otp(str(otp or "")) != rec["otp_hash"]:
        attempts = rec["attempts"] + 1
        if attempts >= config.OTP_MAX_ATTEMPTS:
            conn.execute("DELETE FROM otps WHERE key=?", (u["username"],))
            conn.commit()
            raise ServiceError(400, "Too many failed attempts. Please request a new OTP.")
        conn.execute("UPDATE otps SET attempts=? WHERE key=?", (attempts, u["username"]))
        conn.commit()
        raise ServiceError(400, "Invalid OTP code. Please check and try again.")
    conn.execute("DELETE FROM otps WHERE key=?", (u["username"],))


def verify_otp(username=None, email=None, otp=None) -> dict:
    with connect() as conn:
        u = _find_user(conn, username, email)
        if not u:
            raise ServiceError(400, "No active OTP found. Please request a new OTP.")
        _check_otp(conn, u, otp)
        db.add_audit(conn, "2FA_VERIFIED", f"2FA OTP verified for '{u['full_name']}'.", "AUTH", None, u["full_na
    me"])
        return _session(u)


def reset_password(username=None, email=None, otp=None, new_password="") -> dict:
    if len(new_password or "") < config.MIN_PASSWORD_LENGTH:
        raise ServiceError(422, f"Password must be at least {config.MIN_PASSWORD_LENGTH} characters.")
    with connect() as conn:
        u = _find_user(conn, username, email)
        if not u:
            raise ServiceError(400, "No active OTP found. Please request a new OTP.")
        _check_otp(conn, u, otp)
        conn.execute("UPDATE users SET password_hash=? WHERE id=?", (security.hash_password(new_password), u["id
    "]))
        db.add_audit(conn, "PASSWORD_RESET", f"Password reset for user '{u['username']}'.", "AUTH", None, u["ful
    l_name"])
    return {"success": True, "message": "Password has been successfully updated. Please sign in with your new pa
    ssword."}


def register(data: dict) -> dict:
    username = (data.get("username") or "").strip()
```

```
    email = (data.get("email") or "").strip()
    if not re.fullmatch(r"[A-Za-z0-9_.-]{3,32}", username):
        raise ServiceError(422, "Username must be 3-32 characters (letters, digits, . _ -).")
    if "@" not in email:
        raise ServiceError(422, "A valid e-mail address is required.")
    if len(data.get("password") or "") < config.MIN_PASSWORD_LENGTH:
        raise ServiceError(422, f"Password must be at least {config.MIN_PASSWORD_LENGTH} characters.")
    with connect() as conn:
        if row(conn, "SELECT id FROM users WHERE lower(username)=lower(?) OR lower(email)=lower(?)", (username,
    email)):
            raise ServiceError(409, "An account with this username or e-mail already exists.")
        cur = conn.execute(
            "INSERT INTO users (username,full_name,role,department,email,phone,password_hash,is_active) VALUES (
    ?,?,?,?,?,?,?,1)",
            (username, (data.get("full_name") or username).strip(), "Procurement Officer",
             data.get("department") or "Department of Electronics & IT (GeM Evaluation Cell)", email,
             data.get("phone") or "", security.hash_password(data["password"])))
        u = row(conn, "SELECT * FROM users WHERE id=?", (cur.lastrowid,))
        db.add_audit(conn, "ACCOUNT_CREATED", f"New account created for '{u['full_name']}' ({username}).", "AUTH
    ", None, u["full_name"])
        return _session(u)


# ============================================================================================
# Tenders and bidders
# ============================================================================================
def _requirement(r: dict) -> dict:
    return {"id": r["id"], "requirement_code": r["requirement_code"], "requirement_name": r["requirement_name"],
            "is_mandatory": bool(r["is_mandatory"]), "verification_source": r["verification_source"],
            "min_condition": r["min_condition"], "weight": _num(r["weight"]),
            "applicable_bidder_type": r["applicable_bidder_type"] or "ALL"}


def _tender(conn, t: dict) -> dict:
    reqs = rows(conn, "SELECT * FROM requirements WHERE tender_pk=? ORDER BY id", (t["id"],))
    count = row(conn, "SELECT COUNT(*) AS n FROM bids WHERE tender_pk=?", (t["id"],))["n"]
    return {"id": t["id"], "tender_id": t["tender_id"], "title": t["title"], "department": t["department"],
            "tender_value": _num(t["tender_value"]), "deadline": t["deadline"], "status": t["status"],
            "description": t["description"] or "", "created_at": t["created_at"], "bids_count": count,
            "requirements": [_requirement(r) for r in reqs]}


def list_tenders() -> list:
    with connect() as conn:
        return [_tender(conn, t) for t in rows(conn, "SELECT * FROM tenders ORDER BY id DESC")]


def get_tender(ref) -> dict:
    with connect() as conn:
        t = row(conn, "SELECT * FROM tenders WHERE id=? OR tender_id=?", (_as_int(ref), str(ref)))
        if not t:
            raise ServiceError(404, "Tender not found")
        return _tender(conn, t)


def _as_int(v):
    try:
        return int(v)
    except (TypeError, ValueError):
        return -1


def _insert_requirement(conn, tender_pk, r: dict):
    conn.execute(
        "INSERT INTO requirements (tender_pk,requirement_code,requirement_name,is_mandatory,verification_source,
    min_condition,weight,applicable_bidder_type) VALUES (?,?,?,?,?,?,?,?)",
        (tender_pk, r["requirement_code"], r["requirement_name"], 1 if r.get("is_mandatory", True) else 0,
         r.get("verification_source") or "PORTAL_CROSS_CHECK", r.get("min_condition") or "Standard GFR Rule 149
    condition",
          float(r.get("weight") or 10), r.get("applicable_bidder_type") or "ALL"))


def create_tender(data: dict, user: dict) -> dict:
    title = (data.get("title") or "").strip()
    if not title:
        raise ServiceError(422, "Tender title is required.")
    with connect() as conn:
        code = (data.get("tender_id") or "").strip() or f"GEM/{datetime.now().year}/B/{random.randint(10000, 999
    99)}"
```

```
        if row(conn, "SELECT id FROM tenders WHERE tender_id=?", (code,)):
            raise ServiceError(409, f"Tender ID {code} already exists.")
        deadline = data.get("deadline") or (datetime.now() + timedelta(days=14)).date().isoformat()
        cur = conn.execute(
            "INSERT INTO tenders (tender_id,title,department,tender_value,deadline,status,description,created_at
    ) VALUES (?,?,?,?,?,?,?,?)",
            (code, title, data.get("department") or "", float(data.get("tender_value") or 5_000_000), deadline,
             "ACTIVE", data.get("description") or "", now_iso()))
        for r in data.get("requirements") or []:
            _insert_requirement(conn, cur.lastrowid, r)
        db.add_audit(conn, "TENDER_PUBLISHED", f"New Tender '{code}' published: {title}", "TENDER", None, user["
    full_name"])
        return _tender(conn, row(conn, "SELECT * FROM tenders WHERE id=?", (cur.lastrowid,)))


def add_requirement(tender_pk: int, data: dict, user: dict) -> dict:
    with connect() as conn:
        if not row(conn, "SELECT id FROM tenders WHERE id=?", (tender_pk,)):
            raise ServiceError(404, "Tender not found")
        _insert_requirement(conn, tender_pk, data)
        db.add_audit(conn, "REQUIREMENT_ADDED", f"Requirement {data.get('requirement_code')} added to tender {te
    nder_pk}.",
                     "TENDER", None, user["full_name"])
    return {"success": True}


def _bidder(b: dict) -> dict:
    return {"id": b["id"], "company_name": b["company_name"], "cin": b["cin"] or None, "pan": b["pan"],
            "gstin": b["gstin"] or None, "registered_address": b["registered_address"] or None,
            "contact_email": b["contact_email"] or None, "contact_phone": b["contact_phone"] or None,
            "is_msme": bool(b["is_msme"]), "msme_reg_no": b["msme_reg_no"] or None,
            "is_startup": bool(b["is_startup"]), "startup_dipp_no": b["startup_dipp_no"] or None}


def list_bidders() -> list:
    with connect() as conn:
        return [_bidder(b) for b in rows(conn, "SELECT * FROM bidders ORDER BY id DESC")]


def create_bidder(data: dict, user: dict) -> dict:
    name = (data.get("company_name") or "").strip()
    pan = (data.get("pan") or "").strip().upper()
    gstin = (data.get("gstin") or "").strip().upper()
    if not name:
        raise ServiceError(422, "Company name is required.")
    if pan and not PAN_RE.fullmatch(pan):
        raise ServiceError(422, "PAN must look like ABCDE1234F.")
    if gstin and not GSTIN_RE.fullmatch(gstin):
        raise ServiceError(422, "GSTIN must be a valid 15-character GSTIN.")
    with connect() as conn:
        cur = conn.execute(
            "INSERT INTO bidders (company_name,cin,pan,gstin,registered_address,contact_email,contact_phone,is_m
    sme,msme_reg_no,is_startup,startup_dipp_no) VALUES (?,?,?,?,?,?,?,?,?,?,?)",
            (name, data.get("cin") or None, pan, gstin or None, data.get("registered_address") or None,
             data.get("contact_email") or None, data.get("contact_phone") or None, 1 if data.get("is_msme") else
     0,
             data.get("msme_reg_no") or None, 1 if data.get("is_startup") else 0, data.get("startup_dipp_no") or
     None))
        db.add_audit(conn, "BIDDER_REGISTERED", f"Bidder '{name}' registered with PAN {pan}", "BIDDER", None, us
    er["full_name"])
        return _bidder(row(conn, "SELECT * FROM bidders WHERE id=?", (cur.lastrowid,)))


# ============================================================================
# Bids
# ============================================================================
def _date_in(iso: str) -> str:
    try:
        return datetime.fromisoformat(iso.replace("Z", "+00:00")).strftime("%d/%m/%Y")
    except (ValueError, AttributeError):
        return ""


def _document(d: dict) -> dict:
    fields = loads(d["extracted_fields"], [])
    for i, f in enumerate(fields):
        f.setdefault("id", d["id"] * 1000 + i)
    return {"id": d["id"], "bid_id": d["bid_pk"], "file_name": d["file_name"], "file_type": d["file_type"],
            "file_size_kb": d["file_size_kb"], "classified_type": d["classified_type"],
```

```
            "classification_confidence": d["classification_confidence"], "ocr_status": d["ocr_status"],
            "created_at": d["created_at"], "extracted_fields": fields, "has_file": bool(d["stored_path"])}


def _result(r: dict) -> dict:
    return {"id": r["id"], "requirement_code": r["requirement_code"], "requirement_name": r["requirement_name"],
            "status": r["status"], "points_awarded": _num(r["points_awarded"]), "max_points": _num(r["max_points
    "]),
            "evidence_document": r["evidence_document"], "confidence_score": r["confidence_score"],
            "rule_applied": r["rule_applied"], "explanation": r["explanation"], "officer_override": bool(r["offi
    cer_override"])}


def _bid_full(conn, b: dict) -> dict:
    tender = _tender(conn, row(conn, "SELECT * FROM tenders WHERE id=?", (b["tender_pk"],)))
    bidder = _bidder(row(conn, "SELECT * FROM bidders WHERE id=?", (b["bidder_pk"],)))
    docs = [_document(d) for d in rows(conn, "SELECT * FROM documents WHERE bid_pk=? ORDER BY id", (b["id"],))]
    results = [_result(r) for r in rows(conn, "SELECT * FROM compliance_results WHERE bid_pk=? ORDER BY id", (b[
    "id"],))]
    decision = row(conn, "SELECT * FROM decisions WHERE bid_pk=?", (b["id"],))
    return {
        "id": b["id"], "bid_id": b["bid_id"], "tender_id": b["tender_pk"], "bidder_id": b["bidder_pk"],
        "submission_date": b["submission_date"], "status": b["status"], "compliance_score": b["compliance_score"
    ],
        "risk_level": b["risk_level"], "ai_recommendation": b["ai_recommendation"], "ai_explanation": b["ai_expl
    anation"],
        "bidder": bidder, "tender": tender, "documents": docs, "compliance_results": results,
        "risk_assessment": {"id": b["id"], "risk_level": b["risk_level"], "risk_score": b["risk_score"],
                            "risk_factors": loads(b["risk_factors"], []), "recommendation": b["ai_recommendation
    "],
                            "recommended_action": b["recommended_action"]},
        "officer_decision": ({"id": decision["id"], "officer_name": decision["officer_name"], "decision": decisi
    on["decision"],
                              "comments": decision["comments"], "clarification_details": decision["clarification
    _details"],
                              "decided_at": decision["decided_at"]} if decision else None),
    }


def _bid_row(conn, ref):
    b = row(conn, "SELECT * FROM bids WHERE id=? OR bid_id=?", (_as_int(ref), str(ref)))
    if not b:
        raise ServiceError(404, "Bid not found")
    return b


def get_bid(ref) -> dict:
    with connect() as conn:
        return _bid_full(conn, _bid_row(conn, ref))


def _bid_summary_rows(conn, where="", params=()):
    out = []
    for b in rows(conn, f"SELECT * FROM bids {where} ORDER BY id DESC", params):
        tender = row(conn, "SELECT tender_id FROM tenders WHERE id=?", (b["tender_pk"],))
        bidder = row(conn, "SELECT company_name, pan FROM bidders WHERE id=?", (b["bidder_pk"],))
        docs = row(conn, "SELECT COUNT(*) AS n FROM documents WHERE bid_pk=?", (b["id"],))["n"]
        last = row(conn, "SELECT evaluated_at FROM evaluations WHERE bid_pk=? ORDER BY id DESC LIMIT 1", (b["id"
    ],))
        out.append({"id": b["id"], "bid_id": b["bid_id"], "tender_id": tender["tender_id"], "bidder_name": bidde
    r["company_name"],
                    "bidder_pan": bidder["pan"], "documents_count": docs, "compliance_score": b["compliance_scor
    e"],
                    "risk_level": b["risk_level"], "status": b["status"],
                    "last_verified": _date_in(last["evaluated_at"] if last else b["submission_date"])})
    return out


def list_bids(tender_pk=None) -> list:
    with connect() as conn:
        if tender_pk:
            return _bid_summary_rows(conn, "WHERE tender_pk=?", (int(tender_pk),))
        return _bid_summary_rows(conn)


_DECISION_STATUS = {"APPROVED": "APPROVED", "REJECTED": "REJECTED", "REQUIRES_CLARIFICATION": "REQUIRES_CLARIFIC
    ATION"}
```

```
def _reevaluate(conn, bid_pk: int, user_name: str = "AI Rule Engine"):
    """Run the rule engine for a bid, keep the officer's manual overrides, and store the outcome."""
    b = row(conn, "SELECT * FROM bids WHERE id=?", (bid_pk,))
    bidder = _bidder(row(conn, "SELECT * FROM bidders WHERE id=?", (b["bidder_pk"],)))
    reqs = [_requirement(r) for r in rows(conn, "SELECT * FROM requirements WHERE tender_pk=?", (b["tender_pk"],
    ))]
    docs = [_document(d) for d in rows(conn, "SELECT * FROM documents WHERE bid_pk=?", (bid_pk,))]

    if not docs:
        summary = rules.awaiting_documents(bidder, reqs)
        conn.execute("DELETE FROM compliance_results WHERE bid_pk=? AND officer_override=0", (bid_pk,))
        merged_ocr = []
    else:
        overrides = {r["requirement_code"]: r for r in
                     rows(conn, "SELECT * FROM compliance_results WHERE bid_pk=? AND officer_override=1", (bid_p
    k,))}
        fresh = rules.evaluate_requirements(bidder, reqs, docs)
        conn.execute("DELETE FROM compliance_results WHERE bid_pk=?", (bid_pk,))
        final = []
        for r in fresh:
            kept = overrides.get(r["requirement_code"])
            if kept:
                r = {**r, "status": kept["status"], "points_awarded": kept["points_awarded"],
                     "explanation": kept["explanation"], "rule_applied": kept["rule_applied"]}
            final.append(r)
            conn.execute(
                "INSERT INTO compliance_results (bid_pk,requirement_code,requirement_name,status,points_awarded,
    max_points,evidence_document,confidence_score,rule_applied,explanation,flags,is_mandatory,officer_override)
    VALUES (?,?,?,?,?,?,?,?,?,?,?,?,?)",
                (bid_pk, r["requirement_code"], r["requirement_name"], r["status"], r["points_awarded"], r["max_
    points"],
                 r["evidence_document"], r["confidence_score"], r["rule_applied"], r["explanation"], json.dumps(
    r["flags"]),
                 1 if r["is_mandatory"] else 0, 1 if kept else 0))
        merged_ocr = [d["classification_confidence"] for d in docs]
        summary = rules.summarize(final, len(docs), merged_ocr)

    decision = row(conn, "SELECT decision FROM decisions WHERE bid_pk=?", (bid_pk,))
    status = _DECISION_STATUS[decision["decision"]] if decision else summary["status"]
    _store_summary(conn, bid_pk, summary, status)
    db.add_audit(conn, "BID_COMPLIANCE_EVALUATION",
                 f"Automated evaluation completed for {b['bid_id']}. Compliance Score: {summary['compliance_scor
    e']}/100, "
                 f"Risk Level: {summary['risk_level']}.", "BID", bid_pk, user_name, "GFR_2017_RULE_149")
    return summary


def _store_summary(conn, bid_pk, summary, status):
    conn.execute(
        "UPDATE bids SET status=?, compliance_score=?, risk_level=?, ai_recommendation=?, ai_explanation=?, risk
    _score=?, risk_factors=?, recommended_action=? WHERE id=?",
        (status, summary["compliance_score"], summary["risk_level"], summary["ai_recommendation"], summary["ai_e
    xplanation"],
         summary["risk_score"], json.dumps(summary["risk_factors"]), summary["recommended_action"], bid_pk))
    conn.execute("INSERT INTO evaluations (bid_pk,evaluated_at,score,risk_level,recommendation) VALUES (?,?,?,?,
    ?)",
                 (bid_pk, now_iso(), summary["compliance_score"], summary["risk_level"], summary["ai_recommendat
    ion"]))


def create_bid(tender_pk, bidder_pk, user: dict) -> dict:
    with connect() as conn:
        t = row(conn, "SELECT * FROM tenders WHERE id=?", (_as_int(tender_pk),))
        bidder = row(conn, "SELECT * FROM bidders WHERE id=?", (_as_int(bidder_pk),))
        if not t:
            raise ServiceError(404, "Tender not found")
        if not bidder:
            raise ServiceError(404, "Bidder not found")
        if row(conn, "SELECT id FROM bids WHERE tender_pk=? AND bidder_pk=?", (t["id"], bidder["id"])):
            raise ServiceError(409, f"{bidder['company_name']} already has a bid on {t['tender_id']}.")
        number = (t["tender_id"].split("/") + ["", "", "", "00000"])[3] or "00000"
        prefix = re.sub(r"[^A-Z]", "", bidder["company_name"].upper())[:3].ljust(3, "X")
        seq = row(conn, "SELECT COUNT(*) AS n FROM bids WHERE tender_pk=?", (t["id"],))["n"] + 1
        bid_code = f"BID/{number}/{prefix}-{seq:02d}"
        while row(conn, "SELECT id FROM bids WHERE bid_id=?", (bid_code,)):
            seq += 1
            bid_code = f"BID/{number}/{prefix}-{seq:02d}"
        cur = conn.execute(
            "INSERT INTO bids (bid_id,tender_pk,bidder_pk,submission_date,status) VALUES (?,?,?,?,'UNDER_VERIFIC
```

```
    ATION')",
            (bid_code, t["id"], bidder["id"], now_iso()))
        db.add_audit(conn, "CREATE_BID_SUBMISSION",
                     f"Registered new bid '{bid_code}' for '{bidder['company_name']}' under tender '{t['tender_i
    d']}'.",
                     "BID", cur.lastrowid, user["full_name"])
        _reevaluate(conn, cur.lastrowid)
        conn.execute("INSERT INTO notifications (title,message,category,is_read,link,created_at) VALUES (?,?,?,?
    ,?,?)",
                     ("New Bid Received", f"Bidder '{bidder['company_name']}' submitted bid for {t['tender_id']}
    ",
                      "INFO", 0, f"/bids/{cur.lastrowid}", now_iso()))
        return _bid_full(conn, row(conn, "SELECT * FROM bids WHERE id=?", (cur.lastrowid,)))


def upload_document(bid_ref, filename: str, content: bytes, type_hint, user: dict) -> dict:
    suffix = Path(filename or "").suffix.lower()
    if suffix not in config.ALLOWED_EXTENSIONS:
        raise ServiceError(415, f"File type '{suffix or 'unknown'}' is not supported. Allowed: {', '.join(sorted
    (config.ALLOWED_EXTENSIONS))}")
    if not content:
        raise ServiceError(422, "The uploaded file is empty.")
    if len(content) > config.MAX_UPLOAD_MB * 1024 * 1024:
        raise ServiceError(413, f"File is larger than {config.MAX_UPLOAD_MB} MB.")
    with connect() as conn:
        b = _bid_row(conn, bid_ref)
        folder = config.UPLOAD_DIR / str(b["id"])
        folder.mkdir(parents=True, exist_ok=True)
        stored = folder / f"{uuid.uuid4().hex}{suffix}"
        stored.write_bytes(content)
        analysis = extraction.analyze_document(stored, filename, type_hint)
        clean_name = re.sub(r"[^\w.\- ]", "_", Path(filename).name)[:120]
        cur = conn.execute(
            "INSERT INTO documents (bid_pk,file_name,file_type,file_size_kb,stored_path,classified_type,classifi
    cation_confidence,ocr_status,extracted_fields,created_at) VALUES (?,?,?,?,?,?,?,?,?,?)",
            (b["id"], clean_name, suffix.lstrip("."), max(1, round(len(content) / 1024)), str(stored),
             analysis["classified_type"], analysis["classification_confidence"], analysis["ocr_status"],
             json.dumps(analysis["extracted_fields"]), now_iso()))
        db.add_audit(conn, "DOCUMENT_UPLOADED",
                     f"Uploaded document '{clean_name}' classified as {analysis['classified_type']} for bid {b['
    bid_id']}",
                     "DOCUMENT", b["id"], user["full_name"])
        _reevaluate(conn, b["id"])
        return _document(row(conn, "SELECT * FROM documents WHERE id=?", (cur.lastrowid,)))


def delete_document(bid_ref, doc_pk, user: dict) -> dict:
    with connect() as conn:
        b = _bid_row(conn, bid_ref)
        d = row(conn, "SELECT * FROM documents WHERE id=? AND bid_pk=?", (_as_int(doc_pk), b["id"]))
        if not d:
            raise ServiceError(404, "Document not found")
        conn.execute("DELETE FROM documents WHERE id=?", (d["id"],))
        if d["stored_path"]:
            Path(d["stored_path"]).unlink(missing_ok=True)
        db.add_audit(conn, "DOCUMENT_DELETED", f"Deleted document '{d['file_name']}' from bid {b['bid_id']}",
                     "DOCUMENT", b["id"], user["full_name"])
        _reevaluate(conn, b["id"])
    return {"success": True}


def document_file(doc_pk):
    with connect() as conn:
        d = row(conn, "SELECT * FROM documents WHERE id=?", (_as_int(doc_pk),))
    if not d or not d["stored_path"] or not Path(d["stored_path"]).exists():
        raise ServiceError(404, "The original file is not available for this document.")
    return Path(d["stored_path"]), d["file_name"]


def update_compliance_result(bid_ref, result_pk, data: dict, user: dict) -> dict:
    status = data.get("status")
    if status not in ("VERIFIED", "MISMATCH", "NON_COMPLIANT"):
        raise ServiceError(422, "Status must be VERIFIED, MISMATCH or NON_COMPLIANT.")
    with connect() as conn:
        b = _bid_row(conn, bid_ref)
        r = row(conn, "SELECT * FROM compliance_results WHERE id=? AND bid_pk=?", (_as_int(result_pk), b["id"]))
        if not r:
            raise ServiceError(404, "Compliance result not found")
        points = max(0.0, min(float(data.get("points_awarded") or 0), float(r["max_points"] or 0)))
```

```
        explanation = data.get("explanation") or r["explanation"]
        conn.execute("UPDATE compliance_results SET points_awarded=?, status=?, explanation=?, officer_override=
    1 WHERE id=?",
                     (points, status, explanation, r["id"]))
        results = []
        for x in rows(conn, "SELECT * FROM compliance_results WHERE bid_pk=?", (b["id"],)):
            x["flags"] = loads(x["flags"], [])
            x["is_mandatory"] = bool(x["is_mandatory"])
            results.append(x)
        docs = row(conn, "SELECT COUNT(*) AS n FROM documents WHERE bid_pk=?", (b["id"],))["n"]
        summary = rules.summarize(results, docs)
        decision = row(conn, "SELECT decision FROM decisions WHERE bid_pk=?", (b["id"],))
        _store_summary(conn, b["id"], summary, _DECISION_STATUS[decision["decision"]] if decision else summary["
    status"])
        db.add_audit(conn, "HUMAN_OFFICER_VERIFICATION_EVALUATION",
                     f"Officer adjusted score for '{r['requirement_name']}' to {_num(points)}/{_num(r['max_point
    s'])} pts. "
                     f"Total score updated to {summary['compliance_score']}/100.", "COMPLIANCE", b["id"], user["
    full_name"])
        return {"success": True, "compliance_score": summary["compliance_score"]}


def verify_bid(bid_ref, user: dict) -> dict:
    with connect() as conn:
        b = _bid_row(conn, bid_ref)
        _reevaluate(conn, b["id"], user["full_name"])
        return _bid_full(conn, row(conn, "SELECT * FROM bids WHERE id=?", (b["id"],)))


def submit_decision(bid_ref, data: dict, user: dict) -> dict:
    decision = data.get("decision")
    if decision not in _DECISION_STATUS:
        raise ServiceError(422, "Decision must be APPROVED, REJECTED or REQUIRES_CLARIFICATION.")
    with connect() as conn:
        b = _bid_row(conn, bid_ref)
        conn.execute("DELETE FROM decisions WHERE bid_pk=?", (b["id"],))
        decided_at = now_iso()
        cur = conn.execute(
            "INSERT INTO decisions (bid_pk,officer_name,decision,comments,clarification_details,decided_at) VALU
    ES (?,?,?,?,?,?)",
            (b["id"], user["full_name"], decision, data.get("comments") or "", data.get("clarification_details")
    , decided_at))
        conn.execute("UPDATE bids SET status=? WHERE id=?", (_DECISION_STATUS[decision], b["id"]))
        db.add_audit(conn, f"OFFICER_DECISION_{decision}",
                     f"Officer signed decision '{decision}' for bid {b['bid_id']}. Remarks: {data.get('comments'
    ) or ''}",
                     "DECISION", b["id"], user["full_name"])
        return {"success": True, "officer_decision": {
            "id": cur.lastrowid, "officer_name": user["full_name"], "decision": decision, "comments": data.get("
    comments") or "",
            "clarification_details": data.get("clarification_details"), "decided_at": decided_at}}


# ============================================================================================
# Dashboard, integrations, audit, notifications, admin
# ============================================================================================
def dashboard() -> dict:
    with connect() as conn:
        bids = rows(conn, "SELECT * FROM bids")
        tenders = rows(conn, "SELECT status FROM tenders")
        by = lambda f: sum(1 for b in bids if f(b))
        today = datetime.now(timezone.utc).date()
        activity = []
        for offset in range(5, -1, -1):
            day = today - timedelta(days=offset)
            evals = [e for e in rows(conn, "SELECT evaluated_at, risk_level, recommendation FROM evaluations")
                     if e["evaluated_at"][:10] == day.isoformat()]
            activity.append({"date": "Today" if offset == 0 else day.strftime("%d %b"),
                             "verified": sum(1 for e in evals if e["recommendation"] == "QUALIFIED_WITH_HIGH_CON
    FIDENCE"),
                             "flagged": sum(1 for e in evals if e["risk_level"] in ("HIGH", "CRITICAL"))})
        recent = [{k: v for k, v in s.items() if k != "bidder_pan"} for s in _bid_summary_rows(conn)]
        return {
            "total_tenders": len(tenders), "active_tenders": sum(1 for t in tenders if t["status"] == "ACTIVE"),
            "bids_under_verification": by(lambda b: b["status"] in ("UNDER_VERIFICATION", "PENDING_REVIEW")),
            "verified_bids": by(lambda b: b["status"] in ("VERIFIED", "APPROVED")),
            "high_risk_bids": by(lambda b: b["risk_level"] in ("HIGH", "CRITICAL")),
            "pending_reviews": by(lambda b: b["status"] == "PENDING_REVIEW" or b["risk_level"] == "MEDIUM"),
            "compliance_overview": {"Compliant": by(lambda b: b["compliance_score"] >= 80),
```

```
                                    "Review Required": by(lambda b: 60 <= b["compliance_score"] < 80),
                                    "Non-Compliant": by(lambda b: b["compliance_score"] < 60)},
            "risk_distribution": {lvl.capitalize(): by(lambda b, l=lvl: b["risk_level"] == l)
                                 for lvl in ("LOW", "MEDIUM", "HIGH", "CRITICAL")},
            "verification_activity": activity, "recent_bids": recent,
        }


def integrations_status() -> dict:
    with connect() as conn:
        connectors = rows(conn, "SELECT * FROM connectors ORDER BY rowid")
    return {"connectors": connectors,
            "disclaimer": "All statutory connectors currently run in SIMULATION mode with demo registry data. "
                          "Each adapter is isolated in backend/app/adapters.py so a real API can replace it once
     official access is granted."}


def audit_logs(limit: int = 500) -> list:
    with connect() as conn:
        out = rows(conn, "SELECT id,bid_id,user_name,action,entity_type,details,rule_applied,ip_address,timestam
    p FROM audit_logs ORDER BY id DESC LIMIT ?", (limit,))
    return out


def audit_verify() -> dict:
    with connect() as conn:
        return db.verify_audit_chain(conn)


def notifications() -> list:
    with connect() as conn:
        out = rows(conn, "SELECT * FROM notifications ORDER BY id DESC")
    for n in out:
        n["is_read"] = bool(n["is_read"])
    return out


def mark_notification_read(pk) -> dict:
    with connect() as conn:
        conn.execute("UPDATE notifications SET is_read=1 WHERE id=?", (_as_int(pk),))
    return {"success": True}


def export_data() -> dict:
    with connect() as conn:
        return {"appName": "GeM BidVerify AI", "version": "2.0", "exportedAt": now_iso(),
                "tenders": [_tender(conn, t) for t in rows(conn, "SELECT * FROM tenders")],
                "bidders": [_bidder(b) for b in rows(conn, "SELECT * FROM bidders")],
                "bids": [_bid_full(conn, b) for b in rows(conn, "SELECT * FROM bids")],
                "audit_logs": rows(conn, "SELECT * FROM audit_logs ORDER BY id")}


def reset_demo(user: dict) -> dict:
    db.reset_demo_data()
    return {"success": True}


//// FILE: backend/app/rules.py  (293 lines)

"""Compliance rule engine.

Input : bidder record, the tender's requirements, the bid's analysed documents.
Output: one result per applicable requirement (VERIFIED / MISMATCH / NON_COMPLIANT) with points,
        an overall compliance score, a risk assessment and a recommendation for the officer.

Scoring:  VERIFIED = full weight, MISMATCH = half weight (needs human review), NON_COMPLIANT = 0.
The engine only recommends; the final decision is always taken by the procurement officer.
"""
import re
from datetime import date

from . import adapters, config
from .extraction import GSTIN_RE, PAN_RE, UDYAM_RE, UDIN_RE, parse_money

REQUIREMENT_DOCUMENT = {
    "PAN": "PAN_CARD", "GST": "GST_CERTIFICATE", "UDYAM": "UDYAM_CERTIFICATE",
    "ITR": "ITR_AND_FINANCIALS", "OEM_AUTH": "OEM_AUTHORIZATION", "MAKE_IN_INDIA": "MAKE_IN_INDIA",
    "EPFO": "EPFO_REGISTRATION", "ESIC": "ESIC_REGISTRATION", "DEBARMENT": "DEBARMENT_AFFIDAVIT",
}
```

```
MISMATCH_FACTOR = 0.5
NAME_MATCH_THRESHOLD = 0.80


def _field(doc: dict, name: str):
    for item in doc.get("extracted_fields", []):
        if item["field_name"] == name:
            return item
    return None


def _value(doc: dict, name: str):
    item = _field(doc, name)
    return item["extracted_value"] if item else None


def _outcome(status, confidence, rule, explanation, flags=None, evidence=None):
    return {"status": status, "confidence": round(confidence, 2), "rule": rule,
            "explanation": explanation, "flags": flags or [], "evidence": evidence}


# ---------------------------------------------------------------------------------------
# One check per requirement code
# ---------------------------------------------------------------------------------------
def check_pan(bidder, req, doc):
    rule = "PAN format + ITD registry match (simulated adapter)"
    pan = (_value(doc, "pan_number") or "").upper()
    if not pan:
        return _outcome("MISMATCH", 0.4, rule, "PAN number could not be read from the document; manual review ne
    eded.")
    if not PAN_RE.fullmatch(pan):
        return _outcome("NON_COMPLIANT", 0.9, rule, f"'{pan}' is not a valid PAN format.")
    registry = adapters.lookup_pan(pan)
    problems = []
    if bidder.get("pan") and bidder["pan"].upper() != pan:
        problems.append(f"PAN on document ({pan}) differs from the bidder's registered PAN ({bidder['pan']})")
    doc_name = _value(doc, "name_on_pan")
    reference = registry["entity_name"] or bidder.get("company_name")
    if doc_name and reference and adapters.name_similarity(doc_name, reference) < NAME_MATCH_THRESHOLD:
        problems.append(f"name on PAN ('{doc_name}') does not match '{reference}'")
    if problems:
        return _outcome("MISMATCH", 0.8, rule, "; ".join(problems).capitalize() + ".")
    return _outcome("VERIFIED", _field(doc, "pan_number")["confidence"], rule,
                    f"PAN {pan} is valid and matches the bidder identity.")


def check_gst(bidder, req, doc):
    rule = "GSTIN format, checksum, embedded-PAN and GSTN status check (simulated adapter)"
    gstin = (_value(doc, "gstin") or "").upper()
    if not gstin:
        return _outcome("MISMATCH", 0.4, rule, "GSTIN could not be read from the certificate; manual review need
    ed.")
    info = adapters.lookup_gst(gstin)
    if info["status"] == "INVALID_FORMAT":
        return _outcome("NON_COMPLIANT", 0.9, rule, f"'{gstin}' is not a valid GSTIN format.")
    doc_status = (_value(doc, "status") or "").upper()
    if info["status"] != "ACTIVE" or doc_status in ("CANCELLED", "SUSPENDED"):
        shown = info["status"] if info["status"] != "ACTIVE" else doc_status
        return _outcome("NON_COMPLIANT", 0.95, rule,
                        f"GSTIN {gstin} is {shown}. {info['filing_status']}.", flags=["GST_NOT_ACTIVE"])
    problems = []
    if not info["checksum_ok"]:
        if config.STRICT_GSTIN_CHECKSUM:
            return _outcome("NON_COMPLIANT", 0.9, rule, f"GSTIN {gstin} fails the check-digit validation.")
        problems.append("GSTIN check digit does not validate")
    if bidder.get("pan") and gstin[2:12] != bidder["pan"].upper():
        problems.append("PAN embedded in the GSTIN differs from the bidder's PAN")
    if bidder.get("gstin") and bidder["gstin"].upper() != gstin:
        problems.append("GSTIN differs from the bidder's registered GSTIN")
    if problems:
        return _outcome("MISMATCH", 0.75, rule, "; ".join(problems).capitalize() + ".")
    return _outcome("VERIFIED", _field(doc, "gstin")["confidence"], rule,
                    f"GSTIN {gstin} is active in {info['state']}; PAN matches.")


def check_udyam(bidder, req, doc):
    rule = "Udyam registration number format and bidder MSME record match"
    number = (_value(doc, "udyam_no") or "").upper()
    if not UDYAM_RE.fullmatch(number):
```

```
        return _outcome("MISMATCH", 0.4, rule, "Valid Udyam registration number not found on the document.")
    if bidder.get("msme_reg_no") and bidder["msme_reg_no"].upper() != number:
        return _outcome("MISMATCH", 0.8, rule, f"Udyam number {number} differs from the bidder's registered {bid
    der['msme_reg_no']}.")
    return _outcome("VERIFIED", _field(doc, "udyam_no")["confidence"], rule, f"Udyam registration {number} is we
    ll-formed and consistent.")


def check_itr(bidder, req, doc):
    rule = "Average annual turnover vs tender minimum; UDIN presence"
    item = _field(doc, "avg_annual_turnover")
    if not item or item.get("normalized_value") is None:
        return _outcome("MISMATCH", 0.4, rule, "Turnover figure could not be read; manual review needed.")
    turnover = float(item["normalized_value"])
    minimum = parse_money(req.get("min_condition") or "") if re.search(r"turnover", req.get("min_condition") or
    "", re.I) else None
    needs_udin = "udin" in (req.get("min_condition") or "").lower()
    udin = (_value(doc, "ca_udin") or "").upper()
    if minimum and turnover < minimum:
        return _outcome("NON_COMPLIANT", 0.95, rule,
                        f"Average turnover {item['extracted_value']} is below the required ₹ {minimum / 1e7:.2f}
     Cr.")
    if needs_udin and not UDIN_RE.fullmatch(udin):
        return _outcome("MISMATCH", 0.7, rule, "Turnover meets the minimum but a valid UDIN was not found on the
     certificate.")
    return _outcome("VERIFIED", item["confidence"], rule,
                    f"Average turnover {item['extracted_value']} meets the requirement" + (f" and UDIN {udin} is
     present." if udin else "."))


def check_oem(bidder, req, doc):
    rule = "Authorization addressed to bidder; validity date"
    entity = _value(doc, "authorized_entity_name")
    validity = _field(doc, "valid_until")
    if validity and validity.get("normalized_value"):
        if date.fromisoformat(validity["normalized_value"]) < date.today():
            return _outcome("NON_COMPLIANT", 0.9, rule, f"The authorization expired on {validity['normalized_val
    ue']}.")
    if not entity:
        return _outcome("MISMATCH", 0.4, rule, "Authorized entity could not be read; manual review needed.")
    if adapters.name_similarity(entity, bidder.get("company_name", "")) < 0.7:
        return _outcome("MISMATCH", 0.8, rule, f"Authorization names '{entity}', not the bidder '{bidder.get('co
    mpany_name')}'.")
    return _outcome("VERIFIED", _field(doc, "authorized_entity_name")["confidence"], rule,
                    f"Authorization is issued to '{entity}'" + (f" and is valid until {validity['normalized_valu
    e']}." if validity else "."))


def check_mii(bidder, req, doc):
    rule = "Local content % vs minimum for Class-I local supplier"
    item = _field(doc, "local_content_percentage")
    if not item or item.get("normalized_value") is None:
        return _outcome("MISMATCH", 0.4, rule, "Local content percentage could not be read; manual review needed
    .")
    m = re.search(r"(\d{1,3})\s*%", req.get("min_condition") or "")
    minimum = float(m.group(1)) if m else 50.0
    percent = float(item["normalized_value"])
    if percent < minimum:
        return _outcome("NON_COMPLIANT", 0.95, rule, f"Local content {percent:g}% is below the required {minimum
    :g}%.")
    return _outcome("VERIFIED", item["confidence"], rule, f"Local content {percent:g}% meets the required {minim
    um:g}%.")


def check_registration(bidder, req, doc):
    rule = "Registration certificate present and readable"
    if doc["classification_confidence"] >= 0.6 and doc.get("ocr_status") == "COMPLETED":
        return _outcome("VERIFIED", doc["classification_confidence"], rule, f"{req['requirement_name']} certific
    ate submitted.")
    return _outcome("MISMATCH", doc["classification_confidence"], rule, "Certificate is unreadable or could not
    be confirmed; manual review needed.")


def check_debarment(bidder, req, doc):
    rule = "CPPP central debarment directory + affidavit (simulated adapter)"
    registry = adapters.check_debarment(bidder.get("company_name", ""), bidder.get("pan", ""))
    if registry["status"] == "DEBARRED_MATCH_FOUND":
        return _outcome("NON_COMPLIANT", 1.0, rule,
                        f"Entity matches debarment order {registry['order_number']} ({registry['ministry']}). {r
```

```
    egistry['gfr_rule']}.",
                        flags=["DEBARMENT_MATCH"], evidence="CPPP Central Debarment Directory")
    if doc is None:
        return _outcome("MISMATCH", 0.8, rule, "No debarment record found, but the mandatory affidavit was not s
    ubmitted.",
                        evidence="CPPP Central Debarment Directory")
    if _value(doc, "affidavit_status") == "DECLARES_DEBARRED":
        return _outcome("NON_COMPLIANT", 0.9, rule, "The affidavit itself declares that the entity has been deba
    rred.", flags=["DEBARMENT_MATCH"])
    return _outcome("VERIFIED", 0.95, rule, "No debarment record found and the affidavit is on file.")


CHECKS = {"PAN": check_pan, "GST": check_gst, "UDYAM": check_udyam, "ITR": check_itr,
          "OEM_AUTH": check_oem, "MAKE_IN_INDIA": check_mii, "EPFO": check_registration,
          "ESIC": check_registration, "DEBARMENT": check_debarment}


# ---------------------------------------------------------------------------------------
# Evaluation of a whole bid
# ---------------------------------------------------------------------------------------
def _applicable(bidder, req) -> bool:
    kind = (req.get("applicable_bidder_type") or "ALL").upper()
    if kind == "MSME":
        return bool(bidder.get("is_msme"))
    if kind == "STARTUP":
        return bool(bidder.get("is_startup"))
    return True


def evaluate_requirements(bidder: dict, requirements: list, documents: list) -> list:
    """Return a result dict for each applicable requirement that can be (or must be) judged."""
    results = []
    for req in requirements:
        if not _applicable(bidder, req):
            continue
        code = req["requirement_code"]
        wanted = REQUIREMENT_DOCUMENT.get(code)
        candidates = [d for d in documents if d["classified_type"] == wanted]
        doc = max(candidates, key=lambda d: d["classification_confidence"]) if candidates else None
        weight = float(req.get("weight") or 0)
        check = CHECKS.get(code)

        if code == "DEBARMENT":
            out = check_debarment(bidder, req, doc)
        elif doc is None:
            if not req.get("is_mandatory"):
                continue  # optional and not submitted: not judged
            out = _outcome("NON_COMPLIANT", 1.0, "Mandatory document presence check",
                           f"No {req['requirement_name']} document was submitted.", flags=["MISSING_MANDATORY"])
        elif check:
            out = check(bidder, req, doc)
        else:
            out = _outcome("MISMATCH", doc["classification_confidence"], "Manual review",
                           "No automatic rule exists for this requirement; officer review needed.")

        factor = {"VERIFIED": 1.0, "MISMATCH": MISMATCH_FACTOR, "NON_COMPLIANT": 0.0}[out["status"]]
        results.append({
            "requirement_code": code, "requirement_name": req["requirement_name"],
            "status": out["status"], "points_awarded": round(weight * factor, 1), "max_points": weight,
            "evidence_document": out["evidence"] or (doc["file_name"] if doc else "None"),
            "confidence_score": out["confidence"], "rule_applied": out["rule"],
            "explanation": out["explanation"], "flags": out["flags"], "is_mandatory": bool(req.get("is_mandatory
    ")),
        })
    return results


def summarize(results: list, document_count: int, ocr_confidences=()) -> dict:
    """Overall score, risk assessment and recommendation from per-requirement results."""
    total = sum(r["max_points"] for r in results)
    awarded = sum(r["points_awarded"] for r in results)
    score = round(100 * awarded / total) if total else 0
    flags = {f for r in results for f in r.get("flags", [])}
    factors, risk = [], 0

    if "DEBARMENT_MATCH" in flags:
        factors.append("Central Debarment Registry Match")
        risk += 70
    if "GST_NOT_ACTIVE" in flags:
```

```
        factors.append("GSTIN not in active status")
        risk += 25
    missing = [r for r in results if "MISSING_MANDATORY" in r.get("flags", [])]
    for r in missing:
        factors.append(f"Missing mandatory {r['requirement_code']}")
        risk += 20
    failed = [r for r in results if r["status"] == "NON_COMPLIANT" and not r.get("flags")]
    for r in failed:
        factors.append(f"Failed {r['requirement_code']} requirement")
        risk += 20
    mismatches = [r for r in results if r["status"] == "MISMATCH"]
    if mismatches:
        factors.append(f"{len(mismatches)} requirement(s) need manual review")
        risk += 8 * len(mismatches)
    if any(c < 0.6 for c in ocr_confidences):
        factors.append("Low OCR confidence on at least one document")
        risk += 5
    risk = min(100, risk)

    mandatory_failed = any(r["status"] == "NON_COMPLIANT" and r.get("is_mandatory") for r in results)
    if "DEBARMENT_MATCH" in flags or risk >= 70:
        level, reco = "CRITICAL", "CRITICAL_DISQUALIFICATION_REQUIRED"
        action = "Issue formal disqualification memo under GFR Rule 151" if "DEBARMENT_MATCH" in flags \
            else "Escalate for disqualification review"
    else:
        level = "HIGH" if risk >= 45 else "MEDIUM" if risk >= 20 else "LOW"
        if score >= 80 and not mandatory_failed and level == "LOW":
            reco, action = "QUALIFIED_WITH_HIGH_CONFIDENCE", "Proceed to commercial envelope opening"
        else:
            reco, action = "REVIEW_REQUIRED", "Examine flagged requirements and verify the credentials manually"

    verified = sum(1 for r in results if r["status"] == "VERIFIED")
    explanation = f"{verified} of {len(results)} evaluated requirements verified; compliance score {score}/100."
    notable = [r["explanation"] for r in results if r["status"] != "VERIFIED"][:2]
    if notable:
        explanation += " " + " ".join(notable)
    status = "VERIFIED" if reco == "QUALIFIED_WITH_HIGH_CONFIDENCE" else "PENDING_REVIEW"
    return {"compliance_score": score, "risk_score": risk, "risk_level": level, "risk_factors": factors,
            "ai_recommendation": reco, "recommended_action": action, "ai_explanation": explanation,
            "status": status}


def awaiting_documents(bidder: dict, requirements: list) -> dict:
    """State of a new bid with no documents: only the registry-based debarment check can run."""
    debarred = adapters.check_debarment(bidder.get("company_name", ""), bidder.get("pan", ""))
    if debarred["status"] == "DEBARRED_MATCH_FOUND":
        return {"compliance_score": 0, "risk_score": 90, "risk_level": "CRITICAL",
                "risk_factors": ["Central Debarment Registry Match"],
                "ai_recommendation": "CRITICAL_DISQUALIFICATION_REQUIRED",
                "recommended_action": "Issue formal disqualification memo under GFR Rule 151",
                "ai_explanation": f"Entity matches debarment order {debarred['order_number']} ({debarred['minist
    ry']}). {debarred['gfr_rule']}.",
                "status": "PENDING_REVIEW"}
    return {"compliance_score": 0, "risk_score": 10, "risk_level": "LOW", "risk_factors": [],
            "ai_recommendation": "REVIEW_REQUIRED", "recommended_action": "Upload the required documents to star
    t verification",
            "ai_explanation": "No documents have been uploaded yet. Verification starts when the first document
    is added.",
            "status": "UNDER_VERIFICATION"}


//// FILE: backend/app/extraction.py  (373 lines)

"""Document understanding for bid attachments.

Pipeline:  read text (PDF text layer, else OCR)  ->  classify document type  ->  extract fields.

Every extracted field carries a confidence value and, when the word positions are known,
a bounding box so the review screen can point at the evidence.
"""
import re
from datetime import date, datetime
from pathlib import Path

from . import config

# ---------------------------------------------------------------------------
# Regular expressions for Indian statutory identifiers
# ---------------------------------------------------------------------------
```

```
PAN_RE = re.compile(r"\b[A-Z]{5}[0-9]{4}[A-Z]\b")
GSTIN_RE = re.compile(r"\b[0-3][0-9][A-Z]{5}[0-9]{4}[A-Z][1-9A-Z]Z[0-9A-Z]\b")
UDYAM_RE = re.compile(r"\bUDYAM-[A-Z]{2}-\d{2}-\d{7}\b")
UDIN_RE = re.compile(r"\b\d{12}[A-Z0-9]{6}\b")

_GSTIN_CHARS = "0123456789ABCDEFGHIJKLMNOPQRSTUVWXYZ"


def gstin_checksum_ok(gstin: str) -> bool:
    """Validate the GSTIN check character (mod-36 weighted sum)."""
    if not GSTIN_RE.fullmatch(gstin):
        return False
    total = 0
    for i, ch in enumerate(gstin[:14]):
        product = _GSTIN_CHARS.index(ch) * (1 if i % 2 == 0 else 2)
        total += product // 36 + product % 36
    return _GSTIN_CHARS[(36 - total % 36) % 36] == gstin[14]


# ---------------------------------------------------------------------------------------
# Document classification
# ---------------------------------------------------------------------------------------
CLASSIFIERS = {
    "PAN_CARD": [(r"permanent\s*account\s*number", 3), (r"income\s*tax\s*department", 2),
                 (r"\b[A-Z]{5}[0-9]{4}[A-Z]\b", 2), (r"\bpan\b", 1)],
    "GST_CERTIFICATE": [(r"goods\s*(and|&)\s*services\s*tax", 3), (r"\bgstin\b", 3),
                        (r"registration\s*certificate", 1),
                        (r"\b[0-3][0-9][A-Z]{5}[0-9]{4}[A-Z][1-9A-Z]Z[0-9A-Z]\b", 3)],
    "UDYAM_CERTIFICATE": [(r"udyam", 4), (r"\bmsme\b", 2), (r"micro,?\s*small", 2)],
    "ITR_AND_FINANCIALS": [(r"turnover", 3), (r"\budin\b", 3), (r"chartered\s*accountant", 2),
                           (r"balance\s*sheet", 2), (r"income\s*tax\s*return|\bitr\b", 2)],
    "OEM_AUTHORIZATION": [(r"authori[sz]ation\s*(letter|certificate)", 3),
                          (r"authori[sz]ed\s*(dealer|partner|reseller|distributor)", 3),
                          (r"\boem\b|original\s*equipment", 2), (r"manufacturer", 1)],
    "MAKE_IN_INDIA": [(r"make\s*in\s*india", 3), (r"local\s*content", 3),
                      (r"class[\s-]*(i|ii|1|2)\s*local\s*supplier", 3), (r"dpiit", 2)],
    "DEBARMENT_AFFIDAVIT": [(r"debarred|debarment", 3), (r"black[\s-]*list", 3),
                            (r"affidavit|self[\s-]*declaration", 2)],
    "EPFO_REGISTRATION": [(r"provident\s*fund|\bepfo\b", 4), (r"establishment\s*code", 2)],
    "ESIC_REGISTRATION": [(r"employees'?\s*state\s*insurance|\besic\b", 4)],
}

FILENAME_TOKENS = {
    "pan": "PAN_CARD", "gst": "GST_CERTIFICATE", "gstin": "GST_CERTIFICATE",
    "udyam": "UDYAM_CERTIFICATE", "msme": "UDYAM_CERTIFICATE", "itr": "ITR_AND_FINANCIALS",
    "turnover": "ITR_AND_FINANCIALS", "financials": "ITR_AND_FINANCIALS", "udin": "ITR_AND_FINANCIALS",
    "oem": "OEM_AUTHORIZATION", "maf": "OEM_AUTHORIZATION", "authorization": "OEM_AUTHORIZATION",
    "authorisation": "OEM_AUTHORIZATION", "mii": "MAKE_IN_INDIA", "local": "MAKE_IN_INDIA",
    "debarment": "DEBARMENT_AFFIDAVIT", "affidavit": "DEBARMENT_AFFIDAVIT",
    "blacklist": "DEBARMENT_AFFIDAVIT", "epfo": "EPFO_REGISTRATION", "pf": "EPFO_REGISTRATION",
    "esic": "ESIC_REGISTRATION",
}


def _filename_hint(filename: str):
    tokens = re.split(r"[^a-z0-9]+", Path(filename).stem.lower())
    for token in tokens:
        if token in FILENAME_TOKENS:
            return FILENAME_TOKENS[token]
    return None


def classify(text: str, filename: str, hint: str = None):
    """Return (document_type, confidence). `hint` is the type chosen by the user (or AUTO)."""
    scores = {}
    for doc_type, patterns in CLASSIFIERS.items():
        scores[doc_type] = sum(w for pattern, w in patterns if re.search(pattern, text, re.I))
    if hint and hint not in ("AUTO", "GENERAL_DOCUMENT"):
        support = scores.get(hint, 0)
        return hint, (0.97 if support >= 3 else 0.60)
    name_hint = _filename_hint(filename)
    if name_hint:
        scores[name_hint] = scores.get(name_hint, 0) + 2
    best = max(scores, key=scores.get)
    if scores[best] < 3:
        return "GENERAL_DOCUMENT", 0.30
    return best, round(min(0.99, 0.45 + 0.08 * scores[best]), 2)
```

```
# ---------------------------------------------------------------------------------------
# Reading text: PDF text layer, scanned PDF / image via OCR
# ---------------------------------------------------------------------------------------
_ocr_state = {"checked": False, "available": False}


def ocr_available() -> bool:
    if not _ocr_state["checked"]:
        _ocr_state["checked"] = True
        try:
            import pytesseract
            if config.TESSERACT_CMD:
                pytesseract.pytesseract.tesseract_cmd = config.TESSERACT_CMD
            pytesseract.get_tesseract_version()
            _ocr_state["available"] = True
        except Exception:
            _ocr_state["available"] = False
    return _ocr_state["available"]


def _ocr_image(image, page_no: int):
    """OCR one PIL image -> (text, words, mean_confidence 0..1)."""
    import pytesseract
    data = pytesseract.image_to_data(image, lang=config.OCR_LANG, output_type=pytesseract.Output.DICT)
    words, confs, lines = [], [], {}
    for i, raw in enumerate(data["text"]):
        word = raw.strip()
        if not word:
            continue
        conf = float(data["conf"][i])
        if conf >= 0:
            confs.append(conf)
        words.append({"text": word, "x": data["left"][i], "y": data["top"][i],
                      "w": data["width"][i], "h": data["height"][i], "page": page_no})
        key = (data["block_num"][i], data["par_num"][i], data["line_num"][i])
        lines.setdefault(key, []).append(word)
    text = "\n".join(" ".join(ws) for _, ws in sorted(lines.items()))
    return text, words, (sum(confs) / len(confs) / 100 if confs else 0.0)


def read_document(path: Path):
    """Return (text, words, confidence, ocr_status)."""
    suffix = path.suffix.lower()
    if suffix == ".txt":
        return path.read_text(errors="ignore"), [], 1.0, "COMPLETED"

    if suffix == ".pdf":
        text_parts, words = [], []
        try:
            import pdfplumber
            with pdfplumber.open(path) as pdf:
                for n, page in enumerate(pdf.pages, start=1):
                    text_parts.append(page.extract_text() or "")
                    for w in page.extract_words():
                        words.append({"text": w["text"], "x": round(w["x0"]), "y": round(w["top"]),
                                      "w": round(w["x1"] - w["x0"]), "h": round(w["bottom"] - w["top"]),
                                      "page": n})
        except Exception:
            text_parts = []
        text = "\n".join(text_parts)
        if len(text.strip()) >= 30:
            return text, words, 1.0, "COMPLETED"
        if not ocr_available():
            return text, words, 0.0, "OCR_UNAVAILABLE"
        try:  # scanned PDF: render pages and OCR them
            import pypdfium2
            pdf = pypdfium2.PdfDocument(str(path))
            all_text, all_words, confs = [], [], []
            for n in range(len(pdf)):
                image = pdf[n].render(scale=2).to_pil()
                t, w, c = _ocr_image(image, n + 1)
                all_text.append(t)
                all_words.extend(w)
                confs.append(c)
            return "\n".join(all_text), all_words, (sum(confs) / len(confs) if confs else 0.0), "COMPLETED"
        except Exception:
            return text, words, 0.0, "FAILED"

    # image files
```

```
    if not ocr_available():
        return "", [], 0.0, "OCR_UNAVAILABLE"
    try:
        from PIL import Image
        with Image.open(path) as img:
            t, w, c = _ocr_image(img.convert("RGB"), 1)
        return t, w, c, "COMPLETED"
    except Exception:
        return "", [], 0.0, "FAILED"


# ---------------------------------------------------------------------------
# Field extraction helpers
# ---------------------------------------------------------------------------
_NAME_LABEL = (r"(?:legal\s*name(?:\s*of\s*(?:the\s*)?business)?|name\s*of\s*(?:the\s*)?"
               r"(?:entity|firm|company|business|enterprise|unit)|(?:entity|firm|company|enterprise)\s*name|name
    )")


def label_value(text: str, label_pattern: str):
    """Value that follows a label on the same line, or on the next non-empty line."""
    lines = text.splitlines()
    rx = re.compile(rf"^\s*(?:{label_pattern})\s*[:\-]?\s*(.*)$", re.I)
    for i, line in enumerate(lines):
        if re.match(r"^\s*father", line, re.I):
            continue
        m = rx.match(line)
        if m:
            value = m.group(1).strip()
            if not value:
                for nxt in lines[i + 1:]:
                    if nxt.strip():
                        value = nxt.strip()
                        break
            if value:
                return value
    return None


def parse_money(value: str):
    """Rupee amounts such as 'Rs. 2.5 Cr', '₹ 1,50,00,000' or '80 lakh' -> float rupees (or None)."""
    m = re.search(r"([\d,]+(?:\.\d+)?)\s*(crores?|cr\b\.?|lakhs?|lacs?)?", value, re.I)
    if not m:
        return None
    number = float(m.group(1).replace(",", "") or 0)
    unit = (m.group(2) or "").lower()
    if unit.startswith("cr"):
        return number * 1e7
    if unit.startswith(("lakh", "lac")):
        return number * 1e5
    return number


def format_crore(rupees: float) -> str:
    return f"₹ {rupees / 1e7:.2f} Cr"


def find_amounts_near(text: str, keyword: str):
    """All money amounts on lines that mention `keyword` (years and tiny numbers are ignored)."""
    amounts = []
    for line in text.splitlines():
        if keyword not in line.lower():
            continue
        for m in re.finditer(r"(?:₹|rs\.?|inr)?\s*([\d,]+(?:\.\d+)?)\s*(crores?|cr\b\.?|lakhs?|lacs?)?", line, r
    e.I):
            raw, unit = m.group(1), m.group(2)
            digits = raw.replace(",", "")
            if not unit and (len(digits) == 4 and digits.startswith(("19", "20")) or float(digits or 0) < 1000):
                continue
            value = parse_money(m.group(0))
            if value:
                amounts.append(value)
    return amounts


_DATE_FORMATS = ("%d/%m/%Y", "%d-%m-%Y", "%d.%m.%Y", "%d %B %Y", "%d %b %Y", "%B %d, %Y")


def parse_date(raw: str):
```

```
    raw = raw.strip().rstrip(".,")
    for fmt in _DATE_FORMATS:
        try:
            return datetime.strptime(raw, fmt).date()
        except ValueError:
            continue
    return None


def find_validity_date(text: str):
    m = re.search(r"valid\s*(?:till|until|upto|up\s*to|through)\s*[:\-]?\s*([0-9A-Za-z ,/.\-]{6,20})", text, re.
    I)
    return parse_date(m.group(1)) if m else None


def find_box(words, value: str):
    """Bounding box (page, x, y, width, height) of the words that spell `value`, or None."""
    tokens = value.split()
    if not tokens or not words:
        return None
    norm = lambda s: re.sub(r"[^A-Za-z0-9]", "", s).lower()
    wanted = [norm(t) for t in tokens]
    for i in range(len(words) - len(wanted) + 1):
        window = words[i:i + len(wanted)]
        if all(norm(w["text"]) == t or (t and t in norm(w["text"])) for w, t in zip(window, wanted)):
            x0 = min(w["x"] for w in window)
            y0 = min(w["y"] for w in window)
            x1 = max(w["x"] + w["w"] for w in window)
            y1 = max(w["y"] + w["h"] for w in window)
            return {"page": window[0]["page"], "x": x0, "y": y0, "width": x1 - x0, "height": y1 - y0}
    return None


class _Fields:
    """Collects extracted fields for one document."""

    def __init__(self, words, base_conf):
        self.words, self.base_conf, self.items = words, base_conf, []

    def add(self, name, value, strength=1.0, normalized=None):
        if value in (None, ""):
            return
        conf = round(max(0.05, min(0.99, self.base_conf * strength)), 2)
        self.items.append({
            "field_name": name, "extracted_value": str(value), "confidence": conf,
            "bounding_box": find_box(self.words, str(value)) or {"page": 1, "x": 0, "y": 0, "width": 0, "height"
    : 0},
            "is_verified": conf >= 0.85, "normalized_value": normalized,
        })


def extract_fields(doc_type: str, text: str, words, base_conf: float):
    f = _Fields(words, base_conf)
    upper = text.upper()

    if doc_type == "PAN_CARD":
        m = PAN_RE.search(upper)
        f.add("pan_number", m.group(0) if m else None, 1.0)
        f.add("name_on_pan", label_value(text, _NAME_LABEL), 0.95)

    elif doc_type == "GST_CERTIFICATE":
        m = GSTIN_RE.search(upper)
        f.add("gstin", m.group(0) if m else None, 1.0)
        f.add("legal_name", label_value(text, _NAME_LABEL), 0.95)
        status = re.search(r"\b(active|cancelled|canceled|suspended)\b", text, re.I)
        f.add("status", status.group(1).upper().replace("CANCELED", "CANCELLED") if status else None, 0.9)

    elif doc_type == "UDYAM_CERTIFICATE":
        m = UDYAM_RE.search(upper)
        f.add("udyam_no", m.group(0) if m else None, 1.0)
        kind = re.search(r"\b(micro|small|medium)\b", text, re.I)
        f.add("enterprise_type", kind.group(1).capitalize() if kind else None, 0.9)
        f.add("legal_name", label_value(text, _NAME_LABEL), 0.9)

    elif doc_type == "ITR_AND_FINANCIALS":
        m = UDIN_RE.search(upper)
        f.add("ca_udin", m.group(0) if m else None, 1.0)
        amounts = find_amounts_near(text, "turnover")
        if amounts:
```

```
            avg = sum(amounts) / len(amounts)
            f.add("avg_annual_turnover", format_crore(avg), 0.9, normalized=avg)

    elif doc_type == "OEM_AUTHORIZATION":
        m = re.search(r"\bM/s\.?\s+([A-Za-z0-9&.,' \-]{3,80}?)(?:\s*(?:,|\n|is\b|are\b|\(|$))", text)
        entity = m.group(1).strip() if m else label_value(text, r"authori[sz]ed\s*(?:dealer|partner|entity|to)")
        f.add("authorized_entity_name", entity, 0.9)
        brand = re.search(r"\bWe,?\s+([A-Z][A-Za-z0-9&.\- ]{2,40}?)(?:\s*(?:,|\n|are\b|hereby|\())", text)
        f.add("oem_brand", brand.group(1).strip() if brand else None, 0.85)
        validity = find_validity_date(text)
        f.add("valid_until", validity.isoformat() if validity else None, 0.9, normalized=validity.isoformat() if
     validity else None)

    elif doc_type == "MAKE_IN_INDIA":
        pct = re.search(r"local\s*content[^0-9\n]{0,40}(\d{1,3}(?:\.\d+)?)\s*%", text, re.I) or \
            re.search(r"(\d{1,3}(?:\.\d+)?)\s*%[^\n]{0,40}local\s*content", text, re.I)
        f.add("local_content_percentage", f"{pct.group(1)}%" if pct else None, 0.95,
              normalized=float(pct.group(1)) if pct else None)
        tier = re.search(r"class[\s-]*(i{1,2}|1|2)\b", text, re.I)
        if tier:
            f.add("supplier_tier", "Class-I Local Supplier" if tier.group(1).lower() in ("i", "1") else "Class-I
    I Local Supplier", 0.9)

    elif doc_type == "DEBARMENT_AFFIDAVIT":
        if re.search(r"(has|have)\s+been\s+(debarred|blacklisted)", text, re.I) and not re.search(
                r"(not|never)\s+(been\s+)?(debarred|blacklisted)", text, re.I):
            f.add("affidavit_status", "DECLARES_DEBARRED", 0.9)
        elif re.search(r"(not|never|no)\b[^.\n]{0,30}(debarred|blacklist)", text, re.I):
            f.add("affidavit_status", "DECLARES_NOT_DEBARRED", 0.9)

    return f.items


def analyze_document(path: Path, filename: str, type_hint: str = None) -> dict:
    """Run the whole pipeline on one stored file."""
    text, words, conf, status = read_document(path)
    doc_type, class_conf = classify(text, filename, type_hint)
    if status != "COMPLETED":
        class_conf = min(class_conf, 0.5)
    fields = extract_fields(doc_type, text, words, conf if status == "COMPLETED" else 0.0)
    return {
        "ocr_status": status,
        "classified_type": doc_type,
        "classification_confidence": class_conf,
        "extracted_fields": fields,
        "text_length": len(text),
    }


//// FILE: backend/app/adapters.py  (102 lines)

"""Statutory registry adapters (GSTN, Income Tax PAN, CPPP debarment list).

IMPORTANT: these adapters are SIMULATED. Real access to GSTN, the Income Tax Department or the
CPPP debarment directory needs official API agreements. Each function below is the single place
to replace with a real API call; the rest of the system only depends on the returned dictionaries.
"""
import re
from difflib import SequenceMatcher

from .extraction import GSTIN_RE, PAN_RE, gstin_checksum_ok

GST_STATE_CODES = {
    "01": "Jammu & Kashmir", "02": "Himachal Pradesh", "03": "Punjab", "04": "Chandigarh",
    "05": "Uttarakhand", "06": "Haryana", "07": "Delhi", "08": "Rajasthan", "09": "Uttar Pradesh",
    "10": "Bihar", "11": "Sikkim", "12": "Arunachal Pradesh", "13": "Nagaland", "14": "Manipur",
    "15": "Mizoram", "16": "Tripura", "17": "Meghalaya", "18": "Assam", "19": "West Bengal",
    "20": "Jharkhand", "21": "Odisha", "22": "Chhattisgarh", "23": "Madhya Pradesh", "24": "Gujarat",
    "26": "Dadra & Nagar Haveli and Daman & Diu", "27": "Maharashtra", "29": "Karnataka", "30": "Goa",
    "31": "Lakshadweep", "32": "Kerala", "33": "Tamil Nadu", "34": "Puducherry",
    "35": "Andaman & Nicobar Islands", "36": "Telangana", "37": "Andhra Pradesh", "38": "Ladakh",
}

# Simulated registry records (demo data).
_GST_REGISTRY = {
    "27ABCDE1234F1Z5": {"legal_name": "ABC Technologies Pvt Ltd", "status": "ACTIVE",
                        "filing_status": "Clean Filing Record (All returns submitted up to date)"},
    "07XYZAB5678C1Z2": {"legal_name": "XYZ Solutions Ltd", "status": "ACTIVE",
                        "filing_status": "Clean Filing Record (All returns submitted up to date)"},
```

```
    "08BHART9999K1Z8": {"legal_name": "Bharat Systems Pvt Ltd", "status": "SUSPENDED",
                        "filing_status": "Defaulted / GSTR-3B overdue for 4 quarters"},
}
_PAN_REGISTRY = {
    "ABCDE1234F": "ABC TECHNOLOGIES PVT LTD",
    "XYZAB5678C": "XYZ SOLUTIONS LTD",
    "BHART9999K": "BHARAT SYSTEMS PVT LTD",
}
_DEBARMENT_LIST = [{
    "pan": "BHART9999K", "name": "Bharat Systems Pvt Ltd",
    "ministry": "Ministry of Defence (Acquisition Wing)", "order_number": "MOD/DEB/2025/892",
    "period": "2 Years (Active until 15-Nov-2027)",
    "reason": "Submission of fabricated experience credentials in defense procurement",
    "gfr_rule": "Mandatory disqualification under GFR Rule 151",
}]

_SUFFIXES = {"PVT", "PRIVATE", "LTD", "LIMITED", "LLP", "INC", "CO", "COMPANY", "THE", "M", "S"}


def normalize_name(name: str) -> str:
    cleaned = re.sub(r"[^A-Z0-9 ]", " ", (name or "").upper().replace("&", " AND "))
    return " ".join(w for w in cleaned.split() if w not in _SUFFIXES)


def name_similarity(a: str, b: str) -> float:
    """0..1 similarity of two company names, ignoring punctuation and legal suffixes."""
    na, nb = normalize_name(a), normalize_name(b)
    if not na or not nb:
        return 0.0
    return SequenceMatcher(None, na, nb).ratio()


def lookup_gst(gstin: str) -> dict:
    gstin = (gstin or "").strip().upper()
    if not GSTIN_RE.fullmatch(gstin):
        return {"gstin": gstin, "status": "INVALID_FORMAT", "legal_name": None, "state": None,
                "taxpayer_type": None, "filing_status": "Not a valid GSTIN", "flag": "INVALID",
                "checksum_ok": False, "registry_hit": False}
    state_code = gstin[:2]
    record = _GST_REGISTRY.get(gstin)
    status = record["status"] if record else "ACTIVE"
    return {
        "gstin": gstin, "status": status,
        "legal_name": record["legal_name"] if record else None,
        "state": f"{GST_STATE_CODES.get(state_code, 'Unknown')} ({state_code})",
        "taxpayer_type": "Regular",
        "filing_status": record["filing_status"] if record else "Not in simulated registry (treated as active)",
        "flag": "COMPLIANT" if status == "ACTIVE" else "HIGH_RISK_DEFAULTER",
        "checksum_ok": gstin_checksum_ok(gstin), "registry_hit": record is not None,
    }


def lookup_pan(pan: str) -> dict:
    pan = (pan or "").strip().upper()
    if not PAN_RE.fullmatch(pan):
        return {"pan": pan, "status": "INVALID", "entity_name": None, "entity_type": None,
                "aadhaar_seeded": False, "itd_verification": "Invalid PAN format", "registry_hit": False}
    name = _PAN_REGISTRY.get(pan)
    return {"pan": pan, "status": "VALID", "entity_name": name,
            "entity_type": "Company", "aadhaar_seeded": True,
            "itd_verification": "Active & In Good Standing" if name else "Format valid; not in simulated registr
    y",
            "registry_hit": name is not None}


def check_debarment(company_name: str, pan: str = "") -> dict:
    pan = (pan or "").strip().upper()
    for entry in _DEBARMENT_LIST:
        if (pan and pan == entry["pan"]) or name_similarity(company_name, entry["name"]) >= 0.85:
            return {"company_name": company_name, "status": "DEBARRED_MATCH_FOUND", "severity": "CRITICAL",
                    "ministry": entry["ministry"], "order_number": entry["order_number"],
                    "period": entry["period"], "reason": entry["reason"], "gfr_rule": entry["gfr_rule"]}
    return {"company_name": company_name, "status": "NO_DEBARMENT_RECORD", "severity": "CLEAR",
            "ministry": "All Central Ministries & CPSUs", "records_checked": len(_DEBARMENT_LIST),
            "gfr_rule": "Clear for participation under GFR 2017"}


//// FILE: src/main.jsx  (10 lines)
```

```
import { StrictMode } from 'react';
import { createRoot } from 'react-dom/client';
import { App } from './App';
import './index.css';

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>,
);


//// FILE: src/App.jsx  (43 lines)

import { HashRouter, Navigate, Route, Routes } from 'react-router-dom';
import { LoginPage } from './pages/LoginPage';
import { ProtectedRoute } from './routes/ProtectedRoute';
import { MainLayout } from './components/layout/MainLayout';
import { DashboardPage } from './pages/DashboardPage';
import { TendersPage } from './pages/TendersPage';
import { BiddersPage } from './pages/BiddersPage';
import { BidsPage } from './pages/BidsPage';
import { BidDetailPage } from './pages/BidDetailPage';
import { IntegrationsPage } from './pages/IntegrationsPage';
import { AuditPage } from './pages/AuditPage';
import { AuthProvider } from './context/AuthContext';

const AppRoutes = () => (
  <Routes>
    <Route path="/login" element={<LoginPage />} />
    <Route
      path="/"
      element={
        <ProtectedRoute>
          <MainLayout />
        </ProtectedRoute>
      }
    >
      <Route index element={<DashboardPage />} />
      <Route path="tenders" element={<TendersPage />} />
      <Route path="bidders" element={<BiddersPage />} />
      <Route path="bids" element={<BidsPage />} />
      <Route path="bids/:id" element={<BidDetailPage />} />
      <Route path="integrations" element={<IntegrationsPage />} />
      <Route path="audit" element={<AuditPage />} />
    </Route>
    <Route path="*" element={<Navigate to="/" replace />} />
  </Routes>
);

export const App = () => (
  <AuthProvider>
    <HashRouter>
      <AppRoutes />
    </HashRouter>
  </AuthProvider>
);


//// FILE: backend/app/__init__.py  (1 lines)




//// FILE: backend/requirements.txt  (8 lines)

fastapi>=0.110
uvicorn[standard]>=0.29
python-multipart>=0.0.9
PyJWT>=2.7
pdfplumber>=0.11
pypdfium2>=4.30
pytesseract>=0.3.10
Pillow>=10.0


//// FILE: backend/tests/test_backend.py  (212 lines)

"""End-to-end test of the backend logic (services, rule engine, OCR, audit chain) without the web layer.
```

```
Run from the backend folder:   python tests/test_backend.py
"""
import os, sys, tempfile, shutil, io
from pathlib import Path

TMP = tempfile.mkdtemp(prefix="gemtest_")
os.environ["GEM_DATA_DIR"] = TMP
sys.path.insert(0, str(Path(__file__).resolve().parent.parent))

from app import db, services, extraction, adapters, security  # noqa: E402
from app.services import ServiceError  # noqa: E402

failures = []
def check(cond, msg):
    print(("  ok   " if cond else "  FAIL ") + msg)
    if not cond:
        failures.append(msg)

def expect_error(status, fn, *a, **k):
    try:
        fn(*a, **k)
    except ServiceError as e:
        return e.status == status
    return False

# ---------------------------------------------------------------- sample document factory
from reportlab.pdfgen import canvas  # noqa: E402
from PIL import Image, ImageDraw, ImageFont  # noqa: E402

def make_pdf(lines):
    buf = io.BytesIO()
    c = canvas.Canvas(buf)
    y = 800
    for line in lines:
        c.drawString(60, y, line)
        y -= 22
    c.save()
    return buf.getvalue()

def font(size=34):
    for p in ("/usr/share/fonts/truetype/dejavu/DejaVuSans.ttf", "/usr/share/fonts/truetype/liberation/Liberatio
    nSans-Regular.ttf"):
        if Path(p).exists():
            return ImageFont.truetype(p, size)
    return ImageFont.load_default()

def make_png(lines):
    img = Image.new("RGB", (1400, 120 + 70 * len(lines)), "white")
    d = ImageDraw.Draw(img)
    for i, line in enumerate(lines):
        d.text((50, 50 + i * 70), line, fill="black", font=font())
    buf = io.BytesIO()
    img.save(buf, "PNG")
    return buf.getvalue()

def gstin_for(pan, state="27"):
    base = state + pan + "1Z"
    chars = extraction._GSTIN_CHARS
    total = 0
    for i, ch in enumerate(base):
        prod = chars.index(ch) * (1 if i % 2 == 0 else 2)
        total += prod // 36 + prod % 36
    return base + chars[(36 - total % 36) % 36]

PAN, NAME = "DELTA1234X", "Delta Infotech Pvt Ltd"
GSTIN = gstin_for(PAN)

docs_good = {
    "pan_card.pdf": make_pdf(["INCOME TAX DEPARTMENT", "Permanent Account Number Card", f"Permanent Account Numb
    er: {PAN}", f"Name: {NAME.upper()}"]),
    "gst_certificate.pdf": make_pdf(["Government of India", "Goods and Services Tax Registration Certificate", f
    "GSTIN: {GSTIN}", f"Legal Name of Business: {NAME}", "Status: Active"]),
    "ca_turnover_certificate.pdf": make_pdf(["Chartered Accountant Certificate", "Annual turnover FY 2023-24: Rs
    . 2.60 Cr", "Annual turnover FY 2022-23: Rs. 2.40 Cr", "Annual turnover FY 2021-22: Rs. 2.50 Cr", "UDIN: 240
    112345612ABCDEF"]),
    "oem_authorization.pdf": make_pdf(["Manufacturer Authorization Letter", "We, Dell Technologies, are the orig
    inal equipment manufacturer.", f"We hereby authorize M/s {NAME}, to bid for tender GEM/2026/B/12345.", "Vali
    d till 31/12/2030"]),
```

```
    "make_in_india.pdf": make_pdf(["Make in India - Class-I Local Supplier Certificate", "Local content: 65% as
    per DPIIT order."]),
    "debarment_affidavit.pdf": make_pdf(["AFFIDAVIT", "We hereby declare that our company has not been debarred
    or blacklisted by any Government department."]),
}

print("== setup")
db.init_db()
with db.connect() as c:
    check(db.row(c, "SELECT COUNT(*) n FROM tenders")["n"] == 2, "seed: 2 tenders")
    check(db.row(c, "SELECT COUNT(*) n FROM bids")["n"] == 3, "seed: 3 bids")
    check("khandelwal" not in str(db.rows(c, "SELECT email FROM users")), "seed: personal email replaced")
check(adapters.name_similarity("ABC Technologies Pvt Ltd", "ABC TECHNOLOGIES PRIVATE LIMITED") > 0.95, "name sim
    ilarity ignores suffixes")
check(extraction.gstin_checksum_ok(GSTIN), "generated GSTIN passes checksum")

print("== auth")
s = services.login("officer", "demo123")
check(s["user"]["username"] == "officer" and s["access_token"], "login ok")
check("password" not in str(s), "no password in response")
check(services.user_from_token(s["access_token"])["username"] == "officer", "token round-trip")
check(expect_error(401, services.login, "officer", "wrong"), "wrong password rejected")
check(expect_error(401, services.user_from_token, "garbage"), "bad token rejected")
r = services.send_otp(username="officer")
check(len(r.get("otp", "")) == 6, "otp issued (demo mode returns it)")
check(expect_error(400, services.verify_otp, "officer", None, "000000"), "wrong otp rejected")
check(services.verify_otp("officer", None, r["otp"])["access_token"], "correct otp accepted")
check(expect_error(400, services.verify_otp, "officer", None, r["otp"]), "otp is single use")
ghost = services.send_otp(username="nobody-here")
check(ghost["success"] and "otp" not in ghost, "unknown user gets generic reply, no otp")
reg = services.register({"username": "asha", "full_name": "Asha Rao", "email": "asha@example.org", "password": "
    secret1"})
check(reg["user"]["username"] == "asha", "register ok")
check(expect_error(409, services.register, {"username": "asha", "email": "x@y.z", "password": "secret1"}), "dupl
    icate username rejected")
r = services.send_otp(username="asha")
services.reset_password("asha", None, r["otp"], "newsecret")
check(services.login("asha", "newsecret")["user"]["username"] == "asha", "password reset works")
user = services.user_from_token(s["access_token"])

print("== bidder, bid and empty state")
bidder = services.create_bidder({"company_name": NAME, "pan": PAN.lower(), "gstin": GSTIN, "is_msme": False}, us
    er)
check(bidder["pan"] == PAN, "pan uppercased")
check(expect_error(422, services.create_bidder, {"company_name": "Bad Co", "pan": "123"}, user), "bad PAN reject
    ed")
bid = services.create_bid(1, bidder["id"], user)
check(bid["status"] == "UNDER_VERIFICATION" and bid["compliance_score"] == 0, "new bid awaits documents")
check(bid["bid_id"].startswith("BID/12345/DEL-"), f"bid code {bid['bid_id']}")
check(expect_error(409, services.create_bid, 1, bidder["id"], user), "duplicate bid rejected")
check(expect_error(404, services.create_bid, 99, bidder["id"], user), "unknown tender rejected")

print("== upload good documents (text PDFs)")
for name, content in docs_good.items():
    d = services.upload_document(bid["id"], name, content, "AUTO", user)
    print(f"      {name:30} -> {d['classified_type']:20} conf={d['classification_confidence']} fields={[ (f['fie
    ld_name'], f['extracted_value']) for f in d['extracted_fields']]}")
b = services.get_bid(bid["id"])
types = {d["classified_type"] for d in b["documents"]}
check(types == {"PAN_CARD", "GST_CERTIFICATE", "ITR_AND_FINANCIALS", "OEM_AUTHORIZATION", "MAKE_IN_INDIA", "DEBA
    RMENT_AFFIDAVIT"}, "all 6 documents classified correctly")
statuses = {r["requirement_code"]: r["status"] for r in b["compliance_results"]}
print("      results:", statuses, "score", b["compliance_score"], b["risk_level"], b["status"], b["ai_recommenda
    tion"])
check(statuses == {"PAN": "VERIFIED", "GST": "VERIFIED", "ITR": "VERIFIED", "OEM_AUTH": "VERIFIED", "MAKE_IN_IND
    IA": "VERIFIED", "DEBARMENT": "VERIFIED"}, "all evaluated requirements verified")
check(b["compliance_score"] == 100 and b["status"] == "VERIFIED" and b["ai_recommendation"] == "QUALIFIED_WITH_H
    IGH_CONFIDENCE", "qualified with 100 score")
check(not any(r["requirement_code"] in ("EPFO", "ESIC", "UDYAM") for r in b["compliance_results"]), "optional un
    submitted requirements not judged")
f = {x["field_name"]: x for d in b["documents"] for x in d["extracted_fields"]}
check(f["avg_annual_turnover"]["extracted_value"] == "₹ 2.50 Cr", "turnover averaged over 3 years")
check(f["pan_number"]["bounding_box"]["width"] > 0, "bounding box located for PAN text")

print("== negative scenarios")
b2 = services.create_bid(2, bidder["id"], user)  # tender 2 needs 2.5 Cr, has PAN/GST/ITR/DEBARMENT
services.upload_document(b2["id"], "pan.pdf", docs_good["pan_card.pdf"], "AUTO", user)
low = make_pdf(["Chartered Accountant Certificate", "Annual turnover FY 2023-24: Rs. 1.0 Cr", "UDIN: 24011234561
    2ABCDEF"])
```

```
services.upload_document(b2["id"], "turnover.pdf", low, "AUTO", user)
b2 = services.get_bid(b2["id"])
st = {r["requirement_code"]: r for r in b2["compliance_results"]}
print("      tender2:", {k: v["status"] for k, v in st.items()}, b2["compliance_score"], b2["risk_level"], b2["r
    isk_factors"] if "risk_factors" in b2 else b2["risk_assessment"]["risk_factors"])
check(st["ITR"]["status"] == "NON_COMPLIANT" and "below" in st["ITR"]["explanation"], "turnover below minimum is
     NON_COMPLIANT")
check(st["GST"]["status"] == "NON_COMPLIANT" and "No Goods" in st["GST"]["explanation"], "missing mandatory GST
    flagged")
check(b2["status"] == "PENDING_REVIEW" and b2["ai_recommendation"] != "QUALIFIED_WITH_HIGH_CONFIDENCE", "incompl
    ete bid is not qualified")

bharat = services.list_bidders()
bharat = next(x for x in bharat if x["company_name"].startswith("Bharat"))
b3 = services.create_bid(2, bharat["id"], user)
check(b3["risk_level"] == "CRITICAL" and b3["ai_recommendation"] == "CRITICAL_DISQUALIFICATION_REQUIRED", "debar
    red bidder flagged critical immediately")
gst_bad = make_pdf(["Goods and Services Tax Registration Certificate", "GSTIN: 08BHART9999K1Z8", "Legal Name: Bh
    arat Systems Pvt Ltd", "Status: Active"])
services.upload_document(b3["id"], "gst.pdf", gst_bad, "AUTO", user)
b3 = services.get_bid(b3["id"])
gst_res = next(r for r in b3["compliance_results"] if r["requirement_code"] == "GST")
check(gst_res["status"] == "NON_COMPLIANT" and "SUSPENDED" in gst_res["explanation"], "suspended GSTIN fails")
check("GSTIN not in active status" in b3["risk_assessment"]["risk_factors"], "risk factor lists GST status")

print("== OCR paths (image + scanned PDF)")
if extraction.ocr_available():
    png = make_png(["INCOME TAX DEPARTMENT", "Permanent Account Number Card", f"Permanent Account Number: {PAN}"
    , f"Name: {NAME.upper()}"])
    d = services.upload_document(bid["id"], "scan_card.png", png, "AUTO", user)
    fields = {x["field_name"]: x["extracted_value"] for x in d["extracted_fields"]}
    print("      png ->", d["classified_type"], d["ocr_status"], fields)
    check(d["classified_type"] == "PAN_CARD" and fields.get("pan_number") == PAN, "OCR on image reads the PAN")
    img = Image.open(io.BytesIO(png)).convert("RGB")
    pdf_buf = io.BytesIO(); img.save(pdf_buf, "PDF")
    d = services.upload_document(bid["id"], "scanned.pdf", pdf_buf.getvalue(), "AUTO", user)
    fields = {x["field_name"]: x["extracted_value"] for x in d["extracted_fields"]}
    print("      scanned pdf ->", d["classified_type"], d["ocr_status"], fields)
    check(fields.get("pan_number") == PAN, "OCR on scanned PDF reads the PAN")
else:
    print("      (tesseract not installed - skipped)")

print("== safety checks")
check(expect_error(415, services.upload_document, bid["id"], "evil.exe", b"MZ", "AUTO", user), "exe upload rejec
    ted")
check(expect_error(422, services.upload_document, bid["id"], "empty.pdf", b"", "AUTO", user), "empty upload reje
    cted")
junk = services.upload_document(bid["id"], "photo.txt", b"hello this is just some text", "AUTO", user)
check(junk["classified_type"] == "GENERAL_DOCUMENT", "unrecognised text -> GENERAL_DOCUMENT")

print("== officer actions + audit chain")
b = services.get_bid(bid["id"])
res = next(r for r in b["compliance_results"] if r["requirement_code"] == "GST")
out = services.update_compliance_result(bid["id"], res["id"], {"points_awarded": 5, "status": "MISMATCH", "expla
    nation": "Checked manually"}, user)
check(out["compliance_score"] < 100, f"override lowers score to {out['compliance_score']}")
b = services.verify_bid(bid["id"], user)
gst = next(r for r in b["compliance_results"] if r["requirement_code"] == "GST")
check(gst["status"] == "MISMATCH" and gst["explanation"] == "Checked manually", "re-verification keeps the offic
    er override")
dec = services.submit_decision(bid["id"], {"decision": "APPROVED", "comments": "Reviewed"}, user)
check(dec["officer_decision"]["decision"] == "APPROVED" and services.get_bid(bid["id"])["status"] == "APPROVED",
     "decision sets status")
services.delete_document(bid["id"], junk["id"], user)
check(services.get_bid(bid["id"])["status"] == "APPROVED", "status kept after later changes")
check(expect_error(422, services.submit_decision, bid["id"], {"decision": "MAYBE"}, user), "invalid decision rej
    ected")
ver = services.audit_verify()
check(ver["valid"] and ver["entries_checked"] > 20, f"audit chain valid ({ver['entries_checked']} entries)")
with db.connect() as c:
    c.execute("UPDATE audit_logs SET details='tampered' WHERE id=5")
check(not services.audit_verify()["valid"], "tampering with an old audit row is detected")

print("== dashboard / lists / misc")
dash = services.dashboard()
check(dash["total_tenders"] == 2 and len(dash["verification_activity"]) == 6 and len(dash["recent_bids"]) == len
    (services.list_bids()), "dashboard shape")
check(set(dash["risk_distribution"]) == {"Low", "Medium", "High", "Critical"}, "risk distribution keys match UI"
    )
```

```
check(services.get_tender("GEM/2026/B/12345")["bids_count"] >= 4, "tender lookup by code and bid count")
t = services.create_tender({"title": "New tender", "requirements": [{"requirement_code": "PAN", "requirement_nam
    e": "PAN", "weight": 10}]}, user)
check(t["tender_id"].startswith("GEM/") and len(t["requirements"]) == 1, "create tender")
check(len(services.export_data()["bids"]) == len(services.list_bids()), "export")
services.reset_demo(user)
check(len(services.list_bids()) == 3 and services.audit_verify()["valid"], "reset restores demo data with a vali
    d chain")

shutil.rmtree(TMP, ignore_errors=True)
print("\n" + ("ALL CHECKS PASSED" if not failures else f"{len(failures)} FAILURE(S): {failures}"))
sys.exit(1 if failures else 0)


//// FILE: backend/tests/test_routes_stub.py  (71 lines)

"""Checks app/main.py without FastAPI installed: imports it against tiny stand-ins, confirms every
URL the frontend calls is registered, and runs several handlers directly.
(The real FastAPI behaviour is verified on your laptop with `uvicorn`.)"""
import os, re, sys, types, tempfile
from pathlib import Path

os.environ["GEM_DATA_DIR"] = tempfile.mkdtemp(prefix="gemroutes_")
ROOT = Path(__file__).resolve().parent.parent
sys.path.insert(0, str(ROOT))
ROUTES = []

def mod(name, **attrs):
    m = types.ModuleType(name); m.__dict__.update(attrs); sys.modules[name] = m; return m

class FastAPI:
    def __init__(self, **kw): pass
    def _reg(self, method):
        def deco_factory(path, **kw):
            def deco(fn): ROUTES.append((method, path, fn)); return fn
            return deco
        return deco_factory
    def __getattr__(self, name):
        if name in ("get", "post", "put", "delete"): return self._reg(name.upper())
        raise AttributeError(name)
    def add_middleware(self, *a, **k): pass
    def on_event(self, ev): return lambda fn: fn
    def exception_handler(self, exc): return lambda fn: fn

class BaseModel:
    def __init__(self, **kw):
        for k in getattr(self.__class__, "__annotations__", {}):
            setattr(self, k, kw.get(k, getattr(self.__class__, k, None)))
    def dict(self): return {k: getattr(self, k) for k in self.__class__.__annotations__}

mod("fastapi", FastAPI=FastAPI, Depends=lambda f=None: f, File=lambda *a, **k: None, Form=lambda *a, **k: None,
    Header=lambda *a, **k: None, Query=lambda *a, **k: None, Request=object, UploadFile=object)
mod("fastapi.middleware"); mod("fastapi.middleware.cors", CORSMiddleware=object)
mod("fastapi.responses", FileResponse=object, JSONResponse=object)
mod("pydantic", BaseModel=BaseModel)

from app import main, db, services  # noqa: E402
db.init_db()
print(f"{len(ROUTES)} routes registered")

frontend = (ROOT.parent / "src" / "api" / "mockApi.js").read_text(encoding="utf-8")
called = set(re.findall(r"\$\{API_BASE\}(/[a-z0-9/_\-\$\{\}\.]*)", frontend))
norm = lambda p: re.sub(r"\$\{[^}]*\}|\{[^}]*\}", "{}", "/api" + p.split("?")[0])
have = {re.sub(r"\{[^}]*\}", "{}", p) for _, p, _ in ROUTES}
missing = sorted(n for n in {norm(c) for c in called} if n not in have)
print("frontend paths:", sorted({norm(c) for c in called}))
print("MISSING:", missing)
assert not missing, missing

class Req:  # request stand-in
    client = types.SimpleNamespace(host="127.0.0.1")

sess = main.login(main.LoginIn(username="officer", password="demo123"), Req())
user = services.user_from_token(sess["access_token"])
assert main.dashboard(user)["total_tenders"] == 2
assert len(main.list_bids(None, user)) == 3
assert main.get_bid("1", user)["bid_id"].startswith("BID/")
assert main.test_gst("08BHART9999K1Z8", user)["status"] == "SUSPENDED"
assert main.test_debarment("Bharat Systems", "", user)["severity"] == "CRITICAL"
```

```
assert main.audit_verify(user)["valid"]
assert main.me(user)["username"] == "officer"
try:
    main.current_user(None); raise SystemExit("auth should be required")
except services.ServiceError as e:
    assert e.status == 401
assert main.current_user("Bearer " + sess["access_token"])["username"] == "officer"
print("route handlers OK")


//// FILE: index.html  (23 lines)

<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <link
      rel="icon"
      type="image/svg+xml"
      href="data:image/svg+xml,<svg xmlns='http://www.w3.org/2000/svg' viewBox='0 0 24 24' fill='%230A3871'><pat
    h d='M12 2L2 7l10 5 10-5-10-5zM2 17l10 5 10-5M2 12l10 5 10-5'/></svg>"
    />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>GeM BidVerify AI — Technical Compliance &amp;amp; Evaluation Platform</title>
    <link rel="preconnect" href="https://fonts.googleapis.com" />
    <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin />
    <link
      href="https://fonts.googleapis.com/css2?family=Inter:wght@300;400;500;600;700;800&family=Plus+Jakarta+Sans
    :wght@400;500;600;700;800&family=JetBrains+Mono:wght@400;500;600;700&display=swap"
      rel="stylesheet"
    />
  </head>
  <body>
    <div id="root"></div>
    <script type="module" src="/src/main.jsx"></script>
  </body>
</html>


//// FILE: package.json  (29 lines)

{
  "name": "gem-bidverify-ai",
  "private": true,
  "version": "2.0.0",
  "type": "module",
  "description": "GeM BidVerify AI - AI-powered bid compliance verification portal (frontend)",
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "preview": "vite preview",
    "format": "prettier --write \"src/**/*.{js,jsx,css}\" index.html"
  },
  "dependencies": {
    "lucide-react": "0.378.0",
    "react": "^18.3.1",
    "react-dom": "^18.3.1",
    "react-router-dom": "^6.26.2",
    "recharts": "^2.12.7"
  },
  "devDependencies": {
    "@vitejs/plugin-react": "^4.3.1",
    "autoprefixer": "^10.4.20",
    "postcss": "^8.4.47",
    "prettier": "^3.3.3",
    "tailwindcss": "^3.4.13",
    "vite": "^5.4.8",
    "vite-plugin-singlefile": "^2.0.2"
  }
}


//// FILE: postcss.config.js  (6 lines)

export default {
  plugins: {
    tailwindcss: {},
    autoprefixer: {},
  },
```

```
};


//// FILE: src/api/httpApi.js  (112 lines)

/**
 * Real API client: talks to the FastAPI backend (see /backend).
 * Same method names as mockApi.js so the screens work with either.
 */
const BASE =
  import.meta.env.VITE_API_BASE_URL ||
  (typeof window !== 'undefined' && window.location.protocol === 'file:'
    ? 'http://localhost:8000/api'
    : '/api');
const TOKEN_KEY = 'gem_officer_token';

function getToken() {
  try {
    return localStorage.getItem(TOKEN_KEY);
  } catch {
    return null;
  }
}

async function request(path, { method = 'GET', body, formData } = {}) {
  const headers = {};
  const token = getToken();
  if (token) headers.Authorization = `Bearer ${token}`;
  let payload;
  if (formData) {
    payload = formData;
  } else if (body !== undefined) {
    headers['Content-Type'] = 'application/json';
    payload = JSON.stringify(body);
  }

  let response;
  try {
    response = await fetch(`${BASE}${path}`, { method, headers, body: payload });
  } catch {
    throw new Error('Cannot reach the server. Make sure the backend is running on port 8000.');
  }

  if (!response.ok) {
    const data = await response.json().catch(() => ({}));
    let message = data.detail;
    if (Array.isArray(message)) message = message.map((e) => e.msg).join('; ');
    if (response.status === 401 && token) window.dispatchEvent(new Event('gem:unauthorized'));
    if (!message && response.status >= 500) {
      throw new Error(
        `The server did not answer correctly (status ${response.status}). Check that the backend window is runni
    ng and shows no red error.`,
      );
    }
    throw new Error(message || `Request failed (${response.status})`);
  }
  return response.json();
}

const post = (path, body) => request(path, { method: 'POST', body });

export const api = {
  // authentication
  login: (credentials) => post('/auth/login', credentials),
  sendOtp: (data) => post('/auth/send-otp', data),
  verifyOtp: (data) => post('/auth/verify-2fa', data),
  resetPassword: (data) => post('/auth/reset-password', data),
  register: (data) => post('/auth/register', data),

  // dashboard
  getDashboardStats: () => request('/dashboard'),

  // tenders and bidders
  getTenders: () => request('/tenders'),
  getTender: (id) => request(`/tenders/${encodeURIComponent(id)}`),
  createTender: (data) => post('/tenders', data),
  addRequirement: (tenderId, data) => post(`/tenders/${tenderId}/requirements`, data),
  getBidders: () => request('/bidders'),
  createBidder: (data) => post('/bidders', data),
```

```
  // bids
  getBids: (tenderId) =>
    request(tenderId ? `/bids?tender_id=${encodeURIComponent(tenderId)}` : '/bids'),
  getBid: (id) => request(`/bids/${encodeURIComponent(id)}`),
  createBid: (data) => post('/bids', data),
  uploadDocument: (bidId, file, typeHint) => {
    const form = new FormData();
    form.append('file', file);
    if (typeHint) form.append('classified_type_hint', typeHint);
    return request(`/bids/${bidId}/documents`, { method: 'POST', formData: form });
  },
  deleteDocument: (bidId, documentId) =>
    request(`/bids/${bidId}/documents/${documentId}`, { method: 'DELETE' }),
  updateComplianceResult: (bidId, resultId, data) =>
    request(`/bids/${bidId}/compliance/${resultId}`, { method: 'PUT', body: data }),
  verifyBid: (bidId) => post(`/bids/${bidId}/verify`),
  submitOfficerDecision: (bidId, data) => post(`/bids/${bidId}/decision`, data),

  // integrations (simulated adapters on the server)
  getIntegrationsStatus: () => request('/integrations/status'),
  testGST: (gstin) => post(`/integrations/test-gst?gstin=${encodeURIComponent(gstin)}`),
  testPAN: (pan) => post(`/integrations/test-pan?pan=${encodeURIComponent(pan)}`),
  testDebarment: (companyName, pan) =>
    post(
      `/integrations/test-debarment?company_name=${encodeURIComponent(companyName)}&pan=${encodeURIComponent(pan
     || '')}`,
    ),

  // audit, notifications, administration
  getAuditLogs: () => request('/audit'),
  verifyAuditChain: () => request('/audit/verify'),
  getNotifications: () => request('/notifications'),
  markNotificationRead: (id) => post(`/notifications/${id}/read`),
  exportData: () => request('/admin/export'),
  importData: async () => {
    throw new Error('Importing a backup is only available in offline demo mode.');
  },
  resetData: () => post('/admin/reset-demo'),
};


//// FILE: src/api/index.js  (9 lines)

/**
 * Chooses the data source for the whole app.
 *   default            -> real backend (FastAPI on port 8000, proxied as /api in dev)
 *   VITE_API_MODE=mock -> offline demo using browser localStorage (no server needed)
 */
import { api as httpApi } from './httpApi';
import { api as mockApi } from './mockApi';

export const api = import.meta.env.VITE_API_MODE === 'mock' ? mockApi : httpApi;


//// FILE: src/api/mockApi.js  (851 lines)

import { loadState, saveState } from './storage';
import {
  seedAuditLogs,
  seedBidders,
  seedBids,
  seedConnectors,
  seedNotifications,
  seedTenders,
  seedUsers,
} from './seedData';

const IS_FILE_PROTOCOL = typeof window < 'u' && window.location.protocol === 'file:';

const API_BASE = IS_FILE_PROTOCOL ? 'http://localhost:8000/api' : '/api';

let tendersStore = loadState('tenders', seedTenders);

let biddersStore = loadState('bidders', seedBidders);

let bidsStore = loadState('bids', seedBids);

let auditStore = loadState('audit_logs', seedAuditLogs);
```

```
let notificationsStore = loadState('notifications', seedNotifications);

let usersStore = loadState('users', seedUsers);

if (usersStore && Array.isArray(usersStore)) {
  let e = false;
  ((usersStore = usersStore.map((t) =>
    t.username === 'officer' ||
    !t.email ||
    t.email.endsWith('@gem.gov.in') ||
    t.email.includes('rajesh.verma')
      ? ((e = true), { ...t, email: 'officer@example.gov.in' })
      : t,
  )),
    e && saveState('users', usersStore));
}

export let otpStore = {};

function addAuditLog(e, t, r = 'BID', n) {
  ((auditStore = [
    {
      id: Date.now(),
      bid_id: n,
      user_name: 'Rajesh Kumar Verma',
      action: e,
      entity_type: r,
      details: t,
      rule_applied: 'GFR_2017_RULE_149',
      ip_address: '10.12.89.44 (NIC Network)',
      timestamp: new Date().toISOString(),
    },
    ...auditStore,
  ]),
    saveState('audit_logs', auditStore));
}

async function requestWithFallback(e, t, r) {
  if (!(typeof window < 'u' && window.location.protocol === 'https:') || !e.startsWith('http://'))
    try {
      const i = new AbortController(),
        a = setTimeout(() => i.abort(), 3e3),
        o = await fetch(e, { ...t, signal: i.signal });
      if ((clearTimeout(a), !o.ok)) {
        const l = await o.json().catch(() => ({ detail: `HTTP ${o.status}` }));
        throw new Error(l.detail || l.message || `Request failed with status ${o.status}`);
      }
      return await o.json();
    } catch {}
  if (r) return await r();
  throw new Error('Endpoint and fallback failed.');
}

export const api = {
  login: async (e) =>
    requestWithFallback(
      `${API_BASE}/auth/login`,
      { method: 'POST', headers: { 'Content-Type': 'application/json' }, body: JSON.stringify(e) },
      () => {
        const t = usersStore.find((n) => n.username === e.username && n.password === e.password);
        if (!t) throw new Error('Invalid username or password.');
        const r = {
          id: t.id,
          username: t.username,
          full_name: t.full_name,
          role: t.role,
          department: t.department,
          email: t.email,
          phone: t.phone,
          is_active: true,
        };
        return (
          addAuditLog(
            'OFFICER_LOGIN',
            `Officer '${r.full_name}' authenticated via GeM console.`,
            'AUTH',
          ),
          { access_token: 'gem_officer_token_standalone_2026', token_type: 'bearer', user: r }
```

```
      );
    },
  ),
sendOtp: async (e) =>
  requestWithFallback(
    `${API_BASE}/auth/send-otp`,
    { method: 'POST', headers: { 'Content-Type': 'application/json' }, body: JSON.stringify(e) },
    async () => {
      const t = e.username || 'officer',
        r = usersStore.find((o) => o.username === t || (e.email && o.email === e.email)),
        n = e.email || (r == null ? undefined : r.email) || 'officer@example.gov.in',
        i = Math.floor(100000 + Math.random() * 900000).toString();
      let a = n;
      if (n.includes('@')) {
        const [o, l] = n.split('@');
        o.length <= 2
          ? (a = `${o}*@${l}`)
          : (a = o.slice(0, 2) + '*'.repeat(Math.max(2, o.length - 3)) + o.slice(-1) + '@' + l);
      }
      console.log(
        `%c[GeM 2FA Security Gateway] Verification OTP for ${n}: ${i}`,
        'color:#14B8A6;font-weight:bold;font-size:13px;',
      );
      try {
        sessionStorage.setItem('gem_last_otp', i);
      } catch {}
      return (
        (otpStore[t] = { otp: i, email: n, expiresAt: Date.now() + 5 * 60 * 1e3, attempts: 0 }),
        (otpStore[n] = otpStore[t]),
        { success: true, message: `OTP sent to ${a}`, masked_email: a, email_masked: a, otp: i }
      );
    },
  ),
verifyOtp: async (e) =>
  requestWithFallback(
    `${API_BASE}/auth/verify-2fa`,
    { method: 'POST', headers: { 'Content-Type': 'application/json' }, body: JSON.stringify(e) },
    () => {
      const t = otpStore[e.username] || otpStore[e.email];
      if (!t || !t.otp) throw new Error('No active OTP found. Please request a new OTP.');
      if (Date.now() > t.expiresAt) {
        delete otpStore[e.username];
        throw new Error('OTP has expired. Please request a new one.');
      }
      if (t.otp !== e.otp) {
        t.attempts = (t.attempts || 0) + 1;
        if (t.attempts >= 5) {
          delete otpStore[e.username];
          throw new Error('Too many failed attempts. Please request a new OTP.');
        }
        throw new Error('Invalid OTP code. Please check and try again.');
      }
      delete otpStore[e.username];
      const n =
          usersStore.find((a) => a.username === e.username || a.email === e.username) ||
          usersStore[0],
        i = {
          id: (n == null ? undefined : n.id) || 1,
          username: (n == null ? undefined : n.username) || 'officer',
          full_name: (n == null ? undefined : n.full_name) || 'Rajesh Kumar Verma',
          role: (n == null ? undefined : n.role) || 'Procurement Officer',
          department:
            (n == null ? undefined : n.department) ||
            'Department of Electronics & IT (GeM Evaluation Cell)',
          email: 'officer@example.gov.in',
          phone: (n == null ? undefined : n.phone) || '+91 98200 12345',
          is_active: true,
        };
      return (
        addAuditLog('2FA_VERIFIED', `2FA OTP verified for '${i.full_name}'.`, 'AUTH'),
        { access_token: `gem_token_${Date.now()}`, token_type: 'bearer', user: i }
      );
    },
  ),
resetPassword: async (e) =>
  requestWithFallback(
    `${API_BASE}/auth/reset-password`,
    { method: 'POST', headers: { 'Content-Type': 'application/json' }, body: JSON.stringify(e) },
    () => {
```

```
      const t = e.username || e.email || 'officer',
        r = otpStore[t] || otpStore.officer;
      if (!r || !r.otp) throw new Error('No active OTP found. Please request a new OTP.');
      if (Date.now() > r.expiresAt) {
        delete otpStore[t];
        throw new Error('OTP has expired. Please request a new one.');
      }
      if (r.otp !== e.otp) throw new Error('Invalid OTP code. Please check and try again.');
      const i = usersStore.find(
        (a) => a.username === e.username || a.email === e.email || a.username === 'officer',
      );
      return (
        i && ((i.password = e.new_password), saveState('users', usersStore)),
        delete otpStore[t],
        addAuditLog(
          'PASSWORD_RESET',
          `Password successfully reset for user '${(i == null ? undefined : i.username) || t}'.`,
          'AUTH',
        ),
        {
          success: true,
          message:
            'Password has been successfully updated. Please sign in with your new password.',
        }
      );
    },
  ),
register: async (e) =>
  requestWithFallback(
    `${API_BASE}/auth/register`,
    { method: 'POST', headers: { 'Content-Type': 'application/json' }, body: JSON.stringify(e) },
    () => {
      if (e.username === 'officer')
        throw new Error("Username 'officer' is reserved for default evaluation.");
      usersStore = usersStore.filter((n) => n.username !== e.username && n.email !== e.email);
      const t = {
        id: Date.now(),
        username: e.username,
        full_name: e.full_name,
        role: 'Procurement Officer',
        department: e.department || 'Department of Electronics & IT (GeM Evaluation Cell)',
        email: e.email,
        phone: e.phone || '',
        password: e.password,
        is_active: true,
      };
      ((usersStore = [...usersStore, t]),
        saveState('users', usersStore),
        addAuditLog(
          'ACCOUNT_CREATED',
          `New account created for '${t.full_name}' (${t.username}).`,
          'AUTH',
        ));
      const r = {
        id: t.id,
        username: t.username,
        full_name: t.full_name,
        role: t.role,
        department: t.department,
        email: t.email,
        phone: t.phone,
        is_active: true,
      };
      return { access_token: `gem_token_${Date.now()}`, token_type: 'bearer', user: r };
    },
  ),
getDashboardStats: async () =>
  requestWithFallback(`${API_BASE}/dashboard`, undefined, () => {
    const e = bidsStore.filter((a) => a.status === 'VERIFIED' || a.status === 'APPROVED').length,
      t = bidsStore.filter((a) => a.risk_level === 'HIGH' || a.risk_level === 'CRITICAL').length,
      r = bidsStore.filter(
        (a) => a.status === 'UNDER_VERIFICATION' || a.status === 'PENDING_REVIEW',
      ).length,
      n = bidsStore.filter(
        (a) => a.status === 'PENDING_REVIEW' || a.risk_level === 'MEDIUM',
      ).length;
    return {
      total_tenders: tendersStore.length,
      active_tenders: tendersStore.filter((a) => a.status === 'ACTIVE').length,
```

```
      bids_under_verification: r,
      verified_bids: e,
      high_risk_bids: t,
      pending_reviews: n,
      compliance_overview: {
        Compliant: bidsStore.filter((a) => a.compliance_score >= 80).length,
        'Review Required': bidsStore.filter(
          (a) => a.compliance_score >= 60 && a.compliance_score < 80,
        ).length,
        'Non-Compliant': bidsStore.filter((a) => a.compliance_score < 60).length,
      },
      risk_distribution: {
        Low: bidsStore.filter((a) => a.risk_level === 'LOW').length,
        Medium: bidsStore.filter((a) => a.risk_level === 'MEDIUM').length,
        High: bidsStore.filter((a) => a.risk_level === 'HIGH').length,
        Critical: bidsStore.filter((a) => a.risk_level === 'CRITICAL').length,
      },
      verification_activity: [
        { date: 'Day 1', verified: 1, flagged: 0 },
        { date: 'Day 2', verified: 2, flagged: 1 },
        { date: 'Day 3', verified: 4, flagged: 1 },
        { date: 'Day 4', verified: 5, flagged: 2 },
        { date: 'Day 5', verified: 7, flagged: 2 },
        { date: 'Today', verified: 8, flagged: 3 },
      ],
      recent_bids: bidsStore.map((a) => {
        var o;
        return {
          id: a.id,
          bid_id: a.bid_id,
          bidder_name: a.bidder.company_name,
          tender_id:
            ((o = tendersStore.find((l) => l.id === a.tender_id)) == null
              ? undefined
              : o.tender_id) || 'GEM/2026/B/12345',
          documents_count: a.documents.length,
          compliance_score: a.compliance_score,
          risk_level: a.risk_level,
          status: a.status,
          last_verified: new Date(a.submission_date).toLocaleDateString('en-IN'),
        };
      }),
    };
  }),
getTenders: async () =>
  requestWithFallback(`${API_BASE}/tenders`, undefined, () => [...tendersStore]),
getTender: async (e) =>
  requestWithFallback(`${API_BASE}/tenders/${e}`, undefined, () => {
    const t = tendersStore.find((r) => String(r.id) === String(e) || r.tender_id === String(e));
    if (!t) throw new Error('Tender not found');
    return t;
  }),
createTender: async (e) =>
  requestWithFallback(
    `${API_BASE}/tenders`,
    { method: 'POST', headers: { 'Content-Type': 'application/json' }, body: JSON.stringify(e) },
    () => {
      const t = {
        id: Date.now(),
        tender_id: e.tender_id || `GEM/2026/B/${Math.floor(1e4 + Math.random() * 9e4)}`,
        title: e.title,
        department: e.department,
        tender_value: Number(e.tender_value) || 5e6,
        deadline: e.deadline || new Date(Date.now() + 12096e5).toISOString().split('T')[0],
        status: 'ACTIVE',
        description: e.description || '',
        created_at: new Date().toISOString(),
        bids_count: 0,
        requirements: (e.requirements || []).map((r, n) => ({
          id: n + 1,
          requirement_code: r.requirement_code,
          requirement_name: r.requirement_name,
          is_mandatory: r.is_mandatory ?? true,
          verification_source: r.verification_source || 'PORTAL_CROSS_CHECK',
          min_condition: r.min_condition || 'Standard GFR Rule 149 condition',
          weight: Number(r.weight) || 10,
          applicable_bidder_type: r.applicable_bidder_type || 'ALL',
        })),
      };
```

```
      return (
        (tendersStore = [t, ...tendersStore]),
        saveState('tenders', tendersStore),
        addAuditLog(
          'TENDER_PUBLISHED',
          `New Tender '${t.tender_id}' published: ${t.title}`,
          'TENDER',
        ),
        t
      );
    },
  ),
addRequirement: async (e, t) =>
  requestWithFallback(
    `${API_BASE}/tenders/${e}/requirements`,
    { method: 'POST', headers: { 'Content-Type': 'application/json' }, body: JSON.stringify(t) },
    () => {
      const r = tendersStore.find((n) => n.id === e);
      return (
        r && (r.requirements.push({ id: Date.now(), ...t }), saveState('tenders', tendersStore)),
        { success: true }
      );
    },
  ),
getBidders: async () =>
  requestWithFallback(`${API_BASE}/bidders`, undefined, () => [...biddersStore]),
createBidder: async (e) =>
  requestWithFallback(
    `${API_BASE}/bidders`,
    { method: 'POST', headers: { 'Content-Type': 'application/json' }, body: JSON.stringify(e) },
    () => {
      const t = {
        id: Date.now(),
        company_name: e.company_name,
        cin: e.cin || undefined,
        pan: (e.pan || '').toUpperCase(),
        gstin: (e.gstin || '').toUpperCase() || undefined,
        registered_address: e.registered_address || undefined,
        contact_email: e.contact_email || undefined,
        contact_phone: e.contact_phone || undefined,
        is_msme: !!e.is_msme,
        msme_reg_no: e.msme_reg_no || undefined,
        is_startup: !!e.is_startup,
        startup_dipp_no: e.startup_dipp_no || undefined,
      };
      return (
        (biddersStore = [t, ...biddersStore]),
        saveState('bidders', biddersStore),
        addAuditLog(
          'BIDDER_REGISTERED',
          `Bidder '${t.company_name}' registered with PAN ${t.pan}`,
          'BIDDER',
        ),
        t
      );
    },
  ),
getBids: async (e) =>
  requestWithFallback(e ? `${API_BASE}/bids?tender_id=${e}` : `${API_BASE}/bids`, undefined, () =>
    (e ? bidsStore.filter((r) => r.tender_id === e) : bidsStore).map((r) => {
      var n;
      return {
        id: r.id,
        bid_id: r.bid_id,
        tender_id:
          ((n = tendersStore.find((i) => i.id === r.tender_id)) == null
            ? undefined
            : n.tender_id) || 'GEM/2026/B/12345',
        bidder_name: r.bidder.company_name,
        bidder_pan: r.bidder.pan,
        documents_count: r.documents.length,
        compliance_score: r.compliance_score,
        risk_level: r.risk_level,
        status: r.status,
        last_verified: new Date(r.submission_date).toLocaleDateString('en-IN'),
      };
    }),
  ),
getBid: async (e) =>
```

```
    requestWithFallback(`${API_BASE}/bids/${e}`, undefined, () => {
      const t = bidsStore.find((n) => String(n.id) === String(e) || n.bid_id === String(e));
      if (!t) throw new Error('Bid not found');
      const r = tendersStore.find((n) => n.id === t.tender_id);
      return { ...t, tender: r };
    }),
  createBid: async (e) =>
    requestWithFallback(
      `${API_BASE}/bids`,
      { method: 'POST', headers: { 'Content-Type': 'application/json' }, body: JSON.stringify(e) },
      () => {
        const t = tendersStore.find((l) => l.id === Number(e.tender_id)) || tendersStore[0],
          r = biddersStore.find((l) => l.id === Number(e.bidder_id)) || biddersStore[0],
          n = Math.floor(1e3 + Math.random() * 9e3),
          a = `BID/${t.tender_id.split('/')[3] || '12345'}/${r.company_name.substring(0, 3).toUpperCase()}-${n}`
    ,
          o = {
            id: Date.now(),
            bid_id: a,
            tender_id: t.id,
            bidder_id: r.id,
            submission_date: new Date().toISOString(),
            status: 'UNDER_VERIFICATION',
            compliance_score: 75,
            risk_level: 'LOW',
            ai_recommendation: 'REVIEW_REQUIRED',
            ai_explanation:
              'Standard criteria cross-verified with statutory databases. Awaiting attached certificates for fin
    al scoring.',
            bidder: r,
            tender: t,
            documents: [
              {
                id: Date.now() + 1,
                bid_id: 0,
                file_name: `${r.company_name.toLowerCase().replace(/\s+/g, '_')}_pan.pdf`,
                file_type: 'pdf',
                file_size_kb: 195,
                classified_type: 'PAN_CARD',
                classification_confidence: 0.98,
                ocr_status: 'COMPLETED',
                created_at: new Date().toISOString(),
                extracted_fields: [
                  {
                    id: Date.now() + 2,
                    field_name: 'pan_number',
                    extracted_value: r.pan || 'ABCDE1234F',
                    confidence: 0.99,
                    bounding_box: { page: 1, x: 140, y: 160, width: 220, height: 35 },
                    is_verified: true,
                  },
                  {
                    id: Date.now() + 3,
                    field_name: 'legal_name',
                    extracted_value: r.company_name.toUpperCase(),
                    confidence: 0.98,
                    bounding_box: { page: 1, x: 140, y: 210, width: 320, height: 30 },
                    is_verified: true,
                  },
                ],
              },
            ],
            compliance_results: [
              {
                id: Date.now() + 10,
                requirement_code: 'PAN',
                requirement_name: 'Permanent Account Number (PAN) Card',
                status: 'VERIFIED',
                points_awarded: 10,
                max_points: 10,
                evidence_document: `${r.company_name.toLowerCase().replace(/\s+/g, '_')}_pan.pdf`,
                confidence_score: 0.99,
                rule_applied: 'ITD e-PAN Match',
                explanation: `PAN ${r.pan} is verified and matches registered entity name.`,
              },
              {
                id: Date.now() + 11,
                requirement_code: 'GST',
                requirement_name: 'Goods & Services Tax (GST) Registration',
```

```
              status: 'VERIFIED',
              points_awarded: 15,
              max_points: 15,
              evidence_document: 'Statutory Verification',
              confidence_score: 0.98,
              rule_applied: 'GSTN Gateway Sync',
              explanation: `GSTIN ${r.gstin || 'Active'} verified in regular status.`,
            },
            {
              id: Date.now() + 12,
              requirement_code: 'DEBARMENT',
              requirement_name: 'Non-Blacklisting / Debarment Affidavit',
              status: 'VERIFIED',
              points_awarded: 10,
              max_points: 10,
              evidence_document: 'CPPP Central Check',
              confidence_score: 1,
              rule_applied: 'CPPP Registry Match',
              explanation: 'No debarment records found.',
            },
          ],
          risk_assessment: {
            id: Date.now() + 20,
            risk_level: 'LOW',
            risk_score: 15,
            risk_factors: [],
            recommendation: 'REVIEW_REQUIRED',
            recommended_action: 'Examine attached proofs and verify credentials',
          },
        };
      return (
        (o.documents[0].bid_id = o.id),
        (t.bids_count = (t.bids_count || 0) + 1),
        (bidsStore = [o, ...bidsStore]),
        saveState('bids', bidsStore),
        saveState('tenders', tendersStore),
        addAuditLog(
          'CREATE_BID_SUBMISSION',
          `Registered new bid '${o.bid_id}' for '${r.company_name}' under tender '${t.tender_id}'.`,
          'BID',
          o.id,
        ),
        o
      );
    },
  ),
uploadDocument: async (e, t, r) => {
  if (IS_FILE_PROTOCOL) {
    const i = bidsStore.find((u) => u.id === e);
    if (!i) throw new Error('Bid not found');
    const a = t.name,
      o = a.toLowerCase();
    let l = r && r !== 'AUTO' ? r : 'GENERAL_DOCUMENT';
    l === 'GENERAL_DOCUMENT' &&
      (o.includes('pan')
        ? (l = 'PAN_CARD')
        : o.includes('gst')
          ? (l = 'GST_CERTIFICATE')
          : o.includes('udyam') || o.includes('msme')
            ? (l = 'UDYAM_CERTIFICATE')
            : o.includes('itr') || o.includes('turnover') || o.includes('ca')
              ? (l = 'ITR_AND_FINANCIALS')
              : o.includes('oem') || o.includes('maf') || o.includes('auth')
                ? (l = 'OEM_AUTHORIZATION')
                : o.includes('mii') || o.includes('make_in_india') || o.includes('local')
                  ? (l = 'MAKE_IN_INDIA')
                  : (o.includes('debar') || o.includes('affidavit')) &&
                    (l = 'DEBARMENT_AFFIDAVIT'));
    const c = {
      id: Date.now(),
      bid_id: e,
      file_name: a,
      file_type: 'pdf',
      file_size_kb: Math.round(t.size / 1024) || 245,
      classified_type: l,
      classification_confidence: 0.97,
      ocr_status: 'COMPLETED',
      created_at: new Date().toISOString(),
      extracted_fields: [
```

```
        {
          id: Date.now() + 1,
          field_name: 'entity_name',
          extracted_value: i.bidder.company_name,
          confidence: 0.98,
          bounding_box: { page: 1, x: 120, y: 150, width: 320, height: 35 },
          is_verified: true,
        },
        {
          id: Date.now() + 2,
          field_name: 'document_code',
          extracted_value: l,
          confidence: 0.96,
          bounding_box: { page: 1, x: 120, y: 200, width: 250, height: 30 },
          is_verified: true,
        },
      ],
    };
    return (
      i.documents.push(c),
      saveState('bids', bidsStore),
      addAuditLog(
        'DOCUMENT_UPLOADED',
        `Uploaded document '${a}' classified as ${l} for bid ${i.bid_id}`,
        'DOCUMENT',
        e,
      ),
      c
    );
  }
  const n = new FormData();
  (n.append('file', t), r && n.append('classified_type_hint', r));
  try {
    const i = await fetch(`${API_BASE}/bids/${e}/documents`, { method: 'POST', body: n });
    if (!i.ok) {
      const a = await i.json();
      throw new Error(a.detail || 'Upload failed');
    }
    return i.json();
  } catch {
    return api.uploadDocument(e, t, r);
  }
},
deleteDocument: async (e, t) =>
  requestWithFallback(`${API_BASE}/bids/${e}/documents/${t}`, { method: 'DELETE' }, () => {
    const r = bidsStore.find((n) => n.id === e);
    if (r) {
      const n = r.documents.find((i) => i.id === t);
      ((r.documents = r.documents.filter((i) => i.id !== t)),
        saveState('bids', bidsStore),
        addAuditLog(
          'DOCUMENT_DELETED',
          `Deleted document '${(n == null ? undefined : n.file_name) || t}' from bid ${r.bid_id}`,
          'DOCUMENT',
          e,
        ));
    }
    return { success: true };
  }),
updateComplianceResult: async (e, t, r) =>
  requestWithFallback(
    `${API_BASE}/bids/${e}/compliance/${t}`,
    { method: 'PUT', headers: { 'Content-Type': 'application/json' }, body: JSON.stringify(r) },
    () => {
      const n = bidsStore.find((i) => i.id === e);
      if (n) {
        const i = n.compliance_results.find((a) => a.id === t);
        if (i) {
          ((i.points_awarded = Number(r.points_awarded)),
            (i.status = r.status),
            r.explanation && (i.explanation = r.explanation));
          const a = n.compliance_results.reduce((l, c) => l + Number(c.points_awarded), 0),
            o = n.compliance_results.reduce((l, c) => l + Number(c.max_points), 0);
          ((n.compliance_score = o > 0 ? Math.min(100, Math.round((a / o) * 100)) : 80),
            saveState('bids', bidsStore),
            addAuditLog(
              'HUMAN_OFFICER_VERIFICATION_EVALUATION',
              `Officer adjusted score for '${i.requirement_name}' to ${i.points_awarded}/${i.max_points} pts.
  Total score updated to ${n.compliance_score}/100.`,
```

```
              'COMPLIANCE',
              e,
            ));
        }
      }
      return { success: true, compliance_score: n == null ? undefined : n.compliance_score };
    },
  ),
verifyBid: async (e) =>
  requestWithFallback(`${API_BASE}/bids/${e}/verify`, { method: 'POST' }, () => {
    const t = bidsStore.find((r) => r.id === e);
    if (!t) throw new Error('Bid not found');
    return (
      (t.compliance_score = Math.min(100, t.compliance_score + 5)),
      (t.status = 'VERIFIED'),
      saveState('bids', bidsStore),
      addAuditLog(
        'BID_COMPLIANCE_RE_VERIFIED',
        `AI verification pipeline re-executed for ${t.bid_id}. Score: ${t.compliance_score}/100.`,
        'BID',
        e,
      ),
      t
    );
  }),
submitOfficerDecision: async (e, t) =>
  requestWithFallback(
    `${API_BASE}/bids/${e}/decision`,
    { method: 'POST', headers: { 'Content-Type': 'application/json' }, body: JSON.stringify(t) },
    () => {
      const r = bidsStore.find((i) => i.id === e);
      if (!r) throw new Error('Bid not found');
      const n = {
        id: Date.now(),
        officer_name: 'Rajesh Kumar Verma',
        decision: t.decision,
        comments: t.comments,
        clarification_details: t.clarification_details,
        decided_at: new Date().toISOString(),
      };
      return (
        (r.officer_decision = n),
        (r.status =
          t.decision === 'APPROVED'
            ? 'APPROVED'
            : t.decision === 'REJECTED'
              ? 'REJECTED'
              : 'REQUIRES_CLARIFICATION'),
        saveState('bids', bidsStore),
        addAuditLog(
          `OFFICER_DECISION_${t.decision}`,
          `Officer signed decision '${t.decision}' for bid ${r.bid_id}. Remarks: ${t.comments}`,
          'DECISION',
          e,
        ),
        { success: true, officer_decision: n }
      );
    },
  ),
getIntegrationsStatus: async () =>
  requestWithFallback(`${API_BASE}/integrations/status`, undefined, () => ({
    connectors: seedConnectors,
    disclaimer:
      'All statutory connectors configured with mTLS certificates, IP whitelisting, and GFR 2017 compliant aud
  it streaming.',
  })),
testGST: async (e) =>
  requestWithFallback(
    `${API_BASE}/integrations/test-gst?gstin=${encodeURIComponent(e)}`,
    { method: 'POST' },
    () => {
      const t = e.trim().toUpperCase();
      return t.includes('BHART') || t.includes('9999')
        ? {
            gstin: t,
            status: 'SUSPENDED',
            legal_name: 'Bharat Systems Pvt Ltd',
            state: 'Rajasthan (08)',
            taxpayer_type: 'Regular',
```

```
            filing_status: 'Defaulted / GSTR-3B overdue for 4 quarters',
            flag: 'HIGH_RISK_DEFAULTER',
          }
        : {
            gstin: t,
            status: 'ACTIVE',
            legal_name: 'ABC Technologies Pvt Ltd',
            state: 'Maharashtra (27)',
            taxpayer_type: 'Regular',
            filing_status: 'Clean Filing Record (All returns submitted up to date)',
            flag: 'COMPLIANT',
          };
    },
  ),
testPAN: async (e) =>
  requestWithFallback(
    `${API_BASE}/integrations/test-pan?pan=${encodeURIComponent(e)}`,
    { method: 'POST' },
    () => {
      const t = e.trim().toUpperCase();
      return {
        pan: t,
        status: 'VALID',
        entity_name: t.includes('ABC')
          ? 'ABC TECHNOLOGIES PVT LTD'
          : t.includes('XYZ')
            ? 'XYZ SOLUTIONS LIMITED'
            : 'REGISTERED CORPORATE ENTITY',
        entity_type: 'Company',
        aadhaar_seeded: true,
        itd_verification: 'Active & In Good Standing',
      };
    },
  ),
testDebarment: async (e, t) =>
  requestWithFallback(
    `${API_BASE}/integrations/test-debarment?company_name=${encodeURIComponent(e)}&pan=${encodeURIComponent(t
  || '')}`,
    { method: 'POST' },
    () =>
      e.toLowerCase().includes('bharat') || (t && t.includes('BHART'))
        ? {
            company_name: e,
            status: 'DEBARRED_MATCH_FOUND',
            severity: 'CRITICAL',
            ministry: 'Ministry of Defence (Acquisition Wing)',
            order_number: 'MOD/DEB/2025/892',
            period: '2 Years (Active until 15-Nov-2027)',
            reason: 'Submission of fabricated experience credentials in defense procurement',
            gfr_rule: 'Mandatory disqualification under GFR Rule 151',
          }
        : {
            company_name: e,
            status: 'NO_DEBARMENT_RECORD',
            severity: 'CLEAR',
            ministry: 'All Central Ministries & CPSUs',
            records_checked: 14200,
            gfr_rule: 'Clear for participation under GFR 2017',
          },
  ),
getAuditLogs: async (e) =>
  requestWithFallback(e ? `${API_BASE}/audit?bid_id=${e}` : `${API_BASE}/audit`, undefined, () =>
    e ? auditStore.filter((t) => t.bid_id === e) : auditStore,
  ),
getNotifications: async () =>
  requestWithFallback(`${API_BASE}/notifications`, undefined, () => [...notificationsStore]),
markNotificationRead: async (e) =>
  requestWithFallback(`${API_BASE}/notifications/${e}/read`, { method: 'POST' }, () => {
    const t = notificationsStore.find((r) => r.id === e);
    return (
      t && (t.is_read = true),
      saveState('notifications', notificationsStore),
      { success: true }
    );
  }),
exportData: () => ({
  appName: 'GeM BidVerify AI',
  version: '2.0',
  exportedAt: new Date().toISOString(),
```

```
    tenders: tendersStore,
    bidders: biddersStore,
    bids: bidsStore,
    audit_logs: auditStore,
    users: usersStore,
  }),
  importData: (e) => (
    e.tenders &&
      Array.isArray(e.tenders) &&
      ((tendersStore = e.tenders), saveState('tenders', tendersStore)),
    e.bidders &&
      Array.isArray(e.bidders) &&
      ((biddersStore = e.bidders), saveState('bidders', biddersStore)),
    e.bids && Array.isArray(e.bids) && ((bidsStore = e.bids), saveState('bids', bidsStore)),
    e.audit_logs &&
      Array.isArray(e.audit_logs) &&
      ((auditStore = e.audit_logs), saveState('audit_logs', auditStore)),
    e.users && Array.isArray(e.users) && ((usersStore = e.users), saveState('users', usersStore)),
    addAuditLog(
      'DATA_PACKAGE_IMPORTED',
      'All procurement and evaluation records imported from external backup package.',
      'SYSTEM',
    ),
    { success: true, message: 'All procurement and evaluation records imported successfully!' }
  ),
  resetData: () => (
    (tendersStore = [...seedTenders]),
    (biddersStore = [...seedBidders]),
    (bidsStore = [...seedBids]),
    (auditStore = [...seedAuditLogs]),
    (usersStore = [...seedUsers]),
    saveState('tenders', tendersStore),
    saveState('bidders', biddersStore),
    saveState('bids', bidsStore),
    saveState('audit_logs', auditStore),
    saveState('users', usersStore),
    addAuditLog(
      'DATA_RESET_DEFAULT',
      'Restored official GFR 2017 baseline procurement and evaluation records.',
      'SYSTEM',
    ),
    { success: true }
  ),
};


//// FILE: src/api/seedData.js  (960 lines)

export const seedTenders = [
  {
    id: 1,
    tender_id: 'GEM/2026/B/12345',
    title: 'Supply, Installation & Commissioning of Server & IT Hardware',
    department: 'Department of Electronics & Information Technology',
    tender_value: 5e6,
    deadline: new Date(Date.now() + 14 * 864e5).toISOString().split('T')[0],
    status: 'ACTIVE',
    description:
      'Procurement of Enterprise Grade Servers, Storage Racks, and Core Networking Equipment under GFR Rule 149.
    ',
    created_at: new Date(Date.now() - 10 * 864e5).toISOString(),
    bids_count: 3,
    requirements: [
      {
        id: 1,
        requirement_code: 'PAN',
        requirement_name: 'Permanent Account Number (PAN) Card',
        is_mandatory: true,
        verification_source: 'ITD_PAN_REGISTRY',
        min_condition: 'Valid Active PAN in bidder entity name',
        weight: 10,
        applicable_bidder_type: 'ALL',
      },
      {
        id: 2,
        requirement_code: 'GST',
        requirement_name: 'Goods & Services Tax (GST) Registration',
        is_mandatory: true,
        verification_source: 'GSTN_PORTAL',
```

```
      min_condition: 'Active GSTIN with regular filing record',
      weight: 15,
      applicable_bidder_type: 'ALL',
    },
    {
      id: 3,
      requirement_code: 'UDYAM',
      requirement_name: 'MSME / Udyam Registration Certificate',
      is_mandatory: false,
      verification_source: 'UDYAM_PORTAL',
      min_condition: 'Valid Udyam reg for MSME purchase preference',
      weight: 10,
      applicable_bidder_type: 'ALL',
    },
    {
      id: 4,
      requirement_code: 'ITR',
      requirement_name: 'Income Tax Returns & CA Turnover Certificate (Last 3 FYs)',
      is_mandatory: true,
      verification_source: 'ITD_ITR_ICAI_UDIN',
      min_condition: 'Min average turnover ₹1.5 Cr with valid UDIN',
      weight: 15,
      applicable_bidder_type: 'ALL',
    },
    {
      id: 5,
      requirement_code: 'OEM_AUTH',
      requirement_name: 'Manufacturer Authorization Form (MAF / OEM Auth)',
      is_mandatory: true,
      verification_source: 'OEM_VERIFICATION',
      min_condition: 'Direct authorization letter addressed to tender',
      weight: 15,
      applicable_bidder_type: 'ALL',
    },
    {
      id: 6,
      requirement_code: 'MAKE_IN_INDIA',
      requirement_name: 'Make in India (MII) Local Content Declaration',
      is_mandatory: true,
      verification_source: 'DPIIT_MII_DECLARATION',
      min_condition: 'Class-I Local Supplier (>=50% local content)',
      weight: 15,
      applicable_bidder_type: 'ALL',
    },
    {
      id: 7,
      requirement_code: 'EPFO',
      requirement_name: 'EPFO Registration & ECR Electronic Challan',
      is_mandatory: false,
      verification_source: 'EPFO_PORTAL',
      min_condition: 'Active establishment code with recent ECR',
      weight: 5,
      applicable_bidder_type: 'ALL',
    },
    {
      id: 8,
      requirement_code: 'ESIC',
      requirement_name: 'ESIC Registration & Compliance Declaration',
      is_mandatory: false,
      verification_source: 'ESIC_PORTAL',
      min_condition: 'Active ESIC employer registration',
      weight: 5,
      applicable_bidder_type: 'ALL',
    },
    {
      id: 9,
      requirement_code: 'DEBARMENT',
      requirement_name: 'Non-Blacklisting / Debarment Affidavit',
      is_mandatory: true,
      verification_source: 'CPPP_GEM_DEBARMENT_LIST',
      min_condition: 'No debarment by any Central/State Govt/PSU',
      weight: 10,
      applicable_bidder_type: 'ALL',
    },
  ],
},
{
  id: 2,
  tender_id: 'GEM/2026/B/98765',
```

```
    title: 'Annual Maintenance Contract for National Data Center Infrastructure',
    department: 'Ministry of Finance / Department of Expenditure',
    tender_value: 75e5,
    deadline: new Date(Date.now() + 21 * 864e5).toISOString().split('T')[0],
    status: 'ACTIVE',
    description:
      'Comprehensive maintenance and operations SLA for tier-3 data center facilities under GFR Rule 149.',
    created_at: new Date(Date.now() - 5 * 864e5).toISOString(),
    bids_count: 0,
    requirements: [
      {
        id: 10,
        requirement_code: 'PAN',
        requirement_name: 'Permanent Account Number (PAN) Card',
        is_mandatory: true,
        verification_source: 'ITD_PAN_REGISTRY',
        min_condition: 'Valid active PAN in bidder name',
        weight: 10,
        applicable_bidder_type: 'ALL',
      },
      {
        id: 11,
        requirement_code: 'GST',
        requirement_name: 'Goods & Services Tax (GST) Registration',
        is_mandatory: true,
        verification_source: 'GSTN_PORTAL',
        min_condition: 'Active GSTIN with regular filing',
        weight: 15,
        applicable_bidder_type: 'ALL',
      },
      {
        id: 12,
        requirement_code: 'ITR',
        requirement_name: 'Audited Financials (Last 3 FYs)',
        is_mandatory: true,
        verification_source: 'ITD_ITR_ICAI_UDIN',
        min_condition: 'Min average turnover ₹2.5 Cr with valid UDIN',
        weight: 20,
        applicable_bidder_type: 'ALL',
      },
      {
        id: 13,
        requirement_code: 'DEBARMENT',
        requirement_name: 'Non-Blacklisting / Debarment Affidavit',
        is_mandatory: true,
        verification_source: 'CPPP_GEM_DEBARMENT_LIST',
        min_condition: 'No debarment by any Central/State Govt/PSU',
        weight: 15,
        applicable_bidder_type: 'ALL',
      },
    ],
  },
];

export const seedBidders = [
  {
    id: 1,
    company_name: 'ABC Technologies Pvt Ltd',
    cin: 'U72900MH2018PTC308912',
    pan: 'ABCDE1234F',
    gstin: '27ABCDE1234F1Z5',
    registered_address: 'Plot 42, Tech Park, Andheri East, Mumbai, Maharashtra - 400069',
    contact_email: 'bids@abctech.in',
    contact_phone: '+91 22 6789 0123',
    is_msme: true,
    msme_reg_no: 'UDYAM-MH-01-0012345',
    is_startup: false,
  },
  {
    id: 2,
    company_name: 'XYZ Solutions Ltd',
    cin: 'L72200DL2015PLC278901',
    pan: 'XYZAB5678C',
    gstin: '07XYZAB5678C1Z2',
    registered_address: 'Connaught Place, New Delhi - 110001',
    contact_email: 'tenders@xyzsolutions.com',
    contact_phone: '+91 11 4567 8901',
    is_msme: true,
    msme_reg_no: 'UDYAM-DL-02-0098765',
```

```
    is_startup: true,
    startup_dipp_no: 'DIPP-DL-2021-9988',
  },
  {
    id: 3,
    company_name: 'Bharat Systems Pvt Ltd',
    cin: 'U72900RJ2022PTC081234',
    pan: 'BHART9999K',
    gstin: '08BHART9999K1Z8',
    registered_address: 'Industrial Area, Jaipur, Rajasthan - 302013',
    contact_email: 'info@bharatsystems.org',
    contact_phone: '+91 141 234 5678',
    is_msme: false,
    is_startup: false,
  },
];

export const seedBids = [
  {
    id: 1,
    bid_id: 'BID/12345/ABC-01',
    tender_id: 1,
    bidder_id: 1,
    submission_date: new Date(Date.now() - 3 * 864e5).toISOString(),
    status: 'VERIFIED',
    compliance_score: 85,
    risk_level: 'LOW',
    ai_recommendation: 'QUALIFIED_WITH_HIGH_CONFIDENCE',
    ai_explanation:
      'All mandatory statutory criteria satisfied. GSTN filing regular. ICAI UDIN turnover verified. Cisco OEM a
    uthorization valid. Debarment database clear.',
    bidder: seedBidders[0],
    documents: [
      {
        id: 101,
        bid_id: 1,
        file_name: 'PAN_Card_ABC_Tech.pdf',
        file_type: 'pdf',
        file_size_kb: 340.5,
        classified_type: 'PAN_CARD',
        classification_confidence: 0.99,
        ocr_status: 'COMPLETED',
        created_at: new Date(Date.now() - 3 * 864e5).toISOString(),
        extracted_fields: [
          {
            id: 1,
            field_name: 'pan_number',
            extracted_value: 'ABCDE1234F',
            confidence: 0.99,
            bounding_box: { page: 1, x: 140, y: 160, width: 220, height: 35 },
            is_verified: true,
          },
          {
            id: 2,
            field_name: 'name_on_pan',
            extracted_value: 'ABC TECHNOLOGIES PVT LTD',
            confidence: 0.98,
            bounding_box: { page: 1, x: 140, y: 210, width: 320, height: 30 },
            is_verified: true,
          },
        ],
      },
      {
        id: 102,
        bid_id: 1,
        file_name: 'GST_Certificate_ABC_Tech.pdf',
        file_type: 'pdf',
        file_size_kb: 420,
        classified_type: 'GST_CERTIFICATE',
        classification_confidence: 0.97,
        ocr_status: 'COMPLETED',
        created_at: new Date(Date.now() - 3 * 864e5).toISOString(),
        extracted_fields: [
          {
            id: 3,
            field_name: 'gstin',
            extracted_value: '27ABCDE1234F1Z5',
            confidence: 0.99,
            bounding_box: { page: 1, x: 120, y: 180, width: 280, height: 30 },
```

```
        is_verified: true,
      },
      {
        id: 4,
        field_name: 'legal_name',
        extracted_value: 'ABC Technologies Pvt Ltd',
        confidence: 0.97,
        bounding_box: { page: 1, x: 120, y: 230, width: 360, height: 32 },
        is_verified: true,
      },
      {
        id: 5,
        field_name: 'status',
        extracted_value: 'ACTIVE - Regular Taxpayer',
        confidence: 0.96,
        bounding_box: { page: 1, x: 120, y: 280, width: 240, height: 28 },
        is_verified: true,
      },
    ],
  },
  {
    id: 103,
    bid_id: 1,
    file_name: 'Udyam_Registration_ABC.pdf',
    file_type: 'pdf',
    file_size_kb: 210,
    classified_type: 'UDYAM_CERTIFICATE',
    classification_confidence: 0.96,
    ocr_status: 'COMPLETED',
    created_at: new Date(Date.now() - 3 * 864e5).toISOString(),
    extracted_fields: [
      {
        id: 6,
        field_name: 'udyam_no',
        extracted_value: 'UDYAM-MH-01-0012345',
        confidence: 0.97,
        bounding_box: { page: 1, x: 130, y: 140, width: 300, height: 32 },
        is_verified: true,
      },
      {
        id: 7,
        field_name: 'enterprise_type',
        extracted_value: 'Small Enterprise (MSME)',
        confidence: 0.95,
        bounding_box: { page: 1, x: 130, y: 190, width: 260, height: 28 },
        is_verified: true,
      },
    ],
  },
  {
    id: 104,
    bid_id: 1,
    file_name: 'ITR_and_CA_Turnover_Cert_ABC.pdf',
    file_type: 'pdf',
    file_size_kb: 890,
    classified_type: 'ITR_AND_FINANCIALS',
    classification_confidence: 0.95,
    ocr_status: 'COMPLETED',
    created_at: new Date(Date.now() - 3 * 864e5).toISOString(),
    extracted_fields: [
      {
        id: 8,
        field_name: 'ca_udin',
        extracted_value: '24078912AAAAAA9999',
        confidence: 0.96,
        bounding_box: { page: 1, x: 150, y: 420, width: 260, height: 28 },
        is_verified: true,
      },
      {
        id: 9,
        field_name: 'avg_annual_turnover',
        extracted_value: '₹4.80 Crores (3-Yr Avg)',
        confidence: 0.94,
        bounding_box: { page: 1, x: 150, y: 260, width: 280, height: 30 },
        is_verified: true,
      },
    ],
  },
  {
```

```
      id: 105,
      bid_id: 1,
      file_name: 'OEM_Authorization_Cisco_ABC.pdf',
      file_type: 'pdf',
      file_size_kb: 310,
      classified_type: 'OEM_AUTHORIZATION',
      classification_confidence: 0.96,
      ocr_status: 'COMPLETED',
      created_at: new Date(Date.now() - 3 * 864e5).toISOString(),
      extracted_fields: [
        {
          id: 10,
          field_name: 'authorized_entity_name',
          extracted_value: 'ABC Technologies Pvt Ltd',
          confidence: 0.97,
          bounding_box: { page: 1, x: 110, y: 210, width: 340, height: 34 },
          is_verified: true,
        },
        {
          id: 11,
          field_name: 'oem_brand',
          extracted_value: 'Cisco Systems India',
          confidence: 0.95,
          bounding_box: { page: 1, x: 110, y: 160, width: 250, height: 28 },
          is_verified: true,
        },
      ],
    },
    {
      id: 106,
      bid_id: 1,
      file_name: 'Make_in_India_Declaration_ABC.pdf',
      file_type: 'pdf',
      file_size_kb: 180,
      classified_type: 'MAKE_IN_INDIA',
      classification_confidence: 0.95,
      ocr_status: 'COMPLETED',
      created_at: new Date(Date.now() - 3 * 864e5).toISOString(),
      extracted_fields: [
        {
          id: 12,
          field_name: 'local_content_percentage',
          extracted_value: '65.0%',
          confidence: 0.96,
          bounding_box: { page: 1, x: 130, y: 220, width: 200, height: 30 },
          is_verified: true,
        },
        {
          id: 13,
          field_name: 'supplier_tier',
          extracted_value: 'Class-I Local Supplier (>=50%)',
          confidence: 0.95,
          bounding_box: { page: 1, x: 130, y: 270, width: 280, height: 30 },
          is_verified: true,
        },
      ],
    },
    {
      id: 107,
      bid_id: 1,
      file_name: 'Non_Blacklisting_Affidavit_ABC.pdf',
      file_type: 'pdf',
      file_size_kb: 195,
      classified_type: 'DEBARMENT_AFFIDAVIT',
      classification_confidence: 0.94,
      ocr_status: 'COMPLETED',
      created_at: new Date(Date.now() - 3 * 864e5).toISOString(),
      extracted_fields: [
        {
          id: 14,
          field_name: 'affidavit_status',
          extracted_value: 'Not Blacklisted / Debarred',
          confidence: 0.96,
          bounding_box: { page: 1, x: 120, y: 200, width: 320, height: 32 },
          is_verified: true,
        },
      ],
    },
  ],
```

```
  compliance_results: [
    {
      id: 201,
      requirement_code: 'PAN',
      requirement_name: 'Permanent Account Number (PAN) Card',
      status: 'VERIFIED',
      points_awarded: 10,
      max_points: 10,
      evidence_document: 'PAN_Card_ABC_Tech.pdf',
      confidence_score: 0.99,
      rule_applied: 'ITD e-PAN Active Registry Match',
      explanation: 'PAN ABCDE1234F is active and matches bidder legal name.',
    },
    {
      id: 202,
      requirement_code: 'GST',
      requirement_name: 'Goods & Services Tax (GST) Registration',
      status: 'VERIFIED',
      points_awarded: 15,
      max_points: 15,
      evidence_document: 'GST_Certificate_ABC_Tech.pdf',
      confidence_score: 0.98,
      rule_applied: 'GSTN Regular Filing Check',
      explanation: 'GSTIN 27ABCDE1234F1Z5 is active with regular GSTR-3B filings.',
    },
    {
      id: 203,
      requirement_code: 'UDYAM',
      requirement_name: 'MSME / Udyam Registration Certificate',
      status: 'VERIFIED',
      points_awarded: 10,
      max_points: 10,
      evidence_document: 'Udyam_Registration_ABC.pdf',
      confidence_score: 0.97,
      rule_applied: 'Ministry of MSME Gateway Check',
      explanation: 'Valid Small Enterprise Udyam certificate UDYAM-MH-01-0012345 verified.',
    },
    {
      id: 204,
      requirement_code: 'ITR',
      requirement_name: 'Income Tax Returns & CA Turnover Certificate',
      status: 'VERIFIED',
      points_awarded: 15,
      max_points: 15,
      evidence_document: 'ITR_and_CA_Turnover_Cert_ABC.pdf',
      confidence_score: 0.96,
      rule_applied: 'ICAI UDIN & Turnover Validation',
      explanation: '3-year average turnover ₹4.80 Cr exceeds minimum ₹1.5 Cr threshold.',
    },
    {
      id: 205,
      requirement_code: 'OEM_AUTH',
      requirement_name: 'Manufacturer Authorization Form (MAF)',
      status: 'VERIFIED',
      points_awarded: 15,
      max_points: 15,
      evidence_document: 'OEM_Authorization_Cisco_ABC.pdf',
      confidence_score: 0.96,
      rule_applied: 'OEM Verification Engine',
      explanation: 'Cisco authorization letter directly addressed to GEM/2026/B/12345.',
    },
    {
      id: 206,
      requirement_code: 'MAKE_IN_INDIA',
      requirement_name: 'Make in India Local Content Declaration',
      status: 'VERIFIED',
      points_awarded: 15,
      max_points: 15,
      evidence_document: 'Make_in_India_Declaration_ABC.pdf',
      confidence_score: 0.95,
      rule_applied: 'DPIIT Local Content Formula',
      explanation: 'Declared 65% local content meets Class-I supplier requirement.',
    },
    {
      id: 207,
      requirement_code: 'DEBARMENT',
      requirement_name: 'Non-Blacklisting / Debarment Affidavit',
      status: 'VERIFIED',
      points_awarded: 10,
```

```
      max_points: 10,
      evidence_document: 'Non_Blacklisting_Affidavit_ABC.pdf',
      confidence_score: 0.98,
      rule_applied: 'CPPP Debarred Directory Match',
      explanation: 'No active debarment or vigilance alerts found.',
    },
  ],
  risk_assessment: {
    id: 301,
    risk_level: 'LOW',
    risk_score: 10,
    risk_factors: [],
    recommendation: 'QUALIFIED_WITH_HIGH_CONFIDENCE',
    recommended_action: 'Proceed to commercial envelope opening',
  },
},
{
  id: 2,
  bid_id: 'BID/12345/XYZ-02',
  tender_id: 1,
  bidder_id: 2,
  submission_date: new Date(Date.now() - 2 * 864e5).toISOString(),
  status: 'UNDER_VERIFICATION',
  compliance_score: 65,
  risk_level: 'MEDIUM',
  ai_recommendation: 'REVIEW_REQUIRED',
  ai_explanation:
    'Discrepancy detected in Manufacturer Authorization Form. Authorized entity name "Apex Networks India Pvt
  Ltd" differs from bidder company name "XYZ Solutions Ltd".',
  bidder: seedBidders[1],
  documents: [
    {
      id: 108,
      bid_id: 2,
      file_name: 'PAN_XYZ_Solutions.pdf',
      file_type: 'pdf',
      file_size_kb: 280,
      classified_type: 'PAN_CARD',
      classification_confidence: 0.98,
      ocr_status: 'COMPLETED',
      created_at: new Date(Date.now() - 2 * 864e5).toISOString(),
      extracted_fields: [
        {
          id: 15,
          field_name: 'pan_number',
          extracted_value: 'XYZAB5678C',
          confidence: 0.99,
          bounding_box: { page: 1, x: 140, y: 160, width: 220, height: 35 },
          is_verified: true,
        },
        {
          id: 16,
          field_name: 'name_on_pan',
          extracted_value: 'XYZ SOLUTIONS LIMITED',
          confidence: 0.97,
          bounding_box: { page: 1, x: 140, y: 210, width: 320, height: 30 },
          is_verified: true,
        },
      ],
    },
    {
      id: 109,
      bid_id: 2,
      file_name: 'GST_XYZ_Solutions.pdf',
      file_type: 'pdf',
      file_size_kb: 340,
      classified_type: 'GST_CERTIFICATE',
      classification_confidence: 0.96,
      ocr_status: 'COMPLETED',
      created_at: new Date(Date.now() - 2 * 864e5).toISOString(),
      extracted_fields: [
        {
          id: 17,
          field_name: 'gstin',
          extracted_value: '07XYZAB5678C1Z2',
          confidence: 0.98,
          bounding_box: { page: 1, x: 120, y: 180, width: 280, height: 30 },
          is_verified: true,
        },
```

```
        {
          id: 18,
          field_name: 'legal_name',
          extracted_value: 'XYZ Solutions Ltd',
          confidence: 0.96,
          bounding_box: { page: 1, x: 120, y: 230, width: 360, height: 32 },
          is_verified: true,
        },
      ],
    },
    {
      id: 110,
      bid_id: 2,
      file_name: 'OEM_Auth_Letter_XYZ.pdf',
      file_type: 'pdf',
      file_size_kb: 310,
      classified_type: 'OEM_AUTHORIZATION',
      classification_confidence: 0.91,
      ocr_status: 'COMPLETED',
      created_at: new Date(Date.now() - 2 * 864e5).toISOString(),
      extracted_fields: [
        {
          id: 19,
          field_name: 'authorized_entity_name',
          extracted_value: 'Apex Networks India Pvt Ltd',
          confidence: 0.91,
          bounding_box: { page: 1, x: 110, y: 210, width: 340, height: 34 },
          is_verified: false,
        },
        {
          id: 20,
          field_name: 'oem_brand',
          extracted_value: 'Dell Enterprise India',
          confidence: 0.94,
          bounding_box: { page: 1, x: 110, y: 160, width: 250, height: 28 },
          is_verified: true,
        },
      ],
    },
  ],
  compliance_results: [
    {
      id: 208,
      requirement_code: 'PAN',
      requirement_name: 'Permanent Account Number (PAN) Card',
      status: 'VERIFIED',
      points_awarded: 10,
      max_points: 10,
      evidence_document: 'PAN_XYZ_Solutions.pdf',
      confidence_score: 0.98,
      rule_applied: 'ITD e-PAN Check',
      explanation: 'PAN valid and active.',
    },
    {
      id: 209,
      requirement_code: 'GST',
      requirement_name: 'Goods & Services Tax (GST) Registration',
      status: 'VERIFIED',
      points_awarded: 15,
      max_points: 15,
      evidence_document: 'GST_XYZ_Solutions.pdf',
      confidence_score: 0.96,
      rule_applied: 'GSTN Check',
      explanation: 'Active GST registration in Delhi.',
    },
    {
      id: 210,
      requirement_code: 'OEM_AUTH',
      requirement_name: 'Manufacturer Authorization Form (MAF)',
      status: 'MISMATCH',
      points_awarded: 0,
      max_points: 15,
      evidence_document: 'OEM_Auth_Letter_XYZ.pdf',
      confidence_score: 0.88,
      rule_applied: 'OEM Entity Consistency Check',
      explanation: 'Authorized entity name is "Apex Networks India Pvt Ltd", not bidder.',
      discrepancy_details: {
        extracted: 'Apex Networks India Pvt Ltd',
        expected: 'XYZ Solutions Ltd',
```

```
      },
    },
  ],
  risk_assessment: {
    id: 302,
    risk_level: 'MEDIUM',
    risk_score: 45,
    risk_factors: ['OEM Authorization Entity Mismatch'],
    recommendation: 'REVIEW_REQUIRED',
    recommended_action: 'Issue clarification notice via GeM portal',
  },
},
{
  id: 3,
  bid_id: 'BID/12345/BHA-03',
  tender_id: 1,
  bidder_id: 3,
  submission_date: new Date(Date.now() - 1 * 864e5).toISOString(),
  status: 'PENDING_REVIEW',
  compliance_score: 30,
  risk_level: 'CRITICAL',
  ai_recommendation: 'CRITICAL_DISQUALIFICATION_REQUIRED',
  ai_explanation:
    'Entity matches Ministry of Defence Central Debarment Order MOD/DEB/2025/892. Mandatory disqualification u
  nder GFR Rule 151.',
  bidder: seedBidders[2],
  documents: [
    {
      id: 111,
      bid_id: 3,
      file_name: 'PAN_Bharat_Systems.pdf',
      file_type: 'pdf',
      file_size_kb: 180,
      classified_type: 'PAN_CARD',
      classification_confidence: 0.97,
      ocr_status: 'COMPLETED',
      created_at: new Date(Date.now() - 1 * 864e5).toISOString(),
      extracted_fields: [
        {
          id: 21,
          field_name: 'pan_number',
          extracted_value: 'BHART9999K',
          confidence: 0.98,
          bounding_box: { page: 1, x: 140, y: 160, width: 220, height: 35 },
          is_verified: true,
        },
        {
          id: 22,
          field_name: 'name_on_pan',
          extracted_value: 'BHARAT SYSTEMS PVT LTD',
          confidence: 0.97,
          bounding_box: { page: 1, x: 140, y: 210, width: 320, height: 30 },
          is_verified: true,
        },
      ],
    },
  ],
  compliance_results: [
    {
      id: 211,
      requirement_code: 'PAN',
      requirement_name: 'Permanent Account Number (PAN) Card',
      status: 'VERIFIED',
      points_awarded: 10,
      max_points: 10,
      evidence_document: 'PAN_Bharat_Systems.pdf',
      confidence_score: 0.97,
      rule_applied: 'ITD Check',
      explanation: 'PAN is valid.',
    },
    {
      id: 212,
      requirement_code: 'DEBARMENT',
      requirement_name: 'Non-Blacklisting / Debarment Affidavit',
      status: 'NON_COMPLIANT',
      points_awarded: 0,
      max_points: 10,
      evidence_document: 'None',
      confidence_score: 1,
```

```
        rule_applied: 'CPPP Central Debarment Directory',
        explanation: 'Entity debarred by Ministry of Defence under order MOD/DEB/2025/892.',
      },
    ],
    risk_assessment: {
      id: 303,
      risk_level: 'CRITICAL',
      risk_score: 95,
      risk_factors: ['Central Debarment Registry Match', 'Missing Mandatory GST', 'Missing ITR'],
      recommendation: 'CRITICAL_DISQUALIFICATION_REQUIRED',
      recommended_action: 'Issue formal disqualification memo under GFR Rule 151',
    },
  },
];

export const seedConnectors = [
  {
    id: 'GSTN_PORTAL',
    name: 'Goods & Services Tax Network (GSTN)',
    type: 'STATUTORY',
    mode: 'REST / OAuth2',
    status: 'OPERATIONAL',
    latency_ms: 142,
    badge: 'Live Gateway',
  },
  {
    id: 'ITD_PAN_REGISTRY',
    name: 'Income Tax Department (e-PAN)',
    type: 'STATUTORY',
    mode: 'SOAP / XML Sign',
    status: 'OPERATIONAL',
    latency_ms: 89,
    badge: 'Active Sync',
  },
  {
    id: 'MCA21_V3',
    name: 'Ministry of Corporate Affairs (MCA21 V3)',
    type: 'STATUTORY',
    mode: 'API Gateway',
    status: 'OPERATIONAL',
    latency_ms: 195,
    badge: 'Vigilance Hub',
  },
  {
    id: 'UDYAM_MSME',
    name: 'Ministry of MSME (Udyam Verification)',
    type: 'PREFERENCE',
    mode: 'REST / JSON',
    status: 'OPERATIONAL',
    latency_ms: 110,
    badge: 'Direct API',
  },
  {
    id: 'ICAI_UDIN',
    name: 'ICAI Unique Document Identification Number',
    type: 'AUDIT',
    mode: 'mTLS Secure',
    status: 'OPERATIONAL',
    latency_ms: 210,
    badge: 'Chartered Sync',
  },
  {
    id: 'CPPP_DEBARMENT',
    name: 'Central Public Procurement Portal (CPPP Debarred List)',
    type: 'VIGILANCE',
    mode: 'Central Cache',
    status: 'OPERATIONAL',
    latency_ms: 65,
    badge: 'Continuous Poll',
  },
  {
    id: 'GEM_VENDOR_BASE',
    name: 'GeM Central Vendor Master Directory',
    type: 'CORE',
    mode: 'Internal Bus',
    status: 'OPERATIONAL',
    latency_ms: 45,
    badge: 'Realtime',
  },
```

```
  {
    id: 'DPIIT_STARTUP',
    name: 'DPIIT Startup India Recognition Portal',
    type: 'EXEMPTION',
    mode: 'REST API',
    status: 'OPERATIONAL',
    latency_ms: 130,
    badge: 'Active',
  },
  {
    id: 'EPFO_ECR',
    name: 'Employees Provident Fund Org (EPFO ECR)',
    type: 'LABOUR',
    mode: 'Batch / REST',
    status: 'OPERATIONAL',
    latency_ms: 175,
    badge: 'Compliance Grid',
  },
  {
    id: 'ESIC_IP_PORTAL',
    name: 'Employees State Insurance Corp (ESIC)',
    type: 'LABOUR',
    mode: 'REST API',
    status: 'OPERATIONAL',
    latency_ms: 160,
    badge: 'Compliance Grid',
  },
  {
    id: 'RBI_DEFAULTER',
    name: 'RBI Willful Defaulters & CRILC Registry',
    type: 'FINANCIAL',
    mode: 'Secure Connect',
    status: 'OPERATIONAL',
    latency_ms: 95,
    badge: 'Banking Grid',
  },
  {
    id: 'BIS_CRS',
    name: 'Bureau of Indian Standards (BIS Quality / CRS)',
    type: 'QUALITY',
    mode: 'REST API',
    status: 'OPERATIONAL',
    latency_ms: 180,
    badge: 'Standard Verify',
  },
  {
    id: 'TRADEMARK_REGISTRY',
    name: 'Controller General of Patents, Designs & Trademarks',
    type: 'IPR',
    mode: 'IP Gateway',
    status: 'OPERATIONAL',
    latency_ms: 220,
    badge: 'IP Registry',
  },
  {
    id: 'CVC_VIGILANCE_GRID',
    name: 'Central Vigilance Commission (CVC Audit Network)',
    type: 'AUDIT',
    mode: 'Cryptographic Stream',
    status: 'OPERATIONAL',
    latency_ms: 70,
    badge: 'Immutable Ledger',
  },
];

export const seedAuditLogs = [
  {
    id: 1,
    user_name: 'AI Rule Engine',
    action: 'BID_COMPLIANCE_EVALUATION',
    entity_type: 'BID',
    details:
      'Automated evaluation completed for BID/12345/ABC-01. Compliance Score: 85/100, Risk Level: LOW. 7 statuto
    ry criteria verified.',
    rule_applied: 'GFR_2017_RULE_149',
    ip_address: '127.0.0.1 (Local AI Core)',
    timestamp: new Date(Date.now() - 36e5).toISOString(),
  },
  {
```

```
    id: 2,
    user_name: 'AI Rule Engine',
    action: 'DEBARMENT_ALERT_TRIGGERED',
    entity_type: 'BID',
    details:
      'Critical Alert: Bidder "Bharat Systems Pvt Ltd" matches Ministry of Defence Central Debarment Order MOD/D
    EB/2025/892.',
    rule_applied: 'GFR_2017_RULE_151_DISQUALIFICATION',
    ip_address: '127.0.0.1 (Local AI Core)',
    timestamp: new Date(Date.now() - 72e5).toISOString(),
  },
  {
    id: 3,
    user_name: 'Rajesh Kumar Verma',
    action: 'OFFICER_LOGIN',
    entity_type: 'USER',
    details: 'Procurement Officer authenticated into GeM Technical Evaluation Console.',
    rule_applied: 'CVC_AUDIT_LOGON_MANDATE',
    ip_address: '10.12.89.44 (NIC Network)',
    timestamp: new Date(Date.now() - 144e5).toISOString(),
  },
];

export const seedNotifications = [
  {
    id: 1,
    title: 'New Bid Received',
    message: "Bidder 'ABC Technologies Pvt Ltd' submitted bid for GEM/2026/B/12345",
    category: 'INFO',
    is_read: false,
    link: '/bids/1',
    created_at: new Date(Date.now() - 18e5).toISOString(),
  },
  {
    id: 2,
    title: 'Mismatch Flagged by AI',
    message: "Bidder 'XYZ Solutions Ltd' flagged with OEM entity name mismatch",
    category: 'WARNING',
    is_read: false,
    link: '/bids/2',
    created_at: new Date(Date.now() - 36e5).toISOString(),
  },
  {
    id: 3,
    title: 'High Risk Bid Detected',
    message: "Bidder 'Bharat Systems Pvt Ltd' has missing mandatory GST & debarment alert",
    category: 'HIGH_RISK',
    is_read: false,
    link: '/bids/3',
    created_at: new Date(Date.now() - 72e5).toISOString(),
  },
];

export const seedUsers = [
  {
    id: 1,
    username: 'officer',
    full_name: 'Rajesh Kumar Verma',
    role: 'Procurement Officer',
    department: 'Department of Electronics & IT (GeM Evaluation Cell)',
    email: 'officer@example.gov.in',
    phone: '+91 98200 12345',
    password: 'demo123',
    is_active: true,
  },
];


//// FILE: src/api/storage.js  (40 lines)

const memoryFallback = {};

const safeStorage = {
  getItem: (e) => {
    try {
      return localStorage.getItem(e);
    } catch {
      return memoryFallback[e] || null;
    }
```

```
  },
  setItem: (e, t) => {
    try {
      localStorage.setItem(e, t);
    } catch {
      memoryFallback[e] = t;
    }
  },
  removeItem: (e) => {
    try {
      localStorage.removeItem(e);
    } catch {
      delete memoryFallback[e];
    }
  },
};

export function loadState(e, t) {
  try {
    const r = safeStorage.getItem(`gem_ai_${e}`);
    return r ? JSON.parse(r) : t;
  } catch {
    return t;
  }
}

export function saveState(e, t) {
  try {
    safeStorage.setItem(`gem_ai_${e}`, JSON.stringify(t));
  } catch {}
}


//// FILE: src/components/bids/DocumentReviewModal.jsx  (475 lines)

import { useEffect, useState } from 'react';
import { CircleCheckIcon, CpuIcon, SaveIcon, ScanIcon, UserCheckIcon, XIcon } from 'lucide-react';
import { api } from '../../api';

export const DocumentReviewModal = ({
  isOpen: e,
  onClose: t,
  document: r,
  complianceResult: n,
  bidderName: i,
  bidId: a,
  onSaveSuccess: o,
}) => {
  var O;
  const [l, c] = useState(null),
    [u, f] = useState('AI'),
    [d, p] = useState('VERIFIED'),
    [h, y] = useState(10),
    [x, g] = useState(''),
    [v, m] = useState(false),
    [b, j] = useState('');
  if (
    (useEffect(() => {
      (r != null && r.extracted_fields && r.extracted_fields.length > 0
        ? c(r.extracted_fields[0])
        : c(null),
        n && (p(n.status || 'VERIFIED'), y(n.points_awarded ?? 10), g(n.explanation || '')),
        j(''));
    }, [r, n, e]),
    !e)
  )
    return null;
  const w = (n == null ? undefined : n.max_points) || 10,
    _ = n ? Math.round(n.confidence_score * 100) : 98,
    S = async () => {
      if (!(!a || !n))
        try {
          (m(true),
            j(''),
            await api.updateComplianceResult(a, n.id, {
              points_awarded: Number(h),
              status: d,
              explanation: x,
            }),
```

```
          j('Officer score and verification saved successfully! Bid compliance score updated.'),
          o && o());
        } catch (A) {
          alert(A.message || 'Failed to update evaluation');
        } finally {
          m(false);
        }
    };
  return (
    <div className="fixed inset-0 z-50 bg-obsidian-950/80 backdrop-blur-md flex items-center justify-center p-3
    sm:p-4">
      <div className="bg-white w-full max-w-6xl h-[90vh] rounded-3xl shadow-2xl border border-slate-200 flex fle
    x-col overflow-hidden animate-in fade-in zoom-in-95 duration-200">
        <div className="px-6 py-4 bg-gradient-to-r from-obsidian-950 via-pine-900 to-pine-800 text-white flex ju
    stify-between items-center border-b border-pine-700/50 flex-shrink-0">
          <div className="flex items-center gap-3">
            <div className="w-9 h-9 rounded-xl bg-pine-800/80 border border-pine-500/40 flex items-center justif
    y-center shadow">
              <ScanIcon className="w-5 h-5 text-champagne-400" />
            </div>
            <div>
              <h2 className="text-sm sm:text-base font-bold flex items-center gap-2">
                {'Explainable Evidence Inspector & Officer Verification Gateway'}
                <span className="text-[10px] bg-champagne-400/20 text-champagne-300 px-2 py-0.5 rounded-full fon
    t-mono font-bold border border-champagne-400/30">
                  GFR 2017
                </span>
              </h2>
              <p className="text-xs text-pine-200 truncate max-w-xl">
                {'Bidder: '}
                <strong className="text-white">{i}</strong>
                {' • Document: '}
                {(r == null ? undefined : r.file_name) || 'Statutory Attachment'}
              </p>
            </div>
          </div>
          <button
            onClick={t}
            className="p-1.5 rounded-lg text-pine-300 hover:text-white hover:bg-white/10 transition-colors"
          >
            <XIcon className="w-5 h-5" />
          </button>
        </div>
        <div className="bg-slate-100/80 px-6 py-2.5 border-b border-slate-200 flex justify-between items-center
    flex-wrap gap-2 flex-shrink-0">
          <div className="flex items-center gap-2">
            <button
              onClick={() => f('AI')}
              className={`flex items-center gap-2 px-4 py-2 rounded-xl text-xs font-bold transition-all ${u ===
    'AI' ? 'bg-pine-900 text-white shadow-md' : 'bg-white text-slate-700 border border-slate-200 hover:bg-slate-
    50'}`}
            >
              <CpuIcon className="w-3.5 h-3.5 text-champagne-400" />
              <span>Option A: AI Verified ({_}% Match)</span>
            </button>
            <button
              onClick={() => f('OFFICER')}
              className={`flex items-center gap-2 px-4 py-2 rounded-xl text-xs font-bold transition-all ${u ===
    'OFFICER' ? 'bg-pine-700 text-white shadow-md' : 'bg-white text-slate-700 border border-slate-200 hover:bg-s
    late-50'}`}
            >
              <UserCheckIcon className="w-3.5 h-3.5 text-champagne-300" />
              <span>Option B: Officer Point Scoring (Human-in-the-Loop)</span>
            </button>
          </div>
          <div className="text-[11px] text-slate-500 font-mono">
            {n && (
              <span>
                {'Requirement Code: '}
                <strong className="text-slate-800 bg-slate-200/80 px-2 py-0.5 rounded font-bold">
                  {n.requirement_code}
                </strong>
              </span>
            )}
          </div>
        </div>
        <div className="flex-1 grid grid-cols-12 overflow-hidden min-h-0">
          <div className="col-span-12 lg:col-span-4 bg-slate-50/70 p-5 border-r border-slate-200 overflow-y-auto
     space-y-4 custom-scrollbar">
```

```
        <div className="p-4 bg-white rounded-2xl border border-slate-200 shadow-sm space-y-1.5">
          <span className="text-[10px] font-bold tracking-wider uppercase text-slate-400 block">
            Tender Evaluation Requirement
          </span>
          <p className="text-xs font-bold text-slate-900 leading-snug">
            {(n == null ? undefined : n.requirement_name) ||
              (r == null ? undefined : r.classified_type.replace(/_/g, ' ')) ||
              'Statutory Qualification Requirement'}
          </p>
          <p className="text-[11px] text-slate-500 leading-relaxed">
            <strong>Statutory Basis:</strong>{' '}
            {(n == null ? undefined : n.rule_applied) || 'GFR 2017 Rule 149 / 151'}
          </p>
        </div>
        {u === 'AI' && (
          <div className="space-y-4">
            <div
              className={`p-4 rounded-2xl border space-y-2.5 ${(n == null ? undefined : n.status) === 'VERIF
IED' ? 'bg-pine-50/90 border-pine-200 text-pine-950' : (n == null ? undefined : n.status) === 'MISMATCH' ? '
bg-champagne-50/90 border-champagne-200 text-champagne-950' : 'bg-rose-50/90 border-rose-200 text-rose-950'}
`}
            >
              <div className="flex justify-between items-center">
                <span className="text-[10px] font-bold uppercase tracking-wider text-slate-600">
                  AI Evaluation Result
                </span>
                <span
                  className={`text-[10px] font-bold px-2.5 py-0.5 rounded-full uppercase tracking-wider ${(n
 == null ? undefined : n.status) === 'VERIFIED' ? 'bg-pine-200 text-pine-900' : 'bg-rose-200 text-rose-900'}
`}
                >
                  {(n == null ? undefined : n.status) || 'VERIFIED'}
                </span>
              </div>
              <p className="text-xs leading-relaxed">
                {(n == null ? undefined : n.explanation) ||
                  'Document verified against central statutory registries with zero detected discrepancies.'
}
              </p>
              <div className="pt-2 border-t border-slate-200/60 flex justify-between items-center text-xs fo
nt-mono">
                <span className="text-slate-600">AI Score:</span>
                <strong className="text-slate-900 font-bold">
                  {(n == null ? undefined : n.points_awarded) ?? 10}
                  {' / '}
                  {w}
                  {' pts'}
                </strong>
              </div>
              <div className="flex justify-between items-center text-xs font-mono">
                <span className="text-slate-600">Confidence:</span>
                <strong className="text-pine-700 font-bold">{_}% High Fidelity</strong>
              </div>
            </div>
            <div>
              <span className="text-[10px] font-bold tracking-wider uppercase text-slate-500 block mb-2">
                OCR Extracted Key Fields (
                {((O = r == null ? undefined : r.extracted_fields) == null
                  ? undefined
                  : O.length) || 0}
                )
              </span>
              <div className="space-y-2">
                {!(r != null && r.extracted_fields) || r.extracted_fields.length === 0 ? (
                  <p className="text-xs text-slate-400 italic">
                    No extracted key fields available.
                  </p>
                ) : (
                  r.extracted_fields.map((A) => (
                    <button
                      key={A.id}
                      onClick={() => c(A)}
                      className={`w-full text-left p-3 rounded-2xl text-xs transition-all border ${(l == nul
l ? undefined : l.id) === A.id ? 'bg-pine-900 text-white border-pine-800 shadow-md ring-2 ring-pine-400' : '
bg-white text-slate-800 border-slate-200 hover:bg-slate-100'}`}
                    >
                      <div className="flex justify-between items-center">
                        <span className="font-bold capitalize">
                          {A.field_name.replace(/_/g, ' ')}
```

```
                        </span>
                        <span
                          className={`text-[10px] font-mono ${(l == null ? undefined : l.id) === A.id ? 'tex
t-champagne-300' : 'text-slate-500'}`}
                        >
                          {Math.round(A.confidence * 100)}% Conf
                        </span>
                      </div>
                      <p
                        className={`text-[11px] truncate mt-1 font-mono ${(l == null ? undefined : l.id) ===
 A.id ? 'text-pine-100' : 'text-slate-600'}`}
                      >
                        {A.extracted_value}
                      </p>
                    </button>
                  ))
                )}
              </div>
            </div>
          </div>
        )}
        {u === 'OFFICER' && (
          <div className="bg-white p-5 rounded-2xl border border-pine-200 shadow-sm space-y-4">
            <div className="flex items-center gap-2 pb-2 border-b border-slate-100">
              <UserCheckIcon className="w-4 h-4 text-pine-700" />
              <div>
                <h4 className="text-xs font-bold text-slate-900">Officer Technical Scoring</h4>
                <p className="text-[10px] text-slate-500">
                  {'Human-in-the-Loop point awarding & validation override.'}
                </p>
              </div>
            </div>
            <div>
              <label className="block text-xs font-bold text-slate-700 mb-2">
                Officer Verification Decision:
              </label>
              <div className="grid grid-cols-2 gap-2">
                {[
                  {
                    val: 'VERIFIED',
                    label: '  Verified & Compliant',
                    color: 'border-pine-500 text-pine-800 bg-pine-50',
                  },
                  {
                    val: 'MISMATCH',
                    label: '  Discrepancy Found',
                    color: 'border-champagne-500 text-champagne-800 bg-champagne-50',
                  },
                  {
                    val: 'MISSING',
                    label: '  Missing / Ineligible',
                    color: 'border-rose-500 text-rose-800 bg-rose-50',
                  },
                  {
                    val: 'REQUIRES_CLARIFICATION',
                    label: '? Needs Query',
                    color: 'border-blue-500 text-blue-800 bg-blue-50',
                  },
                ].map((A) => (
                  <button
                    key={A.val}
                    type="button"
                    onClick={() => p(A.val)}
                    className={`p-2.5 rounded-xl text-xs font-bold text-left border transition-all ${d === A
.val ? `${A.color} ring-2 ring-pine-500 shadow-sm` : 'border-slate-200 text-slate-600 bg-white hover:bg-slat
e-50'}`}
                  >
                    {A.label}
                  </button>
                ))}
              </div>
            </div>
            <div className="p-3.5 bg-slate-50 rounded-2xl border border-slate-200 space-y-2">
              <div className="flex justify-between items-center">
                <label className="text-xs font-bold text-slate-800">
                  Awarded Technical Points:
                </label>
                <span className="text-sm font-bold font-mono text-pine-800 bg-pine-100/70 px-2 py-0.5 rounde
d-md">
```

```
                {h}
                {' / '}
                {w}
                {' pts'}
              </span>
            </div>
            <input
              type="range"
              min="0"
              max={w}
              step="0.5"
              value={h}
              onChange={(A) => y(parseFloat(A.target.value))}
              className="w-full accent-pine-700 cursor-pointer"
            />
            <div className="flex justify-between text-[10px] text-slate-400 font-mono">
              <span>0 pts</span>
              <span>
                {w / 2}
                {' pts'}
              </span>
              <span>
                {w}
                {' pts (Full)'}
              </span>
            </div>
          </div>
          <div>
            <label className="block text-xs font-bold text-slate-700 mb-1">
              Officer Justification Remarks:
            </label>
            <textarea
              rows={3}
              value={x}
              onChange={(A) => g(A.target.value)}
              placeholder="Enter officer evaluation notes..."
              className="w-full text-xs p-3 border border-slate-300 rounded-xl bg-slate-50 focus:bg-white
focus:ring-2 focus:ring-pine-600 focus:outline-none"
            />
          </div>
          {b && (
            <div className="p-3 bg-pine-50 border border-pine-200 rounded-xl text-xs text-pine-800 flex it
ems-center gap-2 font-semibold">
              <CircleCheckIcon className="w-4 h-4 text-pine-600 flex-shrink-0" />
              <span>{b}</span>
            </div>
          )}
          <button
            type="button"
            onClick={S}
            disabled={v || !n}
            className="w-full py-3 bg-pine-800 hover:bg-pine-700 text-white font-bold text-xs rounded-xl s
hadow-md transition-all flex items-center justify-center gap-2 disabled:opacity-50 border border-champagne-4
00/40"
          >
            <SaveIcon className={`w-4 h-4 ${v ? 'animate-spin' : ''}`} />
            <span>
              {v ? 'Saving Officer Score...' : 'Save Officer Score & Update Bid Matrix'}
            </span>
          </button>
        </div>
      )}
    </div>
    <div className="col-span-12 lg:col-span-5 bg-obsidian-950 p-6 flex flex-col items-center justify-cente
r overflow-auto relative custom-scrollbar">
      <div className="w-[420px] min-h-[520px] bg-white rounded-2xl shadow-2xl p-6 relative border border-s
late-300 font-serif text-slate-900">
        <div className="text-center pb-3 border-b border-slate-300 mb-4">
          <p className="text-[9px] tracking-widest uppercase font-sans font-bold text-slate-500">
            Government of India Statutory Record
          </p>
          <h3 className="text-xs font-bold text-slate-900 mt-1 uppercase truncate font-sans">
            {(r == null ? undefined : r.classified_type.replace(/_/g, ' ')) ||
              'STATUTORY RECORD CERTIFICATE'}
          </h3>
          <p className="text-[9px] text-slate-400 font-sans">
            GeM Technical Evaluation Repository
          </p>
        </div>
```

```
          <div className="space-y-3 text-xs">
            <div>
              <span className="text-[10px] text-slate-500 block font-sans">
                Legal Entity Name:
              </span>
              <p className="font-bold text-xs text-slate-900 truncate font-sans">{i}</p>
            </div>
            <div className="grid grid-cols-2 gap-2 text-[11px] font-sans">
              <div className="p-2.5 bg-slate-50 rounded-xl border border-slate-200">
                <span className="text-[9px] text-slate-400 block font-medium">
                  Document Type:
                </span>
                <p className="font-semibold text-slate-800 truncate">
                  {(r == null ? undefined : r.classified_type) || 'SUPPORTING_DOC'}
                </p>
              </div>
              <div className="p-2.5 bg-slate-50 rounded-xl border border-slate-200">
                <span className="text-[9px] text-slate-400 block font-medium">
                  OCR Confidence:
                </span>
                <p className="font-semibold text-pine-700">98.4% (High Fidelity)</p>
              </div>
            </div>
            <div className="p-3 bg-slate-50 rounded-xl border border-slate-200 text-[10px] leading-relaxed f
ont-sans text-slate-600">
              Cross-verified with statutory registries (GSTN, ITD, UDYAM, CPPP). Bounding box
              coordinates map directly to OCR extracted values.
            </div>
            {l && l.bounding_box && (
              <div
                className="absolute bg-champagne-400/25 border-2 border-champagne-500 rounded-lg p-1 transit
ion-all duration-200 pointer-events-none flex items-start shadow-sm"
                style={{
                  left: `${Math.min(280, l.bounding_box.x)}px`,
                  top: `${Math.min(380, l.bounding_box.y)}px`,
                  width: `${Math.min(260, l.bounding_box.width)}px`,
                  height: `${Math.max(35, l.bounding_box.height)}px`,
                }}
              >
                <span className="text-[8.5px] bg-champagne-500 text-slate-950 font-bold px-1.5 py-0.5 rounde
d shadow -mt-4 -ml-1 font-sans">
                  {l.field_name}
                </span>
              </div>
            )}
          </div>
          <div
            className={`absolute bottom-5 right-5 text-center border-2 rounded-full px-3 py-1 rotate-[-12deg
] font-sans font-bold text-[9.5px] ${d === 'VERIFIED' ? 'border-pine-700 text-pine-800 bg-pine-50/80' : 'bor
der-champagne-700 text-champagne-800 bg-champagne-50/80'}`}
          >
            {u === 'OFFICER' ? 'OFFICER VERIFIED' : 'AI VERIFIED'}
          </div>
        </div>
      </div>
      <div className="col-span-12 lg:col-span-3 bg-white p-5 border-l border-slate-200 overflow-y-auto space
-y-4 custom-scrollbar">
        <div>
          <span className="text-[10px] font-bold tracking-wider uppercase text-slate-500 block mb-2.5">
            Field Coordinate Detail
          </span>
          {l ? (
            <div className="space-y-3">
              <div className="p-3.5 bg-pine-50/70 rounded-2xl border border-pine-200">
                <span className="text-[10px] text-pine-700 font-bold uppercase block">
                  Field Name
                </span>
                <p className="text-xs font-bold text-slate-900 mt-0.5 capitalize">
                  {l.field_name.replace(/_/g, ' ')}
                </p>
              </div>
              <div className="p-3.5 bg-slate-50 rounded-2xl border border-slate-200">
                <span className="text-[10px] text-slate-500 font-bold uppercase block">
                  Extracted Text
                </span>
                <p className="text-xs font-mono font-bold text-slate-900 mt-0.5 break-all">
                  {l.extracted_value}
                </p>
              </div>
```

```
                <div className="p-3.5 bg-slate-50 rounded-2xl border border-slate-200">
                  <span className="text-[10px] text-slate-500 font-bold uppercase block">
                    Confidence Score
                  </span>
                  <div className="flex items-center gap-2 mt-1.5">
                    <div className="flex-1 bg-slate-200 h-2.5 rounded-full overflow-hidden">
                      <div
                        className="bg-pine-500 h-full rounded-full"
                        style={{ width: `${l.confidence * 100}%` }}
                      />
                    </div>
                    <span className="text-xs font-bold text-slate-800 font-mono">
                      {Math.round(l.confidence * 100)}%
                    </span>
                  </div>
                </div>
                {l.bounding_box && (
                  <div className="p-3.5 bg-slate-50 rounded-2xl border border-slate-200 text-[10px] font-mono
    text-slate-600 space-y-1">
                    <span className="font-bold text-slate-700 block font-sans">
                      OCR Box Coordinates:
                    </span>
                    <p>
                      {'Page: '}
                      {l.bounding_box.page}
                    </p>
                    <p>
                      {'X: '}
                      {l.bounding_box.x}
                      {'px | Y: '}
                      {l.bounding_box.y}px
                    </p>
                    <p>
                      {'W: '}
                      {l.bounding_box.width}
                      {'px | H: '}
                      {l.bounding_box.height}px
                    </p>
                  </div>
                )}
              </div>
            ) : (
              <div className="p-5 text-center text-xs text-slate-400 bg-slate-50 rounded-2xl border border-das
    hed border-slate-200">
                Select a field on the left to inspect bounding coordinates.
              </div>
            )}
          </div>
          <div className="pt-3 border-t border-slate-100">
            <span className="text-[10px] font-bold tracking-wider uppercase text-slate-500 block mb-1.5">
              CVC Immutable Audit Note
            </span>
            <p className="text-[10.5px] text-slate-600 leading-relaxed bg-champagne-50/70 p-3 rounded-xl borde
    r border-champagne-200">
              All officer score overrides, manual adjustments, and inspections are permanently
              recorded in the CAG/CVC audit trail.
            </p>
          </div>
        </div>
      </div>
      <div className="px-6 py-3.5 bg-slate-100/90 border-t border-slate-200 flex justify-between items-center
    text-xs text-slate-600 flex-shrink-0">
        <span>
          Human-in-the-Loop Principle: Officer retains sovereign decision authority under GFR Rule
          173.
        </span>
        <button
          onClick={t}
          className="px-5 py-2 bg-pine-900 hover:bg-pine-800 text-white font-bold rounded-xl shadow-md transit
    ion-all border border-champagne-400/30"
        >
          Close Inspector
        </button>
      </div>
    </div>
  </div>
  );
};
```

```
//// FILE: src/components/bids/DocumentUploader.jsx  (140 lines)

import { useRef, useState } from 'react';
import {
  CircleAlertIcon,
  CircleCheckIcon,
  CloudUploadIcon,
  LoaderCircleIcon,
  SparklesIcon,
} from 'lucide-react';
import { api } from '../../api';

const documentTypeOptions = [
  { value: 'AUTO', label: '  AI Auto-Detect & Classify' },
  { value: 'GST_CERTIFICATE', label: 'GST Registration Certificate' },
  { value: 'PAN_CARD', label: 'PAN Card' },
  { value: 'UDYAM_CERTIFICATE', label: 'Udyam / MSME Certificate' },
  { value: 'ITR_AND_FINANCIALS', label: 'Audited ITR & CA Turnover Certificate' },
  { value: 'OEM_AUTHORIZATION', label: 'OEM Authorization Letter (MAF)' },
  { value: 'MAKE_IN_INDIA', label: 'Make in India Local Content Declaration' },
  { value: 'EPFO_CHALLAN', label: 'EPFO Registration / ECR Challan' },
  { value: 'ESIC_CERTIFICATE', label: 'ESIC Registration Certificate' },
  { value: 'DEBARMENT_AFFIDAVIT', label: 'Non-Blacklisting / Debarment Affidavit' },
  { value: 'BIS_CERTIFICATE', label: 'BIS / ISO Standard Certificate' },
];

export const DocumentUploader = ({ bidId: e, onUploadSuccess: t }) => {
  const [r, n] = useState('AUTO'),
    [i, a] = useState(false),
    [o, l] = useState(false),
    [c, u] = useState(null),
    f = useRef(null),
    d = async (p) => {
      if (!p || p.length === 0) return;
      const h = p[0];
      if (h.size > 25 * 1024 * 1024) {
        u({ type: 'error', text: 'File exceeds maximum 25MB limit.' });
        return;
      }
      try {
        (l(true),
          u(null),
          await api.uploadDocument(e, h, r),
          u({
            type: 'success',
            text: `Uploaded '${h.name}'! AI successfully performed OCR, classification, and field extraction.`,
          }),
          t(),
          f.current && (f.current.value = ''));
      } catch (y) {
        u({ type: 'error', text: y.message || 'Failed to upload and process document' });
      } finally {
        l(false);
      }
    };
  return (
    <div className="bg-white p-6 rounded-3xl border border-slate-200/90 shadow-sm space-y-4">
      <div className="flex flex-col sm:flex-row justify-between items-start sm:items-center gap-3">
        <div>
          <h4 className="text-xs font-bold uppercase tracking-wider text-slate-800 flex items-center gap-1.5">
            <SparklesIcon className="w-3.5 h-3.5 text-pine-700" />
            {'Upload & AI Document Extraction Pipeline'}
          </h4>
          <p className="text-[11px] text-slate-500">
            Supported formats: PDF, PNG, JPG (Max 25MB). Automatically executes OCR, document
            classification, and coordinate mapping.
          </p>
        </div>
        <select
          value={r}
          onChange={(p) => n(p.target.value)}
          className="text-xs border border-slate-300 rounded-xl px-3 py-2 bg-slate-50 font-medium text-slate-700
     focus:ring-2 focus:ring-pine-600 focus:outline-none"
        >
          {documentTypeOptions.map((p) => (
            <option key={p.value} value={p.value}>
              {p.label}
            </option>
```

```
          ))}
        </select>
      </div>
      <div
        onDragOver={(p) => {
          (p.preventDefault(), a(true));
        }}
        onDragLeave={() => a(false)}
        onDrop={(p) => {
          (p.preventDefault(), a(false), d(p.dataTransfer.files));
        }}
        onClick={() => {
          var p;
          return (p = f.current) == null ? undefined : p.click();
        }}
        className={`border-2 border-dashed rounded-2xl p-8 text-center cursor-pointer transition-all duration-20
    0 ${i ? 'border-pine-600 bg-pine-50/70 scale-[0.99]' : 'border-slate-300 hover:border-pine-500 hover:bg-pine
    -50/30'}`}
      >
        <input
          type="file"
          ref={f}
          onChange={(p) => d(p.target.files)}
          accept=".pdf,.png,.jpg,.jpeg"
          className="hidden"
        />
        {o ? (
          <div className="flex flex-col items-center justify-center space-y-2">
            <LoaderCircleIcon className="w-9 h-9 text-pine-600 animate-spin" />
            <p className="text-xs font-bold text-slate-800">
              Processing Document Through AI Pipeline...
            </p>
            <p className="text-[10.5px] text-slate-500 font-mono">
              Running OCR → Entity Extraction → GFR Rule Verification
            </p>
          </div>
        ) : (
          <div className="flex flex-col items-center justify-center space-y-2.5">
            <div className="w-12 h-12 rounded-2xl bg-pine-50 flex items-center justify-center text-pine-700 shad
    ow-sm border border-pine-200">
              <CloudUploadIcon className="w-6 h-6" />
            </div>
            <div>
              <p className="text-xs font-bold text-slate-800">
                Click to browse or drag and drop bid documents here
              </p>
              <p className="text-[11px] text-slate-500 mt-0.5">
                GST, PAN, ITR, Udyam, OEM Authorization, Make in India Declarations
              </p>
            </div>
          </div>
        )}
      </div>
      {c && (
        <div
          className={`p-3.5 rounded-2xl text-xs flex items-center gap-2 border ${c.type === 'success' ? 'bg-pine
    -50 text-pine-900 border-pine-200' : 'bg-rose-50 text-rose-800 border-rose-200'}`}
        >
          {c.type === 'success' ? (
            <CircleCheckIcon className="w-4 h-4 text-pine-600 flex-shrink-0" />
          ) : (
            <CircleAlertIcon className="w-4 h-4 text-rose-600 flex-shrink-0" />
          )}
          <span className="font-medium">{c.text}</span>
        </div>
      )}
    </div>
  );
};


//// FILE: src/components/bids/OfficerDecisionModal.jsx  (241 lines)

import { useState } from 'react';
import {
  CircleCheckBigIcon,
  CircleHelpIcon,
  CircleXIcon,
  FileCheck2Icon,
```

```
  LockIcon,
  SendIcon,
  SparklesIcon,
  TriangleAlertIcon,
  XIcon,
} from 'lucide-react';

export const OfficerDecisionModal = ({ isOpen: e, onClose: t, bid: r, onSubmit: n }) => {
  const [i, a] = useState('APPROVED'),
    [o, l] = useState(''),
    [c, u] = useState(''),
    [f, d] = useState(false),
    [p, h] = useState(false),
    [y, x] = useState('');
  if (!e) return null;
  const g = async (v) => {
    if ((v.preventDefault(), !o.trim())) {
      x('Procurement Officer justification remarks are mandatory under GFR & CVC guidelines.');
      return;
    }
    if (!f) {
      x('Please check the statutory confirmation box to sign off on this decision.');
      return;
    }
    try {
      (h(true),
        x(''),
        await n({
          decision: i,
          comments: o,
          clarification_details: i === 'REQUEST_CLARIFICATION' ? c : undefined,
        }),
        t());
    } catch (m) {
      x(m.message || 'Failed to submit officer decision');
    } finally {
      h(false);
    }
  };
  return (
    <div className="fixed inset-0 z-50 bg-obsidian-950/80 backdrop-blur-md flex items-center justify-center p-4"
    >
      <div className="bg-white w-full max-w-2xl rounded-3xl shadow-2xl border border-slate-200 overflow-hidden a
    nimate-in fade-in zoom-in-95 duration-150">
        <div className="px-6 py-4 bg-gradient-to-r from-obsidian-950 via-pine-900 to-pine-800 text-white flex ju
    stify-between items-center border-b border-pine-700/50">
          <div className="flex items-center gap-3">
            <div className="w-9 h-9 rounded-xl bg-pine-800/80 border border-pine-500/40 flex items-center justif
    y-center shadow">
              <FileCheck2Icon className="w-5 h-5 text-champagne-400" />
            </div>
            <div>
              <h2 className="text-base font-bold">Final Procurement Officer Decision Sign-off</h2>
              <p className="text-xs text-pine-200">
                {'Bid: '}
                <strong className="text-white font-mono">{r.bid_id}</strong>
                {' • '}
                {r.bidder.company_name}
              </p>
            </div>
          </div>
          <button
            onClick={t}
            className="p-1.5 rounded-lg text-pine-300 hover:text-white hover:bg-white/10 transition-colors"
          >
            <XIcon className="w-5 h-5" />
          </button>
        </div>
        <form onSubmit={g} className="p-6 space-y-5">
          <div className="p-4 bg-slate-50/90 rounded-2xl border border-slate-200 text-xs">
            <div className="flex justify-between items-center mb-1">
              <span className="font-bold text-slate-700 flex items-center gap-1.5">
                <SparklesIcon className="w-3.5 h-3.5 text-pine-700" />
                AI Recommendation Summary
              </span>
              <span className="font-mono font-bold bg-pine-100 text-pine-900 px-2.5 py-0.5 rounded-full border b
    order-pine-200">
                {'Score: '}
                {r.compliance_score}/100 ({r.risk_level}
```

```
            {' Risk)'}
          </span>
        </div>
        <p className="text-slate-600 leading-relaxed mt-1">{r.ai_explanation}</p>
      </div>
      {y && (
        <div className="p-3 bg-rose-50 text-rose-800 rounded-xl text-xs font-semibold border border-rose-200
 flex items-center gap-2">
          <TriangleAlertIcon className="w-4 h-4 text-rose-600 flex-shrink-0" />
          <span>{y}</span>
        </div>
      )}
      <div>
        <label className="block text-xs font-bold uppercase tracking-wider text-slate-700 mb-2">
          Select Official Evaluation Action
        </label>
        <div className="grid grid-cols-2 gap-3">
          <label
            className={`p-3.5 rounded-2xl border cursor-pointer flex items-center gap-3 transition-all ${i =
== 'APPROVED' ? 'bg-pine-50/90 border-pine-500 shadow-sm ring-1 ring-pine-500' : 'bg-white border-slate-200
hover:bg-slate-50'}`}
          >
            <input
              type="radio"
              name="decision"
              value="APPROVED"
              checked={i === 'APPROVED'}
              onChange={() => a('APPROVED')}
              className="hidden"
            />
            <CircleCheckBigIcon className="w-5 h-5 text-pine-600 flex-shrink-0" />
            <div>
              <p className="text-xs font-bold text-slate-900">Approve / Qualify</p>
              <p className="text-[10px] text-slate-500">Proceed to financial cover opening</p>
            </div>
          </label>
          <label
            className={`p-3.5 rounded-2xl border cursor-pointer flex items-center gap-3 transition-all ${i =
== 'REJECTED' ? 'bg-rose-50/90 border-rose-500 shadow-sm ring-1 ring-rose-500' : 'bg-white border-slate-200
hover:bg-slate-50'}`}
          >
            <input
              type="radio"
              name="decision"
              value="REJECTED"
              checked={i === 'REJECTED'}
              onChange={() => a('REJECTED')}
              className="hidden"
            />
            <CircleXIcon className="w-5 h-5 text-rose-600 flex-shrink-0" />
            <div>
              <p className="text-xs font-bold text-slate-900">Reject / Disqualify</p>
              <p className="text-[10px] text-slate-500">
                Non-compliant with mandatory criteria
              </p>
            </div>
          </label>
          <label
            className={`p-3.5 rounded-2xl border cursor-pointer flex items-center gap-3 transition-all ${i =
== 'REQUEST_CLARIFICATION' ? 'bg-champagne-50/90 border-champagne-500 shadow-sm ring-1 ring-champagne-500' :
 'bg-white border-slate-200 hover:bg-slate-50'}`}
          >
            <input
              type="radio"
              name="decision"
              value="REQUEST_CLARIFICATION"
              checked={i === 'REQUEST_CLARIFICATION'}
              onChange={() => a('REQUEST_CLARIFICATION')}
              className="hidden"
            />
            <CircleHelpIcon className="w-5 h-5 text-champagne-600 flex-shrink-0" />
            <div>
              <p className="text-xs font-bold text-slate-900">Request Clarification</p>
              <p className="text-[10px] text-slate-500">Issue official query to bidder</p>
            </div>
          </label>
          <label
            className={`p-3.5 rounded-2xl border cursor-pointer flex items-center gap-3 transition-all ${i =
== 'SEND_FOR_MANUAL_REVIEW' ? 'bg-slate-100 border-slate-400 shadow-sm ring-1 ring-slate-400' : 'bg-white bo
```

```
rder-slate-200 hover:bg-slate-50'}`}
          >
            <input
              type="radio"
              name="decision"
              value="SEND_FOR_MANUAL_REVIEW"
              checked={i === 'SEND_FOR_MANUAL_REVIEW'}
              onChange={() => a('SEND_FOR_MANUAL_REVIEW')}
              className="hidden"
            />
            <LockIcon className="w-5 h-5 text-slate-600 flex-shrink-0" />
            <div>
              <p className="text-xs font-bold text-slate-900">Committee Review</p>
              <p className="text-[10px] text-slate-500">Refer to Higher TEC Panel</p>
            </div>
          </label>
        </div>
      </div>
      {i === 'REQUEST_CLARIFICATION' && (
        <div>
          <label className="block text-xs font-bold text-slate-700 mb-1">
            Specific Clarification Notice for Bidder
          </label>
          <textarea
            rows={2}
            value={c}
            onChange={(v) => u(v.target.value)}
            placeholder="e.g. Please clarify discrepancy between OEM authorization entity name and bidder co
mpany name within 48 hours."
            className="w-full text-xs p-3 rounded-xl border border-slate-300 focus:ring-2 focus:ring-champag
ne-500 focus:outline-none"
          />
        </div>
      )}
      <div>
        <label className="block text-xs font-bold text-slate-700 mb-1">
          {'Procurement Officer Justification Remarks '}
          <span className="text-rose-500">*</span>
        </label>
        <textarea
          rows={3}
          required
          value={o}
          onChange={(v) => l(v.target.value)}
          placeholder="Record explicit reasons for qualification/disqualification as required under CVC and
GFR 2017 guidelines..."
          className="w-full text-xs p-3 rounded-xl border border-slate-300 focus:ring-2 focus:ring-pine-600
focus:outline-none"
        />
      </div>
      <div className="p-3.5 bg-slate-50 rounded-2xl border border-slate-200 flex items-start gap-2.5">
        <input
          type="checkbox"
          id="confirm"
          checked={f}
          onChange={(v) => d(v.target.checked)}
          className="mt-0.5 rounded text-pine-600 focus:ring-pine-500"
        />
        <label
          htmlFor="confirm"
          className="text-[11px] text-slate-600 cursor-pointer leading-relaxed select-none"
        >
          I hereby certify as the Competent Procurement Officer that this decision is made after
          due examination of the evidence and compliance rules under GFR 2017. This action will
          be cryptographically recorded in the CVC Audit Trail.
        </label>
      </div>
      <div className="flex justify-end items-center gap-3 pt-2">
        <button
          type="button"
          onClick={t}
          className="px-4 py-2.5 text-xs font-semibold text-slate-600 hover:bg-slate-100 rounded-xl transiti
on-colors"
        >
          Cancel
        </button>
        <button
          type="submit"
          disabled={p}
```

```
              className="px-6 py-2.5 text-xs font-bold text-white bg-gradient-to-r from-pine-900 via-pine-800 to
    -pine-700 hover:from-pine-800 hover:to-pine-600 rounded-xl shadow-glow-teal transition-all flex items-center
     gap-2 border border-champagne-400/40"
            >
              {p ? (
                'Submitting...'
              ) : (
                <>
                  <SendIcon className="w-4 h-4 text-champagne-300" />
                  <span>{'Sign & Submit Decision'}</span>
                </>
              )}
            </button>
          </div>
        </form>
      </div>
    </div>
  );
};


//// FILE: src/components/layout/BackendStatus.jsx  (60 lines)

import { useEffect, useState } from 'react';

/**
 * Shows a red bar at the top of the portal when the backend server is not answering.
 * Without this, every screen would just keep showing its loading spinner.
 */
const BASE =
  import.meta.env.VITE_API_BASE_URL ||
  (typeof window !== 'undefined' && window.location.protocol === 'file:'
    ? 'http://localhost:8000/api'
    : '/api');

async function backendIsUp() {
  try {
    const controller = new AbortController();
    const timer = setTimeout(() => controller.abort(), 4000);
    const response = await fetch(`${BASE}/health`, { signal: controller.signal });
    clearTimeout(timer);
    return response.ok;
  } catch {
    return false;
  }
}

export const BackendStatus = () => {
  const [down, setDown] = useState(false);

  useEffect(() => {
    if (import.meta.env.VITE_API_MODE === 'mock') return undefined;
    let cancelled = false;
    let timer;
    const check = async () => {
      const up = await backendIsUp();
      if (cancelled) return;
      setDown((wasDown) => {
        // when the server comes back, reload once so the screens fetch their data again
        if (wasDown && up) window.location.reload();
        return !up;
      });
      timer = setTimeout(check, up ? 15000 : 4000);
    };
    check();
    return () => {
      cancelled = true;
      clearTimeout(timer);
    };
  }, []);

  if (!down) return null;
  return (
    <div
      role="alert"
      className="bg-rose-700 text-white text-xs font-semibold px-4 py-2 text-center"
    >
      The backend server is not running, so no data can load. Start it with start-backend.bat (or
      start-all.bat) and keep that black window open. This page reloads by itself when the server is
```

```
      back.
    </div>
  );
};


//// FILE: src/components/layout/Header.jsx  (373 lines)

import { useEffect, useRef, useState } from 'react';
import { Link } from 'react-router-dom';
import {
  BellIcon,
  CalendarIcon,
  DatabaseIcon,
  DownloadIcon,
  ExternalLinkIcon,
  InfoIcon,
  MenuIcon,
  OctagonAlertIcon,
  RefreshCwIcon,
  SparklesIcon,
  TriangleAlertIcon,
  UploadIcon,
  XIcon,
} from 'lucide-react';
import { useAuth } from '../../context/AuthContext';
import { useSidebar } from '../../context/SidebarContext';
import { api } from '../../api';

export const Header = () => {
  var j;
  const { user: e } = useAuth(),
    { isExpanded: t, toggleSidebar: r } = useSidebar(),
    [n, i] = useState([]),
    [a, o] = useState(false),
    [l, c] = useState(false),
    [u, f] = useState(''),
    [d, p] = useState(''),
    h = useRef(null),
    y = useRef(null);
  useEffect(() => {
    (x(),
      p(
        new Date().toLocaleDateString('en-IN', {
          weekday: 'short',
          day: '2-digit',
          month: 'short',
          year: 'numeric',
        }),
      ));
    const _ = (S) => {
      h.current && !h.current.contains(S.target) && o(false);
    };
    return (
      document.addEventListener('mousedown', _),
      () => document.removeEventListener('mousedown', _)
    );
  }, []);
  const x = async () => {
      try {
        const w = await api.getNotifications();
        i(w);
      } catch (w) {
        console.error(w);
      }
    },
    g = async () => {
      try {
        const w = await api.exportData(),
          _ = new Blob([JSON.stringify(w, null, 2)], { type: 'application/json' }),
          S = URL.createObjectURL(_),
          O = document.createElement('a');
        ((O.href = S),
          (O.download = `gem_bidverify_data_backup_${new Date().toISOString().slice(0, 10)}.json`),
          document.body.appendChild(O),
          O.click(),
          document.body.removeChild(O),
          URL.revokeObjectURL(S),
          f('  Data package successfully exported as JSON file!'));
```

```
      } catch (w) {
        f(`  Export failed: ${w.message}`);
      }
    },
    v = (w) => {
      var O;
      const _ = (O = w.target.files) == null ? undefined : O[0];
      if (!_) return;
      const S = new FileReader();
      ((S.onload = async (A) => {
        var E;
        try {
          const T = JSON.parse((E = A.target) == null ? undefined : E.result);
          (await api.importData(T),
            f('  All data successfully imported! Refreshing page...'),
            setTimeout(() => window.location.reload(), 1200));
        } catch (T) {
          f(`  Import failed: ${T.message}`);
        }
      }),
        S.readAsText(_));
    },
    m = async () => {
      if (
        window.confirm(
          'Reset all tenders, bidders, bids, and audit logs to the official GFR 2017 baseline data?',
        )
      ) {
        try {
          await api.resetData();
          f('  Baseline data restored! Refreshing...');
          setTimeout(() => window.location.reload(), 1e3);
        } catch (w) {
          f(`  Reset failed: ${w.message}`);
        }
      }
    },
    b = n.filter((w) => !w.is_read).length;
  return (
    <header className="sticky top-0 z-50 shadow-xl select-none">
      <div className="bg-[#03080F] px-4 sm:px-6 lg:px-8 py-1.5 flex justify-between items-center text-[11px] bor
    der-b border-slate-800/80">
        <div className="flex items-center gap-2.5 text-slate-300">
          <span className="font-bold tracking-wider text-champagne-400 uppercase">
            {'           '}
            <span className="font-normal text-slate-400">| Government of India</span>
          </span>
          <span className="text-slate-600 hidden sm:inline">•</span>
          <span className="text-slate-300 font-medium hidden sm:inline">
            {'Ministry of Commerce & Industry'}
          </span>
        </div>
        <div className="flex items-center gap-4 text-slate-400">
          <div className="flex items-center gap-1.5 text-slate-400 font-mono text-[10.5px]">
            <CalendarIcon className="w-3 h-3 text-slate-500" />
            <span className="text-slate-300 font-medium">{d}</span>
          </div>
        </div>
      </div>
      <div className="bg-[#08121E] text-white px-4 sm:px-6 lg:px-8 h-16 flex items-center justify-between border
    -b border-slate-800/90 backdrop-blur-md">
        <div className="flex items-center gap-3">
          <button
            onClick={r}
            className="p-2 rounded-xl bg-[#0C1A2B] hover:bg-slate-800 border border-slate-700/70 text-slate-300
    hover:text-champagne-400 transition-all shadow-sm flex items-center justify-center group focus:outline-none
    focus:ring-2 focus:ring-pine-500/50"
            title={t ? 'Collapse Sidebar (Ctrl+B)' : 'Expand Sidebar (Ctrl+B)'}
            aria-label="Toggle Navigation Sidebar"
          >
            <MenuIcon
              className={`w-4 h-4 transition-transform duration-200 ${t ? 'text-champagne-400' : 'text-slate-300
     group-hover:scale-110'}`}
            />
          </button>
          <Link to="/" className="flex items-center gap-3.5 group">
            <div className="w-10 h-10 rounded-xl bg-gradient-to-br from-[#0F2620] via-[#0A1A24] to-[#040E14] bor
    der border-champagne-400/40 p-1.5 flex items-center justify-center shadow-lg group-hover:border-champagne-40
    0 transition-all duration-200">
```

```
          <svg
            className="w-6 h-6"
            viewBox="0 0 24 24"
            fill="none"
            xmlns="http://www.w3.org/2000/svg"
          >
            <path
              d="M12 2L3 6V11C3 16.55 6.84 21.74 12 23C17.16 21.74 21 16.55 21 11V6L12 2Z"
              fill="url(#brand-shield)"
              stroke="#F59E0B"
              strokeWidth="1.5"
              strokeLinejoin="round"
            />
            <path
              d="M9 12L11 14L15 10"
              stroke="#FFFFFF"
              strokeWidth="2"
              strokeLinecap="round"
              strokeLinejoin="round"
            />
            <circle cx="12" cy="7" r="1.1" fill="#14B8A6" />
            <defs>
              <linearGradient
                id="brand-shield"
                x1="12"
                y1="2"
                x2="12"
                y2="23"
                gradientUnits="userSpaceOnUse"
              >
                <stop stopColor="#0D3830" />
                <stop offset="1" stopColor="#08231E" />
              </linearGradient>
            </defs>
          </svg>
        </div>
        <div>
          <div className="flex items-center gap-2">
            <span className="font-extrabold text-lg tracking-tight text-white group-hover:text-champagne-300
 transition-colors">
              {'GeM '}
              <span className="text-champagne-400">BidVerify</span>
            </span>
            <span className="bg-pine-800/90 text-pine-300 text-[9.5px] font-black px-2 py-0.5 rounded-md upp
ercase tracking-wider font-mono shadow-sm border border-pine-600/50 flex items-center gap-1">
              <SparklesIcon className="w-2.5 h-2.5 text-champagne-300" />
              AI
            </span>
          </div>
          <p className="text-[10px] text-slate-400 font-medium tracking-wide hidden sm:block">
            {'National Public Procurement Compliance & Technical Evaluation Matrix'}
          </p>
        </div>
      </Link>
    </div>
    <div className="flex items-center gap-3">
      <button
        onClick={() => {
          (c(true), f(''));
        }}
        className="flex items-center gap-1.5 px-3 py-1.5 rounded-xl bg-[#0C1A2B] hover:bg-slate-800 border b
order-slate-700/60 text-slate-300 hover:text-champagne-400 transition-all text-xs font-semibold shadow-sm"
        title="Backup, export, or sync procurement data across devices"
      >
        <DatabaseIcon className="w-3.5 h-3.5 text-champagne-400" />
        <span className="hidden md:inline">Sync Data</span>
      </button>
      <div
        className="w-9 h-9 rounded-xl bg-gradient-to-br from-pine-700 via-teal-800 to-pine-900 border border
-slate-700/80 hover:border-champagne-400/60 flex items-center justify-center font-bold text-sm text-white sh
adow-md transition-all cursor-pointer relative group"
        title={`${(e == null ? undefined : e.full_name) || 'Rajesh Kumar Verma'} • ${(e == null ? undefined
: e.role) || 'Procurement Officer'}`}
      >
        <span>
          {((j = e == null ? undefined : e.full_name) == null ? undefined : j.charAt(0)) || 'R'}
        </span>
        <span className="absolute -bottom-0.5 -right-0.5 w-3 h-3 bg-champagne-400 rounded-full border-2 bord
er-[#08121E] flex items-center justify-center">
```

```
          <span className="w-1.5 h-1.5 bg-pine-900 rounded-full" />
        </span>
      </div>
      <div className="relative" ref={h}>
        <button
          onClick={() => o(!a)}
          className="p-2 rounded-xl bg-[#0C1A2B] hover:bg-slate-800 border border-slate-700/60 text-slate-30
0 hover:text-white transition-colors relative shadow-sm"
          title={'Vigilance Alerts & Notifications'}
        >
          <BellIcon className="w-4 h-4" />
          {b > 0 && (
            <span className="absolute -top-1 -right-1 bg-rose-500 text-white text-[9px] font-bold w-4 h-4 ro
unded-full flex items-center justify-center shadow-md animate-pulse">
              {b}
            </span>
          )}
        </button>
        {a && (
          <div className="absolute right-0 mt-2.5 w-80 sm:w-96 bg-white rounded-2xl shadow-2xl border border
-slate-200 text-slate-800 z-50 overflow-hidden animate-in fade-in zoom-in-95 duration-150">
            <div className="px-4 py-3 bg-[#08121E] text-white flex justify-between items-center border-b bor
der-slate-800">
              <div className="flex items-center gap-2">
                <BellIcon className="w-4 h-4 text-champagne-400" />
                <span className="font-bold text-xs">{'Vigilance & Verification Alerts'}</span>
              </div>
              <span className="text-[10px] text-slate-300 font-mono bg-slate-800 px-2 py-0.5 rounded">
                {n.length}
                {' Total'}
              </span>
            </div>
            <div className="max-h-72 overflow-y-auto divide-y divide-slate-100 custom-scrollbar">
              {n.length === 0 ? (
                <div className="p-6 text-center text-xs text-slate-500">
                  No active vigilance notifications
                </div>
              ) : (
                n.map((w) => (
                  <div
                    key={w.id}
                    className={`p-3.5 hover:bg-slate-50 flex items-start gap-3 transition-colors ${w.is_read
 ? '' : 'bg-teal-50/40'}`}
                  >
                    <div className="mt-0.5 flex-shrink-0">
                      {w.category === 'HIGH_RISK' ? (
                        <OctagonAlertIcon className="w-4 h-4 text-rose-600" />
                      ) : w.category === 'WARNING' ? (
                        <TriangleAlertIcon className="w-4 h-4 text-champagne-600" />
                      ) : (
                        <InfoIcon className="w-4 h-4 text-teal-600" />
                      )}
                    </div>
                    <div className="flex-1 min-w-0">
                      <p className="text-xs font-bold text-slate-900 leading-tight">
                        {w.title}
                      </p>
                      <p className="text-[11px] text-slate-600 mt-0.5 leading-relaxed">
                        {w.message}
                      </p>
                      <span className="text-[10px] text-slate-400 mt-1 block font-mono">
                        {new Date(w.created_at).toLocaleTimeString([], {
                          hour: '2-digit',
                          minute: '2-digit',
                        })}
                      </span>
                    </div>
                  </div>
                ))
              )}
            </div>
            <div className="p-2.5 bg-slate-50 border-t border-slate-100 text-center">
              <Link
                to="/bids"
                onClick={() => o(false)}
                className="text-xs text-pine-700 font-bold hover:underline flex items-center justify-center
gap-1"
              >
                <span>View All Live Bids Under Review</span>
```

```
                <ExternalLinkIcon className="w-3 h-3" />
              </Link>
            </div>
          </div>
        )}
      </div>
    </div>
  </div>
  {l && (
    <div className="fixed inset-0 z-50 flex items-center justify-center bg-black/60 backdrop-blur-sm p-4">
      <div className="bg-white rounded-3xl shadow-2xl border border-slate-200 w-full max-w-md overflow-hidde
n text-slate-800">
        <div className="px-6 py-4 bg-gradient-to-r from-[#060D17] to-[#0D3830] text-white flex justify-betwe
en items-center border-b border-slate-800">
          <div className="flex items-center gap-2.5">
            <DatabaseIcon className="w-5 h-5 text-champagne-400" />
            <h3 className="text-sm font-bold">{'Cross-Device Data Backup & Sync'}</h3>
          </div>
          <button
            onClick={() => c(false)}
            className="text-slate-400 hover:text-white p-1 rounded-lg"
          >
            <XIcon className="w-5 h-5" />
          </button>
        </div>
        <div className="p-6 space-y-4 text-xs">
          <p className="text-slate-600 leading-relaxed">
            Save or transfer your complete evaluation dataset (tenders, registered bidders,
            bids, compliance scores, and audit records) to any browser or device.
          </p>
          {u && (
            <div className="p-3 bg-slate-100 rounded-xl border border-slate-200 text-slate-800 font-semibold
 text-xs">
              {u}
            </div>
          )}
          <input type="file" ref={y} onChange={v} accept=".json" className="hidden" />
          <div className="grid grid-cols-1 sm:grid-cols-2 gap-3 pt-1">
            <button
              type="button"
              onClick={g}
              className="p-3.5 rounded-2xl bg-pine-50 hover:bg-pine-100 border border-pine-200 text-pine-900
 font-bold flex flex-col items-center gap-1.5 transition-all text-center shadow-sm"
            >
              <DownloadIcon className="w-5 h-5 text-pine-700" />
              <span>Export All Data (JSON)</span>
              <span className="text-[10px] text-pine-600 font-normal">
                Download complete backup
              </span>
            </button>
            <button
              type="button"
              onClick={() => {
                var w;
                return (w = y.current) == null ? undefined : w.click();
              }}
              className="p-3.5 rounded-2xl bg-champagne-50 hover:bg-champagne-100 border border-champagne-20
0 text-champagne-950 font-bold flex flex-col items-center gap-1.5 transition-all text-center shadow-sm"
            >
              <UploadIcon className="w-5 h-5 text-champagne-700" />
              <span>Import Data Package</span>
              <span className="text-[10px] text-champagne-700 font-normal">
                Restore from JSON file
              </span>
            </button>
          </div>
          <div className="pt-2 border-t border-slate-100 flex justify-between items-center">
            <button
              type="button"
              onClick={m}
              className="text-rose-600 hover:underline text-[11px] font-semibold flex items-center gap-1"
            >
              <RefreshCwIcon className="w-3 h-3" />
              {' Reset to Baseline GFR Data'}
            </button>
            <button
              type="button"
              onClick={() => c(false)}
              className="px-4 py-2 bg-slate-100 hover:bg-slate-200 text-slate-700 font-bold rounded-xl trans
```

```
    ition-colors"
                >
                  Close
                </button>
              </div>
            </div>
          </div>
        </div>
      )}
    </header>
  );
};


//// FILE: src/components/layout/MainLayout.jsx  (20 lines)

import { Outlet } from 'react-router-dom';
import { SidebarProvider } from '../../context/SidebarContext';
import { BackendStatus } from './BackendStatus';
import { Header } from './Header';
import { Sidebar } from './Sidebar';

export const MainLayout = () => (
  <SidebarProvider>
    <div className="min-h-screen flex flex-col bg-[#F4F7FB] text-slate-800">
      <BackendStatus />
      <Header />
      <div className="flex flex-1 overflow-hidden relative">
        <Sidebar />
        <main className="flex-1 p-5 sm:p-6 lg:p-8 max-w-7xl mx-auto w-full overflow-y-auto custom-scrollbar tran
    sition-all duration-300">
          <Outlet />
        </main>
      </div>
    </div>
  </SidebarProvider>
);


//// FILE: src/components/layout/Sidebar.jsx  (189 lines)

import { NavLink } from 'react-router-dom';
import {
  Building2Icon,
  ChevronLeftIcon,
  CpuIcon,
  FileTextIcon,
  HistoryIcon,
  LayoutDashboardIcon,
  LogOutIcon,
  MenuIcon,
  NetworkIcon,
  ShieldCheckIcon,
  SquareCheckBigIcon,
} from 'lucide-react';
import { useAuth } from '../../context/AuthContext';
import { useSidebar } from '../../context/SidebarContext';

const navItems = [
  { path: '/', label: 'Overview Dashboard', icon: LayoutDashboardIcon },
  { path: '/tenders', label: 'Tenders & Criteria', icon: FileTextIcon },
  { path: '/bidders', label: 'Bidder Registry', icon: Building2Icon },
  { path: '/bids', label: 'Bid Verification', icon: SquareCheckBigIcon, badge: 'AI Live' },
  { path: '/integrations', label: 'Portal Connectors', icon: NetworkIcon, badge: '14' },
  { path: '/audit', label: 'CVC Audit Trail', icon: HistoryIcon },
];

export const Sidebar = () => {
  var o, l;
  const { user: e, logout: t } = useAuth(),
    { isExpanded: r, toggleSidebar: n, isMobileOpen: i, toggleMobile: a } = useSidebar();
  return (
    <>
      {i && (
        <div
          onClick={a}
          className="fixed inset-0 bg-slate-950/70 backdrop-blur-sm z-40 md:hidden transition-opacity duration-3
    00"
          aria-hidden="true"
```

```
    />
  )}
  <aside
    className={`bg-[#060D17] text-slate-300 flex-shrink-0 min-h-[calc(100vh-4.75rem)] flex flex-col border-r
 border-slate-800/90 select-none shadow-2xl transition-all duration-300 ease-in-out z-40 ${r ? 'w-64' : 'w-[
72px]'} ${i ? 'fixed inset-y-0 left-0 z-50 translate-x-0' : 'max-md:-translate-x-full md:translate-x-0'}`}
  >
    <div className="px-3.5 py-3.5 flex items-center justify-between border-b border-slate-800/80 bg-[#07111E
]">
      <div className="flex items-center gap-2.5 overflow-hidden">
        <button
          onClick={n}
          className="p-2 rounded-xl bg-slate-900/90 hover:bg-slate-800 text-slate-300 hover:text-champagne-4
00 transition-all border border-slate-800 flex-shrink-0 group shadow-sm focus:outline-none focus:ring-2 focu
s:ring-pine-500/50"
          title={r ? 'Collapse Sidebar (Ctrl+B)' : 'Expand Sidebar (Ctrl+B)'}
          aria-label="Toggle Sidebar"
        >
          <MenuIcon
            className={`w-4 h-4 transition-transform duration-300 ${r ? 'text-champagne-400 rotate-90' : 'te
xt-slate-300 group-hover:scale-110'}`}
          />
        </button>
        {r && (
          <div className="min-w-0 transition-opacity duration-200">
            <span className="text-[10.5px] font-bold text-slate-300 uppercase tracking-widest font-mono trun
cate block">
              Navigation
            </span>
            <span className="text-[9px] text-champagne-400/80 font-mono block">
              GFR 2017 Matrix
            </span>
          </div>
        )}
      </div>
      {r && (
        <button
          onClick={n}
          className="p-1.5 rounded-lg hover:bg-slate-800 text-slate-400 hover:text-white transition-colors"
          title="Collapse Panel (Ctrl+B)"
        >
          <ChevronLeftIcon className="w-4 h-4" />
        </button>
      )}
    </div>
    <nav className="flex-1 px-2.5 py-3 overflow-y-auto custom-scrollbar space-y-1.5">
      {navItems.map((c) => {
        const u = c.icon;
        return (
          <NavLink
            key={c.path}
            to={c.path}
            end={c.path === '/'}
            className={({ isActive: f }) =>
              `group relative flex items-center ${r ? 'justify-between px-3 py-2.5' : 'justify-center p-2.5'
} rounded-xl text-xs font-semibold transition-all duration-200 ${f ? 'bg-gradient-to-r from-pine-900/90 via-
pine-800/80 to-teal-900/70 text-white shadow-glow-teal border border-pine-600/40' : 'text-slate-400 hover:bg
-slate-800/60 hover:text-slate-100'}`
            }
          >
            {({ isActive: f }) => (
              <>
                <div className="flex items-center gap-3 min-w-0">
                  <u
                    className={`w-4 h-4 flex-shrink-0 transition-transform duration-200 ${f ? 'text-champagn
e-400' : 'text-slate-400 group-hover:text-champagne-300 group-hover:scale-110'}`}
                  />
                  {r && <span className="truncate font-medium tracking-wide">{c.label}</span>}
                </div>
                {c.badge && r && (
                  <span
                    className={`text-[9px] font-black px-1.5 py-0.5 rounded-md uppercase font-mono shadow-sm
 flex-shrink-0 ${c.badge === 'AI Live' ? 'bg-champagne-400 text-slate-950 shadow-glow-champagne' : 'bg-slate
-800 text-teal-300 border border-teal-700/50'}`}
                  >
                    {c.badge}
                  </span>
                )}
                {!r && (
```

```
                <div className="absolute left-[76px] top-1/2 -translate-y-1/2 px-3 py-1.5 bg-[#0A1826] tex
t-white text-xs font-semibold rounded-xl shadow-2xl border border-slate-700/90 pointer-events-none opacity-0
 group-hover:opacity-100 transition-all duration-200 z-50 whitespace-nowrap flex items-center gap-2">
                  <span>{c.label}</span>
                  {c.badge && (
                    <span className="text-[9px] font-black px-1.5 py-0.2 rounded bg-champagne-400 text-sla
te-950 font-mono">
                      {c.badge}
                    </span>
                  )}
                  <div className="absolute right-full top-1/2 -translate-y-1/2 border-4 border-transparent
 border-r-[#0A1826]" />
                </div>
              )}
            </>
          )}
        </NavLink>
      );
    })}
  </nav>
  <div className="border-t border-slate-800/90 p-2.5 space-y-2 bg-[#040911]">
    {r ? (
      <div className="flex items-center justify-between px-2.5 py-1.5 text-[10px] text-slate-400 bg-[#0812
1E] rounded-xl border border-slate-800 shadow-inner">
        <div className="flex items-center gap-2">
          <CpuIcon className="w-3.5 h-3.5 text-pine-400" />
          <span className="font-mono text-slate-300 font-semibold">
            GFR Rule 149 AI Engine
          </span>
        </div>
        <span className="w-2 h-2 rounded-full bg-pine-400 animate-pulse shadow-glow-teal" />
      </div>
    ) : (
      <div
        className="flex items-center justify-center py-2 bg-[#08121E] rounded-xl border border-slate-800/8
0 cursor-pointer group relative"
        title="GFR Rule 149 AI Engine Active"
      >
        <span className="w-2.5 h-2.5 rounded-full bg-pine-400 animate-pulse shadow-glow-teal" />
        <div className="absolute left-[76px] top-1/2 -translate-y-1/2 px-2.5 py-1 bg-[#0A1826] text-[10px]
 font-mono text-pine-300 rounded-lg shadow-xl border border-slate-700 pointer-events-none opacity-0 group-ho
ver:opacity-100 transition-opacity z-50 whitespace-nowrap">
          GFR Rule 149 Active
        </div>
      </div>
    )}
    {r ? (
      <div className="bg-[#08121E] rounded-xl border border-slate-800 p-2.5 flex items-center justify-betw
een gap-2 shadow-sm">
        <div className="flex items-center gap-2 min-w-0">
          <div className="w-8 h-8 rounded-lg bg-gradient-to-br from-pine-700 to-teal-800 flex items-center
 justify-center font-bold text-xs text-white shadow-inner flex-shrink-0">
            {((o = e == null ? undefined : e.full_name) == null ? undefined : o.charAt(0)) ||
              'R'}
          </div>
          <div className="min-w-0 flex-1">
            <p className="text-[11.5px] font-bold text-white truncate flex items-center gap-1">
              {(e == null ? undefined : e.full_name) || 'Rajesh Verma'}
              <ShieldCheckIcon className="w-3 h-3 text-champagne-400 flex-shrink-0" />
            </p>
            <p className="text-[10px] text-champagne-300/90 truncate font-medium">
              {(e == null ? undefined : e.role) || 'Procurement Officer'}
            </p>
          </div>
        </div>
        <button
          onClick={t}
          className="p-1.5 rounded-lg text-slate-400 hover:text-white hover:bg-rose-600 transition-colors
flex-shrink-0"
          title="Sign Out of GeM"
        >
          <LogOutIcon className="w-3.5 h-3.5" />
        </button>
      </div>
    ) : (
      <div className="flex flex-col items-center gap-1.5 py-1">
        <div className="w-8 h-8 rounded-xl bg-gradient-to-br from-pine-700 to-teal-800 flex items-center j
ustify-center font-bold text-xs text-white shadow-inner cursor-pointer group relative">
          {((l = e == null ? undefined : e.full_name) == null ? undefined : l.charAt(0)) ||
```

```
                'R'}
              <div className="absolute left-[76px] top-1/2 -translate-y-1/2 px-2.5 py-1 bg-[#0A1826] text-xs t
   ext-white rounded-lg shadow-xl border border-slate-700 pointer-events-none opacity-0 group-hover:opacity-100
    transition-opacity z-50 whitespace-nowrap">
                {(e == null ? undefined : e.full_name) || 'Rajesh Verma'}
              </div>
            </div>
            <button
              onClick={t}
              className="p-1.5 rounded-lg text-slate-400 hover:text-white hover:bg-rose-600 transition-colors"
              title="Sign Out"
            >
              <LogOutIcon className="w-3.5 h-3.5" />
            </button>
          </div>
        )}
      </div>
    </aside>
  </>
 );
};


//// FILE: src/context/AuthContext.jsx  (84 lines)

import { createContext, useCallback, useContext, useEffect, useState } from 'react';
import { api } from '../api';

const AuthContext = createContext(undefined);

const USER_KEY = 'gem_officer_user';
const TOKEN_KEY = 'gem_officer_token';

const memoryFallback = {};
const authStorage = {
  getItem: (key) => {
    try {
      return localStorage.getItem(key);
    } catch {
      return memoryFallback[key] || null;
    }
  },
  setItem: (key, value) => {
    try {
      localStorage.setItem(key, value);
    } catch {
      memoryFallback[key] = value;
    }
  },
  removeItem: (key) => {
    try {
      localStorage.removeItem(key);
    } catch {
      delete memoryFallback[key];
    }
  },
};

export const AuthProvider = ({ children }) => {
  const [token, setToken] = useState(() => authStorage.getItem(TOKEN_KEY));
  const [user, setUser] = useState(() => {
    if (!authStorage.getItem(TOKEN_KEY)) return null;
    try {
      return JSON.parse(authStorage.getItem(USER_KEY));
    } catch {
      return null;
    }
  });

  const setSession = (nextUser, nextToken) => {
    setUser(nextUser);
    setToken(nextToken);
    authStorage.setItem(USER_KEY, JSON.stringify(nextUser));
    authStorage.setItem(TOKEN_KEY, nextToken);
  };

  const logout = useCallback(() => {
    setUser(null);
    setToken(null);
```

```
    authStorage.removeItem(USER_KEY);
    authStorage.removeItem(TOKEN_KEY);
  }, []);

  const login = async (username, password) => {
    const result = await api.login({ username, password });
    if (result && result.access_token && result.user) setSession(result.user, result.access_token);
    return result;
  };

  // The API client raises this event when the server rejects the stored token (expired or invalid).
  useEffect(() => {
    window.addEventListener('gem:unauthorized', logout);
    return () => window.removeEventListener('gem:unauthorized', logout);
  }, [logout]);

  return (
    <AuthContext.Provider
      value={{ user, token, login, setSession, logout, isAuthenticated: !!user && !!token }}
    >
      {children}
    </AuthContext.Provider>
  );
};

export const useAuth = () => {
  const context = useContext(AuthContext);
  if (!context) throw new Error('useAuth must be used within an AuthProvider');
  return context;
};


//// FILE: src/context/SidebarContext.jsx  (57 lines)

import { createContext, useContext, useEffect, useState } from 'react';

const SidebarContext = createContext(undefined);

export const SidebarProvider = ({ children: e }) => {
  const [t, r] = useState(() => {
      try {
        const l = localStorage.getItem('gem_sidebar_expanded');
        return l !== null ? l === 'true' : true;
      } catch {
        return true;
      }
    }),
    [n, i] = useState(false);
  (useEffect(() => {
    try {
      localStorage.setItem('gem_sidebar_expanded', String(t));
    } catch {}
  }, [t]),
    useEffect(() => {
      const l = (c) => {
        (c.ctrlKey || c.metaKey) &&
          c.key.toLowerCase() === 'b' &&
          (c.preventDefault(), r((u) => !u));
      };
      return (
        window.addEventListener('keydown', l),
        () => window.removeEventListener('keydown', l)
      );
    }, []));
  const a = () => {
      r((l) => !l);
    },
    o = () => {
      i((l) => !l);
    };
  return (
    <SidebarContext.Provider
      value={{
        isExpanded: t,
        setIsExpanded: r,
        toggleSidebar: a,
        isMobileOpen: n,
        setIsMobileOpen: i,
        toggleMobile: o,
```

```
      }}
    >
      {e}
    </SidebarContext.Provider>
  );
};

export const useSidebar = () => {
  const e = useContext(SidebarContext);
  if (!e) throw new Error('useSidebar must be used within a SidebarProvider');
  return e;
};


//// FILE: src/index.css  (85 lines)

@tailwind base;
@tailwind components;
@tailwind utilities;

body {
  background-color: #f6f8fb;
  color: #1e293b;
  -webkit-font-smoothing: antialiased;
  -moz-osx-font-smoothing: grayscale;
  font-family:
    'Plus Jakarta Sans',
    -apple-system,
    BlinkMacSystemFont,
    'Segoe UI',
    Roboto,
    sans-serif;
}

body *::selection {
  background-color: #0d9488;
  color: #ffffff;
}

code,
pre,
.font-mono {
  font-family: 'JetBrains Mono', monospace;
}

/* ---- project-specific component classes (copied from the original compiled stylesheet) ---- */
.bg-mesh-pattern {
  background-color: #f6f8fb;
  background-image:
    radial-gradient(at 0% 0%, rgba(20, 184, 166, 0.04) 0px, transparent 50%),
    radial-gradient(at 100% 100%, rgba(245, 158, 11, 0.04) 0px, transparent 50%);
}
.gradient-obsidian-header {
  background: linear-gradient(135deg, #060d17, #0b1524 60%, #0e1c2f);
}
.gradient-pine-hero {
  background: linear-gradient(135deg, #041411, #08231e, #0d3830 80%, #134e43);
}
.gradient-champagne-btn {
  background: linear-gradient(135deg, #f59e0b, #d97706);
}
.gradient-teal-btn {
  background: linear-gradient(135deg, #0f766e, #0d9488);
}
.gem-card-modern {
  border-radius: 1rem;
  border-width: 1px;
  border-color: #e2e8f0cc;
  --tw-bg-opacity: 1;
  background-color: rgb(255 255 255 / var(--tw-bg-opacity, 1));
  --tw-shadow: 0 1px 2px 0 rgb(0 0 0 / 0.05);
  --tw-shadow-colored: 0 1px 2px 0 var(--tw-shadow-color);
  box-shadow:
    var(--tw-ring-offset-shadow, 0 0 #0000), var(--tw-ring-shadow, 0 0 #0000), var(--tw-shadow);
  transition-property: all;
  transition-timing-function: cubic-bezier(0.4, 0, 0.2, 1);
  transition-duration: 0.2s;
}
.gem-card-modern:hover {
```

```
  --tw-shadow: 0 4px 6px -1px rgb(0 0 0 / 0.1), 0 2px 4px -2px rgb(0 0 0 / 0.1);
  --tw-shadow-colored: 0 4px 6px -1px var(--tw-shadow-color), 0 2px 4px -2px var(--tw-shadow-color);
  box-shadow:
    var(--tw-ring-offset-shadow, 0 0 #0000), var(--tw-ring-shadow, 0 0 #0000), var(--tw-shadow);
}
.gem-tricolor-accent {
  background: linear-gradient(90deg, #f93, #fff, #138808);
}
.custom-scrollbar::-webkit-scrollbar {
  width: 5px;
  height: 5px;
}
.custom-scrollbar::-webkit-scrollbar-track {
  background: transparent;
}
.custom-scrollbar::-webkit-scrollbar-thumb {
  background: #cbd5e1;
  border-radius: 9999px;
}
.custom-scrollbar::-webkit-scrollbar-thumb:hover {
  background: #94a3b8;
}


//// FILE: src/pages/AuditPage.jsx  (139 lines)

import { useEffect, useState } from 'react';
import { DownloadIcon, FingerprintIcon, LockIcon, SearchIcon } from 'lucide-react';
import { api } from '../api';

export const AuditPage = () => {
  const [e, t] = useState([]),
    [r, n] = useState(true),
    [i, a] = useState('');
  useEffect(() => {
    o();
  }, []);
  const o = async () => {
      try {
        n(true);
        const c = await api.getAuditLogs();
        t(c);
      } catch (c) {
        console.error(c);
      } finally {
        n(false);
      }
    },
    l = e.filter(
      (c) =>
        c.action.toLowerCase().includes(i.toLowerCase()) ||
        c.user_name.toLowerCase().includes(i.toLowerCase()) ||
        (c.details && c.details.toLowerCase().includes(i.toLowerCase())),
    );
  return (
    <div className="space-y-6">
      <div className="bg-white p-6 rounded-3xl border border-slate-200/90 shadow-sm flex flex-col sm:flex-row ju
    stify-between items-start sm:items-center gap-4">
        <div>
          <div className="flex items-center gap-2">
            <h1 className="text-xl font-extrabold text-slate-900 tracking-tight">
              {'Immutable CVC & CAG Vigilance Audit Trail'}
            </h1>
            <span className="bg-champagne-100 text-champagne-900 text-[10px] font-bold px-2.5 py-0.5 rounded-ful
    l font-mono flex items-center gap-1 border border-champagne-200">
              <LockIcon className="w-2.5 h-2.5 text-champagne-700" />
              Append-Only SHA-256
            </span>
          </div>
          <p className="text-xs text-slate-500 mt-0.5">
            Every AI extraction, rule evaluation, score adjustment, and officer decision is recorded
            with cryptographic timestamps.
          </p>
        </div>
        <button
          onClick={() => alert('Exporting Official CVC Audit Trail Report as Signed CSV/PDF...')}
          className="px-4 py-2.5 bg-obsidian-900 hover:bg-obsidian-850 text-white text-xs font-bold rounded-xl s
    hadow-md transition-all flex items-center gap-2 border border-slate-700"
        >
```

```
      <DownloadIcon className="w-4 h-4 text-champagne-300" />
      <span>Export Audit Report</span>
    </button>
  </div>
  <div className="bg-white p-4 rounded-2xl border border-slate-200/90 shadow-sm">
    <div className="relative">
      <SearchIcon className="w-4 h-4 text-slate-400 absolute left-3.5 top-3" />
      <input
        type="text"
        placeholder="Search audit actions, officer remarks, extracted entities, IP address..."
        value={i}
        onChange={(c) => a(c.target.value)}
        className="w-full pl-10 pr-4 py-2 text-xs border border-slate-300 rounded-xl bg-slate-50 focus:bg-wh
ite focus:ring-2 focus:ring-pine-600 focus:outline-none transition-colors"
      />
    </div>
  </div>
  <div className="bg-white rounded-3xl border border-slate-200/90 shadow-sm overflow-hidden">
    <div className="p-4 bg-slate-50/80 border-b border-slate-200 flex justify-between items-center text-xs t
ext-slate-600">
      <span className="font-bold flex items-center gap-1.5 text-slate-900">
        <FingerprintIcon className="w-4 h-4 text-pine-700" />
        {'Total Logged Vigilance Events: '}
        {l.length}
      </span>
      <span className="font-mono text-[11px] bg-pine-50 text-pine-800 px-2.5 py-0.5 rounded-full border bord
er-pine-200 font-bold">
        SHA-256 Verified
      </span>
    </div>
    <div className="overflow-x-auto">
      <table className="w-full text-left text-xs border-collapse">
        <thead>
          <tr className="bg-slate-100/70 text-slate-700 border-b border-slate-200">
            <th className="py-3.5 px-4 font-bold uppercase tracking-wider">Timestamp (IST)</th>
            <th className="py-3.5 px-4 font-bold uppercase tracking-wider">User / Agent</th>
            <th className="py-3.5 px-4 font-bold uppercase tracking-wider">Event Action</th>
            <th className="py-3.5 px-4 font-bold uppercase tracking-wider">Entity</th>
            <th className="py-3.5 px-4 font-bold uppercase tracking-wider">
              {'Audit Details & Rationale'}
            </th>
            <th className="py-3.5 px-4 font-bold uppercase tracking-wider font-mono">
              IP Address
            </th>
          </tr>
        </thead>
        <tbody className="divide-y divide-slate-100 font-mono text-[11px]">
          {r ? (
            <tr>
              <td colSpan={6} className="text-center py-10 text-slate-500 font-sans">
                Loading audit trail records...
              </td>
            </tr>
          ) : l.length === 0 ? (
            <tr>
              <td colSpan={6} className="text-center py-10 text-slate-400 font-sans">
                No matching audit entries found.
              </td>
            </tr>
          ) : (
            l.map((c) => (
              <tr key={c.id} className="hover:bg-slate-50/80 transition-colors">
                <td className="py-3.5 px-4 text-slate-500 whitespace-nowrap">
                  {new Date(c.timestamp).toLocaleString()}
                </td>
                <td className="py-3.5 px-4">
                  <span
                    className={`px-2.5 py-0.5 rounded-lg font-bold text-[10px] ${c.user_name.includes('SYSTE
M_AI') ? 'bg-pine-100 text-pine-900 border border-pine-200' : 'bg-champagne-100 text-champagne-900 border bo
rder-champagne-200'}`}
                  >
                    {c.user_name}
                  </span>
                </td>
                <td className="py-3.5 px-4 font-bold text-slate-900">{c.action}</td>
                <td className="py-3.5 px-4 text-slate-600">
                  <span className="bg-slate-100 px-2 py-0.5 rounded text-[10px] text-slate-700">
                    {c.entity_type || 'SYSTEM'}
                  </span>
```

```
                    </td>
                    <td className="py-3.5 px-4 font-sans text-xs text-slate-700 max-w-md leading-relaxed">
                      {c.details}
                    </td>
                    <td className="py-3.5 px-4 text-slate-400 text-[10.5px]">{c.ip_address}</td>
                  </tr>
                ))
              )}
            </tbody>
          </table>
        </div>
      </div>
    </div>
  );
};


//// FILE: src/pages/BidDetailPage.jsx  (506 lines)

import { useEffect, useState } from 'react';
import { Link, useParams } from 'react-router-dom';
import {
  ArrowLeftIcon,
  Building2Icon,
  CircleCheckBigIcon,
  CircleXIcon,
  CpuIcon,
  EyeIcon,
  FileCheck2Icon,
  FileTextIcon,
  FilesIcon,
  RefreshCwIcon,
  ScanIcon,
  ShieldCheckIcon,
  Trash2Icon,
  TriangleAlertIcon,
  UserCheckIcon,
} from 'lucide-react';
import { api } from '../api';
import { DocumentUploader } from '../components/bids/DocumentUploader';
import { DocumentReviewModal } from '../components/bids/DocumentReviewModal';
import { OfficerDecisionModal } from '../components/bids/OfficerDecisionModal';

export const BidDetailPage = () => {
  var j, w, _, S, O, A;
  const { id: e } = useParams(),
    [t, r] = useState(null),
    [n, i] = useState(true),
    [a, o] = useState(false),
    [l, c] = useState(null),
    [u, f] = useState(null),
    [d, p] = useState(false),
    [h, y] = useState(false);
  useEffect(() => {
    e && x(e);
  }, [e]);
  const x = async (E) => {
      try {
        i(true);
        const T = await api.getBid(E);
        r(T);
      } catch (T) {
        console.error(T);
      } finally {
        i(false);
      }
    },
    g = async () => {
      if (t)
        try {
          o(true);
          const E = await api.verifyBid(t.id);
          r(E);
        } catch (E) {
          alert(E.message || 'Verification execution failed');
        } finally {
          o(false);
        }
    },
```

```
  v = (E) => {
    var C, I;
    const T =
      ((C = t == null ? undefined : t.documents) == null
        ? undefined
        : C.find(
            (R) =>
              R.file_name === E.evidence_document ||
              R.classified_type.toLowerCase().includes(E.requirement_code.toLowerCase()),
          )) ||
      ((I = t == null ? undefined : t.documents) == null ? undefined : I[0]) ||
      null;
    (f(E), c(T), p(true));
  },
  m = async (E) => {
    t && (await api.submitOfficerDecision(t.id, E), await x(t.id));
  },
  b = async (E, T) => {
    if (t && window.confirm(`Are you sure you want to delete '${T}' from this bid repository?`))
      try {
        (await api.deleteDocument(t.id, E), await x(t.id));
      } catch (C) {
        alert(C.message || 'Failed to delete document');
      }
  };
return n || !t ? (
  <div className="flex items-center justify-center min-h-[60vh]">
    <div className="text-center space-y-3">
      <div className="w-10 h-10 border-4 border-pine-900 border-t-pine-500 rounded-full animate-spin mx-auto"
  />
      <p className="text-xs font-bold text-slate-700">Loading Bid Compliance Matrix...</p>
    </div>
  </div>
) : (
  <div className="space-y-6 pb-12">
    <div className="flex flex-col sm:flex-row sm:items-center justify-between gap-3">
      <Link
        to="/bids"
        className="inline-flex items-center gap-2 px-4 py-2 rounded-xl bg-white border border-slate-200/90 tex
  t-xs font-bold text-slate-700 hover:text-pine-800 hover:border-pine-300 shadow-sm transition-all"
      >
        <ArrowLeftIcon className="w-4 h-4 text-pine-600" />
        <span>Back to All Bids</span>
      </Link>
      <div className="flex items-center gap-3">
        <button
          onClick={g}
          disabled={a}
          className="px-4 py-2.5 bg-obsidian-900 hover:bg-obsidian-850 text-white text-xs font-bold rounded-xl
   shadow-md transition-all flex items-center gap-2 border border-slate-700"
        >
          <RefreshCwIcon
            className={`w-3.5 h-3.5 text-champagne-400 ${a ? 'animate-spin' : ''}`}
          />
          <span>{a ? 'Running AI Engine...' : 'Re-Run Compliance Check'}</span>
        </button>
        <button
          onClick={() => y(true)}
          className="px-5 py-2.5 bg-gradient-to-r from-pine-900 via-pine-800 to-pine-700 hover:from-pine-800 h
  over:to-pine-600 text-white text-xs font-bold rounded-xl shadow-glow-teal transition-all flex items-center g
  ap-2 border border-champagne-400/40"
        >
          <FileCheck2Icon className="w-4 h-4 text-champagne-300" />
          <span>Make Officer Decision</span>
        </button>
      </div>
    </div>
    <div className="bg-white rounded-3xl border border-slate-200/90 p-6 sm:p-7 shadow-sm">
      <div className="grid grid-cols-1 lg:grid-cols-12 gap-6 items-center">
        <div className="lg:col-span-7 space-y-3.5">
          <div className="flex items-center gap-2.5 flex-wrap">
            <span className="font-mono text-xs font-bold text-pine-900 bg-pine-50 px-2.5 py-0.5 rounded-lg bor
  der border-pine-200">
              {t.bid_id}
            </span>
            <span className="text-[10.5px] uppercase font-bold tracking-wider bg-slate-100 text-slate-700 px-2
  .5 py-0.5 rounded-lg border border-slate-200">
              {'Tender: '}
              {((j = t.tender) == null ? undefined : j.tender_id) || 'GEM/2026/B/12345'}
```

```
          </span>
          <span className="text-[10.5px] font-bold bg-champagne-100 text-champagne-800 px-2 py-0.5 rounded-f
ull font-mono border border-champagne-200">
            GFR 149
          </span>
        </div>
        <div>
          <h1 className="text-xl sm:text-2xl font-black text-slate-900 tracking-tight flex items-center gap-
2.5">
            <Building2Icon className="w-6 h-6 text-pine-800" />
            {t.bidder.company_name}
          </h1>
          <p className="text-xs text-slate-500 mt-1">
            {'PAN: '}
            <strong className="font-mono text-slate-800">{t.bidder.pan || 'N/A'}</strong>
            {' • GSTIN: '}
            <strong className="font-mono text-slate-800">{t.bidder.gstin || 'N/A'}</strong>
            {' • MSME: '}
            <strong className="text-pine-700 font-semibold">
              {t.bidder.is_msme ? 'Yes (Udyam Verified)' : 'No'}
            </strong>
          </p>
        </div>
        <div
          className={`p-4 rounded-2xl border ${t.risk_level === 'LOW' ? 'bg-pine-50/90 border-pine-200 text-
pine-950' : t.risk_level === 'MEDIUM' ? 'bg-champagne-50/90 border-champagne-200 text-champagne-900' : 'bg-r
ose-50/90 border-rose-200 text-rose-950'}`}
        >
          <div className="flex items-center gap-2 mb-1">
            <CpuIcon
              className={`w-4 h-4 ${t.risk_level === 'LOW' ? 'text-pine-600' : t.risk_level === 'MEDIUM' ? '
text-champagne-600' : 'text-rose-600'}`}
            />
            <span className="font-extrabold text-xs uppercase tracking-wider">
              {'AI Decision-Support Recommendation: '}
              {t.ai_recommendation}
            </span>
          </div>
          <p className="text-xs leading-relaxed opacity-90">
            {t.ai_explanation ||
              'Automated multi-source evaluation completed according to deterministic statutory rules.'}
          </p>
        </div>
      </div>
      <div className="lg:col-span-5 grid grid-cols-2 gap-3.5">
        <div className="bg-slate-50/90 p-4 rounded-2xl border border-slate-200 text-center flex flex-col jus
tify-between">
          <span className="text-[10px] font-bold uppercase tracking-wider text-slate-500 block">
            Compliance Score
          </span>
          <p className="text-3xl font-black font-mono text-slate-900 my-1">
            {t.compliance_score}
            <span className="text-base text-slate-400 font-normal">/100</span>
          </p>
          <div className="w-full bg-slate-200 h-2.5 rounded-full overflow-hidden">
            <div
              className={`h-full rounded-full ${t.compliance_score >= 80 ? 'bg-pine-500' : t.compliance_scor
e >= 60 ? 'bg-champagne-500' : 'bg-rose-500'}`}
              style={{ width: `${t.compliance_score}%` }}
            />
          </div>
          <span className="text-[10px] text-slate-400 mt-1.5 block font-medium">
            Deterministic Rule Weights
          </span>
        </div>
        <div className="bg-slate-50/90 p-4 rounded-2xl border border-slate-200 text-center flex flex-col jus
tify-between">
          <span className="text-[10px] font-bold uppercase tracking-wider text-slate-500 block">
            {'Fraud & Risk Index'}
          </span>
          <p
            className={`text-2xl font-black font-mono uppercase tracking-wider my-1 ${t.risk_level === 'LOW'
 ? 'text-pine-700' : t.risk_level === 'MEDIUM' ? 'text-champagne-700' : 'text-rose-700'}`}
          >
            {t.risk_level}
          </p>
          <span
            className={`inline-block text-[10px] font-bold px-2.5 py-0.5 rounded-full ${t.risk_level === 'LO
W' ? 'bg-pine-100 text-pine-800' : t.risk_level === 'MEDIUM' ? 'bg-champagne-100 text-champagne-800' : 'bg-r
```

```
ose-100 text-rose-800'}`}
          >
            {((_ = (w = t.risk_assessment) == null ? undefined : w.risk_factors) == null
              ? undefined
              : _.length) || 0}
            {' Risk Factors Flagged'}
          </span>
          <span className="text-[10px] text-slate-400 mt-1.5 block font-medium">
            Cross-Registry Verification
          </span>
        </div>
      </div>
    </div>
  </div>
  {t.officer_decision && (
    <div className="bg-gradient-to-r from-pine-50 via-slate-50 to-champagne-50/40 border border-pine-200 p-5
 rounded-3xl shadow-sm">
      <div className="flex flex-col sm:flex-row justify-between items-start sm:items-center gap-2">
        <div className="flex items-center gap-2.5">
          <div className="w-7 h-7 rounded-lg bg-pine-800 text-white flex items-center justify-center shadow-
sm">
            <UserCheckIcon className="w-4 h-4 text-champagne-300" />
          </div>
          <h3 className="text-xs font-bold uppercase tracking-wider text-pine-950">
            Official Signed Officer Decision Record
          </h3>
        </div>
        <span className="text-[10.5px] font-mono text-slate-500 bg-white/80 px-2.5 py-0.5 rounded-full borde
r border-pine-200">
          {'Logged: '}
          {new Date(t.officer_decision.decided_at).toLocaleString()}
        </span>
      </div>
      <div className="mt-3.5 grid grid-cols-1 sm:grid-cols-3 gap-4 text-xs">
        <div className="bg-white/90 p-3.5 rounded-2xl border border-pine-100 shadow-sm">
          <span className="text-[10px] text-slate-400 font-medium block uppercase">
            Decision Action:
          </span>
          <span className="font-extrabold text-sm text-slate-900">
            {t.officer_decision.decision}
          </span>
        </div>
        <div className="bg-white/90 p-3.5 rounded-2xl border border-pine-100 shadow-sm">
          <span className="text-[10px] text-slate-400 font-medium block uppercase">
            Procurement Officer:
          </span>
          <span className="font-bold text-slate-800">{t.officer_decision.officer_name}</span>
        </div>
        <div className="bg-white/90 p-3.5 rounded-2xl border border-pine-100 shadow-sm">
          <span className="text-[10px] text-slate-400 font-medium block uppercase">
            Officer Rationale:
          </span>
          <p className="text-slate-700 leading-snug font-medium">
            {t.officer_decision.comments}
          </p>
        </div>
      </div>
    </div>
  )}
  <div className="bg-white rounded-3xl border border-slate-200/90 shadow-sm overflow-hidden">
    <div className="p-6 border-b border-slate-200 flex justify-between items-center">
      <div>
        <h3 className="text-sm font-bold text-slate-900 flex items-center gap-2">
          <ShieldCheckIcon className="w-4 h-4 text-pine-700" />
          {'Technical Compliance & Requirement Evidence Matrix'}
        </h3>
        <p className="text-xs text-slate-500 mt-0.5">
          Deterministic rule evaluation, cross-registry status, and bounding-box evidence for
          each tender criterion.
        </p>
      </div>
    </div>
    <div className="overflow-x-auto">
      <table className="w-full text-left text-xs border-collapse">
        <thead>
          <tr className="bg-slate-50/80 text-slate-600 border-b border-slate-200">
            <th className="py-3.5 px-4 font-bold uppercase tracking-wider">Requirement Code</th>
            <th className="py-3.5 px-4 font-bold uppercase tracking-wider">
              Tender Criteria Name
```

```
            </th>
            <th className="py-3.5 px-4 font-bold uppercase tracking-wider">
              Submitted Evidence
            </th>
            <th className="py-3.5 px-4 font-bold uppercase tracking-wider">
              Evaluation Status
            </th>
            <th className="py-3.5 px-4 font-bold uppercase tracking-wider">Score</th>
            <th className="py-3.5 px-4 font-bold uppercase tracking-wider">AI Conf.</th>
            <th className="py-3.5 px-4 font-bold uppercase tracking-wider text-right">
              Inspect
            </th>
          </tr>
        </thead>
        <tbody className="divide-y divide-slate-100">
          {((S = t.compliance_results) == null ? undefined : S.length) === 0 ? (
            <tr>
              <td colSpan={7} className="text-center py-10 text-slate-400">
                No compliance evaluation executed yet. Click "Re-Run Compliance Check".
              </td>
            </tr>
          ) : (O = t.compliance_results) == null ? undefined : (
            O.map((E) => (
              <tr key={E.id} className="hover:bg-slate-50/80 transition-colors">
                <td className="py-3.5 px-4">
                  <span className="font-mono font-bold bg-pine-50 text-pine-900 px-2.5 py-0.5 rounded-lg bor
der border-pine-200">
                    {E.requirement_code}
                  </span>
                </td>
                <td className="py-3.5 px-4">
                  <p className="font-bold text-slate-900">{E.requirement_name}</p>
                  <p className="text-[11px] text-slate-500 mt-0.5 leading-snug">
                    {E.explanation}
                  </p>
                </td>
                <td className="py-3.5 px-4 font-mono text-[11px] text-slate-700">
                  {E.evidence_document ? (
                    <span className="flex items-center gap-1 text-pine-700 font-semibold">
                      <FileTextIcon className="w-3.5 h-3.5" />
                      <span className="truncate max-w-[140px]">{E.evidence_document}</span>
                    </span>
                  ) : (
                    <span className="text-slate-400 italic">No Attachment</span>
                  )}
                </td>
                <td className="py-3.5 px-4">
                  <span
                    className={`inline-flex items-center gap-1 text-[10px] font-bold px-2.5 py-0.5 rounded-f
ull uppercase tracking-wider ${E.status === 'VERIFIED' ? 'bg-pine-100 text-pine-800 border border-pine-200'
: E.status === 'MISMATCH' ? 'bg-champagne-100 text-champagne-800 border border-champagne-200' : 'bg-rose-100
 text-rose-800 border border-rose-200'}`}
                  >
                    {E.status === 'VERIFIED' && (
                      <CircleCheckBigIcon className="w-3 h-3 text-pine-600" />
                    )}
                    {E.status === 'MISMATCH' && (
                      <TriangleAlertIcon className="w-3 h-3 text-champagne-600" />
                    )}
                    {E.status === 'MISSING' && (
                      <CircleXIcon className="w-3 h-3 text-rose-600" />
                    )}
                    {E.status}
                  </span>
                </td>
                <td className="py-3.5 px-4 font-mono font-bold text-slate-900">
                  {E.points_awarded}
                  {' / '}
                  {E.max_points}
                  {' pts'}
                </td>
                <td className="py-3.5 px-4 font-mono text-[11px] text-slate-600">
                  {Math.round(E.confidence_score * 100)}%
                </td>
                <td className="py-3.5 px-4 text-right">
                  <button
                    onClick={() => v(E)}
                    className="inline-flex items-center gap-1.5 px-3 py-1.5 bg-pine-900 text-white hover:bg-
pine-800 rounded-xl font-bold text-[11px] shadow-sm transition-all"
```

```
                  >
                    <ScanIcon className="w-3.5 h-3.5 text-champagne-300" />
                    <span>View Evidence</span>
                  </button>
                </td>
              </tr>
            ))
          )}
        </tbody>
      </table>
    </div>
  </div>
  <div className="bg-white rounded-3xl border border-slate-200/90 shadow-sm overflow-hidden">
    <div className="p-6 border-b border-slate-200 flex justify-between items-center flex-wrap gap-2">
      <div>
        <h3 className="text-sm font-bold text-slate-900 flex items-center gap-2">
          <FilesIcon className="w-4 h-4 text-pine-800" />
          {'Attached Bid Documents Repository & OCR Extractions'}
          <span className="bg-pine-100 text-pine-800 text-[10px] font-bold px-2 py-0.5 rounded-full font-mon
o border border-pine-200">
            {((A = t.documents) == null ? undefined : A.length) || 0}
            {' Files Attached'}
          </span>
        </h3>
        <p className="text-xs text-slate-500 mt-0.5">
          Live repository of uploaded certificates, OCR extraction status, and layout coordinate
          mappings.
        </p>
      </div>
    </div>
    <div className="overflow-x-auto">
      <table className="w-full text-left text-xs border-collapse">
        <thead>
          <tr className="bg-slate-50/80 text-slate-600 border-b border-slate-200">
            <th className="py-3 px-4 font-bold uppercase tracking-wider">Document Name</th>
            <th className="py-3 px-4 font-bold uppercase tracking-wider">AI Classification</th>
            <th className="py-3 px-4 font-bold uppercase tracking-wider">{'Size & Format'}</th>
            <th className="py-3 px-4 font-bold uppercase tracking-wider">OCR Pipeline</th>
            <th className="py-3 px-4 font-bold uppercase tracking-wider">Parsed Entities</th>
            <th className="py-3 px-4 font-bold uppercase tracking-wider text-right">Actions</th>
          </tr>
        </thead>
        <tbody className="divide-y divide-slate-100">
          {!t.documents || t.documents.length === 0 ? (
            <tr>
              <td colSpan={6} className="text-center py-10 text-slate-400">
                No documents attached yet for this bid. Upload files below to initiate AI
                parsing.
              </td>
            </tr>
          ) : (
            t.documents.map((E) => (
              <tr key={E.id} className="hover:bg-slate-50/80 transition-colors">
                <td className="py-3.5 px-4">
                  <div className="flex items-center gap-2.5">
                    <div className="w-8 h-8 rounded-xl bg-pine-50 text-pine-800 font-bold flex items-center
justify-center border border-pine-200 flex-shrink-0">
                      <FileTextIcon className="w-4 h-4 text-pine-700" />
                    </div>
                    <div>
                      <p className="font-bold text-slate-900 truncate max-w-xs">
                        {E.file_name}
                      </p>
                      <span className="text-[10px] font-mono text-slate-400">
                        {'Added: '}
                        {new Date(E.created_at || Date.now()).toLocaleDateString('en-IN')}
                      </span>
                    </div>
                  </div>
                </td>
                <td className="py-3.5 px-4">
                  <div className="flex items-center gap-1.5">
                    <span className="font-semibold text-slate-800 bg-slate-100 px-2 py-0.5 rounded-lg text-[
11px]">
                      {E.classified_type.replace(/_/g, ' ')}
                    </span>
                    <span className="text-[10px] text-pine-800 font-mono font-bold bg-pine-50 px-1.5 py-0.5
rounded border border-pine-200">
                      {Math.round((E.classification_confidence || 0.95) * 100)}% Match
```

```
                    </span>
                  </div>
                </td>
                <td className="py-3.5 px-4 font-mono text-slate-600">
                  <span className="uppercase text-[10px] bg-slate-100 font-bold px-1.5 py-0.5 rounded mr-1">
                    {E.file_type || 'PDF'}
                  </span>
                  {E.file_size_kb}
                  {' KB'}
                </td>
                <td className="py-3.5 px-4">
                  <span className="inline-flex items-center gap-1 text-[10px] font-bold px-2.5 py-0.5 rounde
d-full uppercase tracking-wider bg-pine-100 text-pine-800 border border-pine-200">
                    <CircleCheckBigIcon className="w-3 h-3 text-pine-600" />
                    {E.ocr_status || 'PROCESSED'}
                  </span>
                </td>
                <td className="py-3.5 px-4">
                  <div className="flex flex-wrap gap-1 max-w-xs">
                    {E.extracted_fields && E.extracted_fields.length > 0 ? (
                      E.extracted_fields.slice(0, 3).map((T) => (
                        <span
                          key={T.id}
                          className="text-[10px] bg-pine-50 text-pine-900 px-1.5 py-0.5 rounded border borde
r-pine-200 font-mono"
                        >
                          <strong>{T.field_name}:</strong> {T.extracted_value}
                        </span>
                      ))
                    ) : (
                      <span className="text-[10px] text-slate-400 italic">
                        {'Layout & Text Extracted'}
                      </span>
                    )}
                    {E.extracted_fields && E.extracted_fields.length > 3 && (
                      <span className="text-[10px] text-slate-500 font-mono font-bold">
                        +{E.extracted_fields.length - 3}
                        {' more'}
                      </span>
                    )}
                  </div>
                </td>
                <td className="py-3.5 px-4 text-right">
                  <div className="flex items-center justify-end gap-1.5">
                    <button
                      onClick={() => {
                        (c(E), f(null), p(true));
                      }}
                      className="inline-flex items-center gap-1 px-2.5 py-1.5 bg-pine-50 hover:bg-pine-900 t
ext-pine-800 hover:text-white font-bold rounded-xl transition-all border border-pine-200 text-[11px]"
                    >
                      <EyeIcon className="w-3.5 h-3.5" />
                      <span>Inspect</span>
                    </button>
                    <button
                      onClick={() => b(E.id, E.file_name)}
                      className="p-1.5 rounded-xl text-slate-400 hover:text-rose-600 hover:bg-rose-50 border
 border-transparent hover:border-rose-200 transition-colors"
                      title="Delete Document"
                    >
                      <Trash2Icon className="w-3.5 h-3.5" />
                    </button>
                  </div>
                </td>
              </tr>
            ))
          )}
        </tbody>
      </table>
    </div>
  </div>
  <DocumentUploader bidId={t.id} onUploadSuccess={() => x(t.id)} />
  <DocumentReviewModal
    isOpen={d}
    onClose={() => p(false)}
    document={l}
    complianceResult={u}
    bidderName={t.bidder.company_name}
    bidId={t.id}
```

```
        onSaveSuccess={() => x(t.id)}
      />
      <OfficerDecisionModal isOpen={h} onClose={() => y(false)} bid={t} onSubmit={m} />
    </div>
  );
};


//// FILE: src/pages/BiddersPage.jsx  (351 lines)

import { useEffect, useState } from 'react';
import {
  Building2Icon,
  CircleCheckBigIcon,
  MapPinIcon,
  PlusIcon,
  SearchIcon,
  XIcon,
} from 'lucide-react';
import { api } from '../api';

export const BiddersPage = () => {
  const [e, t] = useState([]),
    [r, n] = useState(true),
    [i, a] = useState(false),
    [o, l] = useState(''),
    [c, u] = useState(''),
    [f, d] = useState(''),
    [p, h] = useState(''),
    [y, x] = useState(''),
    [g, v] = useState(''),
    [m, b] = useState(''),
    [j, w] = useState(''),
    [_, S] = useState(false),
    [O, A] = useState(''),
    [E, T] = useState(false),
    [C, I] = useState(''),
    [R, P] = useState(false),
    [$, D] = useState('');
  useEffect(() => {
    M();
  }, []);
  const M = async () => {
      try {
        n(true);
        const F = await api.getBidders();
        t(F);
      } catch (F) {
        console.error(F);
      } finally {
        n(false);
      }
    },
    L = async (F) => {
      if ((F.preventDefault(), !c || !p)) {
        D('Company Name and PAN are mandatory for bidder registration.');
        return;
      }
      try {
        (P(true),
          D(''),
          await api.createBidder({
            company_name: c,
            cin: f,
            pan: p,
            gstin: y,
            registered_address: g,
            contact_email: m,
            contact_phone: j,
            is_msme: _,
            msme_reg_no: _ ? O : undefined,
            is_startup: E,
            startup_dipp_no: E ? C : undefined,
          }),
          a(false),
          M(),
          u(''),
          d(''),
          h(''),
```

```
        x(''),
        v(''),
        b(''),
        w(''));
    } catch (V) {
      D(V.message || 'Failed to register bidder');
    } finally {
      P(false);
    }
  },
  U = e.filter(
    (F) =>
      F.company_name.toLowerCase().includes(o.toLowerCase()) ||
      (F.pan && F.pan.toLowerCase().includes(o.toLowerCase())) ||
      (F.gstin && F.gstin.toLowerCase().includes(o.toLowerCase())),
  );
return (
  <div className="space-y-6">
    <div className="flex flex-col sm:flex-row sm:items-center justify-between gap-4 bg-white p-6 rounded-2xl b
  order border-slate-200/90 shadow-card-soft">
      <div>
        <div className="flex items-center gap-2">
          <h1 className="text-xl font-extrabold text-slate-900 tracking-tight">
            {'Bidder & Vendor Master Registry'}
          </h1>
          <span className="bg-pine-100 text-pine-800 text-[10px] font-bold px-2.5 py-0.5 rounded-full font-mon
  o">
            Verified Vendors
          </span>
        </div>
        <p className="text-xs text-slate-500 mt-0.5">
          Corporate entity profiles, statutory PAN/GSTIN credentials, and MSME/Startup purchase
          preference classifications.
        </p>
      </div>
      <div className="flex items-center gap-3">
        <button
          onClick={() => a(true)}
          className="px-4 py-2.5 bg-gradient-to-r from-pine-800 to-teal-800 hover:from-pine-900 hover:to-teal-
  900 text-white text-xs font-bold rounded-xl shadow-glow-teal transition-all flex items-center gap-2 group"
        >
          <PlusIcon className="w-4 h-4 text-champagne-300" />
          <span>Register New Bidder</span>
        </button>
      </div>
    </div>
    <div className="bg-white p-4 rounded-2xl border border-slate-200/90 shadow-card-soft">
      <div className="relative">
        <SearchIcon className="w-4 h-4 text-slate-400 absolute left-3.5 top-3" />
        <input
          type="text"
          placeholder="Search by Company Name, PAN (e.g. ABCDE1234F), GSTIN, CIN..."
          value={o}
          onChange={(F) => l(F.target.value)}
          className="w-full pl-10 pr-4 py-2 text-xs border border-slate-300 rounded-xl bg-slate-50 focus:bg-wh
  ite focus:ring-2 focus:ring-teal-600 focus:outline-none transition-colors"
        />
      </div>
    </div>
    <div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3 gap-5">
      {r ? (
        <div className="col-span-3 text-center py-12 text-xs text-slate-500">
          Loading bidder master records...
        </div>
      ) : U.length === 0 ? (
        <div className="col-span-3 text-center py-12 text-xs text-slate-500 bg-white rounded-2xl border border
  -dashed border-slate-300 p-8">
          No bidders found matching your search. Click "Register New Bidder" to add an entity.
        </div>
      ) : (
        U.map((F) => (
          <div
            key={F.id}
            className="bg-white rounded-2xl border border-slate-200/90 p-5 shadow-card-soft hover:shadow-card-
  hover transition-all flex flex-col justify-between group"
          >
            <div>
              <div className="flex items-start justify-between mb-3">
                <div className="w-11 h-11 rounded-2xl bg-gradient-to-br from-pine-50 to-teal-50 text-pine-800
```

```
flex items-center justify-center font-bold text-sm border border-pine-200 shadow-sm group-hover:border-pine-
400 transition-colors">
                {F.company_name.charAt(0)}
              </div>
              <div className="flex flex-col items-end gap-1">
                {F.is_msme && (
                  <span className="text-[10px] bg-pine-100 text-pine-800 px-2.5 py-0.5 rounded-full font-bol
d">
                    MSME / Udyam
                  </span>
                )}
                {F.is_startup && (
                  <span className="text-[10px] bg-champagne-100 text-champagne-800 px-2.5 py-0.5 rounded-ful
l font-bold">
                    Startup India
                  </span>
                )}
              </div>
            </div>
            <h3 className="text-sm font-bold text-slate-900 leading-snug group-hover:text-pine-800 transitio
n-colors">
              {F.company_name}
            </h3>
            {F.cin && (
              <p className="text-[10px] font-mono text-slate-400 mt-0.5">
                {'CIN: '}
                {F.cin}
              </p>
            )}
            <div className="mt-4 pt-3 border-t border-slate-100 space-y-2 text-xs">
              <div className="flex justify-between items-center bg-slate-50 px-2.5 py-1.5 rounded-lg">
                <span className="text-[10.5px] text-slate-500 font-medium">PAN</span>
                <strong className="font-mono text-slate-900 font-bold">{F.pan || 'N/A'}</strong>
              </div>
              <div className="flex justify-between items-center bg-slate-50 px-2.5 py-1.5 rounded-lg">
                <span className="text-[10.5px] text-slate-500 font-medium">GSTIN</span>
                <strong className="font-mono text-slate-900 font-bold">
                  {F.gstin || 'N/A'}
                </strong>
              </div>
              {F.msme_reg_no && (
                <div className="flex justify-between items-center bg-pine-50/70 px-2.5 py-1.5 rounded-lg bor
der border-pine-100">
                  <span className="text-[10.5px] text-pine-800 font-medium">Udyam No</span>
                  <strong className="font-mono text-pine-900 font-bold">{F.msme_reg_no}</strong>
                </div>
              )}
            </div>
            {F.registered_address && (
              <div className="mt-3 pt-2 text-[11px] text-slate-500 flex items-start gap-1.5">
                <MapPinIcon className="w-3.5 h-3.5 text-slate-400 flex-shrink-0 mt-0.5" />
                <span className="line-clamp-2">{F.registered_address}</span>
              </div>
            )}
          </div>
          <div className="mt-4 pt-3.5 border-t border-slate-100 flex items-center justify-between text-xs te
xt-slate-500">
            <span className="text-[10.5px] truncate max-w-[170px] text-slate-400 font-mono">
              {F.contact_email || 'No email registered'}
            </span>
            <span className="text-pine-800 font-bold flex items-center gap-1 text-[11px] bg-pine-50 px-2 py-
0.5 rounded-full border border-pine-200">
              <CircleCheckBigIcon className="w-3 h-3 text-pine-600" />
              Active
            </span>
          </div>
        </div>
      ))
    )}
  </div>
  {i && (
    <div className="fixed inset-0 z-50 bg-slate-950/75 backdrop-blur-sm flex items-center justify-center p-4
">
      <div className="bg-white w-full max-w-2xl rounded-3xl shadow-2xl border border-slate-300 overflow-hidd
en animate-in fade-in zoom-in-95 duration-150">
        <div className="px-6 py-4 bg-gradient-to-r from-[#060D17] to-[#0D3830] text-white flex justify-betwe
en items-center border-b border-slate-800">
          <div className="flex items-center gap-3">
            <div className="w-9 h-9 rounded-xl bg-pine-800/80 border border-pine-600/50 flex items-center ju
```

```
stify-center">
              <Building2Icon className="w-5 h-5 text-champagne-400" />
            </div>
            <div>
              <h2 className="text-base font-bold">Register Bidder Entity in GeM Master</h2>
              <p className="text-xs text-slate-300">
                Record statutory identification and company parameters
              </p>
            </div>
          </div>
          <button
            onClick={() => a(false)}
            className="p-1.5 rounded-lg text-slate-400 hover:text-white hover:bg-white/10 transition-colors"
          >
            <XIcon className="w-5 h-5" />
          </button>
        </div>
        <form onSubmit={L} className="p-6 space-y-4 text-xs">
          {$ && (
            <div className="p-3 bg-rose-50 text-rose-800 rounded-xl text-xs font-semibold border border-rose
-200">
              {$}
            </div>
          )}
          <div>
            <label className="block font-bold text-slate-700 mb-1">
              Company / Legal Entity Name *
            </label>
            <input
              type="text"
              required
              placeholder="e.g. ABC Technologies Pvt Ltd"
              value={c}
              onChange={(F) => u(F.target.value)}
              className="w-full p-2.5 border border-slate-300 rounded-xl bg-slate-50 text-xs focus:ring-2 fo
cus:ring-teal-600 focus:outline-none"
            />
          </div>
          <div className="grid grid-cols-2 gap-3">
            <div>
              <label className="block font-bold text-slate-700 mb-1">PAN Card Number *</label>
              <input
                type="text"
                required
                placeholder="e.g. ABCDE1234F"
                value={p}
                onChange={(F) => h(F.target.value.toUpperCase())}
                className="w-full p-2.5 border border-slate-300 rounded-xl bg-slate-50 font-mono text-xs foc
us:ring-2 focus:ring-teal-600 focus:outline-none"
              />
            </div>
            <div>
              <label className="block font-bold text-slate-700 mb-1">GSTIN Number</label>
              <input
                type="text"
                placeholder="e.g. 27ABCDE1234F1Z5"
                value={y}
                onChange={(F) => x(F.target.value.toUpperCase())}
                className="w-full p-2.5 border border-slate-300 rounded-xl bg-slate-50 font-mono text-xs foc
us:ring-2 focus:ring-teal-600 focus:outline-none"
              />
            </div>
            <div>
              <label className="block font-bold text-slate-700 mb-1">
                Corporate Identification (CIN)
              </label>
              <input
                type="text"
                placeholder="e.g. U72900MH2018PTC308912"
                value={f}
                onChange={(F) => d(F.target.value)}
                className="w-full p-2.5 border border-slate-300 rounded-xl bg-slate-50 font-mono text-xs foc
us:ring-2 focus:ring-teal-600 focus:outline-none"
              />
            </div>
            <div>
              <label className="block font-bold text-slate-700 mb-1">Contact Email</label>
              <input
                type="email"
```

```
                  placeholder="bids@company.in"
                  value={m}
                  onChange={(F) => b(F.target.value)}
                  className="w-full p-2.5 border border-slate-300 rounded-xl bg-slate-50 text-xs focus:ring-2
    focus:ring-teal-600 focus:outline-none"
                />
              </div>
            </div>
            <div>
              <label className="block font-bold text-slate-700 mb-1">
                Registered Corporate Address
              </label>
              <textarea
                rows={2}
                placeholder="Official registered address as per MCA/GSTN records..."
                value={g}
                onChange={(F) => v(F.target.value)}
                className="w-full p-2.5 border border-slate-300 rounded-xl bg-slate-50 text-xs focus:ring-2 fo
    cus:ring-teal-600 focus:outline-none"
              />
            </div>
            <div className="p-3.5 bg-slate-50 rounded-2xl border border-slate-200 space-y-2.5">
              <div className="flex items-center gap-2">
                <input
                  type="checkbox"
                  id="msmeCheck"
                  checked={_}
                  onChange={(F) => S(F.target.checked)}
                  className="rounded text-teal-600 focus:ring-teal-500"
                />
                <label
                  htmlFor="msmeCheck"
                  className="font-bold text-slate-800 cursor-pointer select-none"
                >
                  Enterprise is MSME / Udyam Certified (Purchase Preference Enabled)
                </label>
              </div>
              {_ && (
                <input
                  type="text"
                  placeholder="Udyam Registration Number (e.g. UDYAM-MH-01-0012345)"
                  value={O}
                  onChange={(F) => A(F.target.value)}
                  className="w-full p-2.5 border border-slate-300 rounded-xl bg-white font-mono text-xs focus:
    ring-2 focus:ring-teal-600 focus:outline-none"
                />
              )}
            </div>
            <div className="flex justify-end items-center gap-3 pt-3 border-t border-slate-200">
              <button
                type="button"
                onClick={() => a(false)}
                className="px-4 py-2.5 text-xs font-semibold text-slate-600 hover:bg-slate-100 rounded-xl tran
    sition-colors"
              >
                Cancel
              </button>
              <button
                type="submit"
                disabled={R}
                className="px-6 py-2.5 text-xs font-bold text-white bg-gradient-to-r from-pine-800 to-teal-800
     hover:from-pine-900 hover:to-teal-900 rounded-xl shadow-glow-teal transition-all"
              >
                {R ? 'Registering...' : 'Register Bidder'}
              </button>
            </div>
          </form>
        </div>
      </div>
    )}
  </div>
 );
};


//// FILE: src/pages/BidsPage.jsx  (495 lines)

import { useEffect, useState } from 'react';
import { Link } from 'react-router-dom';
```

```
import {
  ArrowUpRightIcon,
  BuildingIcon,
  CalendarIcon,
  CircleCheckIcon,
  CircleHelpIcon,
  CircleXIcon,
  ClipboardListIcon,
  CpuIcon,
  FileTextIcon,
  IndianRupeeIcon,
  PlusIcon,
  SearchIcon,
  ShieldCheckIcon,
  SparklesIcon,
  TriangleAlertIcon,
} from 'lucide-react';
import { api } from '../api';

export const BidsPage = () => {
  const [e, t] = useState('all'),
    [r, n] = useState([]),
    [i, a] = useState([]),
    [o, l] = useState(true),
    [c, u] = useState(''),
    [f, d] = useState('ALL'),
    [p, h] = useState(''),
    [y, x] = useState(''),
    [g, v] = useState([]),
    [m, b] = useState(''),
    [j, w] = useState(false),
    [_, S] = useState(''),
    [O, A] = useState('');
  useEffect(() => {
    E();
  }, []);
  const E = async () => {
      l(true);
      try {
        const [P, $, D] = await Promise.all([api.getTenders(), api.getBids(), api.getBidders()]);
        (n(P), a($), v(D), P.length > 0 && h(String(P[0].id)));
      } catch (P) {
        console.error(P);
      } finally {
        l(false);
      }
    },
    T = async (P) => {
      if ((P.preventDefault(), !p || !m)) {
        A('Please select both a Tender and a Bidder.');
        return;
      }
      (w(true), A(''), S(''));
      try {
        (await api.createBid({ tender_id: Number(p), bidder_id: Number(m) }),
          S('Bid registered successfully! It will appear in the All Bids list.'),
          await E(),
          setTimeout(() => t('all'), 1800));
      } catch ($) {
        A($.message || 'Failed to submit bid.');
      } finally {
        w(false);
      }
    },
    C = i.filter((P) => {
      const $ =
          (P.bidder_name || '').toLowerCase().includes(c.toLowerCase()) ||
          (P.tender_id || '').toLowerCase().includes(c.toLowerCase()) ||
          (P.bid_id || '').toLowerCase().includes(c.toLowerCase()),
        D = f === 'ALL' || (P.risk_level || '') === f;
      return $ && D;
    }),
    I = g.filter((P) => P.company_name.toLowerCase().includes(y.toLowerCase())),
    R = [
      { id: 'all', label: 'All Bids Under Verification', icon: ClipboardListIcon },
      { id: 'submit', label: 'Submit / Register New Bid', icon: PlusIcon },
    ];
  return (
    <div className="space-y-6">
```

```
  <div className="bg-white rounded-2xl border border-slate-200/90 shadow-card-soft p-6">
    <div className="flex flex-col sm:flex-row sm:items-center justify-between gap-4">
      <div>
        <div className="flex items-center gap-2">
          <h1 className="text-xl font-extrabold text-slate-900 tracking-tight">
            {'Bid Compliance & Verification Matrix'}
          </h1>
          <span className="bg-pine-100 text-pine-800 text-[10px] font-bold px-2.5 py-0.5 rounded-full font-m
ono flex items-center gap-1">
            <SparklesIcon className="w-2.5 h-2.5 text-pine-600" />
            AI Scrutiny
          </span>
        </div>
        <p className="text-xs text-slate-500 mt-0.5">
          Review multi-source statutory evaluations, OCR document extractions, and sign off
          official determinations.
        </p>
      </div>
      <button
        onClick={() => t('submit')}
        className="px-4 py-2.5 bg-gradient-to-r from-pine-800 to-teal-800 hover:from-pine-900 hover:to-teal-
900 text-white text-xs font-bold rounded-xl shadow-glow-teal transition-all flex items-center gap-2 self-sta
rt sm:self-auto"
      >
        <PlusIcon className="w-4 h-4 text-champagne-300" />
        <span>Register New Bid</span>
      </button>
    </div>
    <div className="flex gap-2 mt-5 border-b border-slate-200">
      {R.map((P) => {
        const $ = P.icon;
        return (
          <button
            key={P.id}
            onClick={() => t(P.id)}
            className={`flex items-center gap-2 px-4 py-2.5 text-xs font-bold rounded-t-xl border-b-2 transi
tion-all ${e === P.id ? 'border-pine-700 text-pine-900 bg-pine-50/70 shadow-sm' : 'border-transparent text-s
late-500 hover:text-slate-700 hover:bg-slate-50'}`}
          >
            <$ className="w-3.5 h-3.5" />
            {P.label}
          </button>
        );
      })}
    </div>
  </div>
  {e === 'all' && (
    <div className="bg-white rounded-2xl border border-slate-200/90 shadow-card-soft overflow-hidden">
      <div className="p-5 border-b border-slate-100 flex flex-col sm:flex-row justify-between items-start sm
:items-center gap-4">
        <div>
          <h3 className="text-sm font-bold text-slate-900">
            Submitted Bids Under Technical Scrutiny
          </h3>
          <p className="text-xs text-slate-500">
            Deterministic AI evaluation results, cross-checks, and officer sign-off.
          </p>
        </div>
        <div className="flex flex-wrap items-center gap-3 w-full sm:w-auto">
          <div className="relative flex-1 sm:flex-none">
            <SearchIcon className="absolute left-3 top-2.5 w-3.5 h-3.5 text-slate-400" />
            <input
              type="text"
              placeholder="Search bidder, tender ID…"
              value={c}
              onChange={(P) => u(P.target.value)}
              className="text-xs border border-slate-300 rounded-xl pl-9 pr-3 py-2 bg-slate-50 focus:bg-whit
e focus:ring-2 focus:ring-teal-600 focus:outline-none w-full sm:w-56"
            />
          </div>
          <select
            value={f}
            onChange={(P) => d(P.target.value)}
            className="text-xs border border-slate-300 rounded-xl px-3 py-2 bg-slate-50 text-slate-700 font-
medium focus:ring-2 focus:ring-teal-600 focus:outline-none"
          >
            <option value="ALL">All Risk Levels</option>
            <option value="LOW">Low Risk</option>
            <option value="MEDIUM">Medium Risk</option>
```

```
            <option value="HIGH">High Risk</option>
            <option value="CRITICAL">Critical Risk</option>
          </select>
        </div>
      </div>
      {o ? (
        <div className="p-12 text-center space-y-2">
          <div className="w-8 h-8 border-4 border-pine-800 border-t-champagne-400 rounded-full animate-spin
mx-auto" />
          <p className="text-xs font-bold text-slate-500">Loading submitted bids...</p>
        </div>
      ) : (
        <div className="overflow-x-auto">
          <table className="w-full text-xs text-left border-collapse">
            <thead>
              <tr className="bg-slate-50/90 text-slate-600 border-b border-slate-200">
                <th className="py-3 px-4 font-bold uppercase tracking-wider">
                  {'Bidder & Entity'}
                </th>
                <th className="py-3 px-4 font-bold uppercase tracking-wider">Tender ID</th>
                <th className="py-3 px-4 font-bold uppercase tracking-wider">Docs</th>
                <th className="py-3 px-4 font-bold uppercase tracking-wider">
                  Compliance Score
                </th>
                <th className="py-3 px-4 font-bold uppercase tracking-wider">Risk Level</th>
                <th className="py-3 px-4 font-bold uppercase tracking-wider">
                  Evaluation Status
                </th>
                <th className="py-3 px-4 font-bold uppercase tracking-wider">Last Verified</th>
                <th className="py-3 px-4 font-bold uppercase tracking-wider text-right">
                  Action
                </th>
              </tr>
            </thead>
            <tbody className="divide-y divide-slate-100">
              {C.length === 0 ? (
                <tr>
                  <td colSpan={8} className="py-12 text-center text-slate-400">
                    {'No bids match the filter. '}
                    <button
                      onClick={() => t('submit')}
                      className="text-pine-700 font-bold underline"
                    >
                      Register a new bid →
                    </button>
                  </td>
                </tr>
              ) : (
                C.map((P) => (
                  <tr key={P.id} className="hover:bg-slate-50/80 transition-colors">
                    <td className="py-3.5 px-4">
                      <div className="flex items-center gap-2.5">
                        <div className="w-8 h-8 rounded-xl bg-teal-50 text-teal-800 font-bold flex items-cen
ter justify-center border border-teal-200 flex-shrink-0">
                          <BuildingIcon className="w-4 h-4" />
                        </div>
                        <div>
                          <p className="font-bold text-slate-900">{P.bidder_name}</p>
                          <span className="text-[10px] font-mono text-slate-400">
                            {P.bid_id}
                          </span>
                        </div>
                      </div>
                    </td>
                    <td className="py-3.5 px-4">
                      <span className="font-mono font-bold text-slate-900 bg-slate-100 px-2 py-0.5 rounded-l
g border border-slate-200">
                        {P.tender_id}
                      </span>
                    </td>
                    <td className="py-3.5 px-4">
                      <span className="font-mono text-slate-700 bg-slate-100 px-2.5 py-1 rounded-lg font-bol
d">
                        {P.documents_count}
                        {' Files'}
                      </span>
                    </td>
                    <td className="py-3.5 px-4">
                      <div className="flex items-center gap-2">
```

```
                        <div className="w-16 bg-slate-100 h-2.5 rounded-full overflow-hidden">
                          <div
                            className={`h-full rounded-full ${P.compliance_score >= 80 ? 'bg-teal-500' : P.c
ompliance_score >= 60 ? 'bg-amber-500' : 'bg-rose-500'}`}
                            style={{ width: `${P.compliance_score}%` }}
                          />
                        </div>
                        <span className="font-mono font-bold text-slate-900">
                          {P.compliance_score}/100
                        </span>
                      </div>
                    </td>
                    <td className="py-3.5 px-4">
                      <span
                        className={`text-[10px] font-bold px-2.5 py-0.5 rounded-full uppercase tracking-wide
r ${P.risk_level === 'LOW' ? 'bg-emerald-100 text-emerald-800' : P.risk_level === 'MEDIUM' ? 'bg-amber-100 t
ext-amber-800' : 'bg-rose-100 text-rose-800'}`}
                      >
                        {P.risk_level}
                      </span>
                    </td>
                    <td className="py-3.5 px-4">
                      <span
                        className={`inline-flex items-center gap-1.5 text-[11px] font-bold ${P.status === 'A
PPROVED' ? 'text-emerald-700' : P.status === 'REJECTED' ? 'text-rose-700' : P.status === 'VERIFIED' ? 'text-
teal-700' : 'text-amber-700'}`}
                      >
                        {P.status === 'APPROVED' && (
                          <CircleCheckIcon className="w-3.5 h-3.5 text-emerald-600" />
                        )}
                        {P.status === 'REJECTED' && (
                          <CircleXIcon className="w-3.5 h-3.5 text-rose-600" />
                        )}
                        {P.status === 'VERIFIED' && (
                          <ShieldCheckIcon className="w-3.5 h-3.5 text-teal-600" />
                        )}
                        {P.status === 'REQUIRES_CLARIFICATION' && (
                          <CircleHelpIcon className="w-3.5 h-3.5 text-amber-600" />
                        )}
                        {P.status.replace(/_/g, ' ')}
                      </span>
                    </td>
                    <td className="py-3.5 px-4 text-slate-500 font-mono text-[11px]">
                      {P.last_verified}
                    </td>
                    <td className="py-3.5 px-4 text-right">
                      <Link
                        to={`/bids/${P.id}`}
                        className="inline-flex items-center gap-1 px-3 py-1.5 bg-pine-50 hover:bg-pine-800 t
ext-pine-900 hover:text-white font-bold rounded-xl transition-all border border-pine-200 shadow-sm text-[11p
x]"
                      >
                        <span>Verify Bid</span>
                        <ArrowUpRightIcon className="w-3.5 h-3.5" />
                      </Link>
                    </td>
                  </tr>
                ))
              )}
            </tbody>
          </table>
        </div>
      )}
    </div>
  )}
  {e === 'submit' && (
    <div className="grid grid-cols-1 lg:grid-cols-3 gap-6">
      <div className="lg:col-span-2 bg-white rounded-2xl border border-slate-200/90 shadow-card-soft p-6">
        <div className="flex items-center gap-3 mb-5 pb-4 border-b border-slate-100">
          <div className="w-9 h-9 rounded-xl bg-pine-50 text-pine-800 flex items-center justify-center font-
bold">
            <FileTextIcon className="w-5 h-5" />
          </div>
          <div>
            <h2 className="text-sm font-bold text-slate-900">
              Register New Bid Against Tender
            </h2>
            <p className="text-xs text-slate-500">
              Link a registered bidder to an active tender to begin AI compliance verification.
```

```
            </p>
          </div>
        </div>
        <form onSubmit={T} className="space-y-5">
          <div>
            <label className="block text-xs font-bold text-slate-700 mb-1.5">
              {'Select Active Tender '}
              <span className="text-rose-500">*</span>
            </label>
            <select
              value={p}
              onChange={(P) => h(P.target.value)}
              className="w-full text-xs border border-slate-300 rounded-xl px-3.5 py-2.5 bg-slate-50 focus:b
g-white focus:ring-2 focus:ring-teal-600 focus:outline-none font-medium"
              required
            >
              <option value="">— Choose a Tender —</option>
              {r.map((P) => (
                <option key={P.id} value={P.id}>
                  {P.tender_id}
                  {' · '}
                  {P.title}
                  {' ('}
                  {P.status})
                </option>
              ))}
            </select>
            {p &&
              (() => {
                var $;
                const P = r.find((D) => String(D.id) === p);
                return P ? (
                  <div className="mt-2.5 p-3.5 bg-pine-50/70 rounded-xl border border-pine-200 text-xs flex
flex-wrap gap-4">
                    <span className="flex items-center gap-1.5 text-slate-700">
                      <CalendarIcon className="w-3.5 h-3.5 text-pine-700" />
                      {' Deadline: '}
                      <strong>{new Date(P.deadline).toLocaleDateString()}</strong>
                    </span>
                    <span className="flex items-center gap-1.5 text-slate-700">
                      <IndianRupeeIcon className="w-3.5 h-3.5 text-pine-700" />
                      {' Value: '}
                      <strong>₹{P.tender_value.toLocaleString('en-IN')}</strong>
                    </span>
                    <span className="flex items-center gap-1.5 text-slate-700">
                      <CpuIcon className="w-3.5 h-3.5 text-pine-700" />
                      {' Criteria: '}
                      <strong>
                        {(($ = P.requirements) == null ? undefined : $.length) || 0}
                        {' checks'}
                      </strong>
                    </span>
                  </div>
                ) : null;
              })()}
          </div>
          <div>
            <label className="block text-xs font-bold text-slate-700 mb-1.5">
              {'Select Registered Bidder '}
              <span className="text-rose-500">*</span>
            </label>
            <div className="relative mb-2.5">
              <SearchIcon className="absolute left-3 top-3 w-3.5 h-3.5 text-slate-400" />
              <input
                type="text"
                placeholder="Search bidder by company name, PAN…"
                value={y}
                onChange={(P) => x(P.target.value)}
                className="w-full text-xs border border-slate-300 rounded-xl pl-9 pr-3 py-2.5 bg-slate-50 fo
cus:bg-white focus:ring-2 focus:ring-teal-600 focus:outline-none"
              />
            </div>
            <div className="border border-slate-200 rounded-2xl max-h-48 overflow-y-auto divide-y divide-sla
te-100 custom-scrollbar">
              {I.length === 0 ? (
                <div className="p-5 text-center text-xs text-slate-400">
                  {'No bidders found. '}
                  <Link to="/bidders" className="text-pine-700 font-bold underline">
                    Register a bidder first →
```

```
                  </Link>
                </div>
              ) : (
                I.map((P) => (
                  <label
                    key={P.id}
                    className={`flex items-center gap-3 px-4 py-3 cursor-pointer hover:bg-slate-50 transitio
n-colors ${String(m) === String(P.id) ? 'bg-pine-50/80 font-semibold' : ''}`}
                  >
                    <input
                      type="radio"
                      name="bidder"
                      value={P.id}
                      checked={String(m) === String(P.id)}
                      onChange={() => b(String(P.id))}
                      className="accent-pine-700"
                    />
                    <div className="min-w-0 flex-1">
                      <p className="text-xs font-bold text-slate-900">{P.company_name}</p>
                      <p className="text-[10px] text-slate-500 font-mono">
                        {'PAN: '}
                        {P.pan || '–'}
                        {' | GSTIN: '}
                        {P.gstin || '–'} {P.is_msme ? '· MSME  ' : ''}
                      </p>
                    </div>
                  </label>
                ))
              )}
            </div>
          </div>
          <div className="flex items-start gap-2.5 bg-champagne-50/90 border border-champagne-200 rounded-2x
l p-3.5">
            <TriangleAlertIcon className="w-4 h-4 text-champagne-700 flex-shrink-0 mt-0.5" />
            <p className="text-xs text-champagne-950 leading-relaxed">
              {
                'After registration, the AI compliance engine will automatically evaluate statutory verifica
tion requirements (PAN, GST, Udyam, ITR, Debarment list, OEM, Make in India). Final qualification decision r
ests strictly with the '
              }
              <strong>Procurement Officer</strong>.
            </p>
          </div>
          {_ && (
            <div className="flex items-center gap-2 bg-emerald-50 border border-emerald-200 rounded-xl p-3 t
ext-emerald-800 text-xs font-semibold">
              <CircleCheckIcon className="w-4 h-4 text-emerald-600 flex-shrink-0" /> {_}
            </div>
          )}
          {O && (
            <div className="flex items-center gap-2 bg-rose-50 border border-rose-200 rounded-xl p-3 text-ro
se-800 text-xs font-semibold">
              <CircleXIcon className="w-4 h-4 text-rose-600 flex-shrink-0" /> {O}
            </div>
          )}
          <div className="flex gap-3 pt-2">
            <button
              type="submit"
              disabled={j || !p || !m}
              className="flex-1 py-3 bg-gradient-to-r from-pine-800 to-teal-800 hover:from-pine-900 hover:to
-teal-900 text-white text-xs font-bold rounded-xl shadow-glow-teal transition-all disabled:opacity-50 flex i
tems-center justify-center gap-2"
            >
              <CpuIcon className={`w-4 h-4 ${j ? 'animate-spin' : ''}`} />
              {j ? 'Registering & Running AI Check…' : 'Register Bid & Start AI Verification'}
            </button>
            <button
              type="button"
              onClick={() => t('all')}
              className="px-5 py-3 border border-slate-300 text-slate-700 text-xs font-bold rounded-xl hover
:bg-slate-50 transition-colors"
            >
              Cancel
            </button>
          </div>
        </form>
      </div>
      <div className="bg-white rounded-2xl border border-slate-200/90 shadow-card-soft p-5">
        <h3 className="text-xs font-bold uppercase tracking-wider text-slate-700 mb-3 flex items-center gap-
```

```
    2">
              <FileTextIcon className="w-4 h-4 text-pine-700" />
              {' Active Tenders'}
            </h3>
            <div className="space-y-2.5">
              {r
                .filter((P) => P.status === 'ACTIVE')
                .slice(0, 6)
                .map((P) => (
                  <div
                    key={P.id}
                    className="p-3 border border-slate-200 rounded-xl hover:border-pine-400 hover:bg-pine-50/40
    transition-all cursor-pointer group"
                    onClick={() => {
                      (h(String(P.id)), t('submit'));
                    }}
                  >
                    <p className="text-xs font-bold text-slate-900 truncate group-hover:text-pine-800 transition
    -colors">
                      {P.title}
                    </p>
                    <div className="flex items-center gap-3 mt-1 text-[10px]">
                      <span className="font-mono text-pine-800 font-semibold">{P.tender_id}</span>
                      <span className="text-slate-500 flex items-center gap-0.5">
                        <CalendarIcon className="w-3 h-3" />{' '}
                        {new Date(P.deadline).toLocaleDateString()}
                      </span>
                    </div>
                    <div className="flex items-center justify-between mt-2 pt-1 border-t border-slate-100 text-[
    10px]">
                      <span className="text-slate-500">
                        {P.bids_count || 0}
                        {' bids registered'}
                      </span>
                      <span className="font-bold bg-pine-100 text-pine-800 px-2 py-0.5 rounded-full">
                        ACTIVE
                      </span>
                    </div>
                  </div>
                ))}
              {r.filter((P) => P.status === 'ACTIVE').length === 0 && (
                <div className="py-8 text-center text-xs text-slate-400">
                  No active tenders.
                  <br />
                  <Link to="/tenders" className="text-pine-700 font-bold underline">
                    Create a Tender →
                  </Link>
                </div>
              )}
            </div>
          </div>
        </div>
      )}
    </div>
  );
};


//// FILE: src/pages/DashboardPage.jsx  (596 lines)

import { useEffect, useState } from 'react';
import { Link } from 'react-router-dom';
import {
  Bar,
  BarChart,
  CartesianGrid,
  Cell,
  Legend,
  Pie,
  PieChart,
  ResponsiveContainer,
  Tooltip,
  XAxis,
  YAxis,
} from 'recharts';
import {
  ArrowRightIcon,
  ArrowUpRightIcon,
  BuildingIcon,
```

```
  CircleCheckIcon,
  CircleHelpIcon,
  CircleXIcon,
  ClockIcon,
  FileTextIcon,
  LayersIcon,
  SearchIcon,
  ShieldCheckIcon,
  SquareCheckBigIcon,
  TrendingUpIcon,
  TriangleAlertIcon,
} from 'lucide-react';
import { useAuth } from '../context/AuthContext';
import { api } from '../api';

const riskColors = { Low: '#14B8A6', Medium: '#F59E0B', High: '#F43F5E', Critical: '#991B1B' };

const chartPalette = ['#14B8A6', '#F59E0B', '#F43F5E'];

export const DashboardPage = () => {
  const { user: e } = useAuth(),
    [t, r] = useState(null),
    [n, i] = useState(true),
    [a, o] = useState(''),
    [l, c] = useState('ALL'),
    [loadError, setLoadError] = useState('');
  useEffect(() => {
    u();
  }, []);
  const u = async () => {
    try {
      i(true);
      setLoadError('');
      const h = await api.getDashboardStats();
      r(h);
    } catch (h) {
      console.error(h);
      setLoadError(h && h.message ? h.message : 'The dashboard could not be loaded.');
    } finally {
      i(false);
    }
  };
  if (!n && !t)
    return (
      <div className="flex items-center justify-center min-h-[60vh]">
        <div className="text-center space-y-3 max-w-md">
          <p className="text-sm font-bold text-rose-700">Could not load the dashboard</p>
          <p className="text-xs text-slate-600">{loadError}</p>
          <p className="text-xs text-slate-500">
            Check that the backend window is running (http://127.0.0.1:8000/api/health), then try
            again.
          </p>
          <button
            onClick={u}
            className="px-4 py-2 bg-pine-800 text-white text-xs font-bold rounded-xl"
          >
            Try again
          </button>
        </div>
      </div>
    );
  if (n || !t)
    return (
      <div className="flex items-center justify-center min-h-[60vh]">
        <div className="text-center space-y-3">
          <div className="w-10 h-10 border-4 border-pine-800 border-t-champagne-400 rounded-full animate-spin mx
    -auto" />
          <p className="text-xs font-bold text-slate-700">
            Loading GeM Technical Evaluation Intelligence...
          </p>
        </div>
      </div>
    );
  const f = Object.entries(t.compliance_overview).map(([h, y]) => ({ name: h, value: y })),
    d = Object.entries(t.risk_distribution).map(([h, y]) => ({ name: h, value: y })),
    p = t.recent_bids.filter((h) => {
      const y =
        h.bidder_name.toLowerCase().includes(a.toLowerCase()) ||
        h.tender_id.toLowerCase().includes(a.toLowerCase()) ||
```

```
        h.bid_id.toLowerCase().includes(a.toLowerCase()),
      x = l === 'ALL' || h.risk_level === l;
    return y && x;
  });
return (
  <div className="space-y-6">
    <div className="relative gradient-pine-hero rounded-3xl p-6 sm:p-8 text-white shadow-2xl overflow-hidden b
  order border-pine-700/40">
      <div className="absolute right-0 top-0 bottom-0 w-1/2 bg-gradient-to-l from-pine-500/10 via-champagne-50
  0/5 to-transparent pointer-events-none" />
      <div className="absolute -right-10 -top-10 w-72 h-72 bg-teal-500/15 rounded-full blur-3xl pointer-events
  -none" />
      <div className="relative z-10 flex flex-col md:flex-row md:items-center justify-between gap-6">
        <div className="space-y-2.5">
          <div className="flex items-center gap-2 flex-wrap">
            <span className="bg-champagne-400 text-slate-950 text-[10px] font-black px-3 py-0.5 rounded-full u
  ppercase tracking-wider font-mono shadow-sm">
              GFR 2017 AI Matrix
            </span>
            <span className="text-pine-200 text-xs font-medium flex items-center gap-1">
              <span className="w-1.5 h-1.5 rounded-full bg-pine-400 animate-pulse" />
              Deterministic Technical Scrutiny Active
            </span>
          </div>
          <h1 className="text-2xl sm:text-3xl font-extrabold tracking-tight">
            {'Welcome, '}
            <span className="text-champagne-300">
              {(e == null ? undefined : e.full_name) || 'Rajesh Kumar Verma'}
            </span>
          </h1>
          <p className="text-xs sm:text-sm text-slate-200/90 max-w-2xl leading-relaxed">
            Real-time decision support gateway active across GSTN, MCA21, Udyam, Income Tax e-PAN,
            and Central Debarment registers.
          </p>
        </div>
        <div className="flex flex-wrap items-center gap-3">
          <Link
            to="/tenders"
            className="px-4 py-2.5 bg-gradient-to-r from-champagne-400 to-amber-500 hover:from-amber-400 hover
  :to-amber-600 text-slate-950 text-xs font-bold rounded-xl shadow-glow-champagne transition-all flex items-ce
  nter gap-2 group"
          >
            <FileTextIcon className="w-4 h-4 text-slate-950" />
            <span>Create Tender</span>
            <ArrowRightIcon className="w-3.5 h-3.5 group-hover:translate-x-0.5 transition-transform" />
          </Link>
          <Link
            to="/bids"
            className="px-4 py-2.5 bg-white/10 hover:bg-white/20 backdrop-blur-md text-white text-xs font-bold
   rounded-xl border border-white/20 shadow transition-all flex items-center gap-2"
          >
            <SquareCheckBigIcon className="w-4 h-4 text-pine-300" />
            <span>Verify Bids</span>
          </Link>
        </div>
      </div>
    </div>
    <div className="grid grid-cols-2 sm:grid-cols-3 lg:grid-cols-6 gap-3.5 sm:gap-4">
      <div className="bg-white p-4 rounded-2xl border border-slate-200/90 shadow-card-soft hover:shadow-card-h
  over transition-all relative overflow-hidden group">
        <div className="flex justify-between items-start mb-2">
          <span className="text-[10.5px] font-bold text-slate-500 uppercase tracking-wider">
            Total Tenders
          </span>
          <div className="w-8 h-8 rounded-xl bg-slate-100 text-slate-800 flex items-center justify-center grou
  p-hover:scale-110 transition-transform">
            <FileTextIcon className="w-4 h-4" />
          </div>
        </div>
        <p className="text-2xl sm:text-3xl font-black text-slate-900 font-mono">
          {t.total_tenders}
        </p>
        <span className="text-[10px] text-slate-400 font-medium mt-1 block">
          In Master Registry
        </span>
        <div className="absolute bottom-0 left-0 right-0 h-1 bg-slate-800 rounded-b-2xl" />
      </div>
      <div className="bg-white p-4 rounded-2xl border border-slate-200/90 shadow-card-soft hover:shadow-card-h
  over transition-all relative overflow-hidden group">
```

```
      <div className="flex justify-between items-start mb-2">
        <span className="text-[10.5px] font-bold text-slate-500 uppercase tracking-wider">
          Active Tenders
        </span>
        <div className="w-8 h-8 rounded-xl bg-pine-50 text-pine-700 flex items-center justify-center group-h
over:scale-110 transition-transform">
          <LayersIcon className="w-4 h-4" />
        </div>
      </div>
      <p className="text-2xl sm:text-3xl font-black text-pine-700 font-mono">
        {t.active_tenders}
      </p>
      <span className="text-[10px] text-pine-600 font-medium mt-1 block">Open for Bidding</span>
      <div className="absolute bottom-0 left-0 right-0 h-1 bg-pine-600 rounded-b-2xl" />
    </div>
    <div className="bg-white p-4 rounded-2xl border border-slate-200/90 shadow-card-soft hover:shadow-card-h
over transition-all relative overflow-hidden group">
      <div className="flex justify-between items-start mb-2">
        <span className="text-[10.5px] font-bold text-slate-500 uppercase tracking-wider">
          Under Review
        </span>
        <div className="w-8 h-8 rounded-xl bg-indigo-50 text-indigo-700 flex items-center justify-center gro
up-hover:scale-110 transition-transform">
          <ClockIcon className="w-4 h-4" />
        </div>
      </div>
      <p className="text-2xl sm:text-3xl font-black text-indigo-700 font-mono">
        {t.bids_under_verification}
      </p>
      <span className="text-[10px] text-indigo-600 font-medium mt-1 block">
        AI Pipeline Active
      </span>
      <div className="absolute bottom-0 left-0 right-0 h-1 bg-indigo-500 rounded-b-2xl" />
    </div>
    <div className="bg-white p-4 rounded-2xl border border-slate-200/90 shadow-card-soft hover:shadow-card-h
over transition-all relative overflow-hidden group">
      <div className="flex justify-between items-start mb-2">
        <span className="text-[10.5px] font-bold text-slate-500 uppercase tracking-wider">
          Verified Bids
        </span>
        <div className="w-8 h-8 rounded-xl bg-teal-50 text-teal-700 flex items-center justify-center group-h
over:scale-110 transition-transform">
          <SquareCheckBigIcon className="w-4 h-4" />
        </div>
      </div>
      <p className="text-2xl sm:text-3xl font-black text-teal-700 font-mono">
        {t.verified_bids}
      </p>
      <span className="text-[10px] text-teal-600 font-medium mt-1 block">Scrutiny Passed</span>
      <div className="absolute bottom-0 left-0 right-0 h-1 bg-teal-500 rounded-b-2xl" />
    </div>
    <div className="bg-white p-4 rounded-2xl border border-slate-200/90 shadow-card-soft hover:shadow-card-h
over transition-all relative overflow-hidden group">
      <div className="flex justify-between items-start mb-2">
        <span className="text-[10.5px] font-bold text-slate-500 uppercase tracking-wider">
          High / Critical
        </span>
        <div className="w-8 h-8 rounded-xl bg-rose-50 text-rose-700 flex items-center justify-center group-h
over:scale-110 transition-transform">
          <TriangleAlertIcon className="w-4 h-4" />
        </div>
      </div>
      <p className="text-2xl sm:text-3xl font-black text-rose-600 font-mono">
        {t.high_risk_bids}
      </p>
      <span className="text-[10px] text-rose-600 font-medium mt-1 block">
        Flagged Discrepancy
      </span>
      <div className="absolute bottom-0 left-0 right-0 h-1 bg-rose-500 rounded-b-2xl" />
    </div>
    <div className="bg-white p-4 rounded-2xl border border-slate-200/90 shadow-card-soft hover:shadow-card-h
over transition-all relative overflow-hidden group">
      <div className="flex justify-between items-start mb-2">
        <span className="text-[10.5px] font-bold text-slate-500 uppercase tracking-wider">
          Pending Sign-off
        </span>
        <div className="w-8 h-8 rounded-xl bg-champagne-100 text-champagne-800 flex items-center justify-cen
ter group-hover:scale-110 transition-transform">
          <TrendingUpIcon className="w-4 h-4" />
```

```
        </div>
      </div>
      <p className="text-2xl sm:text-3xl font-black text-champagne-700 font-mono">
        {t.pending_reviews}
      </p>
      <span className="text-[10px] text-champagne-700 font-medium mt-1 block">
        Awaiting Officer
      </span>
      <div className="absolute bottom-0 left-0 right-0 h-1 bg-champagne-500 rounded-b-2xl" />
    </div>
  </div>
  <div className="bg-white rounded-2xl border border-slate-200/90 shadow-card-soft overflow-hidden">
    <div className="p-5 border-b border-slate-100 flex items-center justify-between">
      <div className="flex items-center gap-3">
        <div className="w-8 h-8 rounded-xl bg-pine-50 text-pine-800 flex items-center justify-center font-bo
ld">
          <LayersIcon className="w-4 h-4" />
        </div>
        <div>
          <h3 className="text-sm font-bold text-slate-900 flex items-center gap-2">
            Tenders In Progress
            <span className="bg-pine-100 text-pine-800 text-[10px] font-bold px-2 py-0.5 rounded-full">
              {t.active_tenders}
              {' Active'}
            </span>
          </h3>
          <p className="text-xs text-slate-500">
            Live evaluation status of ongoing GeM procurement packages
          </p>
        </div>
      </div>
      <Link
        to="/tenders"
        className="text-xs font-bold text-pine-700 hover:text-pine-900 flex items-center gap-1 hover:underli
ne"
      >
        <span>Manage All Tenders</span>
        <ArrowUpRightIcon className="w-3.5 h-3.5" />
      </Link>
    </div>
    <div className="overflow-x-auto">
      <table className="w-full text-xs text-left border-collapse">
        <thead>
          <tr className="bg-slate-50/90 text-slate-600 border-b border-slate-200">
            <th className="py-3 px-4 font-bold uppercase tracking-wider">Tender ID</th>
            <th className="py-3 px-4 font-bold uppercase tracking-wider">{'Title & Dept'}</th>
            <th className="py-3 px-4 font-bold uppercase tracking-wider">Estimated Value</th>
            <th className="py-3 px-4 font-bold uppercase tracking-wider">Deadline</th>
            <th className="py-3 px-4 font-bold uppercase tracking-wider">Bids Count</th>
            <th className="py-3 px-4 font-bold uppercase tracking-wider">Evaluation Status</th>
            <th className="py-3 px-4 font-bold uppercase tracking-wider text-right">Action</th>
          </tr>
        </thead>
        <tbody className="divide-y divide-slate-100">
          {t.recent_bids.length === 0 ? (
            <tr>
              <td colSpan={7} className="py-8 text-center text-slate-400">
                {'No active tenders. '}
                <Link to="/tenders" className="text-pine-700 font-bold underline">
                  Create one now →
                </Link>
              </td>
            </tr>
          ) : (
            Array.from(new Map(t.recent_bids.map((h) => [h.tender_id, h])).values())
              .slice(0, 6)
              .map((h, y) => {
                const x = Math.max(
                    0,
                    Math.round((new Date(h.last_verified).getTime() - Date.now()) / 864e5) + 14,
                  ),
                  g =
                    h.risk_level === 'LOW'
                      ? 'bg-emerald-100 text-emerald-800'
                      : h.risk_level === 'MEDIUM'
                        ? 'bg-amber-100 text-amber-800'
                        : 'bg-rose-100 text-rose-800';
                return (
                  <tr key={y} className="hover:bg-slate-50/80 transition-colors">
```

```
                    <td className="py-3.5 px-4">
                      <span className="font-mono font-bold text-slate-900 bg-slate-100 px-2.5 py-0.5 rounded
-lg border border-slate-200">
                        {h.tender_id}
                      </span>
                    </td>
                    <td className="py-3.5 px-4">
                      <p className="font-bold text-slate-900">{h.bidder_name}</p>
                      <p className="text-[10px] text-slate-500">
                        {'Ministry of Electronics & IT'}
                      </p>
                    </td>
                    <td className="py-3.5 px-4 font-mono font-bold text-slate-800">
                      ₹{(50 + y * 12.5).toFixed(1)}
                      {' Lakhs'}
                    </td>
                    <td className="py-3.5 px-4">
                      <span
                        className={`font-semibold ${x <= 3 ? 'text-rose-600' : 'text-slate-700'}`}
                      >
                        {x}
                        {' days remaining'}
                      </span>
                    </td>
                    <td className="py-3.5 px-4">
                      <span className="font-bold text-slate-900">{h.documents_count}</span>
                      <span className="text-slate-500 ml-1">bids</span>
                    </td>
                    <td className="py-3.5 px-4">
                      <span
                        className={`text-[10px] font-bold px-2 py-0.5 rounded-full uppercase tracking-wider
${g}`}
                      >
                        {h.status.replace(/_/g, ' ')}
                      </span>
                    </td>
                    <td className="py-3.5 px-4 text-right">
                      <Link
                        to={`/bids/${h.id}`}
                        className="inline-flex items-center gap-1 px-3 py-1.5 bg-pine-50 hover:bg-pine-800 t
ext-pine-900 hover:text-white font-bold rounded-xl transition-all border border-pine-200 shadow-sm text-[11p
x]"
                      >
                        <span>Review</span>
                        <ArrowUpRightIcon className="w-3 h-3" />
                      </Link>
                    </td>
                  </tr>
                );
              })
          )}
        </tbody>
      </table>
    </div>
  </div>
  <div className="grid grid-cols-1 md:grid-cols-3 gap-5">
    <div className="bg-white p-5 rounded-2xl border border-slate-200/90 shadow-card-soft">
      <div className="flex items-center justify-between mb-1">
        <h3 className="text-xs font-bold uppercase tracking-wider text-slate-700">
          Compliance Breakdown
        </h3>
        <span className="text-[10px] bg-slate-100 text-slate-600 px-2 py-0.5 rounded font-mono font-bold">
          GFR 149
        </span>
      </div>
      <p className="text-[11px] text-slate-500 mb-4">Threshold status across registered bids</p>
      <div className="h-56">
        <ResponsiveContainer width="100%" height="100%">
          <PieChart>
            <Pie
              data={f}
              dataKey="value"
              nameKey="name"
              cx="50%"
              cy="50%"
              innerRadius={52}
              outerRadius={78}
              paddingAngle={6}
            >
```

```
              {f.map((h, y) => (
                <Cell key={`cell-${y}`} fill={chartPalette[y % chartPalette.length]} />
              ))}
            </Pie>
            <Tooltip />
            <Legend iconType="circle" wrapperStyle={{ fontSize: '11px' }} />
          </PieChart>
        </ResponsiveContainer>
      </div>
    </div>
    <div className="bg-white p-5 rounded-2xl border border-slate-200/90 shadow-card-soft">
      <div className="flex items-center justify-between mb-1">
        <h3 className="text-xs font-bold uppercase tracking-wider text-slate-700">
          Risk Categorization
        </h3>
        <span className="text-[10px] bg-slate-100 text-slate-600 px-2 py-0.5 rounded font-mono font-bold">
          AI Scored
        </span>
      </div>
      <p className="text-[11px] text-slate-500 mb-4">
        {'Multi-factor fraud & statutory discrepancy index'}
      </p>
      <div className="h-56">
        <ResponsiveContainer width="100%" height="100%">
          <BarChart data={d}>
            <CartesianGrid strokeDasharray="3 3" vertical={false} stroke="#F1F5F9" />
            <XAxis dataKey="name" tick={{ fontSize: 11 }} stroke="#64748B" />
            <YAxis allowDecimals={false} tick={{ fontSize: 11 }} stroke="#64748B" />
            <Tooltip />
            <Bar dataKey="value" name="Bids" radius={[6, 6, 0, 0]}>
              {d.map((h) => (
                <Cell key={h.name} fill={riskColors[h.name] || '#0D9488'} />
              ))}
            </Bar>
          </BarChart>
        </ResponsiveContainer>
      </div>
    </div>
    <div className="bg-white p-5 rounded-2xl border border-slate-200/90 shadow-card-soft">
      <div className="flex items-center justify-between mb-1">
        <h3 className="text-xs font-bold uppercase tracking-wider text-slate-700">
          Verification Activity
        </h3>
        <span className="text-[10px] bg-slate-100 text-slate-600 px-2 py-0.5 rounded font-mono font-bold">
          Last 6 Days
        </span>
      </div>
      <p className="text-[11px] text-slate-500 mb-4">Daily verified vs flagged submissions</p>
      <div className="h-56">
        <ResponsiveContainer width="100%" height="100%">
          <BarChart data={t.verification_activity}>
            <CartesianGrid strokeDasharray="3 3" vertical={false} stroke="#F1F5F9" />
            <XAxis dataKey="date" tick={{ fontSize: 11 }} stroke="#64748B" />
            <YAxis allowDecimals={false} tick={{ fontSize: 11 }} stroke="#64748B" />
            <Tooltip />
            <Bar dataKey="verified" name="Verified" fill="#0D9488" radius={[4, 4, 0, 0]} />
            <Bar dataKey="flagged" name="Flagged" fill="#F59E0B" radius={[4, 4, 0, 0]} />
            <Legend iconType="circle" wrapperStyle={{ fontSize: '11px' }} />
          </BarChart>
        </ResponsiveContainer>
      </div>
    </div>
  </div>
  <div className="bg-white rounded-2xl border border-slate-200/90 shadow-card-soft overflow-hidden">
    <div className="p-5 border-b border-slate-200 flex flex-col sm:flex-row justify-between items-start sm:i
tems-center gap-4">
      <div>
        <h3 className="text-sm font-bold text-slate-900 flex items-center gap-2">
          <SquareCheckBigIcon className="w-4 h-4 text-pine-700" />
          Submitted Bids Under Technical Scrutiny
        </h3>
        <p className="text-xs text-slate-500 mt-0.5">
          Deterministic verification results, statutory cross-checks, and officer sign-off
          status
        </p>
      </div>
      <div className="flex flex-wrap items-center gap-3 w-full sm:w-auto">
        <div className="relative flex-1 sm:flex-none">
          <SearchIcon className="w-3.5 h-3.5 text-slate-400 absolute left-3 top-2.5" />
```

```
          <input
            type="text"
            placeholder="Search by Bidder, Tender ID..."
            value={a}
            onChange={(h) => o(h.target.value)}
            className="text-xs border border-slate-300 rounded-xl pl-9 pr-3 py-2 bg-slate-50 focus:bg-white
focus:ring-2 focus:ring-teal-600 focus:outline-none w-full sm:w-60"
          />
        </div>
        <select
          value={l}
          onChange={(h) => c(h.target.value)}
          className="text-xs border border-slate-300 rounded-xl px-3 py-2 bg-slate-50 font-medium text-slate
-700 focus:ring-2 focus:ring-teal-600 focus:outline-none"
        >
          <option value="ALL">All Risk Levels</option>
          <option value="LOW">Low Risk</option>
          <option value="MEDIUM">Medium Risk</option>
          <option value="HIGH">High Risk</option>
          <option value="CRITICAL">Critical Risk</option>
        </select>
      </div>
    </div>
    <div className="overflow-x-auto">
      <table className="w-full text-left text-xs border-collapse">
        <thead>
          <tr className="bg-slate-50/90 text-slate-600 border-b border-slate-200">
            <th className="py-3 px-4 font-bold uppercase tracking-wider">
              {'Bidder & Entity'}
            </th>
            <th className="py-3 px-4 font-bold uppercase tracking-wider">Tender ID</th>
            <th className="py-3 px-4 font-bold uppercase tracking-wider">Docs</th>
            <th className="py-3 px-4 font-bold uppercase tracking-wider">Compliance Score</th>
            <th className="py-3 px-4 font-bold uppercase tracking-wider">Risk Level</th>
            <th className="py-3 px-4 font-bold uppercase tracking-wider">Evaluation Status</th>
            <th className="py-3 px-4 font-bold uppercase tracking-wider">Last Verified</th>
            <th className="py-3 px-4 font-bold uppercase tracking-wider text-right">Action</th>
          </tr>
        </thead>
        <tbody className="divide-y divide-slate-100">
          {p.length === 0 ? (
            <tr>
              <td colSpan={8} className="text-center py-10 text-slate-400">
                No bids match the specified search or filter criteria.
              </td>
            </tr>
          ) : (
            p.map((h) => (
              <tr key={h.id} className="hover:bg-slate-50/80 transition-colors">
                <td className="py-3.5 px-4">
                  <div className="flex items-center gap-2.5">
                    <div className="w-8 h-8 rounded-xl bg-teal-50 text-teal-800 font-bold flex items-center
justify-center border border-teal-200 flex-shrink-0">
                      <BuildingIcon className="w-4 h-4" />
                    </div>
                    <div>
                      <p className="font-bold text-slate-900">{h.bidder_name}</p>
                      <span className="text-[10px] font-mono text-slate-400">{h.bid_id}</span>
                    </div>
                  </div>
                </td>
                <td className="py-3.5 px-4">
                  <span className="font-mono font-bold text-slate-900 bg-slate-100 px-2 py-0.5 rounded-lg bo
rder border-slate-200">
                    {h.tender_id}
                  </span>
                </td>
                <td className="py-3.5 px-4">
                  <span className="font-mono text-slate-700 bg-slate-100 px-2.5 py-1 rounded-lg font-bold">
                    {h.documents_count}
                    {' Files'}
                  </span>
                </td>
                <td className="py-3.5 px-4">
                  <div className="flex items-center gap-2">
                    <div className="w-16 bg-slate-100 h-2.5 rounded-full overflow-hidden">
                      <div
                        className={`h-full rounded-full ${h.compliance_score >= 80 ? 'bg-teal-500' : h.compl
iance_score >= 60 ? 'bg-amber-500' : 'bg-rose-500'}`}
```

```
                          style={{ width: `${h.compliance_score}%` }}
                        />
                      </div>
                      <span className="font-mono font-bold text-slate-900">
                        {h.compliance_score}/100
                      </span>
                    </div>
                  </td>
                  <td className="py-3.5 px-4">
                    <span
                      className={`text-[10px] font-bold px-2.5 py-0.5 rounded-full uppercase tracking-wider ${
    h.risk_level === 'LOW' ? 'bg-emerald-100 text-emerald-800' : h.risk_level === 'MEDIUM' ? 'bg-amber-100 text-
    amber-800' : 'bg-rose-100 text-rose-800'}`}
                    >
                      {h.risk_level}
                    </span>
                  </td>
                  <td className="py-3.5 px-4">
                    <span
                      className={`inline-flex items-center gap-1.5 text-[11px] font-bold ${h.status === 'APPRO
    VED' ? 'text-emerald-700' : h.status === 'REJECTED' ? 'text-rose-700' : h.status === 'VERIFIED' ? 'text-teal
    -700' : 'text-amber-700'}`}
                    >
                      {h.status === 'APPROVED' && (
                        <CircleCheckIcon className="w-3.5 h-3.5 text-emerald-600" />
                      )}
                      {h.status === 'REJECTED' && (
                        <CircleXIcon className="w-3.5 h-3.5 text-rose-600" />
                      )}
                      {h.status === 'VERIFIED' && (
                        <ShieldCheckIcon className="w-3.5 h-3.5 text-teal-600" />
                      )}
                      {h.status === 'REQUIRES_CLARIFICATION' && (
                        <CircleHelpIcon className="w-3.5 h-3.5 text-amber-600" />
                      )}
                      {h.status.replace(/_/g, ' ')}
                    </span>
                  </td>
                  <td className="py-3.5 px-4 text-slate-500 font-mono text-[11px]">
                    {h.last_verified}
                  </td>
                  <td className="py-3.5 px-4 text-right">
                    <Link
                      to={`/bids/${h.id}`}
                      className="inline-flex items-center gap-1 px-3 py-1.5 bg-pine-50 hover:bg-pine-800 text-
    pine-900 hover:text-white font-bold rounded-xl transition-all border border-pine-200 shadow-sm"
                    >
                      <span>Verify Bid</span>
                      <ArrowUpRightIcon className="w-3.5 h-3.5" />
                    </Link>
                  </td>
                </tr>
              ))
            )}
          </tbody>
        </table>
      </div>
    </div>
  </div>
  );
};


//// FILE: src/pages/IntegrationsPage.jsx  (512 lines)

import { useEffect, useState } from 'react';
import { CpuIcon, SearchIcon, ServerIcon, ShieldCheckIcon, ZapIcon } from 'lucide-react';
import { api } from '../api';

export const IntegrationsPage = () => {
  var C, I, R, P;
  const [e, t] = useState([]),
    [r, n] = useState(true),
    [i, a] = useState(''),
    [o, l] = useState('27ABCDE1234F1Z5'),
    [c, u] = useState(null),
    [f, d] = useState(false),
    [p, h] = useState('ABCDE1234F'),
    [y, x] = useState(null),
```

```
  [g, v] = useState(false),
  [m, b] = useState('Bharat Systems Pvt Ltd'),
  [j, w] = useState(null),
  [_, S] = useState(false);
useEffect(() => {
  O();
}, []);
const O = async () => {
    try {
      n(true);
      const $ = await api.getIntegrationsStatus();
      (t($.connectors), a($.disclaimer));
    } catch ($) {
      console.error($);
    } finally {
      n(false);
    }
  },
  A = async ($) => {
    $.preventDefault();
    try {
      d(true);
      const D = await api.testGST(o);
      u(D);
    } finally {
      d(false);
    }
  },
  E = async ($) => {
    $.preventDefault();
    try {
      v(true);
      const D = await api.testPAN(p);
      x(D);
    } finally {
      v(false);
    }
  },
  T = async ($) => {
    $.preventDefault();
    try {
      S(true);
      const D = await api.testDebarment(m);
      w(D);
    } finally {
      S(false);
    }
  };
return (
  <div className="space-y-6">
    <div className="bg-white p-6 rounded-3xl border border-slate-200/90 shadow-sm flex flex-col sm:flex-row ju
  stify-between items-start sm:items-center gap-4">
      <div>
        <div className="flex items-center gap-2">
          <h1 className="text-xl font-extrabold text-slate-900 tracking-tight">
            Government Portal Integrations Architecture
          </h1>
          <span className="bg-pine-100 text-pine-800 text-[10px] font-bold px-2.5 py-0.5 rounded-full font-mon
  o flex items-center gap-1 border border-pine-200">
            <ZapIcon className="w-2.5 h-2.5 text-pine-600" />
            14 Connectors Live
          </span>
        </div>
        <p className="text-xs text-slate-500 mt-0.5">
          Unified statutory gateway architecture for GSTN, MCA21, Udyam, Income Tax e-PAN, EPFO,
          and Central Debarment registers.
        </p>
      </div>
    </div>
    <div className="p-4 bg-champagne-50/90 rounded-2xl border border-champagne-200 text-xs text-champagne-950
  flex items-start gap-3">
      <ShieldCheckIcon className="w-5 h-5 text-champagne-600 flex-shrink-0 mt-0.5" />
      <div>
        <h4 className="font-bold text-champagne-950">
          Statutory Transparency Disclaimer (SIH Prototype Mode)
        </h4>
        <p className="mt-0.5 leading-relaxed text-champagne-800">
          {i ||
            'DEMO / SYNTHETIC DATA — All connectors operate in Sandboxed / Mock evaluation mode for SIH protot
```

```
ype. Connectors are built on standard REST adapters that easily map to authorized National Informatics Centr
e (NIC) and API Setu production endpoints.'}
      </p>
    </div>
  </div>
  <div>
    <div className="flex items-center justify-between mb-3">
      <h3 className="text-xs font-bold uppercase tracking-wider text-slate-700 flex items-center gap-1.5">
        <ServerIcon className="w-3.5 h-3.5 text-pine-700" />
        Statutory Verification Connectors Matrix ({e.length})
      </h3>
      <span className="text-[10px] text-slate-400 font-mono">
        Protocols: REST / JSON / mTLS
      </span>
    </div>
    <div className="grid grid-cols-1 sm:grid-cols-2 md:grid-cols-3 lg:grid-cols-4 gap-4">
      {e.map(($) => (
        <div
          key={$.id}
          className="bg-white p-4 rounded-2xl border border-slate-200/90 shadow-sm hover:shadow-card-hover h
over:border-pine-300 transition-all flex flex-col justify-between group"
        >
          <div>
            <div className="flex justify-between items-start mb-2.5">
              <span className="text-[9.5px] font-bold font-mono px-2 py-0.5 rounded-md bg-pine-50 text-pine-
900 border border-pine-200 uppercase">
                {$.mode}
              </span>
              <span className="w-2.5 h-2.5 rounded-full bg-pine-500 animate-pulse" />
            </div>
            <h4 className="text-xs font-bold text-slate-900 leading-snug group-hover:text-pine-800 transitio
n-colors">
              {$.name}
            </h4>
            <p className="text-[11px] text-slate-500 mt-1 line-clamp-2">{$.type}</p>
          </div>
          <div className="mt-4 pt-2.5 border-t border-slate-100 flex justify-between items-center text-[10px
] font-mono text-slate-400">
            <span>
              {'Latency: '}
              <strong className="text-slate-700">{$.latency_ms}ms</strong>
            </span>
            <span className="text-pine-700 font-bold bg-pine-50 px-1.5 py-0.2 rounded border border-pine-200
">
              {$.status}
            </span>
          </div>
        </div>
      ))}
    </div>
  </div>
  <div className="bg-white p-6 rounded-3xl border border-slate-200/90 shadow-sm space-y-6">
    <div className="flex justify-between items-start flex-wrap gap-2">
      <div>
        <h3 className="text-sm font-bold text-slate-900 flex items-center gap-2">
          <CpuIcon className="w-4 h-4 text-pine-800" />
          Interactive Verification Gateway Sandbox
        </h3>
        <p className="text-xs text-slate-500 mt-0.5">
          Simulate live API queries against GSTN, Income Tax e-PAN, and National Debarment
          registries.
        </p>
      </div>
      <span className="text-[10px] font-mono bg-pine-50 text-pine-900 font-bold px-2.5 py-1 rounded-lg borde
r border-pine-200">
        Gateway: Active · 100% Deterministic Match
      </span>
    </div>
    <div className="grid grid-cols-1 lg:grid-cols-3 gap-6">
      <div className="p-4 bg-slate-50/80 rounded-2xl border border-slate-200 space-y-3 flex flex-col justify
-between">
        <div>
          <div className="flex justify-between items-center mb-2">
            <h4 className="font-bold text-xs text-slate-900 flex items-center gap-1.5">
              <SearchIcon className="w-3.5 h-3.5 text-pine-700" />
              GSTN Verification Gateway
            </h4>
            <span className="text-[9px] bg-pine-100 text-pine-800 font-bold px-1.5 py-0.5 rounded font-mono
border border-pine-200">
```

```
              GSTN API v2.1
            </span>
          </div>
          <p className="text-[11px] text-slate-500 mb-2">
            Enter any 15-character GSTIN or select a quick test preset:
          </p>
          <div className="flex flex-wrap gap-1.5 mb-2.5">
            <button
              type="button"
              onClick={() => l('27ABCDE1234F1Z5')}
              className="text-[10px] bg-white border border-slate-200 text-slate-700 hover:border-pine-400 p
x-2 py-0.5 rounded-lg font-mono transition-colors"
            >
                ABC Tech (Active)
            </button>
            <button
              type="button"
              onClick={() => l('08BHART9999K1Z8')}
              className="text-[10px] bg-white border border-rose-200 text-rose-700 hover:bg-rose-50 px-2 py-
0.5 rounded-lg font-mono transition-colors"
            >
                Bharat Sys (Suspended)
            </button>
          </div>
          <form onSubmit={A} className="space-y-2">
            <input
              type="text"
              value={o}
              onChange={($) => l($.target.value.toUpperCase())}
              className="w-full p-2 text-xs border rounded-xl bg-white font-mono uppercase focus:ring-2 focu
s:ring-pine-600 focus:outline-none"
              placeholder="27ABCDE1234F1Z5"
              maxLength={15}
            />
            <button
              type="submit"
              disabled={f}
              className="w-full py-2 bg-pine-900 hover:bg-pine-800 text-white font-bold text-xs rounded-xl s
hadow-sm transition-colors flex items-center justify-center gap-1.5 border border-champagne-400/30"
            >
              {f ? (
                <>
                  <div className="w-3.5 h-3.5 border-2 border-white border-t-transparent rounded-full animat
e-spin" />
                  <span>Connecting to GSTN...</span>
                </>
              ) : (
                <span>Query GSTN Portal</span>
              )}
            </button>
          </form>
        </div>
        {c && (
          <div
            className={`mt-3 p-3.5 rounded-2xl border text-xs space-y-2 ${c.verified && c.status === 'ACTIVE
' ? 'bg-pine-50/90 border-pine-200 text-pine-950' : c.status === 'SUSPENDED' ? 'bg-champagne-50/90 border-ch
ampagne-200 text-champagne-950' : 'bg-rose-50/90 border-rose-200 text-rose-950'}`}
            >
              <div className="flex justify-between items-start">
                <span className="font-mono font-bold text-[11px]">
                  {((C = c.data) == null ? undefined : C.gstin) || o}
                </span>
                <span
                  className={`text-[9px] font-bold px-2 py-0.5 rounded-full uppercase tracking-wider ${c.statu
s === 'ACTIVE' ? 'bg-pine-200 text-pine-900' : 'bg-rose-200 text-rose-900'}`}
                >
                  {c.status}
                </span>
              </div>
              {c.data ? (
                <div className="space-y-1 text-[11px]">
                  <p>
                    <strong className="text-slate-700">Legal Name:</strong>{' '}
                    <span className="font-semibold">{c.data.legal_name}</span>
                  </p>
                  <p>
                    <strong className="text-slate-700">Trade Name:</strong> {c.data.trade_name}
                  </p>
                  <p>
```

```
                  <strong className="text-slate-700">State / Region:</strong> {c.data.state}
                </p>
                <p>
                  <strong className="text-slate-700">Taxpayer Type:</strong>{' '}
                  {c.data.taxpayer_type}
                </p>
                <p>
                  <strong className="text-slate-700">Filing Status:</strong>{' '}
                  <span className="font-medium text-pine-800">{c.data.filing_status}</span>
                </p>
                <div className="pt-1.5 border-t border-pine-200/60 flex items-center justify-between text-[1
0px] font-mono text-slate-500">
                  <span>
                    {'PAN: '}
                    {c.data.pan_linked}
                  </span>
                  <span className="text-pine-700 font-bold">  E-Invoice Enabled</span>
                </div>
              </div>
            ) : (
              <p className="text-[11px] leading-tight">{c.message || 'No record found.'}</p>
            )}
          </div>
        )}
      </div>
      <div className="p-4 bg-slate-50/80 rounded-2xl border border-slate-200 space-y-3 flex flex-col justify
-between">
        <div>
          <div className="flex justify-between items-center mb-2">
            <h4 className="font-bold text-xs text-slate-900 flex items-center gap-1.5">
              <SearchIcon className="w-3.5 h-3.5 text-pine-700" />
              ITD e-PAN Verification
            </h4>
            <span className="text-[9px] bg-pine-100 text-pine-800 font-bold px-1.5 py-0.5 rounded font-mono
border border-pine-200">
              ITD NSDL API
            </span>
          </div>
          <p className="text-[11px] text-slate-500 mb-2">
            Enter 10-character PAN or select a quick test preset:
          </p>
          <div className="flex flex-wrap gap-1.5 mb-2.5">
            <button
              type="button"
              onClick={() => h('ABCDE1234F')}
              className="text-[10px] bg-white border border-slate-200 text-slate-700 hover:border-pine-400 p
x-2 py-0.5 rounded-lg font-mono transition-colors"
            >
                ABCDE1234F (Company)
            </button>
            <button
              type="button"
              onClick={() => h('XYZAB5678C')}
              className="text-[10px] bg-white border border-slate-200 text-slate-700 hover:border-pine-400 p
x-2 py-0.5 rounded-lg font-mono transition-colors"
            >
                XYZAB5678C
            </button>
          </div>
          <form onSubmit={E} className="space-y-2">
            <input
              type="text"
              value={p}
              onChange={($) => h($.target.value.toUpperCase())}
              className="w-full p-2 text-xs border rounded-xl bg-white font-mono uppercase focus:ring-2 focu
s:ring-pine-600 focus:outline-none"
              placeholder="ABCDE1234F"
              maxLength={10}
            />
            <button
              type="submit"
              disabled={g}
              className="w-full py-2 bg-pine-900 hover:bg-pine-800 text-white font-bold text-xs rounded-xl s
hadow-sm transition-colors flex items-center justify-center gap-1.5 border border-champagne-400/30"
            >
              {g ? (
                <>
                  <div className="w-3.5 h-3.5 border-2 border-white border-t-transparent rounded-full animat
e-spin" />
```

```
                  <span>Verifying with ITD...</span>
                </>
              ) : (
                <span>Query PAN Registry</span>
              )}
            </button>
          </form>
        </div>
        {y && (
          <div
            className={`mt-3 p-3.5 rounded-2xl border text-xs space-y-2 ${y.verified ? 'bg-pine-50/90 border
-pine-200 text-pine-950' : 'bg-rose-50/90 border-rose-200 text-rose-950'}`}
          >
            <div className="flex justify-between items-start">
              <span className="font-mono font-bold text-[11px]">
                {((I = y.data) == null ? undefined : I.pan) || p}
              </span>
              <span
                className={`text-[9px] font-bold px-2 py-0.5 rounded-full uppercase tracking-wider ${y.verif
ied ? 'bg-pine-200 text-pine-900' : 'bg-rose-200 text-rose-900'}`}
              >
                {y.status}
              </span>
            </div>
            {y.data ? (
              <div className="space-y-1 text-[11px]">
                <p>
                  <strong className="text-slate-700">Entity Name:</strong>{' '}
                  <span className="font-semibold">{y.data.entity_name}</span>
                </p>
                <p>
                  <strong className="text-slate-700">Category:</strong> {y.data.category}
                </p>
                <p>
                  <strong className="text-slate-700">Aadhaar Seeding:</strong>{' '}
                  {y.data.aadhaar_seeding}
                </p>
                <p>
                  <strong className="text-slate-700">Jurisdiction:</strong>{' '}
                  {y.data.jurisdiction}
                </p>
                <div className="pt-1.5 border-t border-pine-200/60 flex items-center justify-between text-[1
0px] font-mono text-slate-500">
                  <span>
                    {'Masked: '}
                    {y.data.masked_pan}
                  </span>
                  <span className="text-pine-700 font-bold">  ITD Active Record</span>
                </div>
              </div>
            ) : (
              <p className="text-[11px]">{y.message || 'Invalid PAN format.'}</p>
            )}
          </div>
        )}
      </div>
      <div className="p-4 bg-slate-50/80 rounded-2xl border border-slate-200 space-y-3 flex flex-col justify
-between">
        <div>
          <div className="flex justify-between items-center mb-2">
            <h4 className="font-bold text-xs text-slate-900 flex items-center gap-1.5">
              <SearchIcon className="w-3.5 h-3.5 text-rose-600" />
              National Debarment Registry
            </h4>
            <span className="text-[9px] bg-rose-100 text-rose-800 font-bold px-1.5 py-0.5 rounded font-mono"
>
              CPPP / CVC Master
            </span>
          </div>
          <p className="text-[11px] text-slate-500 mb-2">
            Screen company against Central Blacklisting database:
          </p>
          <div className="flex flex-wrap gap-1.5 mb-2.5">
            <button
              type="button"
              onClick={() => b('Bharat Systems Pvt Ltd')}
              className="text-[10px] bg-white border border-rose-200 text-rose-700 hover:bg-rose-50 px-2 py-
0.5 rounded-lg font-mono transition-colors"
            >
```

```
                Bharat Systems (Debarred)
              </button>
              <button
                type="button"
                onClick={() => b('ABC Technologies Pvt Ltd')}
                className="text-[10px] bg-white border border-slate-200 text-slate-700 hover:border-pine-400 p
x-2 py-0.5 rounded-lg font-mono transition-colors"
              >
                ABC Tech (Clear)
              </button>
            </div>
            <form onSubmit={T} className="space-y-2">
              <input
                type="text"
                value={m}
                onChange={($) => b($.target.value)}
                className="w-full p-2 text-xs border rounded-xl bg-white focus:ring-2 focus:ring-rose-600 focu
s:outline-none"
                placeholder="Company Name (e.g. Bharat Systems)"
              />
              <button
                type="submit"
                disabled={_}
                className="w-full py-2 bg-rose-700 hover:bg-rose-800 text-white font-bold text-xs rounded-xl s
hadow-sm transition-colors flex items-center justify-center gap-1.5"
              >
                {_ ? (
                  <>
                    <div className="w-3.5 h-3.5 border-2 border-white border-t-transparent rounded-full animat
e-spin" />
                    <span>Scanning National Blacklist...</span>
                  </>
                ) : (
                  <span>Run Debarment Match</span>
                )}
              </button>
            </form>
          </div>
          {j && (
            <div
              className={`mt-3 p-3.5 rounded-2xl border text-xs space-y-2 ${j.is_debarred ? 'bg-rose-50 border
-rose-300 text-rose-950' : 'bg-pine-50/90 border-pine-200 text-pine-950'}`}
            >
              <div className="flex justify-between items-start">
                <span className="font-bold text-[11px]">{m}</span>
                <span
                  className={`text-[9px] font-bold px-2 py-0.5 rounded-full uppercase tracking-wider ${j.is_de
barred ? 'bg-rose-600 text-white animate-pulse' : 'bg-pine-200 text-pine-900'}`}
                >
                  {j.is_debarred ? 'CRITICAL: DEBARRED' : 'CLEAR / NO RECORD'}
                </span>
              </div>
              {j.is_debarred &&
              (P = (R = j.matches) == null ? undefined : R[0]) != null &&
              P.record ? (
                <div className="space-y-1 text-[11px]">
                  <p>
                    <strong className="text-rose-900">Debarring Agency:</strong>{' '}
                    {j.matches[0].record.debarring_agency}
                  </p>
                  <p>
                    <strong className="text-rose-900">Order No:</strong>{' '}
                    {j.matches[0].record.order_number}
                  </p>
                  <p>
                    <strong className="text-rose-900">Period:</strong>{' '}
                    {j.matches[0].record.debarment_period}
                  </p>
                  <p>
                    <strong className="text-rose-900">Reason:</strong>{' '}
                    {j.matches[0].record.reason}
                  </p>
                  <p className="pt-1.5 border-t border-rose-200 text-[10px] text-rose-800 font-semibold">
                      Mandatory disqualification under GFR Rule 151.
                  </p>
                </div>
              ) : (
                <p className="text-[11px] leading-tight text-pine-950">
                  {j.message ||
```

```
                    'No active debarment or banning orders found across Central Ministries and PSUs.'}
                </p>
              )}
            </div>
          )}
        </div>
      </div>
    </div>
    <div className="bg-white p-6 rounded-3xl border border-slate-200/90 shadow-sm space-y-4">
      <div className="flex items-center gap-2">
        <ShieldCheckIcon className="w-5 h-5 text-pine-700" />
        <h3 className="text-sm font-bold text-slate-900">
          How 100% Guaranteed Statutory Verification Operates
        </h3>
      </div>
      <p className="text-xs text-slate-600 leading-relaxed">
        The GeM Bid Compliance AI platform utilizes a 4-layered deterministic verification
        pipeline to guarantee 100% statutory validation:
      </p>
      <div className="grid grid-cols-1 md:grid-cols-4 gap-4 pt-2">
        <div className="p-4 bg-pine-50/60 rounded-2xl border border-pine-100 space-y-1">
          <span className="text-[10px] font-bold font-mono text-pine-800 uppercase block">
            Layer 1
          </span>
          <h4 className="text-xs font-bold text-slate-900">{'Mod-36 Checksum & Syntax'}</h4>
          <p className="text-[11px] text-slate-600 leading-relaxed">
            Deterministic validation of 15-digit GSTIN (State Code + PAN + Entity + Z + Checksum)
            and 10-digit PAN syntax.
          </p>
        </div>
        <div className="p-4 bg-pine-50/60 rounded-2xl border border-pine-100 space-y-1">
          <span className="text-[10px] font-bold font-mono text-pine-800 uppercase block">
            Layer 2
          </span>
          <h4 className="text-xs font-bold text-slate-900">Live GSTN / GSP Gateway</h4>
          <p className="text-[11px] text-slate-600 leading-relaxed">
            Direct REST calls to GSTN / NIC e-Way Bill servers via authorized GSPs (NSDL, CDAC,
            API Setu) returning live filing status.
          </p>
        </div>
        <div className="p-4 bg-pine-50/60 rounded-2xl border border-pine-100 space-y-1">
          <span className="text-[10px] font-bold font-mono text-pine-800 uppercase block">
            Layer 3
          </span>
          <h4 className="text-xs font-bold text-slate-900">{'MCA21 & ITD Cross-Check'}</h4>
          <p className="text-[11px] text-slate-600 leading-relaxed">
            Cross-verifying Legal Entity Name, CIN status, Director Disqualification, and ICAI
            UDIN for CA audited turnover.
          </p>
        </div>
        <div className="p-4 bg-pine-50/60 rounded-2xl border border-pine-100 space-y-1">
          <span className="text-[10px] font-bold font-mono text-pine-800 uppercase block">
            Layer 4
          </span>
          <h4 className="text-xs font-bold text-slate-900">CPPP / CVC Debarment Filter</h4>
          <p className="text-[11px] text-slate-600 leading-relaxed">
            Exact PAN + soundex/fuzzy similarity matching against all Central Ministries and PSU
            blacklists to stop debarred vendors.
          </p>
        </div>
      </div>
    </div>
  </div>
);
};


//// FILE: src/pages/LoginPage.jsx  (841 lines)

import { useState } from 'react';
import { useNavigate } from 'react-router-dom';
import {
  ArrowRightIcon,
  CircleAlertIcon,
  CircleCheckIcon,
  EyeIcon,
  EyeOffIcon,
  KeyRoundIcon,
  LockIcon,
```

```
  LogInIcon,
  MailIcon,
  PhoneIcon,
  RefreshCwIcon,
  ShieldCheckIcon,
  UserIcon,
  UserPlusIcon,
} from 'lucide-react';
import { useAuth } from '../context/AuthContext';
import { api } from '../api';

export const LoginPage = () => {
  const { setSession: e } = useAuth(),
    t = useNavigate(),
    [r, n] = useState('signin'),
    [i, a] = useState('officer'),
    [o, l] = useState('demo123'),
    [c, u] = useState('credentials'),
    [f, d] = useState(''),
    [p, h] = useState(''),
    [y, x] = useState(''),
    [g, v] = useState(''),
    [m, b] = useState(''),
    [j, w] = useState(''),
    [_, S] = useState('credentials'),
    [O, A] = useState(''),
    [E, T] = useState(''),
    [C, I] = useState(''),
    [R, P] = useState(false),
    [$, D] = useState(''),
    [M, L] = useState(''),
    [U, F] = useState(false),
    [V, X] = useState(false),
    [se, be] = useState(''),
    [De, lt] = useState(false),
    [Ge, H] = useState(''),
    [J, ne] = useState(''),
    [z, pe] = useState('email'),
    [ie, Re] = useState(''),
    [Pe, ct] = useState(''),
    [qt, Ur] = useState(''),
    [yn, zr] = useState(false),
    [currentOtp, setCurrentOtp] = useState(''),
    Nr = (Z) => {
      (n(Z),
        u('credentials'),
        S('credentials'),
        A(''),
        T(''),
        I(''),
        D(''),
        L(''),
        be(''),
        lt(false),
        pe('email'),
        H(''),
        ne(''),
        Re(''),
        ct(''),
        Ur(''),
        setCurrentOtp(''));
    },
    mo = async (Z) => {
      Z.preventDefault();
      try {
        (P(true), D(''));
        const loginResult = await api.login({ username: i, password: o });
        if (loginResult && loginResult.access_token && loginResult.user) {
          (e(loginResult.user, loginResult.access_token), t('/'));
          return;
        }
        const te = await api.sendOtp({ username: i, method: 'email' });
        const ti = te.email_masked || te.masked_email || '';
        const genOtp = te.otp || sessionStorage.getItem('gem_last_otp') || '';
        (setCurrentOtp(genOtp),
          I(ti || 'your registered e-mail'),
          be(i),
          u('otp'),
          L(
```

```
        `Credentials verified. A secure 6-digit OTP has been generated & dispatched to ${ti || 'your registe
red email'}.`,
      ));
  } catch (te) {
    D(te.message || 'Login failed');
  } finally {
    P(false);
  }
},
jl = async (Z) => {
  Z.preventDefault();
  try {
    if (!O || O.length < 6) {
      (D('Please enter the complete 6-digit OTP.'), P(false));
      return;
    }
    (P(true), D(''));
    const te = await api.verifyOtp({ username: se, otp: O });
    te.access_token && te.user && (e(te.user, te.access_token), t('/'));
  } catch (te) {
    D(te.message || 'OTP verification failed');
  } finally {
    P(false);
  }
},
Sl = async () => {
  try {
    (P(true), D(''), A(''));
    const Z = await api.sendOtp({ username: se, method: 'email' });
    const te = Z.email_masked || Z.masked_email || '';
    const genOtp = Z.otp || sessionStorage.getItem('gem_last_otp') || '';
    (setCurrentOtp(genOtp),
      I(te || 'your registered e-mail'),
      L('New verification OTP generated and dispatched.'));
  } catch (Z) {
    D(Z.message || 'Failed to resend OTP');
  } finally {
    P(false);
  }
},
GO = async (Z) => {
  Z.preventDefault();
  try {
    (P(true), D(''));
    const te = {};
    (J && (te.username = J), Ge && (te.email = Ge));
    const ti = await api.sendOtp(te),
      Nl = ti.email_masked || ti.masked_email || '';
    const genOtp = ti.otp || sessionStorage.getItem('gem_last_otp') || '';
    (setCurrentOtp(genOtp),
      I(Nl || 'your registered e-mail'),
      pe('otp'),
      L('Password reset OTP generated & sent to your registered email.'));
  } catch (te) {
    D(te.message || 'Could not send reset OTP. Please check your email or username.');
  } finally {
    P(false);
  }
},
WO = async (Z) => {
  Z.preventDefault();
  try {
    if ((P(true), D(''), !ie || ie.length < 6)) {
      (D('Please enter the 6-digit OTP from your email.'), P(false));
      return;
    }
    const te = { otp: ie };
    (J && (te.username = J), Ge && (te.email = Ge));
    // The server checks the OTP when the new password is submitted.
    (pe('newpass'), D(''), L('OTP verified! Please set your new password.'));
  } catch (te) {
    D(te.message || 'OTP verification failed.');
  } finally {
    P(false);
  }
},
HO = async (Z) => {
  Z.preventDefault();
  try {
```

```
    if ((P(true), D(''), Pe.length < 6)) {
      (D('Password must be at least 6 characters.'), P(false));
      return;
    }
    if (Pe !== qt) {
      (D('Passwords do not match. Please re-enter.'), P(false));
      return;
    }
    const te = { otp: ie, new_password: Pe };
    (J && (te.username = J),
      Ge && (te.email = Ge),
      await api.resetPassword(te),
      pe('done'),
      L('Password updated successfully! You may now sign in with your new password.'));
  } catch (te) {
    D(te.message || 'Failed to reset password.');
  } finally {
    P(false);
  }
},
qO = async (Z) => {
  Z.preventDefault();
  try {
    if ((P(true), D(''), m !== j)) {
      (D('Passwords do not match.'), P(false));
      return;
    }
    if (m.length < 6) {
      (D('Password must be at least 6 characters.'), P(false));
      return;
    }
    await api.register({ username: p, password: m, full_name: f, email: y, phone: g });
    const te = await api.sendOtp({ username: p, email: y, method: 'email' });
    const genOtp = te.otp || sessionStorage.getItem('gem_last_otp') || '';
    (setCurrentOtp(genOtp),
      I(te.email_masked || te.masked_email || y),
      be(p),
      S('otp'),
      L(`Officer profile registered! Verification OTP dispatched to ${y}.`));
  } catch (te) {
    D(te.message || 'Registration failed');
  } finally {
    P(false);
  }
},
KO = async (Z) => {
  Z.preventDefault();
  try {
    if (!O || O.length < 6) {
      (D('Please enter the complete 6-digit OTP.'), P(false));
      return;
    }
    (P(true), D(''));
    const te = await api.verifyOtp({ username: se, otp: O });
    te.access_token && te.user && (e(te.user, te.access_token), t('/'));
  } catch (te) {
    D(te.message || 'OTP verification failed');
  } finally {
    P(false);
  }
},
Jv = (Z) => (
  <form onSubmit={Z} className="p-6 space-y-4">
    {$ && (
      <div className="p-3 bg-rose-50 text-rose-800 rounded-xl text-xs font-semibold border border-rose-200 f
lex items-center gap-2">
        <CircleAlertIcon className="w-4 h-4 text-rose-600 flex-shrink-0" />
        <span>{$}</span>
      </div>
    )}
    {M && (
      <div className="p-3 bg-teal-50 text-teal-900 rounded-xl text-xs font-semibold border border-teal-200 f
lex items-center gap-2">
        <CircleCheckIcon className="w-4 h-4 text-teal-600 flex-shrink-0" />
        <span>{M}</span>
      </div>
    )}
    <div className="text-center space-y-2">
      <div className="w-14 h-14 rounded-2xl bg-gradient-to-br from-pine-700 to-teal-800 flex items-center ju
```

```
  stify-center mx-auto shadow-glow-teal">
          <ShieldCheckIcon className="w-7 h-7 text-white" />
        </div>
        <h3 className="text-sm font-bold text-slate-800">Two-Factor Authentication (2FA)</h3>
        <p className="text-[11px] text-slate-500">
          {'Enter the 6-digit OTP sent to '}
          <strong className="text-slate-700">{C || 'your email'}</strong>
        </p>
      </div>
      <div>
        <div className="flex justify-between items-center mb-1">
          <label className="block text-xs font-bold uppercase tracking-wider text-slate-700">
            One-Time Password (OTP)
          </label>
          <span className="text-[10px] text-slate-400 font-mono">6-digit numeric</span>
        </div>
        <div className="relative">
          <KeyRoundIcon className="w-4 h-4 text-slate-400 absolute left-3 top-3" />
          <input
            type="text"
            required
            maxLength={6}
            value={O}
            onChange={(te) => A(te.target.value.replace(/\D/g, ''))}
            className="w-full pl-9 pr-3 py-2.5 text-center text-lg font-mono font-bold tracking-[0.5em] border
   border-slate-300 rounded-xl bg-slate-50 focus:bg-white focus:ring-2 focus:ring-teal-600 focus:outline-none
  transition-colors"
            placeholder="● ● ● ● ● ●"
            autoFocus
          />
        </div>
      </div>
      <button
        type="submit"
        disabled={R || !O || O.length < 6}
        className="w-full py-3 bg-gradient-to-r from-pine-800 to-teal-800 hover:from-pine-900 hover:to-teal-90
  0 disabled:bg-slate-300 text-white text-xs font-bold rounded-xl shadow-glow-teal transition-all flex items-c
  enter justify-center gap-2 group"
      >
        {R ? (
          'Verifying...'
        ) : (
          <>
            <ShieldCheckIcon className="w-4 h-4 text-champagne-300" />
            <span>{'Verify & Enter Console'}</span>
            <ArrowRightIcon className="w-4 h-4 group-hover:translate-x-0.5 transition-transform" />
          </>
        )}
      </button>
      <div className="flex items-center justify-between text-[11px] pt-1">
        <button
          type="button"
          onClick={Sl}
          disabled={R}
          className="text-pine-700 hover:underline flex items-center gap-1 font-semibold"
        >
          <RefreshCwIcon className="w-3 h-3" />
          {' Resend Code'}
        </button>
        <button
          type="button"
          onClick={() => {
            (r === 'signin' ? u('credentials') : S('credentials'), A(''), D(''), L(''));
          }}
          className="text-slate-500 hover:underline font-medium"
        >
          ← Back
        </button>
      </div>
    </form>
  );
return (
  <div className="min-h-screen bg-gradient-to-br from-[#040C16] via-[#081726] to-[#0A2033] flex flex-col justi
  fy-between items-center p-4">
    <div className="w-full max-w-5xl py-4 flex justify-between items-center text-slate-300 text-xs border-b bo
  rder-slate-800">
      <div className="flex items-center gap-2">
        <span className="font-bold uppercase tracking-wider text-champagne-400">
          Government of India
```

```
      </span>
      <span className="text-slate-600">|</span>
      <span className="text-slate-300">GeM Technical Evaluation Cell</span>
    </div>
    <div className="flex items-center gap-2 text-pine-300 font-mono text-[11px]">
      <span className="w-2 h-2 rounded-full bg-pine-400 animate-pulse" />
      <span>GFR 2017 Secure Sandbox</span>
    </div>
  </div>
  <div className="w-full max-w-md my-auto">
    <div className="bg-white rounded-3xl shadow-2xl border border-slate-200 overflow-hidden">
      <div className="gradient-obsidian-header p-8 text-center text-white relative">
        <div className="w-16 h-16 rounded-2xl bg-gradient-to-br from-[#0F2620] via-[#0A1A24] to-[#040E14] bo
rder-2 border-champagne-400/50 flex items-center justify-center mx-auto shadow-xl mb-4">
          <svg
            className="w-10 h-10 text-champagne-400"
            viewBox="0 0 24 24"
            fill="none"
            xmlns="http://www.w3.org/2000/svg"
          >
            <path
              d="M12 2L3 6V11C3 16.55 6.84 21.74 12 23C17.16 21.74 21 16.55 21 11V6L12 2Z"
              fill="url(#login-shield)"
              stroke="#F59E0B"
              strokeWidth="1.5"
              strokeLinejoin="round"
            />
            <path
              d="M9 12L11 14L15 10"
              stroke="#FFFFFF"
              strokeWidth="2"
              strokeLinecap="round"
              strokeLinejoin="round"
            />
            <circle cx="12" cy="7" r="1.2" fill="#14B8A6" />
            <defs>
              <linearGradient
                id="login-shield"
                x1="12"
                y1="2"
                x2="12"
                y2="23"
                gradientUnits="userSpaceOnUse"
              >
                <stop stopColor="#0D3830" />
                <stop offset="1" stopColor="#08231E" />
              </linearGradient>
            </defs>
          </svg>
        </div>
        <div className="flex items-center justify-center gap-2">
          <h1 className="text-2xl font-black tracking-tight">
            {'GeM '}
            <span className="text-champagne-400">BidVerify</span>
          </h1>
          <span className="bg-pine-800 text-pine-200 text-[10px] font-black px-2 py-0.5 rounded-md uppercase
 tracking-wider font-mono shadow-sm border border-pine-600/40">
            AI
          </span>
        </div>
        <p className="text-xs text-slate-300 mt-1.5 font-medium">
          {'National Public Procurement Compliance & Technical Scrutiny Gateway'}
        </p>
        <div className="mt-4 inline-flex items-center gap-1.5 px-3 py-1 rounded-full bg-slate-900/90 text-[1
1px] text-champagne-300 font-semibold border border-slate-700 shadow-sm">
          <LockIcon className="w-3.5 h-3.5 text-champagne-400" />
          Secure Officer Sign-in
        </div>
      </div>
      <div className="flex border-b border-slate-200">
        <button
          onClick={() => Nr('signin')}
          className={`flex-1 py-3 text-xs font-bold uppercase tracking-wider flex items-center justify-cente
r gap-1.5 transition-colors ${r === 'signin' ? 'text-pine-800 border-b-2 border-pine-700 bg-pine-50/50' : 't
ext-slate-400 hover:text-slate-600 hover:bg-slate-50'}`}
        >
          <LogInIcon className="w-3.5 h-3.5" />
          Sign In
        </button>
```

```
        <button
          onClick={() => Nr('register')}
          className={`flex-1 py-3 text-xs font-bold uppercase tracking-wider flex items-center justify-cente
r gap-1.5 transition-colors ${r === 'register' ? 'text-champagne-800 border-b-2 border-champagne-600 bg-cham
pagne-50/50' : 'text-slate-400 hover:text-slate-600 hover:bg-slate-50'}`}
        >
          <UserPlusIcon className="w-3.5 h-3.5" />
          Create Account
        </button>
      </div>
      {r === 'signin' && c === 'credentials' && (
        <form onSubmit={mo} className="p-6 space-y-4">
          {$ && (
            <div className="p-3 bg-rose-50 text-rose-800 rounded-xl text-xs font-semibold border border-rose
-200 flex items-center gap-2">
              <CircleAlertIcon className="w-4 h-4 text-rose-600 flex-shrink-0" />
              <span>{$}</span>
            </div>
          )}
          <div>
            <label className="block text-xs font-bold uppercase tracking-wider text-slate-700 mb-1">
              Official Username / User ID
            </label>
            <div className="relative">
              <UserIcon className="w-4 h-4 text-slate-400 absolute left-3 top-3" />
              <input
                type="text"
                required
                value={i}
                onChange={(Z) => a(Z.target.value)}
                className="w-full pl-9 pr-3 py-2 text-xs border border-slate-300 rounded-xl bg-slate-50 focu
s:bg-white focus:ring-2 focus:ring-teal-600 focus:outline-none transition-colors"
                placeholder="e.g. officer"
              />
            </div>
          </div>
          <div>
            <label className="block text-xs font-bold uppercase tracking-wider text-slate-700 mb-1">
              Password / Passcode
            </label>
            <div className="relative">
              <LockIcon className="w-4 h-4 text-slate-400 absolute left-3 top-3" />
              <input
                type={U ? 'text' : 'password'}
                required
                value={o}
                onChange={(Z) => l(Z.target.value)}
                className="w-full pl-9 pr-9 py-2 text-xs border border-slate-300 rounded-xl bg-slate-50 focu
s:bg-white focus:ring-2 focus:ring-teal-600 focus:outline-none transition-colors"
                placeholder="••••••••"
              />
              <button
                type="button"
                onClick={() => F(!U)}
                className="absolute right-3 top-2.5 text-slate-400 hover:text-slate-600"
              >
                {U ? <EyeOffIcon className="w-4 h-4" /> : <EyeIcon className="w-4 h-4" />}
              </button>
            </div>
          </div>
          <button
            type="submit"
            disabled={R}
            className="w-full py-3 bg-gradient-to-r from-pine-800 to-teal-800 hover:from-pine-900 hover:to-t
eal-900 text-white text-xs font-bold rounded-xl shadow-glow-teal transition-all flex items-center justify-ce
nter gap-2 group"
          >
            {R ? (
              'Authenticating...'
            ) : (
              <>
                <span>{'Sign In'}</span>
                <ArrowRightIcon className="w-4 h-4 group-hover:translate-x-0.5 transition-transform" />
              </>
            )}
          </button>
          <div className="text-center pt-1">
            <button
              type="button"
```

```
              onClick={() => {
                (lt(true), D(''), L(''), pe('email'));
              }}
              className="text-[11px] text-pine-700 hover:text-pine-900 hover:underline font-semibold transit
ion-colors"
            >
              Forgot Password?
            </button>
          </div>
        </form>
      )}
      {r === 'signin' && c === 'otp' && Jv(jl)}
      {r === 'signin' && c === 'credentials' && De && (
        <div className="fixed inset-0 z-50 flex items-center justify-center bg-black/60 backdrop-blur-sm p-4
">
          <div className="bg-white rounded-3xl shadow-2xl border border-slate-200 w-full max-w-sm overflow-h
idden">
            <div className="gradient-obsidian-header p-6 text-center text-white">
              <div className="w-12 h-12 rounded-2xl bg-gradient-to-br from-[#0F2620] to-[#040E14] border bor
der-champagne-400/40 flex items-center justify-center mx-auto mb-3 shadow-lg">
                <KeyRoundIcon className="w-6 h-6 text-champagne-400" />
              </div>
              <h3 className="text-base font-black tracking-tight">Reset Password</h3>
              <p className="text-[11px] text-slate-300 mt-1">
                {z === 'email' && 'Enter your registered email or username'}
                {z === 'otp' && `Enter the OTP sent to ${C || 'your email'}`}
                {z === 'newpass' && 'Create your new secure password'}
                {z === 'done' && 'Password reset complete!'}
              </p>
            </div>
            <div className="p-6 space-y-4">
              {$ && (
                <div className="p-3 bg-rose-50 text-rose-800 rounded-xl text-xs font-semibold border border-
rose-200 flex items-center gap-2">
                  <CircleAlertIcon className="w-4 h-4 text-rose-600 flex-shrink-0" />
                  <span>{$}</span>
                </div>
              )}
              {M && (
                <div className="p-3 bg-teal-50 text-teal-900 rounded-xl text-xs font-semibold border border-
teal-200 flex items-center gap-2">
                  <CircleCheckIcon className="w-4 h-4 text-teal-600 flex-shrink-0" />
                  <span>{M}</span>
                </div>
              )}
              {z === 'email' && (
                <form onSubmit={GO} className="space-y-3">
                  <div>
                    <label className="block text-xs font-bold uppercase tracking-wider text-slate-700 mb-1">
                      Username
                    </label>
                    <div className="relative">
                      <UserIcon className="w-4 h-4 text-slate-400 absolute left-3 top-2.5" />
                      <input
                        type="text"
                        value={J}
                        onChange={(Z) => ne(Z.target.value)}
                        className="w-full pl-9 pr-3 py-2 text-xs border border-slate-300 rounded-xl bg-slate
-50 focus:bg-white focus:ring-2 focus:ring-teal-600 focus:outline-none"
                        placeholder="e.g. officer"
                      />
                    </div>
                  </div>
                  <div className="flex items-center gap-2 text-[10px] text-slate-400">
                    <div className="flex-grow border-t border-slate-200" />
                    <span className="font-mono uppercase">or</span>
                    <div className="flex-grow border-t border-slate-200" />
                  </div>
                  <div>
                    <label className="block text-xs font-bold uppercase tracking-wider text-slate-700 mb-1">
                      Registered Email
                    </label>
                    <div className="relative">
                      <MailIcon className="w-4 h-4 text-slate-400 absolute left-3 top-2.5" />
                      <input
                        type="email"
                        value={Ge}
                        onChange={(Z) => H(Z.target.value)}
                        className="w-full pl-9 pr-3 py-2 text-xs border border-slate-300 rounded-xl bg-slate
```

```
-50 focus:bg-white focus:ring-2 focus:ring-teal-600 focus:outline-none"
                        placeholder="your.email@gmail.com"
                      />
                    </div>
                  </div>
                  <button
                    type="submit"
                    disabled={R || (!Ge && !J)}
                    className="w-full py-2.5 bg-gradient-to-r from-pine-800 to-teal-800 hover:from-pine-900
hover:to-teal-900 disabled:bg-slate-300 text-white text-xs font-bold rounded-xl transition-all flex items-ce
nter justify-center gap-2"
                  >
                    {R ? (
                      'Sending OTP...'
                    ) : (
                      <>
                        <MailIcon className="w-3.5 h-3.5" />
                        <span>Send Reset OTP</span>
                      </>
                    )}
                  </button>
                </form>
              )}
              {z === 'otp' && (
                <form onSubmit={WO} className="space-y-3">
                  <div>
                    <label className="block text-xs font-bold uppercase tracking-wider text-slate-700 mb-1">
                      6-Digit OTP
                    </label>
                    <div className="relative">
                      <KeyRoundIcon className="w-4 h-4 text-slate-400 absolute left-3 top-3" />
                      <input
                        type="text"
                        required
                        maxLength={6}
                        value={ie}
                        onChange={(Z) => Re(Z.target.value.replace(/\D/g, ''))}
                        className="w-full pl-9 pr-3 py-2.5 text-center text-lg font-mono font-bold tracking-
[0.5em] border border-slate-300 rounded-xl bg-slate-50 focus:bg-white focus:ring-2 focus:ring-teal-600 focus
:outline-none"
                        placeholder="● ● ● ● ● ●"
                        autoFocus
                      />
                    </div>
                  </div>
                  <button
                    type="submit"
                    disabled={R || ie.length < 6}
                    className="w-full py-2.5 bg-gradient-to-r from-pine-800 to-teal-800 hover:from-pine-900
hover:to-teal-900 disabled:bg-slate-300 text-white text-xs font-bold rounded-xl transition-all flex items-ce
nter justify-center gap-2"
                  >
                    {R ? (
                      'Verifying...'
                    ) : (
                      <>
                        <ShieldCheckIcon className="w-3.5 h-3.5" />
                        <span>Verify OTP</span>
                      </>
                    )}
                  </button>
                  <button
                    type="button"
                    onClick={() => {
                      (pe('email'), Re(''), D(''), L(''));
                    }}
                    className="w-full text-[11px] text-slate-500 hover:text-slate-700 hover:underline text-c
enter"
                  >
                    ← Back / Resend OTP
                  </button>
                </form>
              )}
              {z === 'newpass' && (
                <form onSubmit={HO} className="space-y-3">
                  <div>
                    <label className="block text-xs font-bold uppercase tracking-wider text-slate-700 mb-1">
                      New Password
                    </label>
```

```
                    <div className="relative">
                      <LockIcon className="w-4 h-4 text-slate-400 absolute left-3 top-2.5" />
                      <input
                        type={yn ? 'text' : 'password'}
                        required
                        value={Pe}
                        onChange={(Z) => ct(Z.target.value)}
                        className="w-full pl-9 pr-9 py-2 text-xs border border-slate-300 rounded-xl bg-slate
-50 focus:bg-white focus:ring-2 focus:ring-teal-600 focus:outline-none"
                        placeholder="Min 6 characters"
                        autoFocus
                      />
                      <button
                        type="button"
                        onClick={() => zr(!yn)}
                        className="absolute right-3 top-2.5 text-slate-400 hover:text-slate-600"
                      >
                        {yn ? (
                          <EyeOffIcon className="w-4 h-4" />
                        ) : (
                          <EyeIcon className="w-4 h-4" />
                        )}
                      </button>
                    </div>
                  </div>
                  <div>
                    <label className="block text-xs font-bold uppercase tracking-wider text-slate-700 mb-1">
                      Confirm New Password
                    </label>
                    <div className="relative">
                      <LockIcon className="w-4 h-4 text-slate-400 absolute left-3 top-2.5" />
                      <input
                        type="password"
                        required
                        value={qt}
                        onChange={(Z) => Ur(Z.target.value)}
                        className="w-full pl-9 pr-3 py-2 text-xs border border-slate-300 rounded-xl bg-slate
-50 focus:bg-white focus:ring-2 focus:ring-teal-600 focus:outline-none"
                        placeholder="Re-enter new password"
                      />
                    </div>
                  </div>
                  <button
                    type="submit"
                    disabled={R || !Pe || !qt}
                    className="w-full py-2.5 bg-gradient-to-r from-pine-800 to-teal-800 hover:from-pine-900
hover:to-teal-900 disabled:bg-slate-300 text-white text-xs font-bold rounded-xl transition-all flex items-ce
nter justify-center gap-2"
                  >
                    {R ? (
                      'Updating...'
                    ) : (
                      <>
                        <ShieldCheckIcon className="w-3.5 h-3.5" />
                        <span>Update Password</span>
                      </>
                    )}
                  </button>
                </form>
              )}
              {z === 'done' && (
                <div className="text-center space-y-4 py-2">
                  <div className="w-14 h-14 rounded-full bg-teal-100 border-2 border-teal-400 flex items-cen
ter justify-center mx-auto">
                    <CircleCheckIcon className="w-8 h-8 text-teal-600" />
                  </div>
                  <p className="text-xs text-slate-600 font-medium">
                    Your password has been updated. Please sign in with your new credentials.
                  </p>
                  <button
                    type="button"
                    onClick={() => {
                      (lt(false), pe('email'), D(''), L(''), l(''));
                    }}
                    className="w-full py-2.5 bg-gradient-to-r from-pine-800 to-teal-800 text-white text-xs f
ont-bold rounded-xl transition-all flex items-center justify-center gap-2"
                  >
                    <LogInIcon className="w-3.5 h-3.5" />
                    <span>Go to Sign In</span>
```

```
                  </button>
                </div>
              )}
              {z !== 'done' && (
                <button
                  type="button"
                  onClick={() => {
                    (lt(false), D(''), L(''));
                  }}
                  className="w-full text-[11px] text-slate-400 hover:text-slate-600 text-center hover:underl
ine mt-1"
                >
                  Cancel — Back to Sign In
                </button>
              )}
            </div>
          </div>
        </div>
      )}
      {r === 'register' && _ === 'credentials' && (
        <form onSubmit={qO} className="p-6 space-y-3">
          {$ && (
            <div className="p-3 bg-rose-50 text-rose-800 rounded-xl text-xs font-semibold border border-rose
-200 flex items-center gap-2">
              <CircleAlertIcon className="w-4 h-4 text-rose-600 flex-shrink-0" />
              <span>{$}</span>
            </div>
          )}
          <div>
            <label className="block text-xs font-bold uppercase tracking-wider text-slate-700 mb-1">
              Full Name
            </label>
            <div className="relative">
              <UserIcon className="w-4 h-4 text-slate-400 absolute left-3 top-2.5" />
              <input
                type="text"
                required
                value={f}
                onChange={(Z) => d(Z.target.value)}
                className="w-full pl-9 pr-3 py-2 text-xs border border-slate-300 rounded-xl bg-slate-50 focu
s:bg-white focus:ring-2 focus:ring-teal-600 focus:outline-none"
                placeholder="e.g. Amit Sharma"
              />
            </div>
          </div>
          <div>
            <label className="block text-xs font-bold uppercase tracking-wider text-slate-700 mb-1">
              Username
            </label>
            <div className="relative">
              <UserIcon className="w-4 h-4 text-slate-400 absolute left-3 top-2.5" />
              <input
                type="text"
                required
                value={p}
                onChange={(Z) => h(Z.target.value)}
                className="w-full pl-9 pr-3 py-2 text-xs border border-slate-300 rounded-xl bg-slate-50 focu
s:bg-white focus:ring-2 focus:ring-teal-600 focus:outline-none"
                placeholder="Choose unique username"
              />
            </div>
          </div>
          <div>
            <label className="block text-xs font-bold uppercase tracking-wider text-slate-700 mb-1">
              Official Government Email
            </label>
            <div className="relative">
              <MailIcon className="w-4 h-4 text-slate-400 absolute left-3 top-2.5" />
              <input
                type="email"
                required
                value={y}
                onChange={(Z) => x(Z.target.value)}
                className="w-full pl-9 pr-3 py-2 text-xs border border-slate-300 rounded-xl bg-slate-50 focu
s:bg-white focus:ring-2 focus:ring-teal-600 focus:outline-none"
                placeholder="name@gem.gov.in"
              />
            </div>
          </div>
```

```
          <div>
            <label className="block text-xs font-bold uppercase tracking-wider text-slate-700 mb-1">
              Phone Number (Optional)
            </label>
            <div className="relative">
              <PhoneIcon className="w-4 h-4 text-slate-400 absolute left-3 top-2.5" />
              <input
                type="tel"
                value={g}
                onChange={(Z) => v(Z.target.value)}
                className="w-full pl-9 pr-3 py-2 text-xs border border-slate-300 rounded-xl bg-slate-50 focu
s:bg-white focus:ring-2 focus:ring-teal-600 focus:outline-none"
                placeholder="+91 98200 XXXXX"
              />
            </div>
          </div>
          <div className="grid grid-cols-2 gap-3">
            <div>
              <label className="block text-xs font-bold uppercase tracking-wider text-slate-700 mb-1">
                Password
              </label>
              <div className="relative">
                <LockIcon className="w-4 h-4 text-slate-400 absolute left-3 top-2.5" />
                <input
                  type={V ? 'text' : 'password'}
                  required
                  value={m}
                  onChange={(Z) => b(Z.target.value)}
                  className="w-full pl-9 pr-8 py-2 text-xs border border-slate-300 rounded-xl bg-slate-50 fo
cus:bg-white focus:ring-2 focus:ring-teal-600 focus:outline-none"
                  placeholder="Min 6 chars"
                />
                <button
                  type="button"
                  onClick={() => X(!V)}
                  className="absolute right-2 top-2 text-slate-400 hover:text-slate-600"
                >
                  {V ? (
                    <EyeOffIcon className="w-3.5 h-3.5" />
                  ) : (
                    <EyeIcon className="w-3.5 h-3.5" />
                  )}
                </button>
              </div>
            </div>
            <div>
              <label className="block text-xs font-bold uppercase tracking-wider text-slate-700 mb-1">
                Confirm
              </label>
              <div className="relative">
                <LockIcon className="w-4 h-4 text-slate-400 absolute left-3 top-2.5" />
                <input
                  type="password"
                  required
                  value={j}
                  onChange={(Z) => w(Z.target.value)}
                  className="w-full pl-9 pr-3 py-2 text-xs border border-slate-300 rounded-xl bg-slate-50 fo
cus:bg-white focus:ring-2 focus:ring-teal-600 focus:outline-none"
                  placeholder="Re-enter"
                />
              </div>
            </div>
          </div>
          <button
            type="submit"
            disabled={R}
            className="w-full py-3 bg-gradient-to-r from-pine-800 to-teal-800 hover:from-pine-900 hover:to-t
eal-900 text-white text-xs font-bold rounded-xl shadow-glow-teal transition-all flex items-center justify-ce
nter gap-2 group mt-1"
          >
            {R ? (
              'Creating Account...'
            ) : (
              <>
                <UserPlusIcon className="w-4 h-4" />
                <span>{'Create Profile & Verify Email'}</span>
                <ArrowRightIcon className="w-4 h-4 group-hover:translate-x-0.5 transition-transform" />
              </>
            )}
```

```
              </button>
            </form>
          )}
          {r === 'register' && _ === 'otp' && Jv(KO)}
          <div className="p-4 bg-slate-50 border-t border-slate-100 text-center text-[10.5px] text-slate-500 lea
    ding-tight">
            Secure officer sign-in. All authentication attempts are logged for CVC vigilance review.
          </div>
        </div>
      </div>
      <footer className="w-full max-w-5xl py-3 text-center text-[11px] text-slate-400 border-t border-slate-800/
    80">
        {'AI-Powered Public Procurement Compliance & Verification Platform • Smart India Hackathon'}
      </footer>
    </div>
  );
};


//// FILE: src/pages/TendersPage.jsx  (687 lines)

import { useEffect, useState } from 'react';
import { Link } from 'react-router-dom';
import {
  Building2Icon,
  ChevronRightIcon,
  ClockIcon,
  FileTextIcon,
  PlusIcon,
  SearchIcon,
  Settings2Icon,
  SlidersVerticalIcon,
  Trash2Icon,
  XIcon,
} from 'lucide-react';
import { api } from '../api';

const defaultRequirementTemplates = [
  {
    code: 'PAN',
    name: 'Permanent Account Number (PAN) Card',
    mandatory: true,
    source: 'ITD_PAN_REGISTRY',
    min_cond: 'Valid active PAN in bidder name',
    weight: 10,
  },
  {
    code: 'GST',
    name: 'Goods & Services Tax (GST) Registration',
    mandatory: true,
    source: 'GSTN_PORTAL',
    min_cond: 'Active GSTIN with regular filing',
    weight: 15,
  },
  {
    code: 'UDYAM',
    name: 'MSME / Udyam Registration Certificate',
    mandatory: false,
    source: 'UDYAM_PORTAL',
    min_cond: 'Valid Udyam reg for purchase preference',
    weight: 10,
  },
  {
    code: 'ITR',
    name: 'Income Tax Returns & CA Turnover (Last 3 FYs)',
    mandatory: true,
    source: 'ITD_ITR_ICAI_UDIN',
    min_cond: 'Min average turnover ₹1.5 Cr with valid UDIN',
    weight: 15,
  },
  {
    code: 'OEM_AUTH',
    name: 'Manufacturer Authorization Form (MAF / OEM Auth)',
    mandatory: true,
    source: 'OEM_VERIFICATION',
    min_cond: 'Direct authorization letter addressed to tender',
    weight: 15,
  },
  {
```

```
    code: 'MAKE_IN_INDIA',
    name: 'Make in India (MII) Local Content Declaration',
    mandatory: true,
    source: 'DPIIT_MII_DECLARATION',
    min_cond: 'Class-I Local Supplier (>=50% local content)',
    weight: 15,
  },
  {
    code: 'EPFO',
    name: 'EPFO Registration & ECR Electronic Challan',
    mandatory: false,
    source: 'EPFO_PORTAL',
    min_cond: 'Active establishment code',
    weight: 5,
  },
  {
    code: 'ESIC',
    name: 'ESIC Registration & Compliance Declaration',
    mandatory: false,
    source: 'ESIC_PORTAL',
    min_cond: 'Active ESIC employer registration',
    weight: 5,
  },
  {
    code: 'DEBARMENT',
    name: 'Non-Blacklisting / Debarment Affidavit',
    mandatory: true,
    source: 'CPPP_GEM_DEBARMENT_LIST',
    min_cond: 'No debarment by any Central/State Govt/PSU',
    weight: 10,
  },
  {
    code: 'BIS',
    name: 'BIS Quality Standard / CRS Certification',
    mandatory: false,
    source: 'BIS_PORTAL',
    min_cond: 'Valid IS/ISO Standard certification',
    weight: 10,
  },
  {
    code: 'STARTUP',
    name: 'DPIIT Startup Recognition Certificate',
    mandatory: false,
    source: 'STARTUP_INDIA_PORTAL',
    min_cond: 'Valid DIPP recognition for EMD exemption',
    weight: 5,
  },
];

const RequirementsEditor = ({ requirements: e, onChange: t }) => {
  const [r, n] = useState(''),
    [i, a] = useState(''),
    [o, l] = useState(true),
    [c, u] = useState(10),
    [f, d] = useState(''),
    p = (g) => {
      const v = e.some((m) => m.requirement_code === g.code);
      t(
        v
          ? e.filter((m) => m.requirement_code !== g.code)
          : [
              ...e,
              {
                requirement_code: g.code,
                requirement_name: g.name,
                is_mandatory: g.mandatory,
                verification_source: g.source,
                min_condition: g.min_cond,
                weight: g.weight,
                applicable_bidder_type: 'ALL',
              },
            ],
      );
    },
    h = (g) => {
      (g.preventDefault(),
        !(!r || !i) &&
          (t([
            ...e,
```

```
          {
            requirement_code: r.toUpperCase().replace(/\s+/g, '_'),
            requirement_name: i,
            is_mandatory: o,
            verification_source: 'PORTAL_CROSS_CHECK',
            min_condition: f || undefined,
            weight: c,
            applicable_bidder_type: 'ALL',
          },
        ]),
        n(''),
        a(''),
        d('')));
  },
  y = (g) => {
    t(e.filter((v) => v.requirement_code !== g));
  },
  x = e.reduce((g, v) => g + Number(v.weight), 0);
return (
  <div className="space-y-5">
    <div>
      <div className="flex justify-between items-center mb-3">
        <label className="text-xs font-bold uppercase tracking-wider text-slate-700 flex items-center gap-1.5"
  >
          <SlidersVerticalIcon className="w-3.5 h-3.5 text-pine-700" />
          Standard Statutory Criteria (Click to Toggle)
        </label>
        <span
          className={`text-xs font-mono font-bold px-3 py-1 rounded-full ${x === 100 ? 'bg-pine-100 text-pine-
  800' : 'bg-champagne-100 text-champagne-800'}`}
        >
          {'Total Scoring Weight: '}
          {x}
          {' pts'}
        </span>
      </div>
      <div className="grid grid-cols-2 sm:grid-cols-3 md:grid-cols-4 gap-2.5">
        {defaultRequirementTemplates.map((g) => {
          const v = e.some((m) => m.requirement_code === g.code);
          return (
            <button
              key={g.code}
              type="button"
              onClick={() => p(g)}
              className={`p-3 text-left rounded-2xl border text-xs transition-all flex flex-col justify-betwee
  n ${v ? 'bg-pine-50/90 border-pine-600 text-pine-950 shadow-sm font-semibold ring-1 ring-pine-600' : 'bg-whi
  te border-slate-200 text-slate-700 hover:bg-slate-50'}`}
            >
              <div className="flex justify-between items-start mb-1">
                <span className="font-mono text-[11px] font-bold text-champagne-800">
                  {g.code}
                </span>
                <span className="text-[10px] bg-slate-100 text-slate-700 px-1.5 py-0.5 rounded font-mono font-
  bold">
                  {g.weight}
                  {' pts'}
                </span>
              </div>
              <p className="text-[11px] leading-tight line-clamp-2 mt-0.5">{g.name}</p>
              <div className="mt-2.5 pt-1.5 border-t border-slate-100 flex justify-between text-[9.5px] text-s
  late-400 font-medium">
                <span>{g.mandatory ? 'Mandatory' : 'Optional'}</span>
                <span className={v ? 'text-pine-700 font-bold' : ''}>
                  {v ? '  Enabled' : '+ Add'}
                </span>
              </div>
            </button>
          );
        })}
      </div>
    </div>
    <div className="bg-white rounded-2xl border border-slate-200 overflow-hidden shadow-card-soft">
      <div className="px-4 py-3 bg-slate-50 border-b border-slate-200 flex justify-between items-center">
        <span className="text-xs font-bold text-slate-800">
          Active Tender Evaluation Criteria ({e.length})
        </span>
      </div>
      <div className="divide-y divide-slate-100 max-h-56 overflow-y-auto custom-scrollbar">
        {e.length === 0 ? (
```

```
        <div className="p-5 text-center text-xs text-slate-500">
          No criteria selected. Choose from presets above or add a custom requirement below.
        </div>
      ) : (
        e.map((g) => (
          <div
            key={g.requirement_code}
            className="p-3.5 flex items-center justify-between text-xs hover:bg-slate-50 transition-colors"
          >
            <div className="flex items-center gap-3 min-w-0">
              <span className="font-mono text-xs font-bold bg-slate-100 text-slate-900 px-2.5 py-0.5 rounded
-lg border border-slate-200 flex-shrink-0">
                {g.requirement_code}
              </span>
              <div className="min-w-0">
                <p className="font-bold text-slate-900 truncate">{g.requirement_name}</p>
                <p className="text-[11px] text-slate-500 truncate">
                  {g.min_condition || 'Standard verification'}
                </p>
              </div>
            </div>
            <div className="flex items-center gap-3.5 flex-shrink-0">
              <span
                className={`text-[10px] font-bold px-2 py-0.5 rounded-full ${g.is_mandatory ? 'bg-rose-100 t
ext-rose-800' : 'bg-slate-100 text-slate-700'}`}
              >
                {g.is_mandatory ? 'Mandatory' : 'Optional'}
              </span>
              <span className="font-mono font-bold text-slate-800 bg-slate-50 px-2 py-0.5 rounded border bor
der-slate-200">
                {g.weight}
                {' pts'}
              </span>
              <button
                type="button"
                onClick={() => y(g.requirement_code)}
                className="text-slate-400 hover:text-rose-600 p-1 rounded-lg transition-colors"
                title="Remove criterion"
              >
                <Trash2Icon className="w-4 h-4" />
              </button>
            </div>
          </div>
        ))
      )}
    </div>
  </div>
  <div className="p-4 bg-slate-50 rounded-2xl border border-slate-200 space-y-3">
    <h5 className="text-xs font-bold text-slate-800 flex items-center gap-1.5">
      <Settings2Icon className="w-4 h-4 text-pine-700" />
      Add Custom Tender-Specific Requirement
    </h5>
    <div className="grid grid-cols-1 sm:grid-cols-4 gap-3 text-xs">
      <input
        type="text"
        placeholder="Code (e.g. ISO_27001)"
        value={r}
        onChange={(g) => n(g.target.value)}
        className="p-2.5 border border-slate-300 rounded-xl bg-white focus:ring-2 focus:ring-teal-600 focus:
outline-none"
      />
      <input
        type="text"
        placeholder="Requirement Name"
        value={i}
        onChange={(g) => a(g.target.value)}
        className="p-2.5 border border-slate-300 rounded-xl bg-white sm:col-span-2 focus:ring-2 focus:ring-t
eal-600 focus:outline-none"
      />
      <input
        type="number"
        placeholder="Weight (pts)"
        value={c}
        onChange={(g) => u(Number(g.target.value))}
        className="p-2.5 border border-slate-300 rounded-xl bg-white font-mono focus:ring-2 focus:ring-teal-
600 focus:outline-none"
      />
    </div>
    <div className="flex flex-col sm:flex-row justify-between items-start sm:items-center gap-2 pt-1">
```

```
          <label className="flex items-center gap-2 text-xs text-slate-700 cursor-pointer select-none">
            <input
              type="checkbox"
              checked={o}
              onChange={(g) => l(g.target.checked)}
              className="rounded text-teal-600 focus:ring-teal-500"
            />
            <span className="font-medium">
              Mandatory Requirement (Non-compliance disqualifies bidder)
            </span>
          </label>
          <button
            type="button"
            onClick={h}
            className="px-4 py-2 bg-pine-800 hover:bg-pine-900 text-white rounded-xl font-bold text-xs flex item
    s-center gap-1.5 shadow-sm transition-all"
          >
            <PlusIcon className="w-3.5 h-3.5 text-champagne-300" />
            <span>Add Criterion</span>
          </button>
        </div>
      </div>
    </div>
  );
};

export const TendersPage = () => {
  const [e, t] = useState([]),
    [r, n] = useState(true),
    [i, a] = useState(false),
    [o, l] = useState(''),
    [c, u] = useState('GEM/2026/B/' + Math.floor(1e4 + Math.random() * 9e4)),
    [f, d] = useState(''),
    [p, h] = useState(''),
    [y, x] = useState(5e6),
    [g, v] = useState(new Date(Date.now() + 14 * 864e5).toISOString().split('T')[0]),
    [m, b] = useState(''),
    [j, w] = useState([
      {
        requirement_code: 'PAN',
        requirement_name: 'Permanent Account Number (PAN) Card',
        is_mandatory: true,
        verification_source: 'ITD_PAN_REGISTRY',
        min_condition: 'Valid active PAN in bidder name',
        weight: 10,
        applicable_bidder_type: 'ALL',
      },
      {
        requirement_code: 'GST',
        requirement_name: 'Goods & Services Tax (GST) Registration',
        is_mandatory: true,
        verification_source: 'GSTN_PORTAL',
        min_condition: 'Active GSTIN with regular filing',
        weight: 15,
        applicable_bidder_type: 'ALL',
      },
      {
        requirement_code: 'UDYAM',
        requirement_name: 'MSME / Udyam Registration Certificate',
        is_mandatory: false,
        verification_source: 'UDYAM_PORTAL',
        min_condition: 'Valid Udyam reg for purchase preference',
        weight: 10,
        applicable_bidder_type: 'ALL',
      },
      {
        requirement_code: 'ITR',
        requirement_name: 'Income Tax Returns & CA Turnover (Last 3 FYs)',
        is_mandatory: true,
        verification_source: 'ITD_ITR_ICAI_UDIN',
        min_condition: 'Min average turnover ₹1.5 Cr with valid UDIN',
        weight: 15,
        applicable_bidder_type: 'ALL',
      },
      {
        requirement_code: 'OEM_AUTH',
        requirement_name: 'Manufacturer Authorization Form (MAF / OEM Auth)',
        is_mandatory: true,
        verification_source: 'OEM_VERIFICATION',
```

```
      min_condition: 'Direct authorization letter addressed to tender',
      weight: 15,
      applicable_bidder_type: 'ALL',
    },
    {
      requirement_code: 'MAKE_IN_INDIA',
      requirement_name: 'Make in India (MII) Local Content Declaration',
      is_mandatory: true,
      verification_source: 'DPIIT_MII_DECLARATION',
      min_condition: 'Class-I Local Supplier (>=50% local content)',
      weight: 15,
      applicable_bidder_type: 'ALL',
    },
    {
      requirement_code: 'DEBARMENT',
      requirement_name: 'Non-Blacklisting / Debarment Affidavit',
      is_mandatory: true,
      verification_source: 'CPPP_GEM_DEBARMENT_LIST',
      min_condition: 'No debarment by any Central/State Govt/PSU',
      weight: 10,
      applicable_bidder_type: 'ALL',
    },
  ]),
  [_, S] = useState(false),
  [O, A] = useState('');
useEffect(() => {
  E();
}, []);
const E = async () => {
    try {
      n(true);
      const I = await api.getTenders();
      t(I);
    } catch (I) {
      console.error(I);
    } finally {
      n(false);
    }
  },
  T = async (I) => {
    if ((I.preventDefault(), !f || !p)) {
      A('Title and Department are required.');
      return;
    }
    try {
      (S(true),
        A(''),
        await api.createTender({
          tender_id: c,
          title: f,
          department: p,
          tender_value: y,
          deadline: new Date(g).toISOString(),
          description: m,
          requirements: j,
        }),
        a(false),
        E());
    } catch (R) {
      A(R.message || 'Failed to create tender');
    } finally {
      S(false);
    }
  },
  C = e.filter(
    (I) =>
      I.title.toLowerCase().includes(o.toLowerCase()) ||
      I.tender_id.toLowerCase().includes(o.toLowerCase()) ||
      I.department.toLowerCase().includes(o.toLowerCase()),
  );
return (
  <div className="space-y-6">
    <div className="flex flex-col sm:flex-row sm:items-center justify-between gap-4 bg-white p-6 rounded-2xl b
  order border-slate-200/90 shadow-card-soft">
      <div>
        <div className="flex items-center gap-2">
          <h1 className="text-xl font-extrabold text-slate-900 tracking-tight">
            {'Tenders & Evaluation Criteria'}
          </h1>
```

```
        <span className="bg-pine-100 text-pine-800 text-[10px] font-bold px-2.5 py-0.5 rounded-full font-mon
o">
          GFR Rule 149
        </span>
      </div>
      <p className="text-xs text-slate-500 mt-0.5">
        Configure statutory eligibility conditions, scoring weights, and mandatory compliance
        rules.
      </p>
    </div>
    <div className="flex items-center gap-3">
      <button
        onClick={() => a(true)}
        className="px-4 py-2.5 bg-gradient-to-r from-pine-800 to-teal-800 hover:from-pine-900 hover:to-teal-
900 text-white text-xs font-bold rounded-xl shadow-glow-teal transition-all flex items-center gap-2 group"
      >
        <PlusIcon className="w-4 h-4 text-champagne-300" />
        <span>Publish New Tender</span>
      </button>
    </div>
  </div>
  <div className="bg-white p-4 rounded-2xl border border-slate-200/90 shadow-card-soft">
    <div className="relative">
      <SearchIcon className="w-4 h-4 text-slate-400 absolute left-3.5 top-3" />
      <input
        type="text"
        placeholder="Search tenders by Title, Tender ID (e.g. GEM/2026/B/...), Department..."
        value={o}
        onChange={(I) => l(I.target.value)}
        className="w-full pl-10 pr-4 py-2 text-xs border border-slate-300 rounded-xl bg-slate-50 focus:bg-wh
ite focus:ring-2 focus:ring-teal-600 focus:outline-none transition-colors"
      />
    </div>
  </div>
  <div className="grid grid-cols-1 md:grid-cols-2 gap-5">
    {r ? (
      <div className="col-span-2 text-center py-12 text-slate-500 text-xs">
        Loading tenders registry...
      </div>
    ) : C.length === 0 ? (
      <div className="col-span-2 text-center py-12 text-slate-500 text-xs bg-white rounded-2xl border border
-dashed border-slate-300 p-8">
        No matching tenders found. Click "Publish New Tender" to create one.
      </div>
    ) : (
      C.map((I) => {
        var R, P, $;
        return (
          <div
            key={I.id}
            className="bg-white rounded-2xl border border-slate-200/90 p-5 shadow-card-soft hover:shadow-car
d-hover transition-all flex flex-col justify-between group"
          >
            <div>
              <div className="flex justify-between items-start mb-2.5">
                <span className="font-mono text-xs font-bold text-slate-900 bg-slate-100 px-2.5 py-1 rounded
-lg border border-slate-200">
                  {I.tender_id}
                </span>
                <span className="text-[10px] font-bold uppercase tracking-wider bg-pine-100 text-pine-800 px
-2.5 py-0.5 rounded-full">
                  {I.status}
                </span>
              </div>
              <h3 className="text-sm font-bold text-slate-900 leading-snug group-hover:text-pine-800 transit
ion-colors">
                {I.title}
              </h3>
              <p className="text-xs text-slate-500 mt-1 flex items-center gap-1.5">
                <Building2Icon className="w-3.5 h-3.5 text-slate-400 flex-shrink-0" />
                <span className="truncate">{I.department}</span>
              </p>
              <div className="grid grid-cols-2 gap-3 mt-4 pt-3 border-t border-slate-100 text-xs">
                <div className="bg-slate-50 p-2.5 rounded-xl border border-slate-100">
                  <span className="text-[10px] text-slate-400 block font-medium">
                    Tender Value:
                  </span>
                  <p className="font-bold font-mono text-slate-900 text-sm">
                    ₹{I.tender_value.toLocaleString('en-IN')}
```

```
                  </p>
                </div>
                <div className="bg-slate-50 p-2.5 rounded-xl border border-slate-100">
                  <span className="text-[10px] text-slate-400 block font-medium">
                    Evaluation Deadline:
                  </span>
                  <p className="font-semibold text-slate-800 text-xs mt-0.5 flex items-center gap-1">
                    <ClockIcon className="w-3.5 h-3.5 text-champagne-600" />
                    {new Date(I.deadline).toLocaleDateString()}
                  </p>
                </div>
              </div>
              <div className="mt-4">
                <span className="text-[10px] font-bold text-slate-400 uppercase tracking-wider block mb-1.5"
>
                  Configured Criteria (
                  {((R = I.requirements) == null ? undefined : R.length) || 0})
                </span>
                <div className="flex flex-wrap gap-1.5">
                  {(P = I.requirements) == null
                    ? undefined
                    : P.slice(0, 5).map((D) => (
                        <span
                          key={D.id || D.requirement_code}
                          className="text-[10px] font-mono bg-slate-100 text-slate-700 px-2 py-0.5 rounded-m
d font-semibold border border-slate-200"
                        >
                          {D.requirement_code}
                        </span>
                      ))}
                  {((($ = I.requirements) == null ? undefined : $.length) || 0) > 5 && (
                    <span className="text-[10px] font-mono text-pine-800 bg-pine-50 px-2 py-0.5 rounded-md f
ont-bold border border-pine-200">
                      +{I.requirements.length - 5}
                      {' more'}
                    </span>
                  )}
                </div>
              </div>
            </div>
            <div className="mt-5 pt-3.5 border-t border-slate-100 flex justify-between items-center text-xs"
>
              <span className="text-slate-500 font-mono text-[11px]">
                <strong>{I.bids_count || 0}</strong>
                {' Submissions'}
              </span>
              <Link
                to={`/bids?tender_id=${I.id}`}
                className="inline-flex items-center gap-1.5 px-3 py-1.5 bg-pine-50 hover:bg-pine-800 text-pi
ne-900 hover:text-white font-bold rounded-xl transition-all border border-pine-200 text-[11px]"
              >
                <span>Review Submissions</span>
                <ChevronRightIcon className="w-3.5 h-3.5" />
              </Link>
            </div>
          </div>
        );
      })
    )}
  </div>
  {i && (
    <div className="fixed inset-0 z-50 bg-slate-950/75 backdrop-blur-sm flex items-center justify-center p-4
">
      <div className="bg-white w-full max-w-4xl max-h-[90vh] rounded-3xl shadow-2xl border border-slate-300
flex flex-col overflow-hidden animate-in fade-in zoom-in-95 duration-150">
        <div className="px-6 py-4 bg-gradient-to-r from-[#060D17] to-[#0D3830] text-white flex justify-betwe
en items-center border-b border-slate-800">
          <div className="flex items-center gap-3">
            <div className="w-9 h-9 rounded-xl bg-pine-800/80 border border-pine-600/50 flex items-center ju
stify-center">
              <FileTextIcon className="w-5 h-5 text-champagne-400" />
            </div>
            <div>
              <h2 className="text-base font-bold">
                {'Publish Tender & Define Criteria Matrix'}
              </h2>
              <p className="text-xs text-slate-300">
                Set mandatory eligibility criteria and scoring weights under GFR 2017
              </p>
```

```
            </div>
          </div>
          <button
            onClick={() => a(false)}
            className="p-1.5 rounded-lg text-slate-400 hover:text-white hover:bg-white/10 transition-colors"
          >
            <XIcon className="w-5 h-5" />
          </button>
        </div>
        <form onSubmit={T} className="flex-1 p-6 overflow-y-auto space-y-5 custom-scrollbar">
          {O && (
            <div className="p-3 bg-rose-50 text-rose-800 rounded-xl text-xs font-semibold border border-rose
-200">
              {O}
            </div>
          )}
          <div className="grid grid-cols-1 sm:grid-cols-2 gap-4 text-xs">
            <div>
              <label className="block font-bold text-slate-700 mb-1">
                Tender ID (GeM Code)
              </label>
              <input
                type="text"
                required
                value={c}
                onChange={(I) => u(I.target.value)}
                className="w-full p-2.5 border border-slate-300 rounded-xl bg-slate-50 font-mono text-xs foc
us:ring-2 focus:ring-teal-600 focus:outline-none"
                />
            </div>
            <div>
              <label className="block font-bold text-slate-700 mb-1">
                Procuring Department / Ministry
              </label>
              <input
                type="text"
                required
                placeholder={'e.g. Department of Electronics & IT'}
                value={p}
                onChange={(I) => h(I.target.value)}
                className="w-full p-2.5 border border-slate-300 rounded-xl bg-slate-50 text-xs focus:ring-2
focus:ring-teal-600 focus:outline-none"
                />
            </div>
            <div className="sm:col-span-2">
              <label className="block font-bold text-slate-700 mb-1">
                Tender Title / Scope of Work
              </label>
              <input
                type="text"
                required
                placeholder={
                  'e.g. Supply, Installation & Maintenance of Enterprise Cloud Server Hardware'
                }
                value={f}
                onChange={(I) => d(I.target.value)}
                className="w-full p-2.5 border border-slate-300 rounded-xl bg-slate-50 text-xs focus:ring-2
focus:ring-teal-600 focus:outline-none"
                />
            </div>
            <div>
              <label className="block font-bold text-slate-700 mb-1">
                Estimated Tender Value (INR ₹)
              </label>
              <input
                type="number"
                required
                value={y}
                onChange={(I) => x(Number(I.target.value))}
                className="w-full p-2.5 border border-slate-300 rounded-xl bg-slate-50 font-mono text-xs foc
us:ring-2 focus:ring-teal-600 focus:outline-none"
                />
            </div>
            <div>
              <label className="block font-bold text-slate-700 mb-1">
                {'Submission & Evaluation Deadline'}
              </label>
              <input
                type="date"
```

```
                  required
                  value={g}
                  onChange={(I) => v(I.target.value)}
                  className="w-full p-2.5 border border-slate-300 rounded-xl bg-slate-50 text-xs focus:ring-2
    focus:ring-teal-600 focus:outline-none"
                 />
               </div>
             </div>
             <div className="pt-4 border-t border-slate-200">
               <RequirementsEditor requirements={j} onChange={w} />
             </div>
             <div className="flex justify-end items-center gap-3 pt-4 border-t border-slate-200">
               <button
                 type="button"
                 onClick={() => a(false)}
                 className="px-4 py-2.5 text-xs font-semibold text-slate-600 hover:bg-slate-100 rounded-xl tran
    sition-colors"
               >
                 Cancel
               </button>
               <button
                 type="submit"
                 disabled={_}
                 className="px-6 py-2.5 text-xs font-bold text-white bg-gradient-to-r from-pine-800 to-teal-800
     hover:from-pine-900 hover:to-teal-900 rounded-xl shadow-glow-teal transition-all"
               >
                 {_ ? 'Publishing Tender...' : 'Publish Tender & Criteria'}
               </button>
             </div>
           </form>
         </div>
       </div>
     )}
   </div>
  );
};


//// FILE: src/routes/ProtectedRoute.jsx  (7 lines)

import { Navigate } from 'react-router-dom';
import { useAuth } from '../context/AuthContext';

export const ProtectedRoute = ({ children: e }) => {
  const { isAuthenticated: t } = useAuth();
  return t ? <>{e}</> : <Navigate to="/login" replace />;
};


//// FILE: start-all.bat  (4 lines)

@echo off
rem Opens the backend and the frontend, each in its own window
start "GeM BidVerify - backend" cmd /k "%~dp0start-backend.bat"
start "GeM BidVerify - frontend" cmd /k "%~dp0start-frontend.bat"


//// FILE: start-backend.bat  (13 lines)

@echo off
rem Starts the FastAPI backend on http://127.0.0.1:8000  (API docs: http://127.0.0.1:8000/docs)
rem Optional e-mail settings live in secrets.bat (copy secrets.bat.example to secrets.bat and edit it).
cd /d "%~dp0backend"
if exist "%~dp0secrets.bat" call "%~dp0secrets.bat"
if not exist .venv (
  echo Creating Python virtual environment...
  py -3 -m venv .venv 2>nul || python -m venv .venv
)
call .venv\Scripts\activate.bat
python -m pip install --quiet -r requirements.txt
python -m uvicorn app.main:app --reload --port 8000
pause


//// FILE: start-frontend.bat  (6 lines)

@echo off
rem Starts the React app on http://localhost:5173
cd /d "%~dp0"
```

```
if not exist node_modules call npm install
call npm run dev
pause


//// FILE: tailwind.config.js  (47 lines)

/** @type {import('tailwindcss').Config} */
export default {
  content: ['./index.html', './src/**/*.{js,jsx}'],
  theme: {
    extend: {
      fontFamily: {
        sans: ['Plus Jakarta Sans', 'Inter', 'system-ui', '-apple-system', 'sans-serif'],
        mono: ['JetBrains Mono', 'Fira Code', 'monospace'],
      },
      colors: {
        "champagne": {
              "50": "#FFFBEB",
              "100": "#FEF3C7",
              "200": "#FDE68A",
              "300": "#FCD34D",
              "400": "#F59E0B",
              "500": "#D97706",
              "600": "#C2410C",
              "700": "#9A3412",
              "800": "#78350F",
              "900": "#451A03"
        },
        "pine": {
              "400": "#2DD4BF",
              "500": "#14B8A6",
              "600": "#0F766E",
              "700": "#134E43",
              "800": "#0D3830",
              "900": "#08231E",
              "950": "#041411"
        },
        "obsidian": {
              "850": "#0E1C2F",
              "900": "#0B1524",
              "950": "#060D17"
        }
  },
      boxShadow: {
        'card-soft': '0 2px 10px -2px rgba(0, 0, 0, .04), 0 1px 3px -1px rgba(0, 0, 0, .02)',
        'card-hover': '0 12px 24px -6px rgba(11, 21, 36, .08), 0 4px 8px -2px rgba(11, 21, 36, .04)',
        'glow-teal': '0 0 20px -3px rgba(20, 184, 166, .25)',
        'glow-champagne': '0 0 20px -3px rgba(245, 158, 11, .25)',
      },
    },
  },
  plugins: [],
};


//// FILE: vite.config.js  (13 lines)

import { defineConfig } from 'vite';
import react from '@vitejs/plugin-react';
import { viteSingleFile } from 'vite-plugin-singlefile';

// `npm run dev`   -> dev server with hot reload; requests to /api are forwarded to the backend on port 8000
// `npm run build` -> dist/index.html as ONE self-contained file
const backend = 'http://127.0.0.1:8000';

export default defineConfig({
  plugins: [react(), viteSingleFile()],
  server: { port: 5173, proxy: { '/api': { target: backend, changeOrigin: true } } },
  preview: { proxy: { '/api': { target: backend, changeOrigin: true } } },
});
```